

医院信息化

原理、架构与工程实践

以 HIP 平台为贯穿案例

徐剑 著

送审稿 · v0.1

2026 年 8 月

目 录

前言与导读	1
这本书讲什么	1
读者对象	1
全书结构	1
体例约定	2
配套资源	2
致谢与说明	3
第 1 章 医院信息化概论与政策环境	4
学习目标	4
1.1 医院信息化的演进：三次浪潮	4
1.2 电子病历系统应用水平分级评价	6
1.3 医院信息互联互通标准化成熟度测评	7
1.4 智慧服务与智慧管理分级评估	8
1.5 网络安全与数据合规	9
1.6 医保支付改革：当前最强的驱动力	10
1.7 建设模式与产业格局	11
1.8 本书案例平台 HIP 概览	12
本章小结	16
思考与练习	16
第 2 章 医院业务流程全景	17
学习目标	17
2.1 医院组织结构与信息系统角色模型	17
2.2 门诊诊疗全流程	19
2.3 急诊流程：预检分诊与留观	22
2.4 住院诊疗全流程	23
2.5 医技科室业务：科室视角	26
2.6 药事流程	27
2.7 运营后勤流程	28
2.8 患者视角的就医旅程	28
本章小结	29
思考与练习	30
第 3 章 标准与规范	31

学习目标	31
3.1 分类编码体系	31
3.2 卫生信息数据元与值域代码	34
3.3 电子病历文档标准：CDA 与 WS/T 500	35
3.4 消息交换标准：HL7 V2 与 MLLP	37
3.5 HL7 FHIR R4：新一代互操作	37
3.6 医保编码与接口规范	39
3.7 标准落地策略：内建 vs 事后改造	39
本章小结	41
思考与练习	41
第 4 章 总体架构与技术选型	43
学习目标	43
4.1 医院信息系统架构演进	43
4.2 关键抉择：模块化单体 vs 微服务	44
4.3 五层总体架构	46
4.4 技术栈选型方法	48
4.5 数据库架构：单一主库与命名空间的谱系	49
4.6 工程结构：Monorepo 与模块边界	50
4.7 前端架构	52
4.8 架构决策记录（ADR）方法论	54
本章小结	55
思考与练习	55
第 5 章 平台底座	57
学习目标	57
5.1 底座先行原则	57
5.2 认证与授权	58
5.3 组织机构与主数据	61
5.4 字典与术语管理	61
5.5 EMPI 患者主索引	62
5.6 系统参数中心：sys_config	65
5.7 审计日志	66
5.8 数据库迁移纪律：Flyway 版本管理	67
本章小结	68
思考与练习	69
第 6 章 集成平台	70
学习目标	70
6.1 医院集成的问题域	70
6.2 商用 ESB、开源引擎与轻量自研	71
6.3 HL7 V2/MLLP 实战	72
6.4 适配器与 SPI 设计模式	74
6.5 文件/DB 适配器注册与内容路由	77
6.6 事务发件箱与事件驱动：不引入消息中间件的跨模块通信	77
6.7 危急值管理闭环	78

6.8 外部条件工程化	80
本章小结	81
思考与练习	81
第 7 章 门急诊系统	83
学习目标	83
7.1 号源与排班模型	83
7.2 门诊医生站	85
7.3 统一订单行	85
7.4 合理用药与 CDSS	87
7.5 门诊收费	88
7.6 聚合支付	89
7.7 药房发药与审方	90
7.8 预约与叫号体系	92
7.9 急诊预检分诊与留观	93
本章小结	93
思考与练习	94
第 8 章 住院与护理	95
学习目标	95
8.1 住院登记与床位管理	95
8.2 记账式医嘱体系	97
8.3 住院电子病历与 CA 签名冻结	98
8.4 护理执行闭环	99
8.5 护理风险评估	101
8.6 护理白板与交接班	101
8.7 出院结算	102
8.8 移动护理与移动查房	104
8.9 专科场景：ICU 与临床路径	104
本章小结	105
思考与练习	105
第 9 章 医技系统	107
学习目标	107
9.1 LIS 检验流程：标本流转状态机	107
9.2 检验设备直连与危急值触发	109
9.3 RIS 检查报告	110
9.4 PACS 与影像	111
9.5 病理系统	112
9.6 手术麻醉	113
9.7 用血管理	114
9.8 检查检验时段预约	114
本章小结	115
思考与练习	115
第 10 章 医保接口与 DRG	117
学习目标	117

10.1 医保业务模型：三大目录与目录对照	117
10.2 费用分割引擎	118
10.3 结算流程：上传、冲正与一个真实教训	120
10.4 智能审核	122
10.5 对账体系	123
10.6 CHS-DRG 入组与支付分析	124
10.7 省级 SDK 适配与实施工艺	127
10.8 延伸：基金监管侧视角	128
本章小结	129
思考与练习	129
第 11 章 临床数据中心与患者 360	131
学习目标	131
11.1 CDR 的定位：与业务库分离的理由	131
11.2 数据同步：增量、CDC 与事件驱动的取舍	133
11.3 患者 360 视图：跨域聚合与时间轴	135
11.4 全文检索：病历检索的实现路径	136
11.5 CDA 文档生成：从业务数据到标准骨架	137
11.6 数据订阅与推送：向外供数的服务化	138
本章小结	139
思考与练习	140
第 12 章 数据治理、质控与统计上报	141
学习目标	141
12.1 数据治理框架	141
12.2 病案管理	143
12.3 病案统计	144
12.4 医疗质控	145
12.5 医院感染监测	145
12.6 单病种上报与评审指标	147
12.7 自定义报表引擎：SELECT 白名单的安全设计	148
12.8 运营分析	151
本章小结	151
思考与练习	152
第 13 章 安全、合规与运维	153
学习目标	153
13.1 等保三级工程落地：从条款到清单	153
13.2 数据脱敏与隐私	155
13.3 备份与恢复	156
13.4 运维中心	158
13.5 性能工程	159
13.6 测试体系	160
本章小结	163
思考与练习	163
第 14 章 产品化与实施方法论	165

学习目标	165
14.1 从单院系统到标准产品	165
14.2 四层标准化模型	166
14.3 单院独立部署 vs 多租户 SaaS	167
14.4 种子数据分层	168
14.5 医院初始化工程	168
14.6 模块开关与品牌配置化	169
14.7 实施定制层规约	170
14.8 发布工艺	171
14.9 实施全流程	171
14.10 需求响应与偏离管理	172
14.11 试点运行与反馈闭环	173
14.12 验收方法论：虚拟医院全量彩排	173
本章小结	174
思考与练习	175
附录 A AI 在医院信息化中的应用	177
A.1 两个“智能”的真实形态：读码确认	177
A.2 边界设计：为什么 AI 能力放平台外	177
A.3 医院 AI 应用的接入形态谱系	178
A.4 「AI 属外部条件」的工程化	178
附录 B 实训环境搭建指南	180
B.1 环境要求	180
B.2 装机步骤与常见坑	180
B.3 初始化一所虚拟医院	181
B.4 验证清单：跑通一条门诊全流程	181
B.5 各章实践题与实训环境对应表	182
附录 C 术语表	184
C.1 业务与临床	184
C.2 架构与工程	186
C.3 标准与数据	188
C.4 医保与 DRG	188
C.5 安全与运维	189
C.6 政策与评价体系	190
后记	191

前言与导读

这本书讲什么

这是一本讲医院信息化如何真正落地的教材。它有两条线：一条是原理线——医院业务如何运转、行业标准如何约束、架构如何取舍、医保如何计算；另一条是工程线——以一个真实迭代到 v1.0.0 定版的医院信息平台（书中称 HIP）为贯穿案例，把每个原理落到可对照源码的实现上。

市面上的医院信息化读物大致两类：概念型教材把术语讲全但落不了地，厂商文档把操作讲细但讲不出为什么。本书想占住中间那块空地：每一节先讲这类问题的通用解法与业界方案，再剖析一种真实落地的做法——包括它的取舍、简化，以及踩过的坑。书中的“教训案例”（如医保结算号未持久化、增量同步水位线漏抓）都是真实发生的返工，我们选择原样呈现而非事后美化，因为返工的逻辑比成功的结论更有教学价值。

读者对象

- 医院信息科工程师：把全书当作系统建设与选型的参考框架，重点读架构篇、工程篇；
- HIS 开发与实施人员：业务篇与第 14 章实施方法论是为你写的，案例代码可直接对照工作；
- 医学信息类专业高年级本科生与研究生：按篇章顺序通读，配合附录 B 的实训环境完成各章练习。

阅读前置：了解基本的数据库与 Web 开发概念即可；代码示例以 Java/SQL 为主，但重点从来不是语法，而是模型与取舍。

全书结构

篇	章	主题
基础篇	第 1 章	概论与政策环境（评价体系、合规框架、医保改革）
	第 2 章	医院业务流程全景（三流合一的分析框架）
	第 3 章	标准与规范（编码、数据元、CDA、HL7、FHIR、贯标）
架构篇	第 4 章	总体架构与技术选型（约束驱动的架构决策）
	第 5 章	平台底座（认证授权、主数据、EMPI、配置、迁移纪律）
	第 6 章	集成平台（HL7/MLLP、适配器 SPI、危急值闭环）
业务篇	第 7 章	门急诊系统（号源、订单行、CDSS、收费、发药、分诊）
	第 8 章	住院与护理（占床、记账式医嘱、护理闭环、出院结算）
	第 9 章	医技系统（LIS/RIS/病理/手麻/用血：状态机与实物流对齐）
	第 10 章	医保接口与 DRG（分割、结算冲正、对账、入组与支付模拟）
数据篇	第 11 章	临床数据中心与患者 360（CDR、同步、CDA 生成）
	第 12 章	数据治理、质控与统计上报（病案、院感、报表引擎）
工程篇	第 13 章	安全、合规与运维（安全基线、备份恢复、测试体系）
	第 14 章	产品化与实施方法论（四层模型、初始化、发布、彩排）
附录	A/B/C	AI 应用 实训环境搭建 术语表

推荐读法：教学按篇章顺序；工程师可从第 4 章切入，遇到业务背景不足再回第 2 章；实施人员可先读第 14 章建立全局，再按需下钻各业务章。各章自成闭环（学习目标—正文—案例框—小结—习题），跨章依赖均以「(第 X 章)」显式标注。

体例约定

- 案例框：编号为「案例框 N-x」的引用块是 HIP 真实实现的剖析，类名、表名、代码片段均可在配套仓库中对照源码；
- 业务场景框（第 2 章）：以具体患者与角色的叙事串联流程，场景对应真实功能页能力；
- 【成稿核对：xx】：涉及政策文号、标准版本、评级条目数等外部事实，写作时无法当场核实的，以此标注保守处理——正式出版前须逐条对照最新官方文件核定，全书共约 60 处；
- 诚实披露：凡 HIP 的实现相对行业完整形态有简化（如目录为示例集、某链路未任务化），正文以「当前简化/演进方向」口径明示，不虚构能力。部分真实缺陷被有意保留为教学点与习题（对应修复工作在平台侧另行进行）。

配套资源

- 实训环境：附录 B 给出完整搭建指南（Java 21 + Spring Boot 3.5 + PostgreSQL 16 + Vue 3），一台普通开发机即可运行全部实验；
- 各章习题：概念、计算、设计、排障、论述、实践六类，实践题均可在实训环境中完成，附录 B 附有习题与模块的对应表。

致谢与说明

本书全部案例源自 HIP 平台从零到 v1.0.0 的真实开发历程——55 个功能页、44 个数据库迁移版本、19 套端到端测试套件，以及贯穿其中的架构决策记录与版本规划文档。感谢这段工程实践本身：它证明了中小医院的信息化可以在有限资源下做到工程上的体面，这也是本书最想传递的信念。

第 1 章 医院信息化概论与政策环境

本章属第一篇「基础篇」，是全书的起点。医院信息化是一个被政策深度塑造的工程领域——不理解评价体系与合规要求，就无法理解医院信息系统为什么长成今天这个样子，也无法理解一份采购需求里为什么会出现那些条目。本章先梳理三十余年演进的内在逻辑，再逐一拆解三大评价体系（电子病历分级评价、互联互通测评、智慧服务与智慧管理评估）与两条硬约束线（安全合规、医保支付），最后引出贯穿全书的案例平台 HIP，给出全书的阅读地图。后续各章会反复引用本章建立的政策坐标系。

学习目标

学完本章，你应当能够：

1. 概述医院信息化三次浪潮的内容与各自的驱动力，解释“叠加而非替代”的演进特征；
2. 说出电子病历系统应用水平分级评价的框架（0 - 8 级）与 4 级的标志性要求，理解“评级要求如何变成系统需求清单”；
3. 区分互联互通标准化成熟度测评与分级评价的定位差异，说出测评考察的主要板块与四级甲等的技术要点；
4. 描述“三位一体”智慧医院的评价格局（电子病历 / 智慧服务 / 智慧管理）；
5. 列出医院信息系统面临的核心合规要求（等保三级、数据安全法、个人信息保护法）及其在系统设计上的落点；
6. 解释医保支付改革（贯标、DRG/DIP、基金监管）为什么是当前医院信息化最强的驱动力；
7. 比较自研、外购、混合三种建设模式的适用条件，说出信创趋势对技术选型的影响；
8. 描绘 HIP 案例平台的整体结构（十域 55 功能页），知道全书各章与平台各部分的对应关系。

1.1 医院信息化的演进：三次浪潮

1.1.1 为什么从收费开始

医院是业务流、资金流、物流耦合最紧密的组织之一：一张处方同时是医疗行为记录（业务流）、收费依据（资金流）和药品出库指令（物流）。信息化从哪一端切入，取决于哪一端的痛最急、账最清楚。

答案在所有国家几乎一致：从钱开始。二十世纪九十年代，我国医院信息化的第一批系统是财务电算化与“划价收费”系统——挂号、划价、收费、发药、药库进销存。这一代系统被后人称为“以收费为中心的 HIS”，其原始驱动力是财务管理：手工划价慢且错漏多、日结对账靠算盘、药品跑冒滴漏无从追查。信息系统首先解决的是“钱和药算得清”的问题。原卫生部在这一时期组织的信息化推

进工程与《医院信息系统基本功能规范》为第一代 HIS 提供了官方框架【成稿核对：工程名称（“金卫工程”）与《医院信息系统基本功能规范》发布年份（2002 年版）】。

不要轻视这一代系统。它完成了两件影响深远的事：其一，把医院最核心的主数据（药品目录、收费项目、科室人员）第一次数字化；其二，确立了“收费主线”这条至今仍是一切 HIS 骨架的业务链——挂号→开单→收费→执行→发药。本书第 2 章讲医院业务流程、第 7 章讲门急诊系统时，你会看到这条主线仍然是系统设计的中轴。

1.1.2 第二次浪潮：临床信息化

进入二十一世纪，矛盾转移了。收费系统里只有“项目和金额”，没有“病情和过程”——医生开了什么药系统知道，为什么开、患者对什么过敏、检验结果是多少，系统一无所知。医疗质量与安全成为第二次浪潮的驱动力，信息化的口号从“以收费为中心”转向“以患者（临床）为中心”。

这一时期建设的主角是临床系统家族：电子病历（EMR）、检验信息系统（LIS）、医学影像存档与传输系统（PACS/RIS）、医生工作站与护士工作站、移动护理、手术麻醉、重症监护。与第一代系统“一套软件包打天下”不同，临床系统天然多源——检验、影像、病理各有专业厂商，设备各有接口协议，“系统孤岛”问题由此埋下（第 6 章集成平台所要解决的问题域，正是这一时期形成的）。

临床信息化的制度化标志是电子病历系统应用水平分级评价的建立（1.2 节详述）：政府第一次用一把统一的尺子度量医院临床信息化的深度，并将其与医院等级评审挂钩。评价指挥棒使临床信息化从“先进医院的自选动作”变成“所有医院的规定动作”。

1.1.3 第三次浪潮：平台化与数据驱动

第三次浪潮大约始于 2010 年代中后期，至今仍在进行。它有三个相互交织的主题：

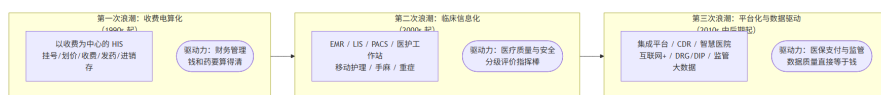
平台化整合。孤岛系统的代价在这一阶段集中爆发：一个患者的数据散落在十几个系统里，调阅要开十几个窗口，上报要人工汇总。解法是集成平台 + 临床数据中心（CDR）：用标准化的消息与文档把各系统连起来，把临床数据汇聚到独立的数据中心供调阅、质控、上报与科研（第 6 章、第 11 章）。国家层面的互联互通标准化成熟度测评（1.3 节）正是这一方向的指挥棒。

数据驱动的管理与支付。数据不再只是业务的副产品，而成为管理与结算的直接依据：DRG/DIP 医保支付按病案数据付费（第 10 章），公立医院绩效考核按上报指标排名，医保基金监管用大数据筛查违规。数据质量第一次直接等于钱——这是第三次浪潮最深刻的变化，它把信息化的压力从信息科传导到了病案室、临床科室和院长办公会。

智慧医院与互联网+。面向患者的服务（预约、缴费、报告查询、互联网诊疗）与面向管理的智慧运营被纳入评价体系（1.4 节），信息化的服务对象从院内工作人员扩展到患者和管理者。

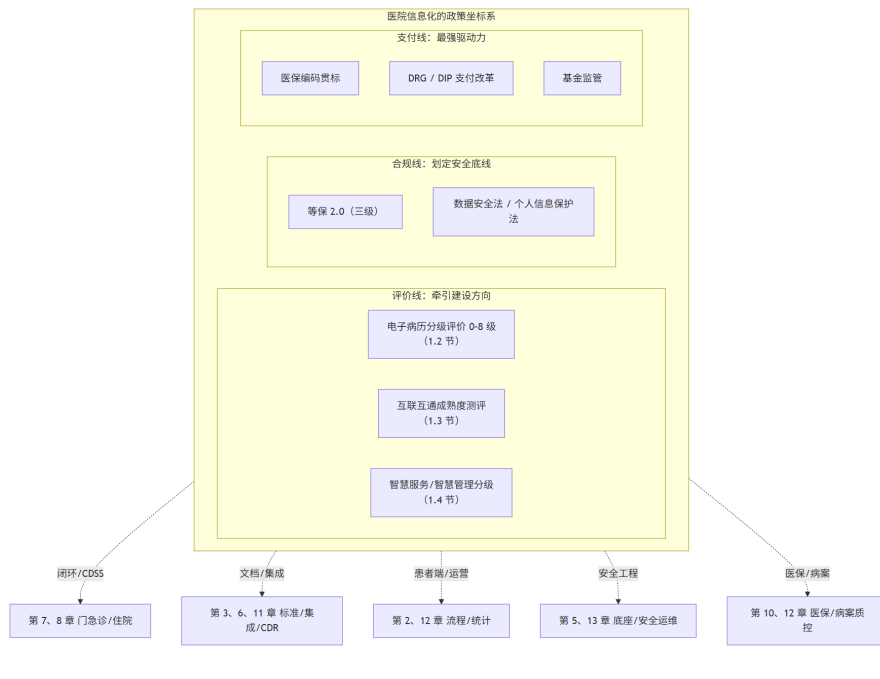
1.1.4 演进的内在逻辑

把三次浪潮放在一起看（图 1-1），有三条规律值得记住：



1. 驱动力一路从“院内管理”走向“外部约束”：财务管理是院内需求，医疗质量一半靠评价指挥棒，医保支付则完全是外部规则——医院信息化越来越像一场“应试”，读懂考纲（政策）成了做系统的前提。
2. 浪潮是叠加而非替代。第三次浪潮并没有淘汰收费主线和临床系统，而是在其上加盖平台层与数据层。今天任何一家医院的系统截面里，三代建设成果同时在运行——这也是医院信息系统复杂性的来源：你面对的不是一个时代的产物，而是三十年地层的堆积。
3. 数据的角色一路升格：从记账凭证（第一代），到临床记录（第二代），到生产要素与结算依据（第三代）。相应地，对数据的要求从“算得对”到“记得全”再到“标准化、可交换、经得起稽核”。

本章余下部分讲的政策环境，可以归纳为三条线（图 1-2）：评价线牵引建设方向（1.2 - 1.4），合规线划定安全底线（1.5），支付线提供最强驱动力（1.6）。三条线共同构成医院信息化的“考纲”。



1.2 电子病历系统应用水平分级评价

1.2.1 定位：度量临床信息化的深度

电子病历系统应用水平分级评价（业内简称“分级评价”或“电子病历评级”）由国家卫生健康委医政部门主导，评的是临床信息化应用的深度：不是“买没买系统”，而是“系统在临床工作中被真实使用到什么程度”。评价制度于 2011 年建立试行，2018 年发布修订版评价标准并要求全国二级以上医院全员参评【成稿核对：2011 年试行文件与 2018 年《电子病历系统应用水平分级评价管理办法（试行）》及标准的文号；“到 2020 年三级医院达到 4 级以上、二级医院达到 3 级以上”的目标出处】。

三大评价体系中，分级评价对医院信息系统功能建设的牵引最直接——它逐条规定了医嘱、病历、检验检查、用药等环节应达到的电子化与智能化水平，是 HIS/EMR 采购需求最主要的政策来源。

1.2.2 分级框架：0-8 级

分级评价满分为 8 级，从 0 到 8 共九个等级，刻画从“无电子病历”到“数据整合与持续改进”的完整阶梯：

级别	主题（概括）
0 级	未形成电子病历系统
1 级	独立医疗信息系统建立
2 级	医疗信息部门内部交换
3 级	部门间数据交换
4 级	全院信息共享，初级医疗决策支持
5 级	统一数据管理，中级医疗决策支持
6 级	全流程医疗数据闭环管理，高级医疗决策支持
7 级	医疗安全质量管控，区域医疗信息共享
8 级	健康信息整合，医疗安全质量持续提升

【成稿核对：各级主题表述以 2018 版评价标准原文为准】

读这张阶梯要抓住两条主线的递进：

- 共享范围：单系统 → 部门内 → 部门间 → 全院 → 区域。这条线与互联互通测评（1.3）在方向上重叠，但分级评价关心的是“临床用起来了没有”；
- 智能程度：无 → 初级决策支持（如药品剂量提示）→ 中级（如基于知识库的用药审查）→ 高级（如闭环管理中的自动拦截与质量管控）。

评价方法上，分级评价按临床工作角色划分若干评价项目（如病房医嘱处理、检查检验报告、药品配置等），每个项目分别评“电子病历系统功能状态”与“有效应用比例”——功能有但没人用，不得分。总分之外还有“基本项目”约束：申报某级别必须满足该级别全部基本项目，防止靠长板凑分【成稿核对：角色与项目数量（2018 版为 10 类角色 39 个评价项目）及基本项/选择项规则】。“有效应用比例”这个设计值得注意：它使得评级材料必须来自生产系统的真实运行数据，从制度上排除了“为评级搭演示系统”的做法。

1.2.3 4 级：多数医院的第一道坎

4 级（全院信息共享，初级医疗决策支持）是政策要求二级以上医院普遍达到的水平，也是系统建设含金量的分水岭。它的两个关键词各自对应一组硬性的系统能力：

全院信息共享：医生站能看到检验、检查、用药、护理的全部结果；数据在挂号、收费、药房、医技之间自动流转而非人工转录。这意味着全院系统必须打通——要么一体化平台，要么集成平台把异构系统连起来（第 6 章）。

初级医疗决策支持：系统在关键环节给出自动提示——药品剂量与相互作用审查、危急值提醒、过敏警示。这意味着必须有结构化的字典与规则库支撑（第 7 章合理用药五重拦截即为此类能力的完整案例）。

再往上，5 级要求全院统一的数据管理与更强的知识库支持，6 级要求全流程闭环管理——医嘱从开立、审核、执行到结果反馈的每个环节都被系统记录且可追溯，任何一环缺失即闭环断裂（第 8 章的医嘱执行、第 9 章的标本流转状态机，都是闭环建模的具体形态）。

1.2.4 评级要求如何变成需求清单

对工程师，分级评价最实用的读法是把它当作需求清单的生成器。以“危急值闭环”这一典型评价内容为例，一条评价要求展开后是这样一串系统需求：

评价要求（概括）	展开的系统需求	本书章节
检验危急值及时提醒与处理	LIS 判定危急值（HH/LL 界值维护） → 自动推送医生站 → 医生确认与处理记录 → 超时升级 → 全程留痕可查	第 6、9 章
用药决策支持	药品知识库、过敏/相互作用/剂量规则、开单环节实时拦截、拦截记录	第 7 章
医嘱闭环	医嘱状态机、执行扣库存、执行人与时间留痕、未执行拦截出院	第 8 章
病历质量管控	时限质控、环节质控、签名与版本冻结	第 8、12 章

医院申报评级前的“差距分析”，本质上就是拿评价标准逐条对照现有系统生成这样一张表——这也是信息科向院领导要预算时最有力的语言。反过来，厂商的产品规划也围绕评价条款展开：评价标准是这个行业事实上的公共需求规格说明书。

1.3 医院信息互联互通标准化成熟度测评

1.3.1 定位：度量“连接”的标准化程度

互联互通标准化成熟度测评（业内简称“互联互通测评”）由国家卫生健康委统计信息中心组织，评的是医院信息平台建设与数据交换的标准化程度。与分级评价“评临床应用深度”不同，互联互通测评回答的问题是：“你的数据是否按国家标准组织？你的系统之间、你与外部（区域平台）之间，是否通过标准化的服务交换数据？”

两者的关系可以概括为一纵一横：分级评价纵向钻临床业务的深度，互联互通横向查数据交换的宽度与规范度。一家医院通常两个都要考，且建设内容大量复用——按标准建的临床数据中心，既是互联互通的核心考点，也是分级评价高级别的基础设施。

1.3.2 测评框架

互联互通测评的等级从低到高为：一级、二级、三级、四级乙等、四级甲等、五级乙等、五级甲等，共七档【成稿核对：等级划分与各级要求以现行《医院信息互联互通标准化成熟度测评方案》为准】。四级甲等是多数大中型医院的现实目标，五级乙等及以上目前仅少数头部医院达到。

测评内容分四大板块：

1. 数据资源标准化建设：医院数据是否对齐国家卫生信息标准——数据元与值域代码（WS 363/364/365 系列）、电子病历共享文档（WS/T 500 系列 CDA 文档）。通俗地说：你的“性别”字段是不是用国家标准代码，你的出院小结能不能导出成标准 CDA 文档。这部分的技术细节在第 3 章（标准与规范）展开，工程落地在第 11 章（CDA 文档生成）；
2. 互联互通标准化建设：基于信息平台的标准化交互服务——文档注册与调阅、术语与字典服务、患者主索引服务等。文档注册/调阅正是第 11 章临床数据中心的核心理功能；
3. 基础设施建设：信息平台、硬件、网络与信息安全（与等保要求呼应，见 1.5 节与第 13 章）；
4. 互联互通应用效果：平台是否真实支撑业务——联通的业务系统数量、与区域平台的对接、面向患者与医务人员的应用。

四级甲等的技术要点可以概括为：建成基于标准的医院信息平台，实现一定数量的标准共享文档与标准服务，主要业务系统通过平台交换数据，并与外部（区域/上级平台）实现联通【成稿核对：四级甲等对共享文档类数、交互服务数、联通系统数的具体量化要求】。

1.3.3 对系统建设的实际牵引

互联互通测评对建设的牵引集中在三处，也是实施中最常见的三类改造：

- 数据标准化改造：字典对齐国家值域代码、关键表补齐标准数据元——对老系统这是伤筋动骨的工程，对新建系统则应“标准内建”（第 3 章讨论这两种策略的成本差异，HIP 采用后者）；
- 平台与文档服务建设：建集成平台与 CDR，业务数据生成标准 CDA 文档并提供注册、检索、调阅服务（第 6 章、第 11 章）；
- 留痕与可验证性：测评要求交互服务可实测、报文可查验——集成平台的出入站留痕（第 6 章）在这里第二次兑现价值（第一次是排障，第三次是医保对账，见第 10 章）。

一个务实的提醒：互联互通测评以“平台”为中心的框架，隐含假设是医院存在多个异构系统需要集成。对采用一体化平台的中小医院，数据天然在一个库里，“互联互通”的重点就从“连接改造”转移为“标准表达”——按标准输出文档与服务。HIP 的定位正属后者，这也是第 3 章“标准内建”路线的政策注脚。

1.4 智慧服务与智慧管理分级评估

1.4.1 “三位一体”的智慧医院评价格局

2019 年起，国家卫生健康委在电子病历分级评价之外陆续建立医院智慧服务分级评估（面向患者的服务能力，0-5 级）与医院智慧管理分级评估（面向医院运营管理，0-5 级），与电子病历评级

共同构成“三位一体”的智慧医院评价体系【成稿核对：智慧服务评估标准发布时间（2019 年）与智慧管理评估标准发布时间（2021 年）及文号】。三者分工明确：

评价体系	面向	级别	关键词
电子病历分级评价	医务人员（临床）	0 - 8 级	病历、医嘱、闭环、决策支持
智慧服务分级评估	患者	0 - 5 级	预约、缴费、报告、全程服务
智慧管理分级评估	管理者（运营）	0 - 5 级	人财物、绩效、决策分析

1.4.2 智慧服务：患者视角的评价

智慧服务评估围绕患者就医旅程展开：诊前（预约、导诊）、诊中（signage 导航、移动缴费、信息推送）、诊后（报告查询、随访、药品配送）以及全程的基础与安全要求【成稿核对：评估项目的类别划分与数量】。它把第三次浪潮的“互联网+医疗”从可选项变成了评价项——患者端 H5/小程序、线上支付、电子发票、报告线上查询等能力由此进入采购需求。HIP 的患者端（/portal：预约挂号、待缴费支付、报告查询与分享短链，见第 2 章患者旅程与功能清单第九域）即是对这组要求的最小完整响应。

1.4.3 智慧管理：运营视角的评价

智慧管理评估覆盖医疗护理管理、人财物资源管理、运营决策分析等维度，牵引的是 HRP（医院资源计划：财务、人事、物资、设备、固定资产）与运营数据分析类系统的建设（HIP 第四域“运营管理”七个功能页、第 12 章运营分析）。与公立医院绩效考核（“国考”）的指标上报要求叠加，智慧管理评估使运营数据的系统化采集从“财务科的内部事务”上升为全院工程。

对三大评价体系做一个工程视角的总结：分级评价定义了临床系统的功能深度，互联互通定义了数据的标准化与连接方式，智慧服务/管理定义了服务与运营的覆盖面。三者相加，几乎就是一份完整的医院信息化建设总纲——这也是为什么本书的业务篇（第 7 - 10 章）、数据篇（第 11 - 12 章）的内容组织，处处能找到对应的评价条款。

1.5 网络安全与数据合规

1.5.1 等级保护：安全建设的基线框架

网络安全等级保护制度（“等保”）是我国网络安全的基本制度，《网络安全法》（2017 年施行）确立其法律地位；2019 年发布的 GB/T 22239-2019《网络安全等级保护基本要求》标志等保进入 2.0 时代。等保将信息系统按重要程度与破坏后果定为一至五级，级别越高要求越严。

医院的核心业务系统（HIS、EMR 等承载诊疗与患者数据的系统）普遍定为第三级——三级意味着系统破坏会对社会秩序与公共利益造成严重损害，须落实“一个中心、三重防护”（安全管理中心，安全通信网络、安全区域边界、安全计算环境）的技术要求，并定期开展等级测评【成稿核对：三级系统测评频率（现行要求为每年一次）】。

对系统建设者，等保三级不是一份抽象的合规文件，而是一张可以逐条核对的工程清单。典型条款到工程落点的映射：

等保要求（概括）	工程落点	本书章节
身份鉴别、访问控制	认证与 RBAC 权限、密码复杂度策略、登录失败锁定	第 5 章
安全审计	写操作全量审计日志、敏感操作留痕、日志防篡改	第 5、13 章
数据保密性/完整性	传输加密(HTTPS)、密钥外置管理、敏感字段处理	第 13 章
数据备份恢复	每日备份任务化、恢复演练留档、备份时效监控	第 13 章
通信与边界防护	网络分区、端口最小开放、反向代理	第 13 章

第 13 章将完整展开“从条款到清单”的工程化方法。这里先立一个原则：合规能力必须内建于产品，而不是实施时补贴膏药——审计、脱敏、密码策略这类能力，事后加装的成本与风险都远高于设计期内建（HIP 的“安全合规内建”设计原则即源于此）。

1.5.2 数据安全法与个人信息保护法

2021 年施行的《数据安全法》（9 月 1 日）与《个人信息保护法》（11 月 1 日）把数据处理从“安全问题”上升为“法律责任”。对医院数据处理，框架性的要求如下：

数据安全法要求数据分类分级管理、重要数据保护、数据处理全流程的安全制度。医疗数据体量大、敏感度高，是各行业数据分类分级实践的重点领域【成稿核对：卫生健康行业数据分类分级的行业标准/指南名称与状态】。

个人信息保护法与医院关系更直接：医疗健康信息被明确列为敏感个人信息，处理须具备特定目的和充分必要性，并取得个人单独同意；同时确立最小必要原则、公开透明原则，以及去标识化/匿名化的法律区分（去标识化后仍是个人信息，匿名化后才脱离个保法约束）。此外《民法典》将隐私权与个人信息保护写入人格权编，《基本医疗卫生与健康促进法》亦有患者隐私保护条款——医院泄露患者信息面临的是多部法律的叠加责任。

这些法条落到系统设计，形成一组具体的设计约束（第 13 章详述工程实现）：

- 按角色脱敏：身份证号、联系方式等敏感字段默认脱敏展示，仅特定角色可见明文（HIP 的 EMPI 即按此设计，见第 5 章）；
- 最小必要的数据流出：对外分享必须匿名化、脱敏、限时（HIP 患者端的报告分享短链是一个完整案例：匿名访问 + 脱敏内容 + 限时失效，见第 13 章）；
- 审计可追溯：谁在何时看过、改过哪个患者的什么数据，全量留痕——这既是等保要求，也是发生泄露事件时的举证基础；
- 面向患者的知情同意：患者端注册、数据使用授权的交互设计要能支撑“单独同意”的合规要求。

合规线的完整工程化——从法条到清单到代码——是第 13 章的主题；本节建立的框架（等保定基线、两法定责任、条款落设计）会在那里逐项兑现【成稿核对：《网络数据安全条例》施行时间（2025 年 1 月 1 日）及其对医疗数据处理的补充要求，视篇幅决定是否纳入】。

1.6 医保支付改革：当前最强的驱动力

1.6.1 为什么是医保

评价体系讲的是“荣誉与达标”，医保讲的是钱。公立医院收入中医保基金占比高，医保规则的任何变化都直接改变医院的收入函数——这使医保成为当前医院信息化最强、最不容讨价还价的驱动力。近年的医保改革沿三条线推进，每条线都对应一组信息化的硬需求：

第一条线：编码贯标。国家医保局自 2019 年起推行医保信息业务编码标准（药品、诊疗项目、耗材等 15 项编码）的统一贯标，并建设全国统一的医保信息平台。对医院系统，贯标意味着院内字典必须与国家医保编码建立完整映射（目录对照），医保接口按全国统一规范（各省 SDK 形态落地）改造——第 3 章讲编码标准，第 10 章讲目录对照与省 SDK 适配的完整工艺。

第二条线：DRG/DIP 支付方式改革。国家医保局于 2021 年底发布支付方式改革三年行动计划，要求到 2025 年底 DRG/DIP 支付方式覆盖所有符合条件的开展住院服务的医疗机构【成稿核对：《DRG/DIP 支付方式改革三年行动计划》文号与目标表述】。按病组/病种付费把医院收入与病案首页质量、费用明细质量直接挂钩：漏填一个并发症诊断，病例入组下降，支付标准应声下跌。信息系统由此新增三类需求——入组与支付模拟、医保结算清单生成、病案质量治理，第 10 章（10.6 节）与第 12 章（病案管理）分别展开。

第三条线：基金监管。《医疗保障基金使用监督管理条例》（2021 年 5 月施行）为基金监管提供了行政法规依据，飞行检查、大数据筛查常态化。对医院，这意味着“事前提醒—事中审核—事后稽核”三道防线中，自建事前审核能力（第 10 章 10.4 节）从可选项变成自保项；同时，行级费用明细、完整留痕、可追溯的结算链路成为接受检查时的原始证据。

1.6.2 支付改革如何重塑系统需求

把三条线合起来看，医保改革对信息系统的重塑有一个共同指向：从“结算准确”升级为“数据可稽核”。第一代系统只要算对患者付多少钱；今天的系统要保证每一行费用可归类（行级分割）、每一次上传可对账（留痕）、每一份清单可入组（病案质量）。第 10 章会用整章的篇幅展开这条链路的工程实现，包括两个真实的返工教训——它们都源于用“演示能跑”的标准对待“必须经得起稽核”的领域。

对读者的提示：如果你只有时间精读本书的一章业务章节，选第 10 章——医保是政策、业务、工程三条线交汇最密、教训最真实的领域，也是检验一个 HIS 团队工程成熟度的试金石。

1.7 建设模式与产业格局

1.7.1 自研、外购、混合：决策框架

医院获得信息系统的方式有三种基本模式，决策要素可归纳为四个维度：

维度	自研	外购（商用套装）	混合
能力门槛	需稳定的开发团队（多数医院不具备）	低，依赖厂商	需较强的信息科（能定义边界、管集成）
前期成本	高且不确定	明确（采购价）	中
长期控制力	最强，需求响应快	弱，受制于厂商排期与商务	核心受制、外围自主
生命周期风险	团队流失即系统失养	厂商退出市场/停止维护	集成界面的长期维护成本

现实中的分布大致是：极少数头部医院自研（有百人级开发团队的大型三甲）；绝大多数医院外购；混合模式（外购核心 HIS + 自研或第三方外围应用 + 自建集成）在有一定技术力量的医院中渐成主流。对多数医院，决策的关键不是“要不要自研系统”，而是要不要掌握数据主权与集成主导权——数据库结构与接口文档是否交付、集成平台由谁掌控、换厂商时数据能否完整迁出。这些条款在采购合同里的分量，远大于功能清单里多几个模块。

从供给侧看，本书案例 HIP 所代表的形态——面向县市级医院的标准化产品 + 配置化实施——是外购模式下的一种产品化路线：不为单家医院定制分叉，医院差异全部收敛到配置层、适配器层与受控的定制层（第 14 章四层标准化模型详述）。这条路线能否成立，取决于产品对“按需建设”的支持程度，1.8 节的模块开关设计即为此而生。

1.7.2 产业格局的粗线条

医疗信息化产业可以粗略分为几个梯队（本书不评价具体厂商，只描述结构）：

- 综合 HIS 厂商：提供覆盖医院主要业务的套装产品，全国性大厂商与深耕一省一域的区域厂商并存。区域厂商的本地化服务能力（驻场、响应速度）往往是其对抗全国性厂商的核心武器——医院信息系统是“三分软件七分实施”的行业，实施与服务能力比软件功能本身更能决定项目成败；
- 专科系统厂商：LIS、PACS、手术麻醉、重症、病理等领域各有专业厂商，产品深度是综合厂商难以覆盖的，这也决定了“综合平台 + 专科系统 + 集成”是大中型医院的常态结构（第 6 章集成平台的问题域由此而来）；
- 平台与新势力：集成平台/数据中台厂商、互联网大厂（患者服务与云入口）、保险与医药产业链玩家从各自入口切入医疗信息化；
- 配套生态：CA 认证、合理用药知识库、医保审核规则库、电子发票等专项服务商——这类能力通常以“外部条件”形态进入项目（第 6 章、第 10 章多次出现这个概念）。

1.7.3 信创趋势

信息技术应用创新（“信创”）指基础软硬件的国产化替代：芯片、整机、操作系统、数据库、中间件、办公软件。在医疗行业，信创的推进节奏总体上是从党政机关向重点行业渐进、从外围系统向核心系统渐进【成稿核对：医疗行业信创的政策要求现状与时间表】。对医院信息系统建设的现实影响集中在选型层：

- 数据库是最关键的替代点。国产数据库（达梦、人大金仓等）多数提供 PostgreSQL 或 Oracle 兼容模式，因此选择 PostgreSQL 技术路线的系统天然拥有低成本的信创迁移路径——这是 HIP 选型 PostgreSQL 的理由之一（第 4 章详述）；
- 操作系统与中间件：Java 技术栈对国产 OS（麒麟、统信）适配成熟，风险可控；
- 采购要求：越来越多的政府采购需求书出现信创适配条款，产品是否具备国产化适配能力正在从加分项变为门槛项。

对工程团队，信创的正确应对不是等风来再改造，而是在架构上保持可替换性：不绑定特定数据库的私有特性、不依赖不可替代的商用中间件——这与第 4 章的选型方法论一脉相承。

1.8 本书案例平台 HIP 概览

1.8.1 定位与形态

HIP（Hospital Information Platform）是贯穿本书的案例平台：面向县市级综合医院（二级为主）的一体化信息平台，覆盖 HIS + EMR + 医技 + 管理 + 互联网患者端，以“平台底座 + 可插拔业务模块”方式交付。技术形态一句话概括：Java 21 + Spring Boot 3 模块化单体（12 个后端模块），Vue 3 + Element Plus 前端工作台，PostgreSQL 16 单库（各模块以表前缀组织，schema 隔离为演进预留，见第 4 章），一院一实例一库独立部署（v1.0.0 定版）。

选择这个定位作为教学案例，有三个理由：其一，县市级医院是我国医院的数量主体，其信息化的资源约束（无专职 DevOps、预算有限、依赖外部条件）最具代表性；其二，一体化平台让读者能在一套连贯的代码与数据模型中看到全部业务域，而不必在多个异构系统间跳跃；其三，HIP 是真实迭代出来的系统——本书引用的案例包括真实的返工教训（如第 10 章的医保结算号事故），而非事后美化的设计。

需要预先声明的是本书的双层写法：每一章先讲这类问题的通用原理与业界方案，再以 HIP 为案例讲一种真实落地的做法。HIP 的选择不是唯一解，很多设计（如模块化单体、Mock 适配器先行）是在“县市级医院”这个约束下的解，换约束条件解就会变——各章会明确交代这些约束。

1.8.2 十域 55 功能页全景

HIP 的功能按十个业务域组织，共 55 个功能页（另有患者端 H5 与打印页），8 个内置角色（管理员/医生/护士/收费/药师/医技/质控院感/运营后勤）按页授权。全景如下（据功能清单 v1.0，各域代表性能力择要）：

#	业务域	功能页	代表能力	备注
一	患者服务	患者建档、患者管理	EMPI 主索引、按角色脱敏；临床路径、随访、体检、满意度	
二	门诊业务	18 页	挂号排班、医生站（合理用药五重拦截）、收费与医保联动、扫码支付、药房发药、审方、LIS/RIS 工作台、叫号、预约、急诊分诊、专科流程	CDSS 可关闭
三	住院业务	8 页	住院登记（原子占床）、记账式医嘱、护士执行与体温单、护理白板、出院结算、手麻、用血、移动工作台	手麻/用血可关闭
四	运营管理	7 页	固定资产、设备、物资与 OA、人事、药事分析、日结报表、科研	人事可关闭
五	基础数据	3 页	药品目录、收费项目、药库管理，均支持 CSV 批量导入	
六	数据中心	11 页	患者 360（CDR）、DRG 分析、病案统计、质控中心、院感专项、单病种上报、数据治理、报表引擎	DRG 可关闭
七	集成平台	3 页	HL7 V2/MLLP 集成监控、医保管理（对照/分割/审核/对账）、适配器路由	医保可关闭
八	系统管理	5 页	用户/科室/角色权限、审计日志、运维中心	
九	患者端	H5（/portal）	预约挂号、缴费出码、报告查询、分享短链、智能导诊	微信实名为外部条件
十	平台横切	—	机构参数化、模块开关（六域）、JWT+RBAC、审计、备份、PWA	

表中两类标注值得注意，它们是产品化设计的关键机制：可整体关闭（模块开关，实施时按院启停）与依赖外部条件（省医保 SDK、CHS-DRG 官方目录、CA 产品、聚合支付商等八项——接入点已

就绪，等资质与合同到位即接，完整清单见功能清单附录）。前者见案例框 1-1；后者的工程方法（接入点先行、Mock 实现、条件到位即接）是第 6 章 6.8 节的主题。

案例框 1-1：模块开关——“按需建设”的产品化表达

政策与预算共同决定了没有两家医院会建设完全相同的功能集：未评三级的医院可能不上 DRG，无手术资质的医院不需要手麻，人事另有系统的医院要关掉人事模块。传统做法是按院裁剪代码——每家医院一个分支，三年后没人说得清哪家跑的是什么。

HIP 的做法是模块开关：六个业务域（DRG / CDSS / 医保 / 用血 / 人事 / 手麻）各设一个配置开关，实施初始化时按医院采购范围置位。开关是“双路”生效的：菜单过滤（未启用域的页面不出现在任何角色的菜单树）+ API 拦截（对应接口直接返回 404）——只藏菜单不拦 API 是安全幻觉，懂技术的用户仍可直连接口。上线检查清单里因此有一条硬核验证：“未采购模块直连 API 实测 404”。

全部医院运行同一个标准发布产物，差异只存在于配置——这是第 14 章“四层标准化模型”的第一层机制，也是“按需建设”从销售话术变成工程事实的支点。

案例框 1-2：一份真实的采购需求长什么样

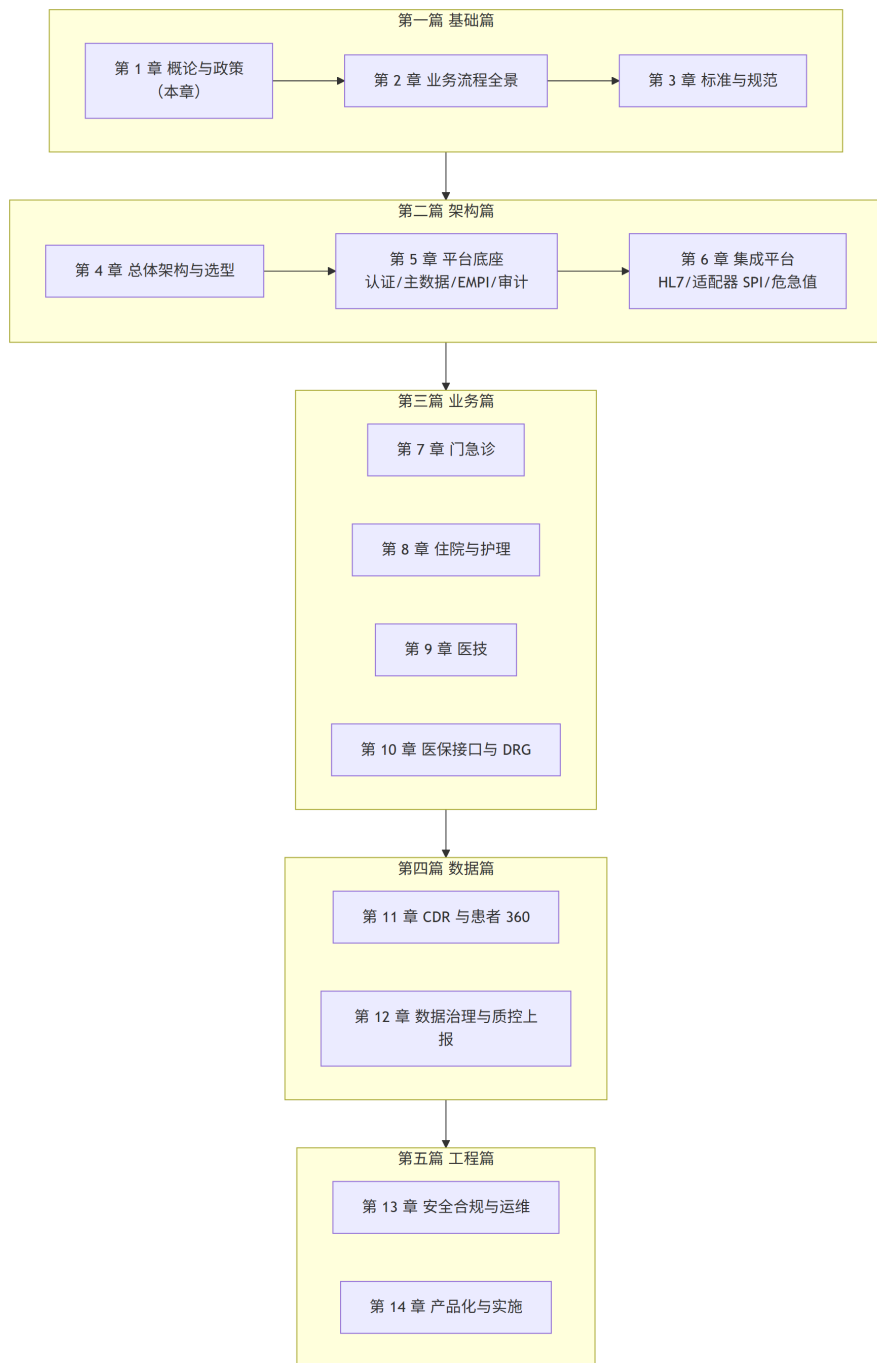
县市级医院的信息化采购通常以公开招标进行，需求书的量级值得初学者建立直观感受：HIP 对标的一份采购需求包含 107 个功能模块、4062 条技术参数——每条参数都是一句“系统应支持……”的可验证要求，投标方须逐条应答（满足/部分满足/不满足，即“正偏离/部分响应/负偏离”）。

HIP 团队对这 4062 条参数做了完整的响应矩阵（v17 终版口径）：3388 条正偏离（产品直接满足）、152 条部分响应、210 条属配套产品范围（如重型 ESB、区域平台）、28 条依赖外部条件（省医保 SDK 等）、43 条属非参数条目（资质、服务承诺等），另有 241 条完成证据包待复核。每条“正偏离”背后要求可演示的证据（页面截图、接口实测、E2E 断言）——需求响应不是打勾游戏，而是一场按条目组织的验收预演。

这张矩阵在本书中会两次再现：第 14 章（14.10 节）讲它的管理方法（证据包签认、技术偏离表），而各业务章节的功能设计，几乎都能在矩阵中找到对应的参数条目——采购需求是行业需求的化石层，读懂它就读懂了这个行业对系统的全部期待。

1.8.3 全书章节地图

最后给出全书结构与 HIP 平台各部分的对应关系（图 1-3），这是你阅读本书的导航图：



阅读路径建议：

- 教学顺序（推荐初学者）：按章节顺序——先业务与标准（基础篇），再架构（架构篇），然后按业务域展开（业务篇、数据篇），最后是工程与实施（工程篇）；
- 实施工程师：第 1、2 章建立全景后可直接跳第 14 章（产品化与实施方法论）与第 13 章（安全运维），再按项目涉及的业务域回读业务篇；
- 政策条线的读者：本章的三条政策线各有工程落地章——评价线去第 3、6、11 章，合规线去第 5、13 章，支付线去第 10、12 章。

本章建立的政策坐标系是全书的隐含背景：后文凡写到“这是分级评价 X 级要求”“这是等保条款”“这是医保稽核依据”，都在回指本章。政策会更新，但“评价牵引建设、合规划定底线、支付驱动质量”的结构在可见的未来不会变——这是比任何具体条文都更值得记住的东西。

本章小结

- 医院信息化三十余年经历三次浪潮：收费电算化（驱动力：财务管理）→ 临床信息化（驱动力：医疗质量与评价指挥棒）→ 平台化与数据驱动（驱动力：医保支付与监管）。浪潮叠加而非替代，数据的角色从记账凭证升格为生产要素与结算依据。
- 政策环境可归纳为三条线：评价线（分级评价 0-8 级评临床应用深度、互联互通七档评标准化与连接、智慧服务/管理 0-5 级评服务与运营）、合规线（等保三级基线 + 数据安全法/个人信息保护法责任）、支付线（贯标、DRG/DIP、基金监管）。
- 电子病历分级评价是功能需求最主要的政策来源，“有效应用比例”设计使评级必须基于真实运行；评价标准的正确读法是把条款逐条翻译成需求清单（闭环、CDSS、危急值）。
- 互联互通测评与分级评价一横一纵：前者查数据标准化与交换（数据元/CDA 文档/平台服务），对新建系统的启示是“标准内建”优于事后改造。
- 医疗健康信息是法定敏感个人信息；等保三级与两法的要求必须内建于产品设计（按角色脱敏、最小必要流出、全量审计），而非实施时补装。
- 医保支付改革把系统要求从“结算准确”升级为“数据可稽核”：行级费用、完整留痕、病案质量直接决定收入与稽核风险，是当前最强的信息化驱动力。
- 建设模式的现实光谱是“少数自研、多数外购、混合渐多”，决策关键在数据主权与集成主导权；信创的正确应对是架构上保持可替换性（PostgreSQL 兼容路线）。
- HIP 是本书贯穿案例：县市级综合医院定位、十域 55 功能页、模块开关支撑按需建设、107 模块 4062 参数的需求矩阵是行业期待的完整快照。全书按“基础—架构—业务—数据—工程”五篇组织，各章与平台各部分一一对应。

思考与练习

1. 概念：分别说出三次浪潮的核心系统与驱动力，并举一个你身边医院的例子说明“浪潮叠加而非替代”——同一家医院里三代系统同时运行的证据。
2. 调研：查询你所在省（或任选一省）通过互联互通标准化成熟度测评四级甲等及以上的医院名单（国家卫生健康委统计信息中心定期公布），分析级别分布与医院规模、区域经济水平的关系，写一页调研笔记。
3. 调研：选择一家你熟悉的医院，对照电子病历分级评价各级主题，判断其大致所处级别；列出它升到 4 级还缺哪些系统能力，并标注每项能力对应本书哪一章。
4. 分析：以“检验危急值闭环管理”为例，把评价要求翻译成一份需求清单：写出涉及的系统（LIS/医生站/消息通道）、状态流转（发布→通知→确认→处理→留痕）与超时升级机制，比较你的清单与第 6 章、第 9 章的实现有何异同（可在读完相应章节后回头修订）。
5. 论述：患者在手机上查看自己的检验报告并分享给外院医生——这个看似简单的功能同时触碰等保三级、个人信息保护法的哪些要求？给出你的设计要点，并与 HIP 的“报告分享短链”方案（匿名、脱敏、限时，第 13 章）对比。
6. 论述：为什么说医保支付改革是当前医院信息化最强的驱动力？从“数据质量直接等于钱”的角度，分析 DRG 付费如何改变医院对病案编码岗位与临床数据质量的投入逻辑。
7. 实践：一家二级综合医院未开展手术、人事另有集团统一系统、暂不参与 DRG 付费。据 1.8 节的模块开关设计，写出这家医院的开关配置（六域各取 true/false）并逐项说明理由；再对照功能清单，列出该配置下患者端与集成平台还剩哪些“外部条件”需要在实施中排队。
8. 延伸调研：检索你所在省医保局关于 DRG 或 DIP 付费的最新实施文件，弄清本省选择的是哪条技术路线、当前覆盖到哪类医院；带着这个答案去读第 10 章 10.6 节，体会“统筹区选择决定系统需求”的含义。

第 2 章 医院业务流程全景

本章属第一篇「基础篇」。在动手设计任何医院信息系统之前，必须先把医院当作一个业务有机体看懂：科室与角色如何分工，患者与数据如何流动，钱与物如何流转。本章沿门诊、急诊、住院、医技、药事、运营六条业务线做一次全景导览，并引入贯穿全书的三流视角——信息流、资金流、实物流的分与合。本章只讲业务不讲实现；各业务线的系统设计分别在第 7 - 9 章展开。

学习目标

学完本章，你应当能够：

1. 描述医院的科室分类（临床/医技/职能/后勤）与信息系统八角色模型的对应关系，解释系统为什么按角色而不是按科室组织功能；
2. 画出门诊诊疗全流程（挂号→接诊→开单→收费→执行→发药）的泳道图，并标出「先收费、后执行」的控制点；
3. 说出急诊预检分诊 I - IV 级的分级含义，解释急诊流程与门诊流程在资金流上的根本差异；
4. 画出住院全流程图，解释医嘱为什么是住院信息流的主轴，以及「记账式医嘱」如何把信息流与资金流合并为一条链；
5. 从科室视角描述检验、检查、病理、手术、输血五类医技业务的流转骨架，指出各自的实物流主角；
6. 在同一张流程图上勾勒信息流、资金流、实物流三条轨迹，指出它们的分叉点、汇合点与闭合点；
7. 沿三流轨迹分析一个逆向流程（退号/退费/退药）的正确执行顺序及其拦截条件。

2.1 医院组织结构与信息系统角色模型

2.1.1 科室：医院的组织单元

无论规模大小，医院的科室都可分为四类：

- 临床科室：直接为患者提供诊疗服务的科室，分门诊与住院（病区）两种形态。内科、外科、儿科、妇产科等按学科划分，急诊科则按服务时效划分。注意科室与病区不是一回事：科室是学科建制，病区（护理单元）是床位管理单元——一个大科可辖多个病区，床位紧张时一个病区也可能混合收治多个科的患者。系统建模时科室与病区必须是两个独立维度，医生按科室归属、床位按病区归属，混为一谈会在转科转床与护理管理上留下长期隐患（第 5 章主数据建模会回到这一点）；
- 医技科室：不直接管患者、但为临床提供技术支撑的科室——检验科、医学影像科（放射/超声/心电图/内镜）、病理科、手术麻醉科、输血科、药学部等。它们是「平台科室」：一个检验科同时服务全院所有门诊与病区；

- 职能科室：医务、护理、质控、院感、病案、财务、医保办、信息科等管理部门，不产生诊疗行为，但制定与监督规则；
- 后勤保障：设备、物资、总务、消毒供应中心（CSSD，Central Sterile Supply Department）等，维持医院这台机器的物理运转。

这个分类直接映射为医院信息系统的模块版图：临床科室对应门诊、住院系统（第7、8章），医技科室对应医技系统（第9章），职能科室对应质控统计与医保（第10、12章），后勤对应运营管理（HRP，Hospital Resource Planning）。

把科室按业务重新编组，可以得到贯穿本章的五条业务线，它们与本书案例平台 HIP 的十个业务域一一对应：

业务线	覆盖流程	本章节次	HIP 对应业务域	系统详述
门急诊线	挂号→接诊→开单 →收费→执行	2.2、2.3	患者服务、门诊业务	第7章
住院线	入院→医嘱→护理 →出院	2.4	住院业务	第8章
医技线	申请→执行→报告	2.5	门诊业务(医技页)、 专科流程	第9章
药事线	药库→药房→患者	2.6	基础数据、门诊业务 (药房页)	第7章
运营线	资产/物资/人事/财 务日结	2.7	运营管理、系统管理	第12、14章

职能科室的质量管理与医保业务横切在五条线之上（对应 HIP 的数据中心、集成平台两域），在第10、12章展开；患者端（2.8）则是五条线朝向患者的统一出口。

2.1.2 角色：信息系统的用户模型

科室是行政单元，角色才是信息系统的功能单元——同一个检验科里有医生、技师、护士，用的功能完全不同；而门诊医生与病区医生虽分属不同科室，用的功能高度同构。因此成熟的医院信息系统按「角色×功能页」组织权限，而不是按科室。本书采用与案例平台 HIP 一致的八角色模型：

角色	典型岗位	主要业务动作	承载业务线
管理员	信息科	用户/科室/权限维护、系统参数、审计	平台底座
医生	门诊/病区/急诊医生	接诊、病历、诊断、开单、医嘱	门诊、住院
护士	门诊/病区/急诊护士	皮试输液、医嘱执行、体征、护理评估	门诊、住院
收费	挂号收费窗口	挂号退号、收费退费、日结	资金流入口
药师	药房、药库	审方、发药退药、进销存、药事分析	药事
医技	检验/检查/病理技师与医生	标本流转、结果录入、报告审核发布	医技
质控院感	质控科、院感科、病案室	时限质控、院感监测、报卡审核、病案统计	质量管理
运营后勤	设备科、总务、人事	资产设备、物资、人事、OA	运营保障

八角色不是理论抽象，而是可以直接落到培训现场的划分：给每个角色 15 - 20 分钟，就能沿其日常动线把系统走一遍。在 HIP 中，这一模型体现为 8 个内置角色对 55 个功能页的绑定关系，用户登录后菜单只显示本角色权限内的功能页（权限机制见第5章）。

角色与人不是一一对应：一名夜班护士可能兼做收费（小医院常态），一名科主任同时是医生角色与质控角色——系统允许一人多角色，权限取并集。反过来，同一角色在不同科室的可见范围也应不同（呼吸科医生不该看到妇产科的候诊队列），这靠角色之外的科室归属维度限定。角色定功能、科室定范围、患者授权定隐私边界，三个维度叠加构成医院信息系统权限模型的业务需求——实现留到第5章，这里只需记住：权限模型是业务分工的镜像，画不清业务分工就配不对权限。

2.1.3 三流视角：本章的分析框架

把医院业务拆到底层，流动的东西只有三种：

- 信息流：单据与数据的流动——挂号记录、病历、医嘱、申请单、报告、报文。它跑在网络上，速度最快、成本最低；
- 资金流：费用的发生、收缴与结算——挂号费、药费、检查费的应收与实收，医保与患者的分担，日终的对账与交款。它受财务与医保规则的强约束；
- 实物流：物理对象的移动——药品从药库到药房到患者手中，标本从采集处到检验科，血液从血库到病区，消毒包从CSSD到手术室。它最慢、最贵，且一旦交付往往不可逆。

三条流的性格差异决定了流程设计的基本矛盾：信息流希望越快越好，资金流要求笔笔可核，实物流讲究安全可追溯——一个环节同时服务三个目标时，必然要排优先级（急诊把救治的信息流排在资金流前面，就是最典型的一次排序）。三流在流程的某些环节并行分离，在另一些环节强制汇合。汇合点就是控制点：门诊收费窗口是信息流与资金流的汇合点（不缴费则后续环节不启动），药房发药窗口是三流的汇合点（处方信息、已收费状态、药品实物三者对齐才能交付）。信息化的大量设计工作，本质上是把这些汇合点从「靠人核对」变成「靠系统对齐」——这正是后续章节反复出现的闭环概念的地基：一个业务闭环，就是三流各自走完且互相对平的一段流程。本章每节都会点出该流程中三流的分与合。

三流视角还给了我们一把评估信息化成熟度的尺子：早期的收费电算化只管资金流；HIS/EMR分立时代信息流与资金流各管一段、靠接口缝合；平台化阶段（第1章）追求的正是三流在同一数据底座上的原生对齐。读完本章再回看第1章的演进史，会多一层理解。

2.2 门诊诊疗全流程

门诊是医院业务量最大、节奏最快的流程，主链条六个环节：挂号→接诊→开单→收费→执行→发药。

2.2.1 挂号：就诊的起点与容量的闸门

挂号做两件事：为患者建立本次就诊的身份（一次挂号对应一次就诊，后续病历、处方、费用都挂在它下面），并占用一个号源。号源是医生出诊能力的量化——排班决定某医生某时段放多少号，号满即止。挂号因此是门诊的容量闸门：它决定了整个下游流程的流量。首次就诊的患者需先建档，进入患者主索引（EMPI，Enterprise Master Patient Index），保证「一人一档」——同一患者多次就诊、跨门诊住院的数据能纵向串联，靠的就是这份唯一档案；档案信息中的过敏史、参保类型会被后续的用药拦截与医保结算反复引用（第5章详述）。

挂号的渠道近年持续多元化：窗口、自助机、电话、患者端线上预约——渠道不同，但都在消耗同一池号源。这带来门诊流程的第一个并发难题：多个渠道同时抢同一个号，系统必须保证「一号一患」，宁可让后来者挂号失败，不能让两位患者拿到同一个号（业务上的超挂意味着医生超负荷与患者纠纷）。号别（普通号/专家号/专病门诊号）与午别（上午/下午）则是号源池的业务维度，定价与排班随之而异。

三流在这里第一次同时启动：信息流建立就诊记录并扣减号源，资金流产生第一笔应收（挂号费）。在 HIP 中，这一环节由「患者建档」「医生排班」「门诊挂号」三个功能页承载，挂号时原子占号防止超挂（并发实现见第7章）。

2.2.2 接诊：病历与诊断

患者候诊，医生按队列（或叫号）接诊。候诊秩序由叫号体系维护：按挂号顺序（或预约时段）排队，过号顺延、可重呼——叫号看似小功能，却是门诊现场秩序感的主要来源。接诊的核心产出是两样：门诊病历（主诉、现病史、查体）与诊断（以 ICD-10 编码记录，第一诊断为主诊断——它是后续医保结算、病案统计、DRG 入组共同引用的关键数据，第3、10、12章会反复回到它）。初诊与复诊在流程上同构，差别在信息流的起点：复诊医生首先要看的是历史——上次的病历、检验结果、用药记录，这正是患者纵向视图（患者360，第11章）的业务价值所在。这一环节几乎是纯信息流：不收钱、不动实物，却生产整个就诊中最有价值的数据。在 HIP 中由「门诊医生站」「叫号大屏」承载，保存病历时 CDSS（Clinical Decision Support System，临床决策支持系统）按主诊断给出检查与用药路径提示。

2.2.3 开单：诊疗意图的单据化

医生的诊疗决定必须变成可流转的单据才能驱动下游：处方（药品）、检验申请单、检查申请单、治疗单。业务上它们性质相同——都是「医生开出的、待收费待执行的费用行」，因此现代系统把它们统一为订单行模型（第7章详述其设计）。开单是三流的分叉点：一张单据从此拆出三条轨迹——信息流（单据流转至执行科室）、资金流（每一行都携带价格，形成应收）、实物流（药品行将驱动库存扣减，检验行将驱动标本采集）。开单环节还有第一道质量闸门：合理用药审查在开立瞬间拦截过敏、重复用药、配伍禁忌等问题——在费用发生之前拦住错误，成本最低。此外，检查检验申请单不只是「点菜单」：申请单上的临床诊断与检查目的是医技医生解读结果的重要上下文，「申请单信息不全」是医技科室对临床最常见的抱怨——信息流的质量在源头就决定了。

2.2.4 收费：三流的第一汇合点

门诊的基本财务规则是先收费、后执行（急诊绿色通道除外，见2.3）。收费窗口把患者本次未缴费的订单行汇总为结算单，选择支付方式收款；医保患者在此处同步完成费用分割与医保上传（第10章详述）。收清费用后，订单行状态翻转为「已收费」——这是发给全院执行科室的放行信号：药房据此调剂，医技据此接单。收费是信息流与资金流的第一汇合点，也是门诊流程最重要的控制点。

支付方式的构成决定了资金流的后续走向：现金进钱箱、移动支付（微信/支付宝等聚合支付）进通道账户、医保部分由统筹基金与患者按分割结果分担——每一类都要在日结时单独轧账（2.7），移动支付还要与支付服务商的通道流水对账。收费同时产生票据：财政医疗票据是患者报销与维权的凭证，票据号与结算单的对应关系必须严格留痕。在 HIP 中，本环节由「门诊收费」「扫码支付台」承载，退费与重收、退号联动拦截在同页处理。

2.2.5 执行与发药：三流的第二汇合点

已收费的订单行分流到各执行点：药品行到药房（审方→调剂→按叫号发药），检验行到采血/采样处，检查行到相应医技科室，治疗行（输液、注射）到门诊护士站。以静脉输液为例可以看清执行环节的质量控制密度：需皮试的药物（如青霉素类）必须先做皮试且结果为阴性方可开始；输液中护士按规范巡视并记录；换瓶、拔针各有核对动作——每一步都是「查对制度」在门诊场景的落点，也都是系统需要留痕的状态节点。药房发药窗口是三流的第二汇合点：处方信息（信息流）、已收费状态（资金流）、药品实物（实物流）三者对齐，药品交付患者，库存同步扣减。检验检查的执行则把实物流（标本、患者本人）送入医技科室，报告发布后信息流逆向回流到医生站——医生看到结果，可能复诊、调整诊断、再次开单，流程进入第二圈循环。

图 2-1 用泳道图汇总全流程：

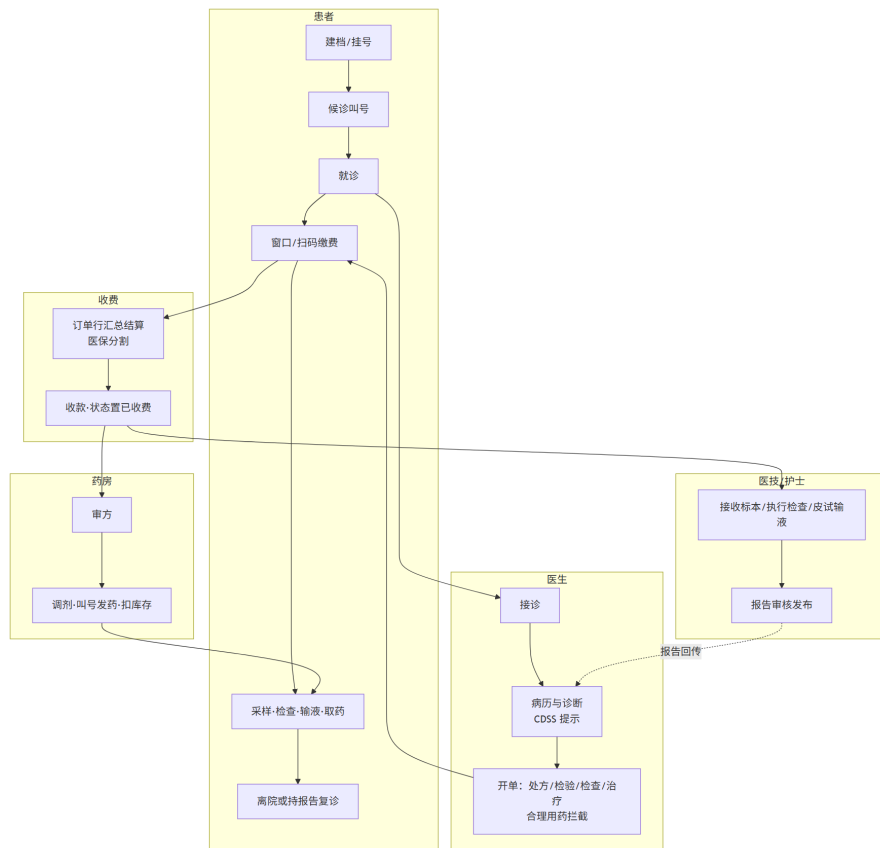


图 2-1 门诊诊疗全流程泳道图（报告回传的虚线是流程中唯一的逆向信息流，它驱动复诊闭环）

2.2.6 三流合一：在同一张图上看门诊

现在把 2.2.1 - 2.2.5 的三流线索抽出来单独画一遍（图 2-2）。这张图是本章的核心：后续每条业务线的分析，都是它的变体。

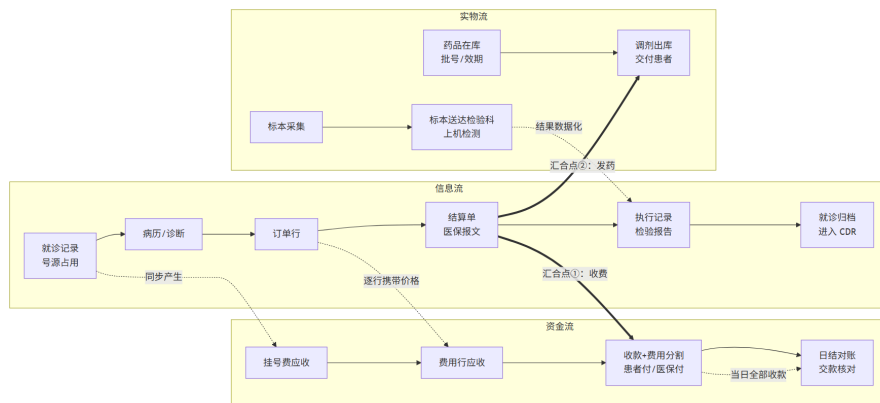


图 2-2 门诊流程的信息流—资金流—实物流三流合一图

读这张图要抓住四个节点：

1. 分叉点（开单）：一份医嘱意图拆出三条轨迹，此后各自独立推进；
2. 汇合点①（收费）：资金流结清是信息流继续向执行环节推进的前提；
3. 汇合点②（发药/执行）：实物交付必须同时核对信息（处方内容）与资金（已收费）——「凭票取药」的本质就是人工核对汇合点，信息化把它变成系统状态核对；
4. 闭合点（日结与归档）：资金流以日结对账闭合（当日各支付方式实收与账面核对，见 2.7），信息流以就诊归档闭合（汇入临床数据中心 CDR，第 11 章）。三条流各自闭合且能互相印证，这次就诊才算「对得上账」。

逆向流程是三流的镜像。退费不是删除一笔收费，而是沿原路逆行且顺序相反：实物流未发生（未发药）可直接退费；实物流已发生（已发药）必须先退药（药品回库、库存回补），资金流才允许退费；资金流退清后，信息流才允许退号（释放号源）。每一步的拦截条件都来自「下游流未回退则上游流不得回退」这条通用规则——请在习题2中把这条逆向路径完整画出来。

业务场景框 2-1：一位上呼吸道感染患者的门诊两小时

周一 8:05，李女士因发热咽痛到某县医院。她去年建过档，收费窗口按身份证号调出档案，挂了呼吸内科王医生的号——排班表显示上午还剩12个号。8:40叫号屏叫到她，王医生接诊，录入主诉「发热咽痛2天」，拼音首字母检出诊断「急性上呼吸道感染」；保存病历时系统按主诊断提示了血常规检查路径。王医生开出血常规一项、口服药两种——其中一种与李女士档案中的青霉素过敏史无冲突，系统未拦截。

8:50 李女士在自助机扫码缴费（检验费与药费一单结清），采血窗口的队列里随即出现她的名字。9:00 采血，标本送检验科。9:40 检验报告审核发布，王医生工作站上她的申请单自动转为「已执行」并显示结果：白细胞轻度升高，与临床判断相符。李女士回诊室复诊确认后，到药房叫号窗口取药——药师核对处方与实物，发药完成，库存同步扣减。10:05 离院。

复盘这两小时：信息流走了「就诊→病历→订单→报告回传→复诊」一个整圈；资金流在8:50一次汇合结清；实物流两条——标本进了检验科，药品出了药房。在HIP中，这条动线依次经过患者建档、门诊挂号、门诊医生站（含CDSS提示与用药拦截）、扫码支付台、LIS工作台、药房发药六个功能页；李女士本人只在三个物理位置停留过——诊室、采血窗、药房窗，其余流转全部发生在系统里。

2.3 急诊流程：预检分诊与留观

2.3.1 急诊与门诊的根本差异

急诊处理的是「不能等」的患者，流程设计的第一原则从门诊的「秩序优先」变为时间优先，由此产生两点根本差异：

1. 按病情定顺序，而不是按到达定顺序——这就是预检分诊；
2. 资金流后置——危急患者先救治后缴费（「绿色通道」），费用先记账、事后补收。这是三流关系的一次刻意倒置：门诊里资金流是信息流的闸门，急诊里信息流（救治指令）先行，资金流退居其后。倒置的代价是欠费风险，因此急诊记账需要更严格的事后追缴与核销流程——流程设计里没有免费的例外，只有把代价挪到别处的例外。

2.3.2 预检分诊：I-IV级

患者到达急诊，分诊护士依据生命体征与主诉在数分钟内完成分级。我国急诊普遍采用四级分诊【成稿核对：急诊预检分诊分级的现行规范名称与文号（原卫生部2011年《急诊病人病情分级指导原则》及后续行业标准）】：

级别	定义	响应要求	去向
I级	濒危：随时危及生命	立即抢救	复苏室/抢救室
II级	危重：短时间内可能恶化	尽快安排，密切监护	抢救室
III级	急症：有潜在恶化风险	优先于IV级诊治	优先诊室
IV级	非急症	顺序候诊	普通诊室

分级形成四条优先级不同的候诊队列。分诊本身是高价值的信息流起点：分级结果、分诊时的生命体征都是后续医疗行为与质量追溯的依据——一旦发生纠纷，「到达时刻—分诊级别—首次处置时

刻」这条时间链是核心证据。急诊分诊还承担传染病预检职责：发热、疑似传染病患者在分诊环节即被识别分流至发热门诊或隔离区域，这是公共卫生对流程的强制要求。

绿色通道的记账机制值得展开：I、II级患者的检查、用药先执行、费用挂账，形成「负债运行」的就诊；抢救结束后由家属补缴，无人补缴的部分进入欠费管理（催缴、核销）。系统必须支持这种「信息流先行、资金流挂账」的模式并给出欠费的全程台账——绿色通道不是不要钱，而是把资金流汇合点从执行前移到执行后，风险由医院的坏账制度兜底。

2.3.3 留观与去向

急诊处置后患者有四种去向：离院、留观、收入院、转院。留观是急诊特有的中间形态——病情尚不明朗、不满足住院指征又不宜离院的患者占用观察床位，护士定时记录观察情况，原则上在72小时内明确去向【成稿核对：急诊留观时限的现行规范表述】。留观在业务上是「微型住院」：要占床、要连续观察记录、要记账，但不走入院登记。图2-3汇总急诊流程：

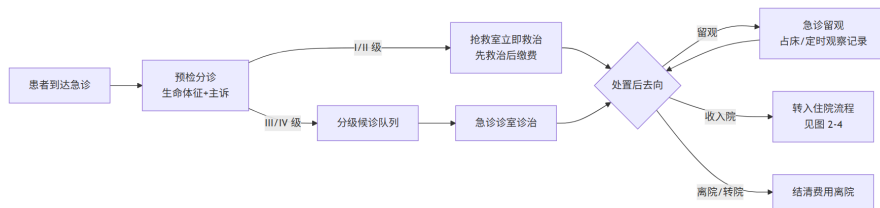


图 2-3 急诊预检分诊与留观流程

在HIP中，本节流程由「急诊预检分诊」功能页承载：I-IV级分诊队列、留观占床与观察记录、去向流转（急诊系统设计见第7章）。

2.4 住院诊疗全流程

门诊以「次」为单位，一次就诊几小时；住院以「天」为单位，一次住院几天到几十天。时间维度的拉长改变了一切：诊疗指令要素化为可反复执行的医嘱，费用从「先收后执行」变为「边执行边记账、出院总结算」，并出现了门诊没有的角色重心——护理。

2.4.1 入院：登记、押金与占床

住院的入口有三个：门诊医生开住院证（择期）、急诊直接收入院（急症）、转院接收。床位紧张的医院还有一段「待床」缓冲——住院证开出但无床可占，患者进入预约队列，由病区按床位周转安排召入。患者凭住院证办理入院登记：分配住院号（本次住院的唯一标识）、预收押金、在目标病区占用一张床位。三流同时启动且各有特点：信息流建立住院就诊；资金流以押金形式预收——这是住院资金流的独特设计，用预收对冲「边执行边欠费」的风险；实物流占用床位——床位是住院流程中最稀缺的「准实物」资源，占床与退床需要与号源同级别的严格管理（第8章详述）。在HIP中由「住院登记」功能页承载，原子占床防止一床两派。

2.4.2 医嘱：住院信息流的主轴

住院期间医生的一切诊疗指令都以医嘱形式下达，医嘱分两类：

- 长期医嘱：自开立起每日循环执行直至停止（如「头孢呋辛 1.5g 静滴 每日两次」、护理级别、饮食）；
- 临时医嘱：执行一次即完成（如某项检查、术前用药、「今日出院」）。

医嘱有自己的生命周期：开立→（护士核对）→执行→停止/作废，每一步记录操作人与时刻。手工时代护士要把医嘱「转抄」到各种执行单上，转抄是差错的温床；信息化把转抄变成系统流转，医嘱从开立到执行的每一环可查可溯，这是住院信息化最早也最扎实的收益之一。医嘱是住院信息流的主轴：护理执行看医嘱，费用记账跟医嘱，医技检查源于医嘱，出院结算校验医嘱。记账式医嘱是理解住院资金流的关键：每条医嘱执行时，同步生成对应的费用记录（药品行、治疗行、床位费行），信息流与资金流在医嘱执行这一动作上合并为一条链——不存在「执行了但没记账」或「记账了但没执

行」的缝隙。这个模型的价值到第10章会再次显现：DRG时代医保要求行级费用明细，记账式医嘱天然满足（医嘱体系设计见第8章）。

2.4.3 护理执行与病房日常

病区的日常节律由护理驱动：按医嘱给药（药品执行同步扣减库存——实物流与信息流的又一合并点）、定时测量生命体征（体温、脉搏、呼吸、血压，汇成体温单）、按分级护理要求巡视（特级、一、二、三级护理对应不同的观察频次与照护强度【成稿核对：护理分级行业标准 WS/T 431 现行版本】）、进行护理风险评估（压疮、跌倒等量表评估与分级预警）。医生侧的节律则是查房与病程记录：三级医师查房（住院医师—主治医师—主任医师逐级查房）是医疗质量安全核心制度之一【成稿核对：《医疗质量安全核心制度要点》文号】，查房结论落为病程记录并驱动医嘱调整——「查房→调整医嘱→执行→再查房」构成住院信息流的每日循环。病区还有一个以「班」为单位的节律：交接班。护理白板（在院患者总览：床位、护理级别、风险预警、当日重点）是交接班的现场工具，接班者据此在几分钟内接管全病区状态——白板信息不准，交接就有漏项，这类「状态总览」页面的数据实时性要求因此高于一般报表。在HIP中，护理侧由「护士执行」「护理白板」「移动工作台」承载，医生侧由「住院医生站」承载。

2.4.4 在院期间的流程变化：转科、会诊与临床路径

住院不总是「一个病区躺到底」。三类常见变化：

- 转科转床：病情变化需换专科（如内科转外科手术），或病区内调床。转科是床位资源的一放一占，必须原子完成并全程留痕——患者「此刻在哪张床」是护理执行、费用记账（床位费按天按床计）、探视管理共同依赖的事实；
- 会诊：主管医生请他科医生对本科患者提出诊疗意见，形成「申请—应邀—意见—采纳」的小闭环。会诊是跨科室的纯信息流协作，时限要求（如急会诊须在规定时间内到位）使其成为质控的监测对象；
- 临床路径：对诊断明确、诊疗方案成熟的病种（如单纯性阑尾炎），预先定义标准化的按天诊疗计划，患者「入径」后医嘱按路径模板批量生成，实际执行偏离模板则记录变异及原因。临床路径把个体化的医嘱流变成可比对的标准流，既是质量工具（减少随意性）也是成本工具（DRG时代控制资源消耗的抓手，第10章）。在HIP中由「患者管理」的路径入径/变异功能承载。

2.4.5 出院：三流的总清算

出院是住院三流的集中闭合点，顺序不能乱：医生下达出院医嘱并完成出院小结→护理停止全部长期医嘱→费用清理（校验有无已开立未执行的医嘱——若有，要么执行要么撤销，否则会出现「收了钱没干活」或「干了活没收钱」）→出院结算（全部住院费用汇总，医保患者做行级费用分割，押金冲抵，多退少补）→床位释放→病案整理、编码、归档（第12章）。图2-4汇总全流程：

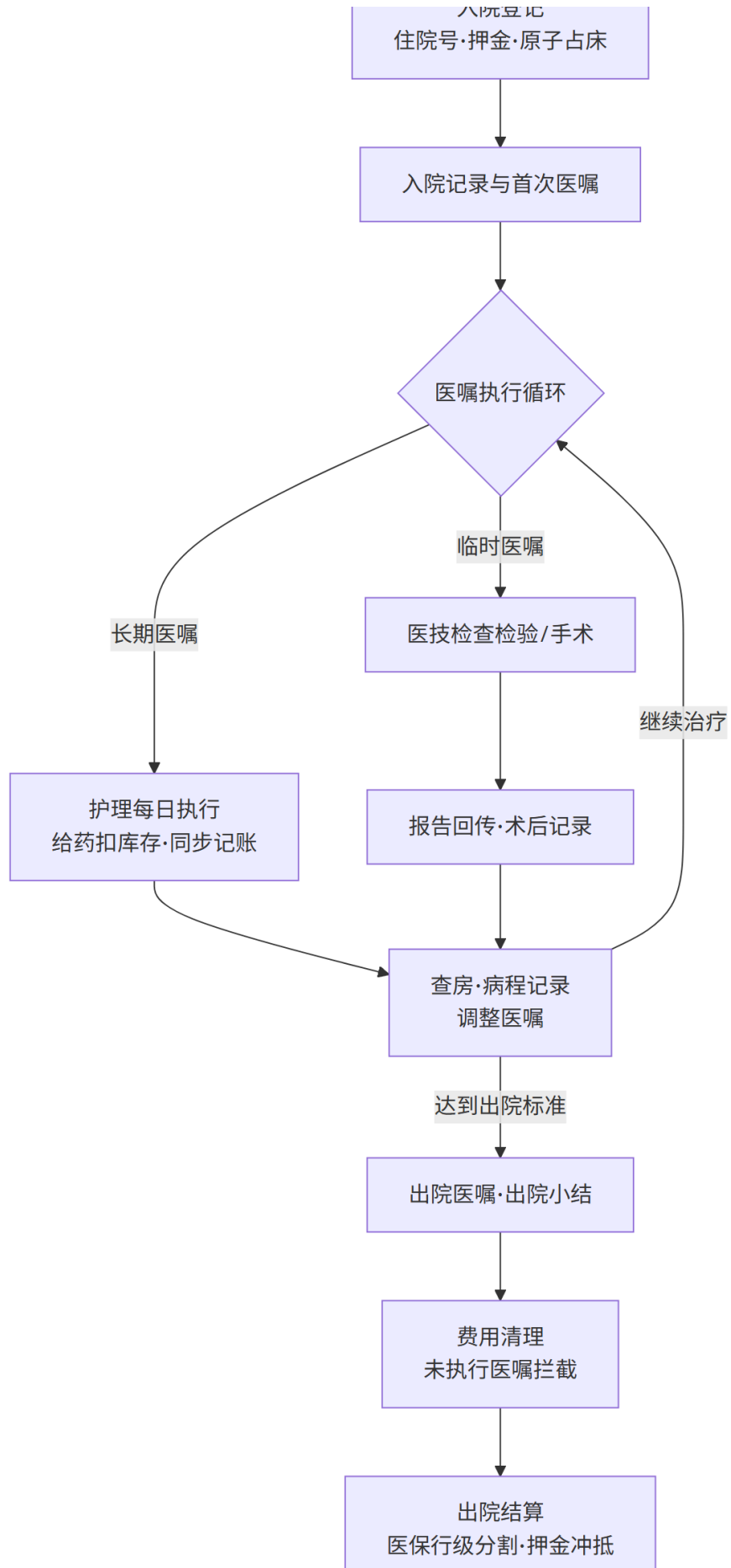


图 2-4 住院诊疗全流程图（中部的医嘱执行循环以「天」为周期运转，是住院与门诊在流程形态上的本质区别）

业务场景框 2-2：一位阑尾炎患者的 72 小时

周三 21:40，大学生小陈因转移性右下腹痛到急诊。分诊护士测得体温 38.2℃，评估为 III 级，进入优先候诊队列。22:00 急诊外科医生接诊，开立血常规与腹部超声——两小时后结果回传：白细胞 $14 \times 10^9/L$ ，超声提示阑尾增粗。诊断急性阑尾炎，建议手术，开具住院证。

23:30 家属办理入院登记：分配住院号、预交押金五千元、普外科病区占床。值班医生下达术前医嘱：禁食（长期）、术前抗菌药皮试（临时）、急诊手术申请（临时）。周四凌晨 2:10 手术——腹腔镜阑尾切除，麻醉记录逐时段留痕，术毕经麻醉复苏室评估达标后返回病房。术后医嘱切换：一级护理、抗菌药静滴每日两次、补液。此后两天病区进入日常节律：护士按医嘱执行给药（每次执行同步扣库存、记费用），每四小时录一次生命体征汇入体温单；医生每日查房写病程，周五查房评估恢复良好，将护理级别降为二级并停补液。

周六上午，医生下达出院医嘱、完成出院小结并签名。护士站停止全部长期医嘱；结算窗口清点费用时系统拦截了一条已开立未执行的换药医嘱——护士确认为出院当日已无需执行，予以撤销后放行。总费用九千余元，医保行级分割后统筹支付六千余，押金冲抵个人部分，退回差额。床位释放，病案在规定时间内归档编码。

这 72 小时里，小陈的信息流跨越了急诊、检验、超声、住院、手术、护理、病案七个环节；资金流从急诊缴费、住院押金到出院总清算；实物流有标本一份、术中耗材若干、药品数十次执行、床位一张占用 60 小时。在 HIP 中，这条动线依次落在急诊预检分诊、LIS 工作台、RIS 报告、住院登记、住院医生站、手术麻醉、护士执行、出院结算、病案统计等功能页上——没有任何一个单独的「系统」能覆盖它，这正是医院信息化必须平台化的业务根源（第 4 章）。

2.5 医技科室业务：科室视角

前两节从患者动线看流程，医技科室看到的是同一流程的另一个侧面：源源不断到达的申请单，以及各自科室内部的一套流转工艺。申请来自门诊与住院两个方向，医技科室必须把两股流量合并排程——设备产能、人员班次、急诊优先级共同决定「今天能做多少、谁先做」，这使医技科室天然是全院的资源调度节点。医技业务的共同骨架是「申请→执行→报告」，各科室的差异在于实物流的主角不同。

检验（LIS, Laboratory Information System）：实物流主角是标本。标本从采集（贴条码建立「标本—患者—申请单」的绑定，这一绑定是检验质量的生命线）、送达接收（核收，拒收不合格标本）、上机检测到结果审核发布，走一条严格的状态链。检验科室的核心效率指标是 **TAT**（Turnaround Time，标本周转时间）——从采集或接收到报告发布的时长，急诊生化的 TAT 承诺常以分钟计，它逼着流程把每个状态节点的时刻都记录下来（没有时间戳就没有 TAT 管理）。发布是信息流的关键一跳：结果回传医生站、申请单自动转已执行；超出危急界限的结果（危急值）触发危急值管理——立即通知临床、要求确认处理并留痕，这是信息流中的「紧急通道」，绕过常规节奏直达责任医生（第 6、9 章详述）。在 HIP 中由「LIS 工作台」承载，替检需登记替检人以保全标本绑定的可追溯。

检查（RIS, Radiology Information System; PACS, Picture Archiving and Communication System）：实物流主角是患者本人——患者要到设备前完成检查，因此时段预约、改约、叫号是检查科室的日常（预约让患者流与设备产能对齐）。产出是双份的：影像归 PACS 管理（以 DICOM 标准存储传输），诊断报告经书写、审核两级后发布——「书写与审核分离」是医技报告质量的通用制度，

审核者对报告负责签发。心电、内镜、超声等在业务上同构，按项目归类管理；住院患者外出检查还涉及陪检安排与病情许可评估，是病区与医技之间的协作细节。检查报告同样有危急界限（如影像发现的急性大出血征象），危急值管理机制与检验共用一套通道。在 HIP 中由「预约与叫号」「RIS 报告」承载。

病理：实物流主角是组织标本，流转链为取材→核收→制片诊断→报告发布。与检验标本「检测后废弃」不同，病理标本的蜡块与切片须长期保存备查，实物流的终点不是消耗而是归档。病理报告常是肿瘤诊断的「金标准」，时效以天计（术中冰冻切片除外——外科医生在台上等结果决定术式，时效以分钟计），是医技中节奏最慢、责任最重的一支。

手术麻醉：手术室是全院实物流密度最高的场所——患者、器械包（来自 CSSD 的消毒灭菌循环）、耗材、血液在此汇聚。业务链为手术申请→排台（手术室与麻醉资源的排程，一间手术室一天的台次安排是典型的资源调度问题）→术前核对（麻醉实施前、手术开始前、离室前的三方安全核查，核对患者、部位、术式【成稿核对：手术安全核查制度的现行规范表述】）→术中（麻醉记录单按时间轴记录用药与生命体征，是麻醉质量与纠纷举证的关键文书）→复苏（PACU，麻醉后恢复室，评分达标方可出室）→术后记录。手术还是费用大户：术中耗材与用药的记录完整性直接决定收费与医保结算的准确性——「台上用了、账上没有」是手术室成本管理的经典漏洞。

输血：血液是受法规最严格管制的实物——用血申请须逐级审批，发血、输血双人核对，输血过程与反应全程记录，形成「申请→审批→发血→输血」的完整状态链，每一步可追溯【成稿核对：《医疗机构临床用血管理办法》现行版本】。

医技的三流特征可以一句话概括：信息流双向（申请下行、报告上行），实物流单向且不可逆（标本消耗、血液输注），资金流则完全依附于门诊收费或住院记账——医技科室自己不收钱，这也是把医技执行与收费状态联动校验（未收费不执行）的业务原因。各科室的系统设计在第 9 章展开。

2.6 药事流程

药品是医院最大宗的实物流，药事管理的链条纵贯「库—房—窗—人」四级。

药库：进销存的源头。采购入库（登记批号、效期、供应商——批号与效期是药品追溯与安全的关键属性）、库存养护、盘点调整、向药房下送。药库业务的铁律是一切库存变动必留流水：入库、出库、盘点、退药各记一笔，任意时点的账面库存都能由流水重演出来——这既是管理要求，也是「账实相符」的技术前提。特殊管理药品（麻醉药品、精神药品等）在此之上叠加更严的管制：专库专柜、专册登记、处方与空安瓿回收核对，流程要求逐支可追溯【成稿核对：麻醉药品和精神药品管理条例的现行要求表述】。在 HIP 中，常规进销存由「药库管理」承载，进销存统一流水、低库存预警。

审方：发药前的专业闸门。药师对方剂进行适宜性审查（处方审核），发现问题的处方拒绝并说明理由，医生须重新开立。审方是合理用药的第二道防线——第一道是开单时的系统自动拦截（2.2.3），系统拦截规则化、审方专业判断化，两者互补。在 HIP 中由「审方工作台」承载，拒绝即处方作废。

发药：三流汇合的窗口。已收费处方进入调剂队列，药师调配、核对、按叫号发药，库存同步扣减（2.2.5 已述）。西药房、中药房（饮片按剂调配，常配套代煎服务）在流程上同构、在计量与调配工艺上迥异，系统需以不同的药房类型分别管理。逆向的退药同样严格：药品实物回库、库存回补并记退药流水，然后才允许退费——实物流回退先于资金流回退的又一实例；已拆封、冷链等不满足回库条件的药品不得退回，拦截条件源自药品安全而非财务规则。

处方点评：与逐张审方互补的事后抽样机制——按月抽取处方样本，由药师依据点评规范评价用药合理性并通报结果，问题处方追溯到医生个人。审方拦在事前、点评查在事后，两者共同构成处方质量的 PDCA 循环。

合理用药监测与药事分析：宏观层面，药学部门定期分析用药结构——用药金额排名、抗菌药物使用强度（AUD，以 DDD 即限定日剂量为单位折算）等指标，既是药事管理与药物治疗学委员会的决策依据，也是国家绩效考核的监测项【成稿核对：抗菌药物使用强度考核参考线的现行口径】。在 HIP 中由「药事分析」承载（相关统计分析见第 12 章）。

业务场景框 2-3：门诊药师的一个上午

8:00，张药师到岗先看审方队列：夜班积压的处方里有一张同时开了头孢类注射剂与含酒精成分的口服液，提示配伍风险，她点了拒绝并注明理由——处方作废，医生站会收到重开提示。8:30 起进入发药高峰，叫号屏按处方聚合叫号，她与同事按「调配—核对—发药」双人流水作业，每发一张，库存自动扣减。10:10 来了一位退药患者：上午刚取的口服药，医生调整了方案。她核对药品未拆封、在退药时限内，办理退药——库存回补、记退药流水，并告知患者现在可以去收费窗口退费了。10:40 空隙里她处理了两条低库存预警，向药库提交请领。11:30 交班前，她顺手看了眼本月药事分析：抗菌药使用强度比上月降了两个点。

这个上午覆盖了药师角色的完整动线：审方、发药、退药、库存、分析。在 HIP 中分别由审方工作台、药房发药、药库管理、药事分析四个功能页承载；注意退药流程里「实物先回、资金后退」的顺序——她无权退费，只能在实物流回退完成后，把资金流的回退交还给收费角色。

2.7 运营后勤流程

运营后勤是医院的「第二战场」：不接触患者，却托着临床运转的底。其业务可分四块：

- 资产与设备：固定资产台账与折旧、调拨报废；医疗设备的维修工单（报修→派工→完成）、保养计划、计量器具到期检定、供应商资质证照管理。设备管理的核心诉求是可用性——一台故障的 CT 影响的是整条检查业务线，因此报修响应时长、保养按期率是设备科的硬指标；大型设备还要算效益账（检查量、单机收入与折旧维保成本的对比），这是运营分析的题材之一；
- 物资：耗材与低值易耗品的进销存（与药库同构的流水化管理）、CSSD 消毒包的「回收—清洗—灭菌—发放」循环追溯——手术器械包是在院内闭环循环的特殊实物流，每个灭菌包带批次标识，一旦某灭菌批次质控不合格，须能反查该批次全部包的去向并召回；
- 人事与 OA：员工档案、工资、培训与继续教育学分、考勤，以及请假、用章、采购等审批流——职能科室的信息流大多以「审批」形态存在；
- 财务日结与对账：这是资金流的每日闭合仪式，分两级。第一级是收款员个人日结：每位收费员日终汇总自己经手的收费与退费，按支付方式分列（现金/移动支付/医保），账面应交与实际清点核对——对不上即产生长款或短款，当日查明原因（常见如退费票据漏登、找零差错）；第二级是全院日结：财务汇总全部收款员的交款，与收入账面轧平，移动支付部分还要与支付服务商的通道流水逐日核对。日结把一天内成千上万笔零散收费压缩为一组可核对的汇总数——资金流不闭合到日，差错就会滚动累积到无法追查。第 10 章的医保对账、第 12 章的运营分析都建立在日结口径之上。

在 HIP 中，本节业务由「固定资产」「设备管理」「物资与 OA」「人事管理」「日结报表」等七个功能页承载。运营域的三流特征与临床相反：实物流与资金流是主角，信息流为它们服务——这也是运营系统（HRP）与临床系统在设计气质上迥异的原因：临床系统争分夺秒，运营系统斤斤计较；前者的失败模式是「慢了」，后者的失败模式是「对不上」。

2.8 患者视角的就医旅程

前七节都站在医院内部看流程；换到患者一侧，看到的不是流程而是旅程——一段由排队、移动、等待串起来的体验。完整的旅程分三段：诊前（感到不适、选择医院与科室、预约）、诊中（院内的挂号—就诊—检查—缴费—取药）、诊后（康复、复诊提醒、慢病随访）。传统信息化只覆盖诊中，诊前靠患者自己摸索、诊后基本失联；「以患者为中心」的建设方向，就是把信息流向诊前（导诊、预约）与诊后（报告推送、随访计划）两端延伸。

诊中一段的传统痛点被概括为「三长一短」：挂号排队长、缴费排队长、取药排队长，看病时间短。用三流视角诊断：患者排的三个长队，恰好都是三流的汇合点（挂号建立就诊、收费结清费用、发药交付实物）——汇合点靠窗口人工核对，队伍就长在核对上。

「互联网+医疗」的改造思路由此清晰：把信息流与资金流从物理窗口上移到线上，实物流留在线下但精确预约到点。患者端（手机 H5/小程序）承载的典型能力：

- 预约挂号：号源上线，患者按时段预约，到院即诊——挂号队伍消失；
- 线上缴费：诊间开单后手机出码支付——缴费队伍消失，且资金流汇合点提前到诊室门口；
- 报告查询：检验检查报告发布后线上可查，无需专程返院取单。这里有一个值得注意的伦理设计：危急值不在线上直接展示，而是提示「请尽快回院复诊」——危急结果必须在有医生解释与处置能力的场景下告知，这是信息流设计里的医学伦理约束；
- 智能导诊：按症状推荐科室，替代分诊台的部分咨询功能。导诊的边界必须清晰——它推荐「去哪个科」，不给出诊断与用药建议；任何面向患者的智能功能都要在「有用」与「越界」之间画好线（AI 应用的架构与边界见附录 A）。

诊后一端的延伸以随访为代表：出院或门诊后按计划回访（术后复查提醒、慢病定期随访、满意度回访），把一次性的就诊拉长为连续的健康管理关系。随访是纯信息流业务，成本低而黏性高，也是公立医院绩效考核与患者满意度指标的来源之一。在 HIP 中由「患者管理」的随访与满意度功能承载。

线上化不改变流程骨架——挂号、收费、执行的控制点原样保留，只是执行介质从窗口变为手机。因此患者端在架构上是院内系统的一个新入口而非新系统（第7章）。在 HIP 中由患者端（/portal）承载：预约挂号、待缴费出码支付、检验/检查/病理报告查询（危急值遮蔽为回院提示）、报告分享短链（匿名脱敏限时）、智能导诊；微信实名认证与电子健康卡属外部条件。

业务场景框 2-4：同一位患者的两次就医

李女士上次门诊（业务场景框 2-1）在院内停留约两小时，其中就诊与采血合计不到二十分钟，其余是排队与等待。三个月后她复查血常规，这次用了医院的患者端：前一晚手机预约了 9:00 - 9:30 时段的号；到院直接候诊，诊间医生开单后她在椅子上完成扫码缴费，径直去采血；报告发布后手机收到通知，线上查看结果正常，无需再回诊室。在院总时长四十分钟。

对比两次旅程：流程环节一个没少——挂号、接诊、开单、收费、执行、报告全在；变化的只是信息流与资金流的发生地点从窗口移到了指尖。而假如这次报告出现危急值，她在手机上只会看到「请尽快回院复诊」——系统把最关键的一条信息流，刻意留在了线下。

本章小结

- 医院科室分临床/医技/职能/后勤四类，信息系统则按八角色（管理员/医生/护士/收费/药师/医技/质控院感/运营后勤）组织功能——角色而非科室才是系统的功能单元。

- 三流视角是分析医院流程的通用框架：信息流（单据报文）、资金流（费用收缴）、实物流（药品/标本/血液/床位）在开单处分叉、在收费与发药处汇合、在日结与归档处闭合；汇合点即控制点，信息化的核心工作是把汇合点的核对从人工变为系统。业务闭环 = 三流各自走完且互相对平。
- 门诊主链六环节（挂号→接诊→开单→收费→执行→发药），基本规则是先收费后执行；逆向流程（退药→退费→退号）遵循「下游流未回退则上游流不得回退」。
- 急诊以时间优先重排流程：预检分诊 I - IV 级按病情顺序，绿色通道将资金流后置——三流关系的刻意倒置，代价由事后追缴机制承担；留观是「微型住院」的中间形态。
- 住院以医嘱为信息流主轴，长期/临时医嘱驱动「查房→调整→执行」的每日循环；记账式医嘱把信息流与资金流合并为一条链；押金预收对冲欠费风险；转科转床、会诊、临床路径是在院期间的三类流程变化；出院是三流总清算，未执行医嘱拦截保证「活」与「钱」对平。
- 医技业务共享「申请→执行→报告」骨架，差异在实物流主角（标本/患者/组织/器械/血液）；信息流双向，报告回传与危急值通道是逆向信息流的两种形态；医技不收钱，资金流依附于门诊收费与住院记账。
- 药事纵贯「库一房一窗一人」，铁律是库存变动全流水；审方与系统拦截构成合理用药双防线；运营后勤域实物流与资金流是主角，日结对账是资金流的每日闭合仪式。
- 患者旅程分诊前/诊中/诊后三段，信息化正从诊中向两端延伸（导诊预约、报告推送与随访）；诊中「三长一短」长在三流汇合点的人工核对上，互联网+的改造是把信息流资金流上移线上、实物流精确预约，流程控制点不变；危急值不外显是信息流设计中的医学伦理约束。
- 带着三流地图进入后续章节：第 7 - 9 章逐条业务线展开系统设计时，每一个「原子操作」「状态机」「拦截校验」，都能在本章的某个汇合点或闭合点上找到它的业务出处。

思考与练习

1. 概念：门诊医生与病区医生分属不同科室，为什么在信息系统中常映射为同一大类角色？举出一个「同科室不同角色」和一个「同角色跨科室」的例子，说明按角色授权相对按科室授权的优势与局限。
2. 画图：以图 2-1 为底，画出「已发药、已收费、已挂号」状态下患者要求全额退费的完整逆向路径，标出每一步的拦截条件（提示：实物流→资金流→信息流的回退顺序）。
3. 画图：参照图 2-2 的画法，为急诊绿色通道画一张三流合一图，用醒目标记标出它与门诊三流图的差异点（资金流后置到何处闭合？）。
4. 流程分析：出院结算前的「未执行医嘱拦截」若被取消，分别推演「已开立未执行的药品医嘱」与「已执行未停止的长期医嘱」两种遗留情形下，三流会出现什么样的对不平？由哪个岗位在何时发现？
5. 画图：画出检验标本从医生开单到报告回传的泳道图（泳道：医生/护士或采血处/检验科/系统），并用旁路箭头标出危急值的紧急通道与常规报告路径的差异。
6. 分析：部分地区试点「先诊疗后付费」（诊疗全程记账、离院前一次结清）。对照 2.2 节「先收费后执行」的门诊流程，分析这一模式移动了哪些三流汇合点、新增了什么风险、需要哪些配套机制（可参考 2.3 急诊记账与 2.4 住院押金的既有做法）。
7. 调研：实地走访（或回忆）一次你所在城市医院的就医经历，按 2.8 节的旅程视角列出全部排队与等待环节，判断每个环节属于哪类汇合点，并提出至少三条信息化改进建议，说明各自改造的是哪条流。
8. 拓展：住院押金在会计上是预收款而非收入。从资金流角度分析：押金在「入院—在院—出院」三个阶段各处于什么状态？若患者中途欠费（费用累计超过押金），流程上应在哪个环节、由哪个角色触发什么动作？

第 3 章 标准与规范

本章属第一篇「基础篇」。医院信息系统的每一次对接、每一次上报、每一次测评，背后都是标准在起作用。本章自底向上讲解医院信息化的标准体系：分类编码、数据元与值域、临床文档、交换消息、医保编码，最后讨论标准落地的工程策略。与后续章节的分工是：本章讲标准本身的结构与设计思想，实现细节引向对应章——HL7 通信实战在第 6 章，医保贯标与接口在第 10 章，CDA 文档生成在第 11 章，病案编码质量管理在第 12 章。

学习目标

学完本章，你应当能够：

1. 画出医院信息标准的四层全景图（分类编码 / 数据元与值域 / 文档 / 消息），说出每层代表性标准与它解决的问题；
2. 解释 ICD-10 的类目—亚目—扩展码结构与国家临床版的关系，给定一段诊断文本能说出编码的查找思路；
3. 说出卫生信息数据元的三要素（标识、定义、表示），读懂一条 WS 363 数据元并把它落成一行元数据记录；
4. 描述 CDA 文档的层次结构（头/体/节/条目）与三级结构化程度，解释「文档」与「消息」的本质区别；
5. 读懂一条 HL7 V2 报文的段结构，解释事件驱动模型与 ACK 应答机制；
6. 说出 FHIR 资源模型与 RESTful 交换的要点，能对给定场景论证选 V2 还是 FHIR；
7. 解释医保 15 项编码贯标的意义，理解「标准内建」相对「事后改造」的工程优势。

3.1 分类编码体系

3.1.1 标准的四层全景

初学者面对医疗信息标准，最大的困惑是「这么多标准到底谁管谁」。一个有效的心智模型是按粒度分四层（图 3-1）：最底层是分类编码——给单个概念（一个诊断、一个药品、一个检验项目）一个全局唯一的码；其上是数据元与值域——规定一个数据项叫什么、什么类型、取值从哪个代码表里选；再上是文档——规定一次诊疗产生的成套记录（出院小结、检验报告）如何组织；最上层是消息——规定系统之间实时传递数据的报文格式与交互行为。上层标准引用下层：一条 HL7 消息的字段里装着 LOINC 编码，一份 CDA 文档的条目里装着 ICD 编码。

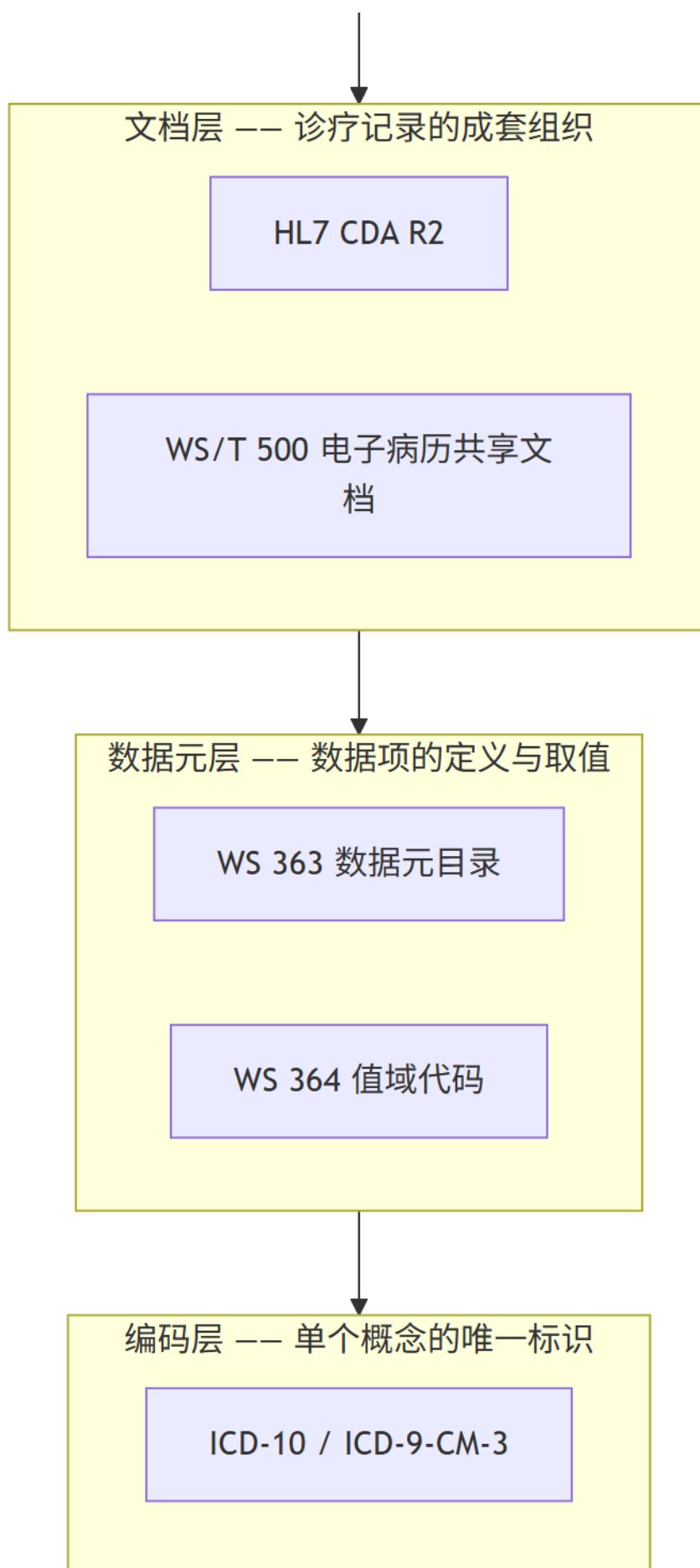


图 3-1 医院信息标准四层全景：上层标准的字段里装着下层标准的码

本章 3.1—3.6 依次走完这四层（医保编码单列一节，因为它自成体系且直接关联收入），3.7 讨论怎么把它们落进一个真实系统。

3.1.2 ICD-10：疾病分类

ICD (International Classification of Diseases, 国际疾病分类) 由世界卫生组织维护，是全球通用的疾病和有关健康问题的统计分类。当前我国病案与医保场景使用的是第十次修订本 ICD-10 (ICD-11 已由 WHO 发布，国内处于试点推进阶段，【成稿核对：ICD-11 国内应用的最新政策口径】)。

ICD-10 的编码是字母开头的层级结构：

- 章：全书按疾病系统与特殊目的分为 22 章，每章占用若干字母段（如第十章呼吸系统疾病为 J00 - J99）；
- 类目：3 位码（字母 + 两位数字），如 J06 多部位急性上呼吸道感染；
- 亚目：4 位码（类目 + 小数点后 1 位），如 J06.9 急性上呼吸道感染，未特指；
- 扩展码：我国临床使用的版本在亚目之后再扩展 2 位，形成 6 位码，如 J06.900 ——这是国家临床版（国家卫生健康委组织编制的《疾病分类代码国家临床版》，【成稿核对：国临版现行版本号与发布年份】）的形态。此外国家医保局另有医保版疾病诊断分类代码（贯标编码之一，见 3.6），两版同源 ICD-10 但扩展码不完全一致，病案与医保上报时须按各自口径。

ICD-10 还有两个编码机制值得知道：剑号/星号双重分类（†/*）——病因编码与临床表现编码成对出现，如结核性脑膜炎以剑号编病因、星号编表现；肿瘤形态学码（M 码）——肿瘤除部位编码外另编组织学形态与动态（良性/恶性/原位）。

给一段诊断文本编码的基本思路（详细的编码规则与主要诊断选择原则属病案专业，第 12 章展开）：先从诊断中找主导词（通常是疾病名称的核心，如「肺炎」而非部位词「左下肺」），查字母顺序索引得到候选类目，再回类目表核对包含与排除注释，编到可编的最细一级。切忌拿着诊断全文在字典里做字符串模糊匹配——「胃溃疡穿孔」与「胃溃疡」是不同亚目，编码的粒度直接决定 DRG 入组（第 10 章）与统计口径。

3.1.3 ICD-9-CM-3：手术与操作分类

诊断用 ICD-10，手术与操作用 ICD-9-CM-3 (ICD-9 临床修订本第三卷) ——这是历史形成的搭配：WHO 的 ICD-10 没有配套的操作分类，各国沿用或自建，我国选择在 ICD-9-CM 第三卷基础上扩展为国家临床版（【成稿核对：ICD-9-CM-3 国临版现行版本】）。其编码为纯数字 xx.xx 形态（如阑尾切除术 47.0x），按解剖系统分章。病案首页的手术操作编码质量与诊断编码同等重要——DRG 分组的外科/内科分流正是「是否有手术操作编码」驱动的（第 10 章 10.6）。

3.1.4 药品、检验与收费项目编码

诊断之外，医院日常数据里还有三类高频编码对象：

- 药品。院内通常自编药品字典码；行业层面有多套编码并存：药品批准文号（监管身份）、国家药品编码本位码（流通条码体系）、WHO 的 ATC 分类（Anatomical Therapeutic Chemical，解剖学治疗学化学分类，7 位层级码如 J01CA04 阿莫西林——它标识的是「成分类别」而非具体药品，适合用药分析与 DDD 统计，第 12 章）、以及医保药品编码（一药一码，见 3.6）。多套编码各有用途，系统设计要点是院内码为主键、标准码为映射属性。
- 检验项目。国际通用编码是 LOINC (Logical Observation Identifiers Names and Codes)，如血红蛋白 718-7。LOINC 解决的是「不同医院的『血红蛋白』是不是同一个项目」——区域平台做检验结果互认时，本地项目码必须映射到 LOINC 或省级统一目录才能跨院比对。
- 收费项目。依据《全国医疗服务价格项目规范》与各省价格文件形成收费目录，医保侧另有医疗服务项目贯标编码。收费编码是「政策敏感」编码——调价、新增、停用频繁，字典必须支持启停与生效日期。

案例框 3-1：编码在 HIP 里落在哪

HIP 的编码落点体现「院内码为主键、标准码贯穿业务」的结构：

- ICD-10 字典表 `md_icd10` (`code/name/pinyin`, `V4__masterdata.sql`) 是一张简表——种子只有几十条常见诊断 (`J06.900` 急性上呼吸道感染等)，实施时由医院导入全量国临版字典 (第 14 章的字典 CSV 模板家族)。表结构上 `code` 即主键，6 位国临版码原样承载；
- 诊断落库时码与名一起冗余：`outp_diagnosis` 表存 `icd_code + icd_name + primary_diag` (主诊断标记)。冗余名称不是偷懒——字典年年更新，诊断记录必须保留开立当时的名称口径，这是病历的法律属性决定的；
- 编码进而成为下游逻辑的驱动键：DRG 分组表 `drg_group_def` 按 `icd_prefixes` (ICD 前缀清单) 命中病例，单病种上报 `sd_disease_def` 按 `icd_prefix` 自动入组——前缀匹配之所以可行，正是因为 ICD 是层级编码，前缀天然代表上位分类。编码不只是上报字段，而是可计算的数据资产——这是「标准内建」(3.7) 最直接的红利。

3.2 卫生信息数据元与值域代码

3.2.1 元数据方法：数据元三要素

编码解决「概念怎么标识」，数据元解决「数据项怎么定义」。数据元 (Data Element) 是不可再分的最小数据单元，国家卫生行业标准用统一的元数据方法描述它，核心是三要素：

1. 标识：全国唯一的数据元标识符，形如 `DE02.01.040.00` ——DE 前缀 + 分类段 (大类.小类) + 序号 + 版本段，标识符本身携带分类信息；
2. 定义：名称与语义定义 (「性别代码」= 本人生理性别的代码标识)；
3. 表示：数据类型 + 表示格式 + 允许值。数据类型分字符型 (S)、数值型 (N)、日期型 (D)、布尔型 (B)、编码型 (C) 等；表示格式用紧凑记法，如 `A..50` (可变长字符最长 50)、`AN..18` (字母数字混合最长 18)、`YYYYMMDD` (日期)、`N1` (1 位数字)；编码型数据元的允许值指向一张值域代码表。

这套方法的行业标准载体是：**WS/T 303**《卫生信息数据元标准化规则》定方法论，**WS 363**《卫生信息数据元目录》给出全国统一的数据元清单 (分主题多个部分，【成稿核对：WS 363 的部分数与现行版本】)，**WS 364**《卫生信息数据元值域代码》配套给出各编码型数据元的代码表。例如「性别代码」的值域引用国标 GB/T 2261.1 (0 未知、1 男、2 女、9 未说明)，而不是各家系统自造「M/F」。

数据元方法的价值在上报与互认场景兑现：互联互通测评、区域平台对接、各类国家级上报，本质都是把院内字段映射到国家数据元。院内建表时若字段命名、类型、值域与数据元对齐，映射就是一张薄薄的对照表；若各业务系统自造口径，每次上报都是一场字段考古。

3.2.2 值域代码：字典的字典

数据元向上组装成数据集 (Dataset)：一个业务主题所需数据元的集合，如「住院病案首页数据集」「门诊摘要数据集」。行业标准为数据集的分类编码与元数据描述定了规则 (WS/T 306 等)，各类国家级数据集标准 (电子病历基本数据集 WS 445、健康档案相关数据集) 都遵循同一元数据框架。于是形成一条清晰的组装链：数据元 → 数据集 → 共享文档——3.3 的 WS/T 500 文档正是把 WS 445 数据集套进 CDA 结构的产物。理解这条链，读任何一份上报接口文档都会快很多：接口文档的字段清单本质就是一个数据集，每个字段背后是一条数据元。

值域代码表回答「这个编码型字段可以取哪些值」。它与业务字典的关系值得辨析：院内字典 (科室表、药品表) 是实例数据，各院不同；值域代码 (性别、婚姻状况、证件类型、民族) 是标准枚举，

全国统一。系统设计的常见错误是把两者混在一张「通用字典表」里，导致标准枚举被实施人员随手改值。正确姿态是：值域代码表随产品发布、锁定修改；业务字典开放维护（第5章的字典管理、第14章的种子数据分层讲的正是这个边界）。

案例框 3-2：一条 WS 363 数据元怎么落进 HIP

HIP 的数据治理模块（`datagov`）用 `dg_data_element` 表承载数据元登记（`V23__phase26_datastd.sql`），列结构就是三要素的直译：`code`（数据元标识，唯一约束）、`name`、`datatype`（S/N/D/B/C 五型）、`format`（表示格式）、`value_set`（允许值说明）、`std_ref`（引用标准出处）。种子数据示范了标准的读法：

code	name	datatype	format	value_set	std_ref
DE02.01.039.00	本人姓名	S	A..50	—	WS 363.2
DE02.01.040.00	性别代码	C	N1	GB/T 2261.1（0-9）	WS 363.2
DE02.01.005.01	出生日期	D	YYYYMMDD	—	WS 363.2
DE08.10.052.00	危急值标识	B	T/F	是/否	WS/T 303

配套的 `dg_term` 表做术语映射：本地术语 → 标准编码，键为「类别 + 本地名称 + 标准体系」，`std_system` 支持 ICD10 / LOINC / ATC / ICD9CM3 多体系并存——同一个本地药品可以既映射 ATC 又映射医保码。种子里的三行（急性上呼吸道感染 → J06.900 @ICD10、血红蛋白 → 718-7 @LOINC、阿莫西林胶囊 → J01CA04 @ATC）恰好对应 3.1.4 的三类编码对象。管理入口在 `DataStdController.java`（重复登记返回专用错误码 4630/4631）。

要诚实指出当前的局限：这两张表是登记册形态——数据元没有反向驱动业务表的建表与校验，术语映射也尚未接入上报/同步链路，它们此刻的价值是把标准对照留在系统里而不是实施人员的 Excel 里。演进方向（第12章数据治理）是让质量规则引用数据元做校验、让 CDA 生成与数据订阅查询 `dg_term` 完成编码转换——登记册先行，消费链路渐进接入，这本身就是标准落地的务实节奏。

3.3 电子病历文档标准：CDA 与 WS/T 500

3.3.1 文档与消息的本质区别

检验结果既可以装在 HL7 消息里（3.4），也可以装在 CDA 文档里，初学者常问何必两套。区别在语义：消息是「事件通知」——瞬时、面向系统、传完即完成使命；文档是「诊疗记录」——持久、面向人的阅读、具有完整性与法律属性。CDA 规范对文档的定义强调六个特征：持久性、可管辖性（由机构负责保管）、可认证性（可签名）、完整性（整体为单位）、人可读性、上下文性。一份出院小结签名归档后是不可再拆的整体；而一条「结果已发布」的消息不需要签名，也没人会十年后调阅它。

3.3.2 CDA 的层次结构

CDA（Clinical Document Architecture，临床文档架构）是 HL7 组织的文档标准，现行为 R2 版本，基于 HL7 V3 的 RIM 模型，用 XML 表达。一份 CDA 文档的骨架自上而下四层（图 3-2）：

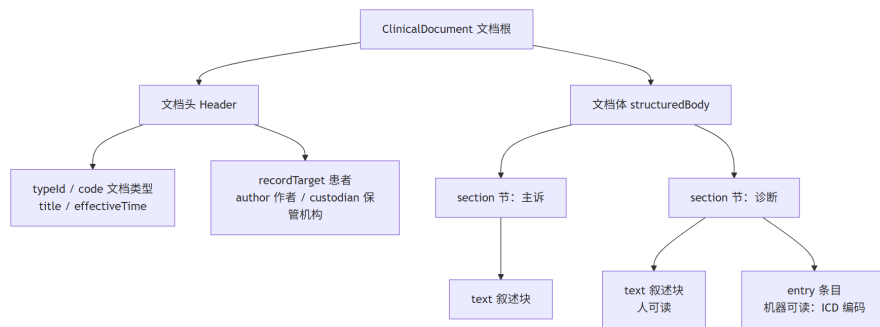


图 3-2 CDA 文档层次：头/体/节/条目，*text* 给人读、*entry* 给机器读

- 文档头 (Header)：文档类型、标识、时间，以及三类参与者——患者 (recordTarget)、作者 (author)、保管机构 (custodian)。头部是全结构化的，保证「不打开正文也能管理文档」；
- 文档体 (Body)：由若干节 (section) 组成，每节对应临床叙述的一个自然段落 (主诉、现病史、诊断、用药)；
- 每节内部双轨并行：**text** 叙述块是人可读的富文本，**entry** 条目是机器可读的编码语句 (一条诊断 entry 里装着 ICD 码)。

由此定义了三级结构化程度：**Level 1** 只有头结构化、体为整段叙述；**Level 2** 体分节、节有编码标题；**Level 3** 节内含 entry 条目级编码。三级不是优劣而是成本阶梯——Level 1 一天能做出来，Level 3 要求上游业务数据本身是结构化的。

CDA 的标识体系用 **OID** (Object Identifier, 对象标识符)——层级化的数字串，HL7 的根为 2.16.840.1.113883。文档类型、编码体系、机构都以 OID 全局定位，这是跨机构交换不撞车的前提。

3.3.3 WS/T 500 系列：中国的共享文档规范

我国在 CDA R2 基础上制定了本土化的文档规范族：**WS 445**《电子病历基本数据集》定义各类病历记录该有哪些数据元 (承接 3.2 的数据元方法)，**WS/T 500**《电子病历共享文档规范》定义如何把数据集组装成 CDA 形态的共享文档 (分数十个部分，覆盖病历概要、门急诊病历、检查检验报告、住院病案首页、出院小结、手术记录等)，【成稿核对：WS 445 / WS/T 500 的部分数与发布年份】；另有面向公卫的 WS/T 483 健康档案共享文档规范)。互联互通标准化成熟度测评 (第 1 章) 的核心技术要求，正是医院能按 WS/T 500 生成规定种类的共享文档并提供检索服务——这是「文档标准」从教科书概念变成医院刚需的现实驱动。

案例框 3-3: HIP 的 CDA 骨架生成

HIP 的临床数据中心 (CDR, 第 11 章) 为每份患者文档提供 CDA 样式输出：GET /api/cdr/documents/{id}/cda 返回 XML，生成逻辑在 CdrSyncService.java 的 toCda()。对照 3.3.2 的层次读这段模板：根元素 ClinicalDocument (命名空间 urn:h17-org:v3)，typeid 的 OID 根 2.16.840.1.113883.1.3，头部有 code/title/effectiveTime 与 recordTarget/patientRole (患者标识、姓名、性别、出生日期取自 EMPI)，体部是 structuredBody → component → section，节内容以 <text><![CDATA[...]]></text> 装载文档 JSON——严格说这是 **Level 1**：头结构化、体为叙述块，没有 entry 条目。

这个「骨架先行」的取舍值得体会：CDR 的文档内容本身是结构化 JSON (诊断带 ICD 码、检验带结果行)，升到 Level 3 缺的不是数据而是映射工作量——把 JSON 字段逐一 translate 成 entry 的 XML 语句、编码走 dg_term 转换。骨架把「文档类型、患者、层次结构」这些接口形态先钉住，测评或区域平台要求到来时，升级发生在 toCda() 一个方法内部，调用方无感。这与第 10 章「结构完整、目录简化」的 DRG 分组器是同一个方法论。

3.4 消息交换标准：HL7 V2 与 MLLP

3.4.1 报文文法

HL7 V2 是 1980 年代末诞生、至今仍统治设备与科室系统接口的消息标准。它的报文是管道分隔的分段文本：报文由段（Segment）组成，段名 3 个大写字母；段内字段以 `|` 分隔，字段内组件以 `^` 分隔，再加重复符 `~`、转义符 `\`、子组件符 `&`，五个分隔符在报文头 `MSH|^~\&|...` 中自我声明。常用段：**MSH** 消息头（收发方、时间、消息类型、控制号、版本）、**PID** 患者标识、**OBR** 申请/医嘱信息、**OBX** 观察结果（一行一个结果项）。一条检验结果报文的具体样例、字段释义与著名的「MSH-1 是分隔符本身」索引陷阱，第 6 章 6.3 已随 HIP 实现详解，此处不重复。

3.4.2 事件驱动模型与应答

V2 的交互哲学是事件驱动：真实世界发生一件事（患者入院、医嘱开立、结果发布），触发一条对应类型的消息。消息类型 = 消息类别 + 触发事件，写在 MSH-9，如 `ADT^A01`（入院）、`ADT^A08`（患者信息更新）、`ORM^O01`（医嘱）、`ORU^R01`（检验结果）。接收方处理后回 **ACK** 应答：`MSA|AA`（接受）或 `MSA|AE`（应用错误，带原因），控制号原样回带以便发送方匹配。「事件—消息—应答」这个三元组是理解一切 V2 接口文档的钥匙。

传输承载的事实标准是 **MLLP**（Minimal Lower Layer Protocol）：TCP 长连接上以控制字符定帧（`0x0B + 报文 + 0x1C 0x0D`）。HIP 的 `MllpServer.java` 与解析器 `HL7V2Message.java` 是一份不到 200 行的完整参考实现（第 6 章案例框 6-2）。

3.4.3 版本演进与「方言」问题

V2 从 2.1 一路演进到 2.9，惯例上小版本向后兼容：新版本增段增字段，不改既有字段语义，所以 2.3 的解析器大体能读 2.5 的报文。国内设备与 LIS 中间件最常见的是 2.3.1 与 2.5.x。V2 的最大特点也是最大问题是宽松：大量字段可选、允许自定义 Z 段，每家厂商都说自己「支持 HL7」，但字段用法各不相同——业内自嘲「见过一个 HL7 接口，就只见过这一个 HL7 接口」。这决定了 V2 对接的工程重心不在解析器而在逐家联调与字段映射表，也解释了为什么 HIP 选择宽容取值的轻量解析器而非强类型类库（第 6 章）。V2 报文本质是「电报」——紧凑、高效、但要拿着码本才能读；这为 3.5 的 FHIR 出场做了铺垫。

3.5 HL7 FHIR R4：新一代互操作

3.5.1 资源模型

FHIR（Fast Healthcare Interoperability Resources，快速医疗互操作资源）是 HL7 组织的新一代标准，现行主流版本 **R4**（4.0.1，2019 年发布）。它的核心创新是把医疗信息拆成约一百五十个资源（Resource）：`Patient`（患者）、`Encounter`（就诊）、`Observation`（观察/检验结果）、`Condition`（诊断）、`MedicationRequest`（用药申请）……每个资源是一个有标准字段定义的 JSON/XML 对象，资源之间以引用（reference）连接——一条 `Observation` 通过 `subject` 引用它的 `Patient`。本地化扩展不再靠 Z 段，而是规范化的 **extension** 机制与 **profile**（对资源的约束裁剪）。

对比 V2 的「电报」，FHIR 资源是自描述的：

```
{
  "resourceType": "Observation",
  "status": "final",
  "code": { "coding": [{ "system": "http://loinc.org", "code": "718-7", "display":
"Hemoglobin" }] },
  "subject": { "reference": "Patient/P00000002" },
```

```

"valueQuantity": { "value": 45, "unit": "g/L" },
"interpretation": [{ "coding": [{ "code": "LL" }] }]
}

```

注意 `code.system` 指向 LOINC——四层全景（图 3-1）中「消息层引用编码层」在 FHIR 里是显式的 URI，而 V2 的 OBX-3 只是裸编码，靠接口文档约定体系。

3.5.2 RESTful 交换形态

FHIR 的第二个创新是交换方式：不再定义专用传输协议，直接采用 Web 的 **RESTful** 风格——每个资源有 URL（`GET /Patient/123` 读取、`POST /Observation` 创建、`PUT` 更新），支持标准化检索（`GET /Observation?subject=Patient/123&code=718-7`），批量操作用 `Bundle` 资源打包，服务端以 `CapabilityStatement` 声明自己支持什么。这让医疗接口与互联网技术栈汇合：HTTPS、OAuth2 授权（SMART on FHIR 应用生态）、JSON、前端可直连。两代标准的交换形态对比见图 3-3：

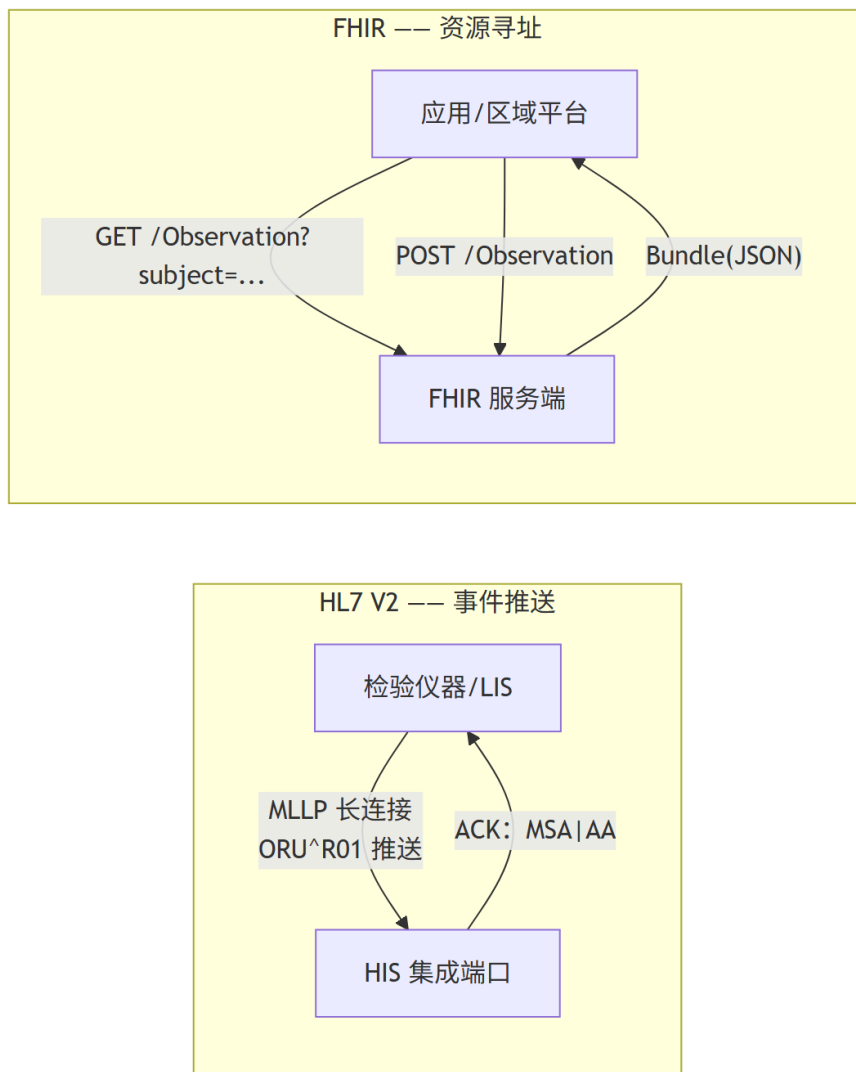


图 3-3 V2 与 FHIR 的交换形态：点对点事件推送 vs 资源化按需读写

3.5.3 与 V2/CDA 的关系与迁移趋势

FHIR 不会短期取代 V2——设备与 LIS 中间件的存量生态太深，V2 在「科室系统/设备接口」这一层还会长期存在；CDA 所代表的「文档」概念在 FHIR 中由 `DocumentReference` 与文档型 `Bundle` 承接。务实的图景是分层共存：设备层 V2、文档交换层 CDA/WS 500、新建的应用层与区域互联层渐进采用 FHIR，中间靠集成平台做转换。国内现状：互联互通测评的技术底座仍是 WS 系列文档与服

务规范，FHIR 多见于互联网医疗、科研数据平台与新建区域项目（【成稿核对：FHIR 在国内测评/行业规范中的最新采纳情况】）。

给工程师的选型直觉：对方是设备或传统 LIS/PACS，用 V2；对外提供数据服务、对接互联网应用或新建平台，用 FHIR；要交换有签名归档语义的成套病历，用文档标准。HIP 当前未内置 FHIR 服务端——它面向的县市级单院场景暂无此需求，但 CDR 的患者文档模型（第 11 章）与 FHIR 的资源思想同构，未来在 CDR 之上加一层 FHIR Facade 是清晰的演进路径。

3.6 医保编码与接口规范

3.6.1 15 项贯标编码

国家医保局自 2019 年起推行全国统一的医保信息业务编码（业内称贯标），共 15 项，可分三组理解（【成稿核对：15 项编码的完整枚举与现行文号】）：

组	编码项（举要）	对医院的意义
目录类	医保疾病诊断分类与代码、手术操作分类与代码、医保药品、医疗服务项目、医用耗材	与院内字典逐条对照，决定每行费用的报销口径
机构人员类	定点医疗机构、医保医师、医保护士、定点零售药店等	结算与处方上传的身份标识，人员变动须同步维护
业务类	医保结算清单、门诊慢特病病种、按病种结算病种等	上报文书与支付方式的载体编码

贯标之前，各统筹区自编目录码，医院对接三个统筹区就要维护三套对照；贯标把「N 地 × N 码」收敛为「院内码 × 全国码」一套映射。对医院信息系统，贯标落点就是 3.1.4 讲的映射结构：HIP 的 `yb_catalog_map` 表（V24__phase27_insurance.sql）以 `item_type + item_code`（院内）对 `yb_code`（医保贯标码），附带 `charge_class`（甲乙丙）与 `self_ratio`（先行自付比例），后续迁移又补了 `effective_date` 生效日期——编码对照是有时间维度的（目录年度调整）。对照工程与对照率管理在第 10 章 10.1 详述。

3.6.2 接口规范的形态

医保接口在标准形态上与前几节都不同：它不是公开的行业标准文本，而是国家统一接口规范 + 省级实施包的双层结构。国家医保局发布统一的接口规范文档——报文为 JSON，接口按业务编号组织（人员信息查询、费用明细上传、结算、冲正、对账文件下载等各有编号，【成稿核对：引用接口编号时对照现行版本接口文档】）；落到各省则是不同的接入形态：动态库 SDK、独立前置服务或报文网关，部署于院内前置机并经医保专线接入。这种「规范统一、实施各异」的形态，决定了医院端系统必须用适配器把省差异挡在平台之外——HIP 的 `InsuranceAdapter` SPI 及其书面契约 ADR-0002 是完整的工程答案，第 10 章 10.7 详述。

医保域还有一份值得单列的「文档级」规范：医保结算清单——全国统一格式的出院结算文书，整合病案首页核心项、行级费用明细与医保结算信息，是 DRG/DIP 付费的官方数据载体。把它放进本章的框架看：清单的字段清单是一个数据集（每个字段对应贯标编码或数据元口径），清单本身是一份结构化文档，上传走接口规范——医保域把四层标准完整走了一遍。清单对院内系统的连锁要求（行级费用、编码质量、结算号）在第 10 章 10.6.4 展开。本章只需记住定位：贯标编码是数据标准，接口规范是交换标准，两者合起来构成医保域自成一体的「小四层」。

3.7 标准落地策略：内建 vs 事后改造

3.7.1 两条路线的真实代价

标准进系统有两条路线。事后改造：系统按自己的舒服方式建模，等测评、上报、对接来了，再开发导出与映射。内建：建模阶段就让关键字段对齐标准——诊断表带 ICD 码列、费用表行级可归类、患者表字段对齐数据元、留痕表按交换规范设计。

事后改造的代价常被低估，因为最贵的不是写导出程序，而是当初没存的数据：住院费用只记总额，DRG 要行级明细（第 10 章的真实返工）；诊断只存文本，上报要编码——补录历史数据的人力是数量级于开发的成本。而内建的代价其实很小：标准早已把字段、类型、值域设计好了，照着建模几乎不增加工作量，难的只是建模的人知道标准。这正是本章存在的理由——标准知识的价值兑现在建模的那一刻，而不是测评前的冲刺里。

当然，内建不等于全量吞下标准。务实的内建是分级的：编码字段与行级粒度必须内建（数据没存就永远没了）；文档与接口形态可以骨架先行（结构对、内容简化，如 Level 1 CDA）；映射内容可以渐进补齐（对照率作为运营指标管理）。

3.7.2 HIP 的「标准内建」地图

把本章讲过的标准与 HIP 的模块落点对齐成一张地图，作为四层全景（图 3-1）的落地版：

标准层	标准	HIP 落点	载体
编码	ICD-10 / ICD-9-CM-3	主数据 + 各业务模块	md_icd10 字典、outp_diagnosis 等码名冗余、DRG/单病种前缀驱动
编码	医保贯标码	modules/insurance	yb_catalog_map 对照表 + CSV 导入 + 对照率
数据元	WS 363 / WS 364	modules/datagov	dg_data_element 登记、dg_term 术语映射（ICD10/LOINC/ATC 多体系）
文档	CDA / WS/T 500	datacenter/cdr	toCda() Level 1 骨架、/documents/{id}/cda
消息	HL7 V2 / MLLP	platform/integration	Hl7V2Message 解析、MllpServer 监听、int_message_log 留痕
接口	医保接口规范	platform/integration + insurance	InsuranceAdapter SPI、ADR-0002 契约

案例框 3-4：一份血常规报告的三种标准形态

收束本章，追踪同一份检验报告在 HIP 里穿过的三层标准，体会各层标准回答的问题不同：

1. 消息形态（进院门）：检验仪器经 LIS 中间件把结果推成 ORU^R01 报文，MLLP 帧进 MllpServer，OruProcessingService 从 OBR-2 取申请单号、逐条 OBX 取结果（项目码^名称、值、单位、参考范围、异常标志），发布事件驱动医嘱执行与危急值告警（第 6 章）。此刻标准回答的是「两个系统怎么实时对话」；
2. 数据形态（进数据中心）：CDR 同步任务把已执行的检验医嘱聚合成 LAB_REPORT 类型的患者文档——结构化 JSON 存 results 行（item_name/result_value/unit/ref_range/abnormal_flag），患者维度可检索（第 11 章）。标准（数据元与值域的口径）回答的是「数据项怎么统一定义以便聚合」；
3. 文档形态（出院门）：区域平台来取时，toCda() 把这份文档包成 CDA XML——头部是标准化的患者与文档标识，体部装载报告内容。标准回答的是「一份完整记录怎么跨机构交换与存证」。

同一份数据，三层标准各司其职、在系统里各有载体——这就是「标准体系」四个字的工程含义：不是一摞要应付的文件，而是系统架构里一组各就各位的构件。

本章小结

- 医疗信息标准按粒度分四层：分类编码（概念的唯一标识）→ 数据元与值域（数据项的定义与取值）→ 文档（成套记录的组织）→ 消息（系统间交换），上层引用下层。
- ICD-10 是字母开头的层级编码（类目 3 位→亚目 4 位→国临版扩展至 6 位），手术操作配 ICD-9-CM-3；层级性使前缀匹配可用于 DRG 入组与单病种筛选——编码是可计算的数据资产。药品、检验、收费各有编码体系（ATC/LOINC/价格规范），系统以院内码为主键、标准码为映射。
- 数据元三要素：标识（DE 编号）、定义、表示（类型 + 格式 + 值域）；WS/T 303 定方法、WS 363 给目录、WS 364 配值域。值域代码是全国统一枚举，须与院内业务字典分治。
- CDA 文档四层结构（头/体/节/条目），text 给人读、entry 给机器读，结构化程度分三级；WS/T 500 是中国化的共享文档规范，互联互通测评是其现实驱动。文档与消息的区别在持久性、完整性与法律属性。
- HL7 V2 是管道分隔的事件驱动报文（事件—消息—ACK 三元组），MLLP 承载；宽松性导致「方言」丛生，对接重心在逐家联调。FHIR R4 以资源模型 + RESTful 重塑互操作，编码体系引用显式化；两代标准将长期分层共存，选型看对端形态。
- 医保 15 项贯标编码统一了全国目录口径，接口规范呈「国家统一规范 + 省级实施包」双层结构，医院端以适配器收敛省差异。
- 标准落地内建优于事后改造：编码字段与行级粒度必须建模期内建，文档/接口可骨架先行，映射内容渐进补齐。最贵的返工永远是「当初没存的数据」。

思考与练习

1. 概念：用四层全景图定位以下标准各属哪一层，并说明相邻层如何引用：LOINC、WS 364、ORU^R01、WS/T 500 出院小结、医保药品编码。
2. 编码：患者诊断文本为「左下肺大叶性肺炎」。写出你查 ICD-10 的思路（主导词选什么、索引与类目表如何配合），并说明若病案员简化编为「肺炎，未特指」（J18.9），会对 DRG 入组与疾病谱统计造成什么影响。
3. 设计：参照案例框 3-2，把 WS 363 中「就诊日期时间」类数据元登记为 `dg_data_element` 的一行（自拟合理的 `code/datatype/format/std_ref`），并说明：若要让门诊挂号表的 `visit_date` 字段受这条数据元约束，系统需要增加什么机制？
4. 辨析：某三甲医院要向区域平台提供检验报告。平台提供两种接入方式：按 WS/T 500 上传共享文档，或开放 FHIR `Observation` 查询接口。从数据粒度、实时性、签名存证、改造成本四个维度比较两种方式，给出你的选择与理由。
5. 选型：以下三个对接场景，各选 V2 还是 FHIR？说明理由。（a）新购的血球分析仪回传结果；（b）互联网医院小程序拉取患者历史报告；（c）与邻院互认检验结果、需保留报告原始签发形态。
6. 排障：实施工程师反馈：某设备中间件发来的 ORU 报文全部被 HIP 应答 `MSA|AE|缺少申请单号 (OBR-2)`，但中间件厂商坚称「单号明明放了」。结合 3.4 的报文文法与第 6 章的 MSH 索引陷阱，列出至少三种可能原因与验证方法。
7. 论述：3.7.1 说「内建的难点是建模的人得知道标准」。假设你是一家 HIS 公司的架构师，设计一套机制（评审清单、字典基线、培训、工具皆可）保证新模块建模时标准不被遗漏，并讨论该机制在「快速交付压力」下如何存活。

8. 实践（配套实训环境）：把 HIP 的 `toCda()` 从 Level 1 升级一步：为诊断节生成 `entry` 条目（`<entry><observation>` 内含 ICD 编码，编码体系 OID 自查 ICD-10 的注册值），编码转换查 `dg_term` 表。写出改动的方法、新增的 SQL 与一个验证用例（对比升级前后同一文档的 XML）。

第 4 章 总体架构与技术选型

本章属第二篇「架构篇」。医院信息系统的架构选择，决定了此后数年里每一次上线、每一次升级、每一次半夜排障的成本。本章从行业三十年的架构演进讲起，聚焦一个贯穿性的命题——中小医院没有专职 **DevOps** 团队，这一约束应当如何塑造全部架构决策；然后依次展开五层总体架构、技术栈选型、数据库架构、工程结构与前端架构，最后介绍架构决策记录（ADR）的写作方法。全章以 HIP 平台的真实仓库为案例，包括两处「规划与落地不一致」的诚实剖析。

学习目标

学完本章，你应当能够：

1. 描述医院信息系统从 C/S 单体、SOA/ESB 到微服务与中台的演进脉络，解释每个时代的驱动力与遗留问题，理解「钟摆回摆」现象；
2. 列出微服务架构的隐性成本清单，并针对「县级医院 2-4 台服务器、无专职运维」的场景完成一次模块化单体 vs 微服务的论证；
3. 画出医院信息平台的五层总体架构图，说明每层的职责与典型构件；
4. 掌握技术栈选型的评估维度（生态、人才、生命周期、离线部署、信创路径），解释 HIP 选择 Java + Spring Boot / Vue 3 / PostgreSQL 的理由与备选否决过程；
5. 说出数据库命名空间隔离的谱系（分库—多 schema—表前缀），解释「单一主库、事务一致性优先」的取舍及其保留的拆分口；
6. 理解 Monorepo 工程结构下模块边界的划分依据（业务域内聚、表前缀归属、依赖方向单向），能对一个跨域控制器做出「下沉还是留聚合层」的裁决；
7. 按「背景—决策—备选与否决—影响」结构写出一篇合格的 ADR，判断何时值得写。

4.1 医院信息系统架构演进

4.1.1 三个时代与一次回摆

医院信息系统的架构史，大体经历了三个时代。

C/S 单体时代（约 1990 年代—2000 年代中期）。以收费电算化为起点的第一代 HIS（Hospital Information System，医院信息系统），普遍采用两层 C/S 架构：PowerBuilder、Delphi 等工具开发的胖客户端，直连 Oracle 或 SQL Server 数据库，业务逻辑大量写在存储过程里。它的历史功绩是把医院从手工账带进了电算化；它的遗留问题至今仍在许多医院运行着——客户端逐台安装升级、业务逻辑与数据库强绑定难以重构，以及最深远的一条：LIS、PACS、EMR 各自采购自不同厂商，形成一座座信息烟囱，系统间靠视图开放、中间表对拷勉强互通。

SOA/ESB 时代（约 2000 年代中期—2010 年代中期）。烟囱之痛催生了「集成平台」：以 SOA（Service-Oriented Architecture，面向服务架构）思想为指导、ESB（Enterprise Service Bus，企业服务总线）为载体，把各系统的能力包装成服务挂到总线上，统一注册、路由、编排。互联互通标准化成熟度测评（第 1 章）的技术要求正是这一时代的制度化成果——文档注册、服务注册、统一患者主索引，都以「有一个平台居中」为前提。这个时代解决了「连起来」的问题，但商用 ESB 价格高昂，且总线本身成为新的单点与新的供应商锁定：换 ESB 厂商的代价不亚于换 HIS。

微服务与中台时代（约 2010 年代中期至今）。互联网医疗的兴起把互联网行业的技术栈带进了医院：容器、微服务（Microservices）、注册中心、消息队列，以及随后的「中台」叙事。大型三甲医院与区域平台确有匹配的场景——高并发的互联网入口、数百人的开发团队、独立机房与专职运维。但这套架构被不加裁剪地推向中小医院时，问题出现了：一套「微服务版 HIS」动辄几十个服务、十几个中间件，县级医院的机房与人力根本养不起。

三个时代的得失可以并置对比：

时代	驱动力	解决了什么	遗留了什么
C/S 单体	收费电算化	从手工账到电算化	胖客户端运维、存储过程绑定、信息烟囱
SOA/ESB	互联互通、消除烟囱	系统间标准化集成、统一主索引	总线单点、商用 ESB 成本与锁定
微服务/中台	互联网医疗、大型组织并行开发	独立部署、弹性伸缩、故障隔离	分布式复杂度下沉到不具备承接能力的医院

于是出现了值得注意的钟摆回摆：近年软件行业重新为「模块化单体」（Modular Monolith）正名——在单一部署单元内部保持严格的模块边界，模块间只通过接口与事件通信，需要时再把成熟模块拆出为服务。回摆不是否定微服务，而是承认一个朴素结论：微服务解决的是组织规模问题（几百人如何并行开发部署），不是代码质量问题；组织没有那个规模，就不该支付那份成本。

4.1.2 康威定律：架构是组织的镜像

康威定律（Conway's Law）指出：系统的结构必然映射设计它的组织的沟通结构。它在医院信息化里有两层读法：

- 供方视角：几个人的开发团队维护几十个独立部署的服务，意味着每个人同时对抗十几套配置、日志与发布流程——架构粒度超过了组织粒度，代价是持续的摩擦。反之，几百人共享一个单体仓库又会在合并与发布上互相踩踏。粒度要和团队对齐。
- 需方视角：医院信息科通常只有数名工程师，且以运维与协调为主业。交付给医院的系统，其运维复杂度必须匹配信息科的组织能力——这一条比任何技术偏好都更硬，是下一节的出发点。

4.2 关键抉择：模块化单体 vs 微服务

4.2.1 微服务的隐性成本清单

论证之前，先把微服务的成本摊开。教科书常讲它的收益（独立部署、故障隔离、技术异构），成本却往往用一句「复杂度上升」带过。展开来至少有六项：

1. 分布式事务：收费扣库存这类跨模块强一致操作，从一条本地事务变成 Saga/TCC 补偿编排，正确性论证难度上一个数量级；
2. 网络不可靠：模块间调用从进程内方法变成 RPC，超时、重试、幂等、熔断成为每个调用点的必答题；
3. 可观测性：排障需要分布式链路追踪、集中日志、指标聚合——一套 Prometheus + 日志栈本身就是几台服务器和一个专职岗位；

4. 部署编排：几十个服务需要 Kubernetes 级别的编排，而 K8s 集群的搭建与故障处理是一门独立的手艺；
5. 版本矩阵：服务间接口兼容性随服务数量组合爆炸，需要契约测试与灰度机制护航；
6. 人：以上每一项都要有人值守。这是最贵的一项，也是中小医院最缺的一项。

4.2.2 约束驱动设计：把「没有 DevOps」写进每个决策

HIP 的目标客户是县级、地市二级医院：机房 2 - 4 台服务器，内网部署常年离线，信息科没有专职 DevOps 工程师，未来可能有信创要求。架构设计的正确起点不是「业界最佳实践」，而是这份约束清单。把约束逐条向下传导，会发现它几乎决定了所有关键选择：

约束	传导出的架构决策	反面（被否决的方案）
无专职 DevOps，故障靠电话远程指导	模块化单体：一个 jar 进程，重启=一条命令	几十个服务的启动顺序与健康排查
2 - 4 台服务器，无独立中间件资源	单一 PostgreSQL 主库，缓存仅 Redis	注册中心/配置中心/消息集群各占机器
跨模块通信要可靠但量不大	进程内 Spring 事件 + 数据库留痕，不上消息中间件	RabbitMQ/Kafka 集群的运维与监控
升级由实施工程师远程完成	Flyway 自动前滚：换 jar 重启即完成 DDL 升级	手工执行 SQL 脚本的漏执行风险
部署环境异构（物理机/虚拟机）	Docker Compose 单文件编排，或直接 systemd 跑 jar	K8s 集群
会话不能依赖额外组件	无状态 JWT 认证，任一实例可服务任一请求	依赖 Redis/粘性会话的服务端 session
未来信创可能	PostgreSQL 及其兼容生态（4.4 节）	深度绑定单一商业数据库特性

这张表是本章的灵魂：每一行都是同一个约束的不同侧影。运维面收敛到「一个 jar + 一个 PostgreSQL + 一个 Redis + 一个 Nginx」，意味着实施工程师可以在电话里指导医院网管完成 90% 的操作——这不是技术保守，而是把「谁来运维」当作第一设计输入。第 13 章的备份任务化、第 14 章的升级演练，都是这条主线的延续。

案例框 4-1：ADR-0001 的否决记录

HIP 的第一篇架构决策记录（ADR-0001 技术栈与架构风格）用不到一页纸固定了上述选择。最有教学价值的是它的「备选与否决理由」一节，原文只有三行：

- .NET 8：工程质量同级，但国内 HIS 生态、医保/设备 SDK、招聘池均弱于 Java 一档，仅在团队 C# 明显更强时才值得；
- Python/Node 做主业务：强事务大规模代码库的长期维护成本高，医院内网离线部署不友好，否决；
- 微服务起步：县级医院 2 - 4 台服务器、无 DevOps 团队，分布式复杂度不可接受，否决。

注意否决微服务的理由里没有出现任何技术词汇——只有服务器台数和人。同时决策为拆分留了口：「模块间仅通过接口与事件通信，保留微服务拆分口」，并在影响一节要求第 1 - 2 迭代内定义模块间通信规范、防止模块间直接引用对方 Repository。留口不是空话，它落成了 4.6 节可以验证的依赖规则。

4.2.3 单体不是借口：边界纪律

模块化单体最大的风险是退化为「泥球单体」——反正在一个进程里，谁都能 import 谁。防退化靠的不是意愿而是纪律与机制：模块以独立 Maven module 存在、依赖在 pom 中显式声明且单向（4.6 节）、跨模块协作走接口（SPI）与事件（如检验结果回传事件 `LabResultReceivedEvent`，见第 6 章）、

数据表按前缀归属模块（4.5 节）。一个可检验的判据是：当某个模块需要整体迁出时，成本是否可控——HIP 曾将医保局案件查办系统（bureau）连同底座依赖从仓库整体迁出为独立产品（ADR-0003 医保局案件查办系统后记），迁出只涉及删目录与还原根 pom，这就是边界纪律的兑现。

4.3 五层总体架构

4.3.1 分层的意义

分层架构回答的是「变化隔离」问题：接入形态（Web/H5/自助机）变化最快，基础设施最慢，业务居中。把它们放进不同的层，每层只依赖下层的抽象，一层的替换不波及全局。医院场景里分层还有一个特殊价值：采购与评审的沟通语言——互联互通测评、等保测评、采购需求书都按类似的层次组织条款，架构图与制度文本同构，逐条对应就不费翻译。

HIP 的总体架构分五层（图 4-1，依据总体规划并按落地现状标注）：



图 4-1 HIP 五层总体架构

4.3.2 各层要点

- 接入层：所有入口共享同一后端 API，但会话隔离——患者端使用独立的 PORTAL 令牌，与院内接口完全隔离，防止患者态凭证触达院内业务面（第 13 章脱敏与权限将再次讨论）。
- 应用层：按业务域组织的功能页（门急诊 18 页、住院 8 页、数据中心 11 页等，全景见第 1 章），域的划分同时是后端模块边界（4.6 节）与功能开关粒度（第 14 章 ModuleGate）。
- 平台层：业务模块共用的横切能力。注意 AI 子服务的位置——它以独立 Python FastAPI 进程经 REST 接入，主平台对其优雅降级：AI 不可用时业务照常。把最不成熟、依赖最重的能力放到进程边界之外，是单体架构里控制风险的常用手法。
- 数据层：业务库与 CDR 分离（第 11 章），EMPI 与主数据先行建设且保持稳定（第 5 章「底座先行」原则）。
- 基础设施层：开发与小型部署用 `deploy/docker-compose.yml`（postgres + redis 两个容器），完整部署用 `docker-compose.full.yml` 增加应用与前端；规划中「Docker Compose → K8s」的演进箭头指向区域化场景，单院不启用。

规划文档中的五层图还列有表单引擎、工作流、ODR 运营数据中心等构件，属于规划期的完整蓝图；落地版本以「够用」为度收敛（如报表以 SQL 白名单自定义报表引擎实现，见第 12 章）。架构图应当区分「已建」与「规划」，评审时逐项对账——这也是把规划文档与仓库现状对照阅读的意义。

4.4 技术栈选型方法

4.4.1 选型的评估维度

技术选型容易滑向个人偏好之争。把它变成工程问题的办法是先定维度、后比方案。对医院信息系统，实践中最有区分度的五个维度是：

1. 行业生态：医保 SDK、检验设备接口包、CA 厂商 SDK 提供什么语言的绑定？国内医疗行业事实上以 Java 生态为中心，很多省医保 SDK 只有 Java/C 版本；
2. 人才池：县市级项目的驻场与维护人员从本地招聘，主流技术栈的招聘半径完全不同；
3. 生命周期：医院系统一用十年起步，框架要有长期支持（LTS）版本与清晰的升级路径；
4. 离线部署友好度：院内网常年无外网，依赖管理必须支持完全离线（私服镜像、锁定版本）；
5. 信创路径：未来被要求替换为国产化组件时，迁移成本是否可控。

用一票否决的思路过一遍这五个维度，候选集会迅速收敛——这比「哪个框架性能高 5%」的争论有效得多。选型还有一条元原则：优先选无聊的、被大规模验证过的技术，把创新预算留给业务本身。

4.4.2 HIP 的选型与理由

HIP 的实际技术栈（以仓库 `pom.xml` 与 `README` 为准）：

层	选型	核心理由	备选与信创路径
后端	Java 21 + Spring Boot 3.5, Maven 多模块	国内 HIS 生态主流、医保/设备 SDK 齐全、人才好招; LTS 版本	.NET 8 被否决 (生态弱一档); Python/Node 不做主业务 (强事务大代码库维护成本)
前端	Vue 3 + TypeScript + Element Plus + Vite	国内企业级中后台事实标准, 组件覆盖医疗表单/表格密集场景	—
数据库	PostgreSQL 16	开源无授权费、事务与 SQL 能力完整、JSON 与全文检索内建	信创场景平移达梦/人大金仓 (均提供 PostgreSQL 兼容模式, 迁移成本相对可控)【成稿核对: 两库对 PG 兼容口径与目标版本】
缓存	Redis 7	缓存与轻量队列, 单实例即可	—
AI 子服务	Python FastAPI 独立进程	算法生态在 Python; 进程隔离保证故障不影响主业务	—
部署	Docker Compose; 前端 Nginx	单院一键起; 区域化再考虑 K8s	—
认证	无状态 JWT (jjwt) + RBAC	不依赖会话中间件	—
迁移	Flyway, JPA ddl-auto=validate	DDL 全部版本化, 禁止 Hibernate 自动建表	—

几点展开:

- 版本策略: Java 21 与 Spring Boot 3.5 都取 LTS/主流稳定线。规划初稿写的是 Java 17, 立项时 Java 21 LTS 已稳定即直接采用——选型文档与代码以代码为准, 文档随之更新, 这正是 ADR 要「保鲜」的原因 (4.8 节)。
- PostgreSQL 与信创: 选择 PostgreSQL 的一个隐含收益是国产数据库多数提供 PG 兼容模式, SQL 方言、驱动层面的迁移工作量远小于从商业数据库迁出。但「兼容」不等于「免测试」——存储过程、锁行为、执行计划都可能有差异, 信创迁移应当按一次完整回归对待 (HIP 的 19 套 E2E 在此就是迁移验收工具, 见第 13 章)。
- 离线部署的工程含义: 选型确定后, 还要为「院内网无外网」补一层工艺——后端依赖由 Maven 在构建机解析、交付物是自包含的可执行 jar; 前端依赖锁定在 package-lock.json、交付物是静态 dist 由 Nginx 托管; 数据库与缓存以镜像形式随 Compose 打包。交付物在有网环境构建、在无网环境运行, 院内永远不做依赖解析——这也是为什么解释型语言做主业务在此维度失分: 运行环境即依赖环境, 离线补包苦不堪言。
- 禁止 Hibernate 自动建表: ddl-auto=validate 意味着实体与表结构不一致时启动即失败, 把「代码与库漂移」压制在开发期。生产库的每一条 DDL 都必须走 Flyway 版本文件, 这是 4.5 节迁移纪律的技术抓手。

4.5 数据库架构: 单一主库与命名空间的谱系

4.5.1 隔离谱系: 从分库到表前缀

「模块的数据如何隔离」存在一条谱系, 隔离度递减、一致性与运维成本也递减:

方案	隔离度	跨模块事务	运维成本	适用
每模块独立数据库实例	最高	分布式事务	每库都要备份/监控/调优	微服务体系
单实例、每模块一个 schema	中	同库事务可用	单库运维；权限/迁移按 schema 管理	单体但预留拆库
单实例单 schema、表名前缀	逻辑约定	完全无障碍	最低	中小规模模块化单体

医院业务的特殊之处在于跨模块强一致场景密集：门诊收费同时要写结算单（门诊模块）、扣药房库存（主数据/药房）、写医保分割留痕（医保模块）、落集成留痕（集成模块）——这在单库里是一条本地事务，在分库体系里是一场 Saga 编排。「事务一致性优先」因此成为 HIP 数据层的第一取舍：单一 PostgreSQL 主库，分析型负载则分流到 CDR（第 11 章），不让报表拖垮 OLTP。

案例框 4-2：从「schema 隔离」规划到「表前缀」落地

HIP 的规划与 ADR-0001 均写明「单一主库 + 按模块 schema 隔离，保留拆库能力」；但查看仓库会发现，44 个 Flyway 迁移里没有一条 CREATE SCHEMA ——实际落地是单 schema + 表前缀：sys_（平台核心）、empi_、outp_（门诊）、inp_（住院）、yb_（医保）、int_（集成）、drg_ 等，前缀即模块归属；实施定制层另有 impl_ 前缀与独立版本段（ADR-0003 实施定制层规约）。

偏差的原因是务实的：Flyway 单一版本序列跨 schema 管理繁琐；跨模块查询（驾驶舱统计、患者 360 抽取）在单 schema 下最直接；而 schema 承诺的「拆库能力」，表前缀同样保留了一——前缀本身就是清晰的归属边界，真要拆时按前缀搬表即可。教学上的两点提醒：其一，隔离谱系上的位置可以移动，模块归属的约定不能没有——最怕的不是选了轻量方案，而是表名无规则、归属靠口传；其二，规划与落地出现偏差时，应回头修订 ADR 或补记偏差，让文档保持可信（4.8 节）。

4.5.2 迁移纪律

单库单序列的 Flyway 是这套数据架构的秩序来源，HIP 沉淀了四条纪律：

1. 历史迁移不改：已发布的 V1 - V44 版本文件永不修改，纠错一律新增版本（校验和机制也不允许改）；
2. 版本段分配：平台占 V1 - V999；实施定制层从 V10000 起，两段在同一库中互不冲突地前滚——升级标准产品时平台段自动前滚，实施段随实施仓库演进（第 14 章）；
3. 索引也是版本：38 个迁移后的一次系统性索引审计以新增迁移（V39）落库、只加不删，审计过程用 EXPLAIN ANALYZE 实测 E2E 高频查询（技术债交接单债二，方法细节见第 13 章性能工程）；
4. 种子数据分层：迁移中只保留平台必需数据，行业基线与演示数据剥离到初始化工具（第 14 章种子数据三分层）。

4.6 工程结构：Monorepo 与模块边界

4.6.1 Monorepo 的选择

HIP 采用单仓库（Monorepo）：后端全部模块、前端、部署脚本、E2E 工具、文档（含 ADR）同仓。对小团队，其收益是压倒性的——一次提交可以原子化地同时改后端接口、前端调用与 E2E 断言；依赖用 Maven module 内联而非发布私服构件；全库检索与统一 CI。多仓的收益（独立权限、独立发布节奏）在单产品小团队场景下用不上。当真正出现第二个部署主体时再拆仓——bureau 案件查办系统的迁出（4.2.3）就是在「部署主体、数据边界、发布节奏完全独立」三个条件同时成立后进行的。

根 pom.xml 聚合的模块清单（每行注释来自各模块 pom 的 <name>）：

分组	Maven 模块	职责
平台底座	platform/core	用户/组织/RBAC/JWT 认证/sys_config/审计
	platform/empi	患者主索引
	platform/masterdata	药品/收费/ICD 主数据与库存
	platform/integration	集成引擎：HL7 V2/MLLP、三类适配器 SPI、留痕
业务模块	modules/outpatient	门诊：挂号/医生站/收费/药房
	modules/inpatient	住院：入出转/医嘱/护理/结算
	modules/insurance	医保：目录对照/分割/审核/对账
	modules/medtech	医技：LIS/RIS/预约叫号
	modules/qualitycare	质控：病案/DRG/院感/不良事件
	modules/datagov	数据治理：数据元/填报/质量核查
	modules/hrp	运营：资产/物资/人事/OA
	modules/portal	互联网患者端（独立会话）
数据中心	datacenter/cdr	CDR 文档抽取与患者 360
启动器	server	聚合装配、Flyway、跨模块驾驶舱聚合

（对外口径「12 后端模块」与 pom 的 13 个非启动器模块存在计数口径差【成稿核对：portal 或 cdr 是否计入宣传口径，统一全书数字】。）

4.6.2 依赖关系：方向即架构

模块间依赖关系直接读自各模块 pom（图 4-2）：

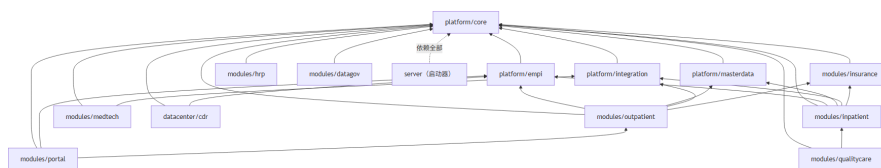


图 4-2 HIP 模块依赖关系（箭头指向被依赖方；server 依赖省略展开）这张图隐含三条规则，也是模块边界的三条划分依据：

1. 业务域内聚：模块按业务域切分，与前端页面分域、功能开关粒度、表前缀四者对齐——一个域的代码、页面、表、开关在同一边界内；
2. 依赖单向、底座居下：业务模块可依赖平台底座，底座绝不依赖业务模块；业务模块之间原则上互不依赖；
3. 横向依赖必须有裁决理由：图中三条业务模块间的边都有明确的业务因果——门诊/住院依赖 insurance 是因为结算必须在事务内调用费用分割引擎（第 10 章）；qualitycare 依赖 inpatient 是因为 DRG 入组读取住院病例；portal 依赖 outpatient 是因为患者端预约复用号源逻辑。没有说得出口的理由，就不允许连线。

4.6.3 边界从哪里来：一次真实的裁决

边界不是一次画对的。HIP 早期把跨模块的统计与聚合接口都放在 server 启动器里，随着功能增长，server 里积累了一批「究竟属于谁」悬而未决的控制器。清理时逐一裁决的记录保留在技术债交接单中，是模块边界方法论最生动的教材：

案例框 4-3：聚合控制器的下沉裁决

裁决的判据只有一条：看它读写哪些表（表前缀即域归属），真跨模块的留 server，单一域的下沉。摘录裁决表：

控制器	裁决	理由
StatsController	留 server	真跨模块驾驶舱聚合
DrgController	下沉 qualitycare	仅依赖 inp_* + drg_*，与病案/质控同域
InsuranceController	下沉新建 modules/insurance	医保域独立——对账虽跨门诊/住院，但表都是 yb_*
QueueCenterController	下沉 outpatient	三队列数据源均在门诊侧
PriceFinanceController	拆：价格→masterdata，交款核对 留 server	价格属主数据域

两点方法论：其一，InsuranceController 的裁决说明「业务上跨域」不等于「架构上跨域」——对账业务横跨门诊与住院，但它读写的全是 yb_* 表，数据归属才是裁决依据，medtech/insurance 等模块正是在这类裁决中被识别并独立出来的；其二，执行工艺是 `git mv` + 包名批量替换 + `pom` 依赖核对，每迁一个控制器跑一次全量测试，五项裁决全部落地时 48 个单元测试与 18 套 E2E 保持全绿、行为零变化。边界重构的安全感来自测试覆盖，不来自小心。

4.7 前端架构

4.7.1 工作台外壳与页面组织

医院前端的特殊性在于角色多、页面密、共享上下文：医生站、护士站、收费台、药房共享同一套登录、菜单与患者语境。业界两条路线：微前端（qiankun 等，各业务前端独立开发部署、外壳聚合）与单应用 + 路由懒加载。微前端解决的同样是组织规模问题——多个团队各管一摊时才值得支付其复杂度。HIP 规划期预留了微前端方向，落地为单一 **Vue** 应用：一个外壳（登录、布局、菜单），全部页面按业务域组织在 `views/` 下（图 4-3）：

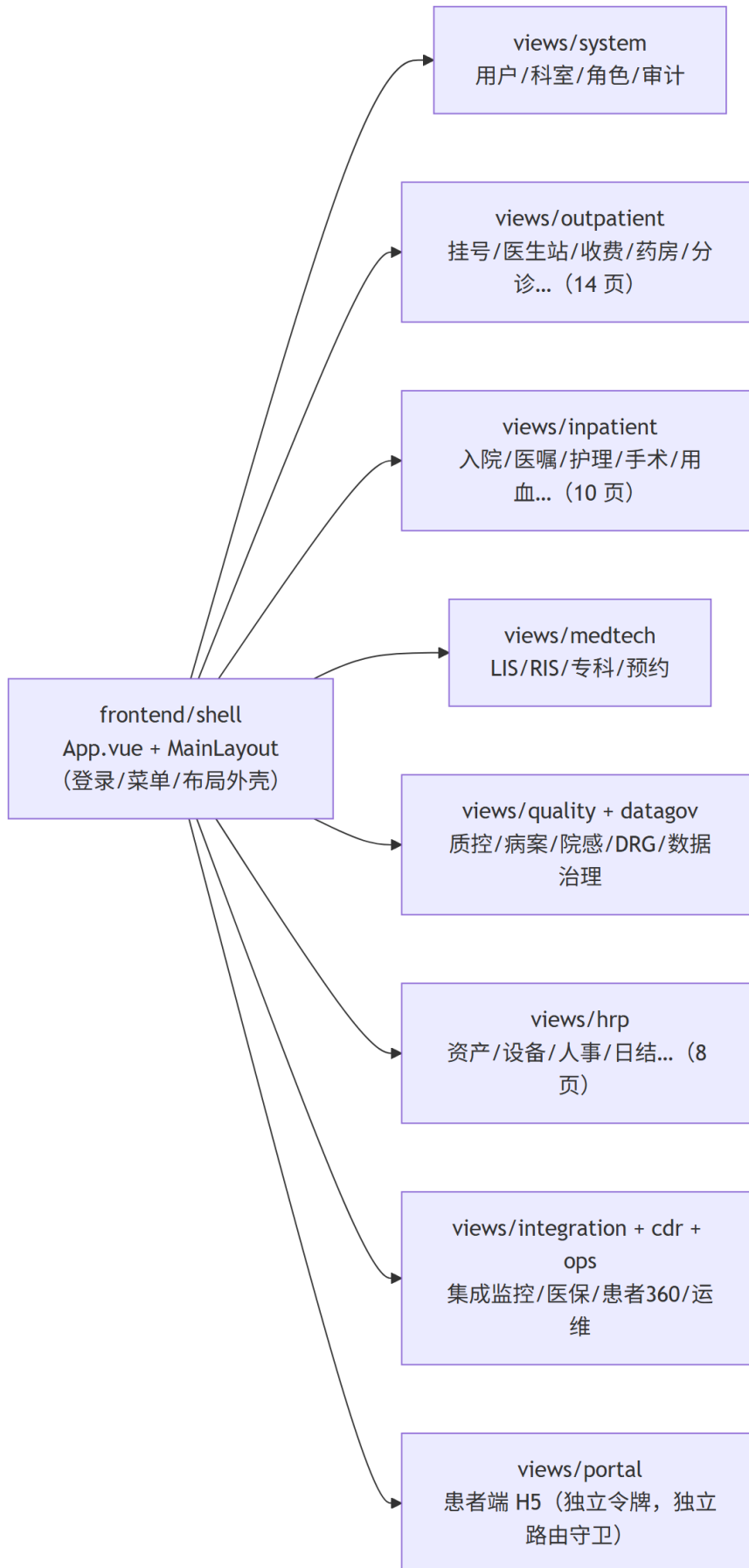


图 4-3 前端页面组织：单外壳 + 按域分目录（约 60 个视图）三个关键机制：

- 菜单由服务端下发：菜单树存于数据库（`sys_menu`），按角色授权过滤后下发，前端 `<component :is>` 动态解析图标。菜单即权限的可见性层，与后端权限点二道防线配合（第 5 章）；也因此模块开关只需在服务端摘掉菜单并让对应 API 返回 404，前端无需改造（第 14 章 `ModuleGate`）。
- 路由级按需加载：`router/index.ts` 中每个路由都是 `() => import(...)` 动态导入，构建时每页一个 `chunk`，首屏只加载外壳与当班角色用到的页面——对「55 功能页但单个用户只用其中几页」的形态，这是免费的性能红利。
- 患者端隔离：`/portal` 路由使用独立令牌与独立守卫，与院内会话完全隔离。

4.7.2 包体积治理：一次真实的摇树排障

案例框 4-4：element-plus 按需引入与 `manualChunks` 陷阱

HIP 前端早期全量引入 Element Plus，该依赖独立 `chunk` 达 1.07MB（gzip 339KB）。治理记录在技术债交接单债三，改造分三步：

1. 引入 `unplugin-vue-components + ElementPlusResolver`，模板中的组件自动按需解析并带样式；
2. 编程式调用的 `ElMessage / ElMessageBox` 不出现在模板里，解析器摸不到，样式须手动引入（`main.ts` 中两行 `style/css import`）——这是按需方案最常见的踩坑点；
3. 最曲折的一步：构建配置原本用 `manualChunks` 把 `element-plus` 归并为单独 `chunk`，结果按需引入后体积纹丝不动。根因写在 `vite.config.ts` 的注释里：`element-plus` 的 `es/index.mjs` 是 `export *` 重导出枢纽，被强制归并单 `chunk` 时跨 `chunk` 重导出会令 `rollup` 放弃摇树、保留全部组件导出（实测 91 vs 49 个组件）——必须把它交还给自动分 `chunk`。改后实际打包 49/91 组件、CSS 251KB。

另有一条反直觉的裁决：`@element-plus/icons-vue` 的全量注册必须保留——菜单图标以字符串存于数据库、运行时动态解析，构建器无法静态分析哪些图标被用到；裁剪的前提是先对库内图标做白名单。教训的一般形式：按需加载的边界由「构建期可静态分析」决定，凡是运行时动态决定的引用（编程式 API、数据库驱动的组件名），都要显式处理。遗留项（`echarts` 改 `echarts/core` 按需注册，约省 600KB）记录在交接单待议区——治理是有台账的持续工作，不是一次冲锋。

4.8 架构决策记录（ADR）方法论

4.8.1 为什么写、写什么

架构知识最大的敌人是蒸发：半年后没人记得「当初为什么不用消息队列」，新成员只能考古聊天记录。ADR（Architecture Decision Record，架构决策记录）用一页纸对抗蒸发，经典结构四段：

- 背景（Context）：决策时面对的约束与事实——这是最有长期价值的一段，因为约束变了决策才需要重议；
- 决策（Decision）：做了什么选择，陈述句，编号列点；
- 备选与否决理由：认真考虑过什么、为什么不选——防止后人「重新发明被否决的方案」；
- 影响（Consequences）：决策带来的义务与代价，包括负面影响。

写的时机有三个信号：决策难以逆转（技术栈、数据库、仓库结构）；决策约束他人（接口契约、版本段分配）；同一个问题被问了第二遍。反过来，能用一次重构撤销的选择不必写 ADR，写进代码注释即可。

4.8.2 三种类型：以 HIP 的真实 ADR 为例

HIP 仓库的三篇 ADR 恰好覆盖了三种典型形态：

选型型——ADR-0001 技术栈与架构风格。全文不足一页：背景四行（团队、客户、部署形态、信创可能），决策六条编号，否决三行，影响两条。示范了 ADR 的正确密度：只记结论与理由，不复述教程。它的「影响」一节值得注意——不是泛泛的展望，而是两条可执行的义务（模块以 `Maven module` 加入并由 `server` 聚合；1-2 迭代内定义模块通信规范），本章 4.6 节验证了它们确实被履行。

行为契约型——ADR-0002 医保适配器实现约定。SPI 的接口签名只有两个方法，但平台的对账、回滚、留痕正确性建立在实现方的行为之上：结算号必须回传、留痕必须带 `api` 关键字、失败必须返回 `ok=false` 而非抛异常、重试必须幂等。这些无法用 Java 类型系统表达的约定，正是最需要落成文字的部分（第 10 章从业务侧完整剖析了这份契约）。当一个模块的正确性建立在另一个模块的实现细节上，该细节就必须升格为书面契约——这类 ADR 的读者是未来的第三方实现团队，文末还附了可操作的替换步骤清单。

规约型——ADR-0003 实施定制层规约。为一整类未来行为立规矩：实施定制代码放哪个 `Maven` 模块、用哪个 `Flyway` 版本段（V10000+）、哪些事情被禁止（改平台表结构）。它约束的不是一次决策而是一条长期边界，是「配置优先、禁止分叉」产品化原则（第 14 章）的法律文本。

仓库里还保留着一篇编号冲突的 ADR——医保局案件查办系统与实施定制层规约同为 0003。这是同日高强度开发留下的真实瑕疵，也是一条朴素提醒：ADR 编号要唯一且只增不复用，新增前先看目录。而案件查办 ADR 的「后记」（系统迁出为独立仓库后补记迁出决定、保留原文）则示范了 ADR 的生命周期管理：决策被推翻或超越时，不删除、不篡改，追加状态与后记——历史决策的失效过程本身就是架构知识。

本章小结

- 医院信息系统架构经历 C/S 单体 → SOA/ESB → 微服务/中台三个时代，当前行业出现向模块化单体的钟摆回摆：微服务解决的是组织规模问题，组织没有那个规模就不该支付那份成本。康威定律的需方读法：系统运维复杂度必须匹配医院信息科的组织能力。
- 「中小医院无专职 DevOps」是 HIP 全部架构决策的第一约束，逐条传导为：模块化单体（一个 jar）、单一 PostgreSQL 主库、进程内事件不上消息中间件、Flyway 自动前滚、Docker Compose、无状态 JWT。运维面收敛到「一 jar 一库一缓存一反代」。
- 单体防退化靠纪律与机制：模块独立 `Maven module`、依赖显式且单向、跨模块走接口与事件、表前缀归属域；可检验判据是模块整体迁出的成本（bureau 迁出实证）。
- 五层架构（接入/应用/平台/数据/基础设施）是与测评、采购条款同构的沟通语言；AI 子服务放在进程边界外并可降级，是单体中控制不成熟依赖的手法。
- 技术选型五维度：行业生态、人才池、生命周期、离线部署、信创路径；优先无聊技术。PostgreSQL 的信创收益在于国产库的 PG 兼容模式，但兼容不等于免回归。
- 数据隔离谱系（分库—多 schema—表前缀）上的位置可以移动，模块归属约定不能没有。HIP 规划写 schema 隔离、落地为表前缀，偏差本身应回写文档。迁移四纪律：历史版本不改、版本段分配（平台 V1 - V999/实施 V10000+）、索引走版本、种子数据分层。
- 模块边界三依据：业务域内聚（代码/页面/表/开关四者对齐）、依赖单向底座居下、横向依赖须有裁决理由。裁决判据是数据归属（表前缀）而非业务话题；边界重构的安全感来自每步全量测试。
- 前端单外壳 + 服务端菜单下发 + 路由级懒加载；包体积治理的边界由「构建期可静态分析」决定，运行时动态引用（程式 API、数据库驱动图标）须显式处理。
- ADR 四段结构（背景/决策/备选否决/影响），三个写作信号（难逆转、约束他人、被问第二遍），三种形态（选型型/行为契约型/规约型）；决策失效时追加后记而非删改。

思考与练习

1. 概念：解释「钟摆回摆」现象。微服务的六项隐性成本中，哪几项在医院内网离线环境下会被进一步放大？为什么？
2. 论证：某市三甲医院（日门诊 8000 人次，信息科 15 人含 3 名开发）计划重建 HIS，供应商 A 报模块化单体方案、供应商 B 报微服务方案。请分别为两家医院——该三甲医院与一家县级二乙医院（信息科 2 人）——写出你的选型建议与论证，说明哪些约束条件的差异改变了结论。
3. 设计：4.5 节给出数据隔离谱系三方案。设某平台已按「单库 + 表前缀」运行三年，现要把医保模块（`yb_*` 表，约 20 张）拆为独立服务与独立库。列出拆分要处理的问题清单（提示：门诊收费事务中同步调用分割引擎、对账依赖 `int_message_log`），并说明哪些当年的设计让拆分变容易或变困难。
4. 分析：图 4-2 中 `qualitycare` 依赖 `inpatient`。有同事提议反转依赖——由 `inpatient` 在出院时调用 `qualitycare` 的入组接口。对比两种依赖方向对模块边界、可关闭性（医院未购 DRG 模块时）与事件化改造的影响，给出你的裁决。
5. 排障：某前端项目按案例框 4-4 的方法改造按需引入后，构建产物体积不降反增，且线上偶发某弹窗组件无样式。列出最可能的两个原因与验证步骤。
6. 论述：「规划文档写 `schema` 隔离、代码落地表前缀」这类偏差在长周期项目中几乎必然发生。论述团队应当建立怎样的机制（文档、评审、工具）让偏差被发现、被裁决（接受并回写 or 纠正代码）、被记录，而不是被遗忘。
7. 实践（配套实训环境）：为「是否引入 Redis 作为叫号队列的发布订阅通道（替代当前轮询）」写一篇完整 ADR：调研 HIP 叫号体系现状（第 7 章），给出背景、决策、至少两个被否决备选（含 PostgreSQL LISTEN/NOTIFY）及否决理由、影响清单；按 4.8 节的密度要求控制在一页以内。

第 5 章 平台底座

本章属第二篇「架构篇」。任何业务模块——挂号、医嘱、检验、医保——都站在一组公共能力之上：用户是谁、能做什么、科室长什么样、患者是哪一个、参数从哪读、操作留没留痕、数据库怎么演进。这组公共能力合称平台底座。本章先论证“底座先行”，再依次展开认证授权、组织主数据、字典术语、患者主索引、系统参数、审计日志与迁移纪律，以 HIP 的 `platform/core` 与 `platform/empi` 为贯穿案例。

学习目标

学完本章，你应当能够：

1. 用“依赖方向”和“变更成本”论证为什么主数据与权限体系必须最先建设且保持稳定；
2. 比较 Session 与 JWT 两种会话形态的取舍，说出无状态令牌“不可吊销”的代价与缓解手段；
3. 画出 RBAC 的用户—角色—菜单授权模型，解释“菜单即授权粒度”的适用边界与防自锁设计；
4. 设计一个防爆破的登录流程（失败计数、锁定、口令策略），并说明各环节对应的安全要求；
5. 说出 EMPI 判重的算法谱系，解释“证件号确定性匹配+不自动合并”为什么是保守而正确的起点；
6. 列举数据脱敏的四个实现层次，并按威胁模型为给定场景选择合适的层次；
7. 独立设计一张系统参数表：键的命名分类、值的类型约定、生效机制与新增键的纪律；
8. 复述 Flyway 版本段规约（V1 - V999 平台段 / V10000+ 实施段）与“历史迁移不改”原则，说明违反它在多医院部署下的后果。

5.1 底座先行原则

5.1.1 什么是平台底座

把一个医院信息平台的表倒出来看，会发现两类截然不同的表：一类是业务表——挂号记录、医嘱、检验标本、结算单，它们随业务功能一批批出现；另一类表在第一天就存在，并且被后来的几乎每张业务表外键引用——用户、角色、菜单、科室、患者。后者加上读它们的服务（认证、授权、参数读取、审计），就是平台底座。

HIP 的住院登记表是现成的观察样本（V8__inpatient.sql）：`inp_admission` 一张表四个外键——`patient_id` 指向 `empi_patient`、`dept_id` 与 `ward_id` 指向 `sys_dept`、`doctor_id` 指向 `sys_user`。业务表的外键箭头几乎全部指向底座表，而底座表不反向依赖任何业务表——这个单向依赖方向，就是“底座”一词的严格定义。

5.1.2 工程论证：为什么先建、为什么求稳

“底座先行”不是习惯，而是由两条工程规律推出的结论：

第一，依赖方向决定建设顺序。被依赖者必须先于依赖者存在：挂号要写 `patient_id`，患者主索引就得先有。反过来做——先写业务、用临时字符串存“科室名”“医生名”，等底座建好再回头改外键——意味着对每张业务表做一次数据迁移和代码返工，成本随业务表数量线性增长。

第二，变更成本与依赖数成正比，所以底座必须求稳。改一张叶子业务表只影响自己；改患者标识模型（比如把患者主键从自增 `id` 换成全局 `GUID`）要动几十张引用表和全部关联代码。依赖越多的东西越改不起，因此越是被广泛依赖的模型，越要在早期把“接口”想清楚，并在此后保持克制。

HIP 的迁移历史是这条原则的实测数据：44 个 Flyway 迁移中，V1 建认证授权与组织机构（V1__init_core.sql），V2 建患者主索引（V2__empi_patient.sql）——底座占据最前两个版本号。此后 42 个迁移里，`sys_user` 只动过三次、全是加列（防爆破计数、锁定时间、密码修改时间），`empi_patient` 列结构未改一列。底座表被引用数百次，结构却近乎冻结——先行且稳定，后面的一切才盖得起来。

需要澄清的是：“先建且稳定”指模型接口稳定，不指内容一次建全。HIP 的菜单树从 V1 的 4 条菜单长到 55 个功能页，每个迁移都在 `insert` 新菜单；角色从 4 个长到 8 个（V36）。内容随版本自然生长，但“用户—角色—菜单三表+两张关联表”的模型骨架从 V1 至今一字未动。求稳的是骨架，不是血肉。

5.2 认证与授权

5.2.1 会话形态：Session 还是 JWT

用户登录后，服务端如何在后续请求中认出他？两条技术路线：

维度	Session（服务端会话）	JWT（JSON Web Token，无状态令牌）
状态存放	服务端（内存/Redis），客户端只持会话 id	全部编码在令牌里，服务端不存
水平扩展	需会话共享或粘性路由	天然无状态，任意实例可验
主动吊销	删会话即失效，即时	做不到——签出的令牌在过期前始终有效
客户端形态	依赖 Cookie，浏览器友好	Header 携带，Web/移动 H5/自助机/患者端通吃
失效控制	服务端说了算	只能靠过期时间兜底

医院场景客户端形态多（工作台、移动 H5、患者端、自助机）而部署规模小（单院单体）：多端统一是 JWT 的强项，其短板“不可吊销”用短有效期缓解。HIP 选择 JWT：`JwtService`（`JwtService.java`）用 HMAC 对称密钥签发，令牌主体（`subject`）只放用户名，有效期默认 12 小时（覆盖一个白班加交接，配置键 `hip.security.jwt-expiry-hours`）。密钥从环境变量 `HIP_JWT_SECRET` 注入，生产必设 32 位以上随机串——密钥外置是把密钥从代码与配置文件里赶出去，属等保整改中最常见的一条（第 13 章）。

令牌验证在 `JwtAuthenticationFilter`（`JwtAuthenticationFilter.java`）完成，其中有一个值得注意的设计：一套过滤器承载两类主体。员工令牌的 `subject` 是用户名，每次请求回库加载角色；患者端令牌的 `subject` 是 `portal:{patientId}` 前缀形态，不关联任何员工账号，只授予 `ROLE_PORTAL`。配套的授权规则（`SecurityConfig.java`）做了双向隔离：`/api/portal/**` 仅 `PORTAL` 角色可入，而其余全部院内接口要求 `isAuthenticated()` and `!hasRole('PORTAL')`——患者令牌打死进不了院内接口，员工令牌也不冒充患者。互联网端与院内端共用一个进程时，这条双向隔离是必须画出的硬边界。

完整的认证链路如图 5-1：

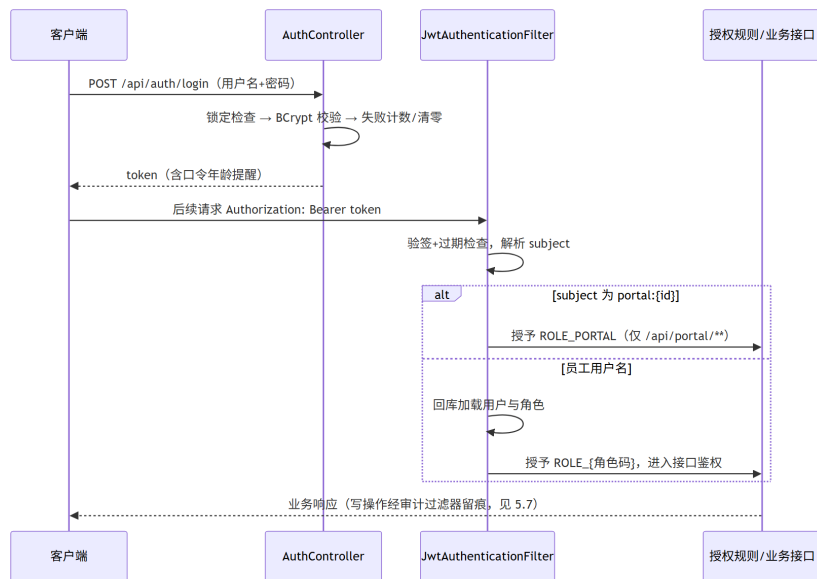


图 5-1 HIP 认证授权链路

5.2.2 RBAC 与菜单树授权

授权模型的谱系从简到繁：**ACL**（直接给用户挂权限清单，人多即失控）→ **RBAC**（Role-Based Access Control，基于角色的访问控制，NIST 模型中的 RBAC0）→ 带角色继承的 RBAC1 → **ABAC**（基于属性动态判定，如“仅主管医师可签出院”）。医院信息系统的主流形态是 RBAC0：岗位划分天然就是角色——医、护、药、技、收费、质控、运营、管理员，HIP 恰好是这 8 个（V36_phase38_role_matrix.sql）；ABAC 式细粒度判定通常落为业务代码里的显式校验（医嘱只能由开单医生撤销），不必进权限框架。

HIP 的 RBAC 落在 V1 的五张表上：`sys_user`、`sys_role`、`sys_menu` 加两张多对多关联 `sys_user_role`、`sys_role_menu`。授权的粒度是菜单：角色绑定一组菜单，登录后 `/api/auth/me` 按角色并集下发菜单树，前端据此渲染导航。菜单表的 `perm` 字段（如 `sys:user:list`）为按钮级权限预留了扩展位，但 HIP 当前止步于菜单级——粒度每细一档，授权管理成本翻一番，按菜单管权限是中小医院运维能力与安全需要的平衡点；要害操作（改配置、管用户、授权）另用方法级注解 `@PreAuthorize("hasRole('ADMIN')")` 二次把门，形成“菜单可见性 + 接口硬校验”双层。

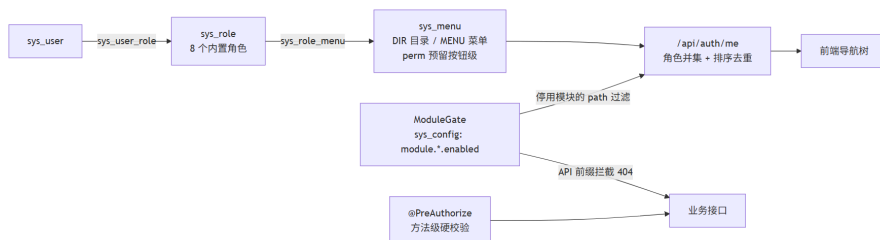


图 5-2 RBAC 菜单树授权模型（含模块开关的双路生效）

图 5-2 中的 `ModuleGate` 是授权链路上的一个旁路输入：医院未采购的模块经配置整体关闭时，菜单树先过滤掉对应路径，API 侧再由前缀拦截兜底返回 404——菜单过滤管“看不见”，API 拦截管“绕不过”，这套模块开关机制在第 14 章展开，其配置载体正是 5.6 节的 `sys_config`。

授权管理接口（`SysRoleController.java`）有两处防御性设计。其一，保存授权采用整表替换而非增量比对——语义简单、天然幂等，低频操作不值得做增量。其二，内置 **ADMIN** 角色的授权不可修改（错误码 1302），注释写得直白：“防止把自己锁在门外”——管理员误操作摘掉自己的系统管理菜单，就再也进不去授权页了；这类“自锁”是权限系统上线初期的高发事故，必须在机制上焊死，同类设计还有“不能停用内置管理员”（1104）。

菜单树还有一个易踩的细节：角色只绑叶子菜单时，父级目录（DIR）不一起绑定，前端树就渲染不出来。HIP 在两处处理——授权矩阵迁移末尾一条 SQL 为绑了子菜单的角色自动补父目录，/me 下发前再清理因模块过滤而空掉的父目录。一补一清对应“绑定时”与“下发时”两个时点，缺一处树就缺枝或长空枝。

5.2.3 密码策略与防爆破

口令是等保测评的重点检查项。HIP 的口令防线分四环，全部落在真实代码中：

1. 存储：BCrypt 单向散列（自适应成本、内置盐，彩虹表免疫），库中只有密文（`SysUser.password` 注释即“BCrypt 密文”）；
2. 强度：创建与改密时校验“至少 8 位且同时含字母与数字”（`SysUserController.java` 的 `passwordPolicyError`）。【成稿核对：等保三级对口令复杂度的具体要求（长度、字符种类数、更换周期）以现行测评口径为准，行文避免给出绝对数值】
3. 防爆破：连续失败 5 次锁定 15 分钟，计数持久化在 `sys_user.failed_attempts / locked_until` 两列（V17 加列），成功登录清零；
4. 时效：`password_updated_at` 记录最近改密时间，登录响应携带口令年龄，超过 90 天返回 `passwordExpireWarning` 提醒。注意这是提醒而非强制——强制到期改密会在早高峰的挂号窗口制造集中改密拥堵，HIP 把“是否强制”留给管理制度，系统提供数据与提示。

案例框 5-1：HIP 的登录防爆破实现

登录接口（`AuthController.java`）把锁定检查放在密码校验之前，失败计数写在 `BadCredentialsException` 分支里：

```
if (u.getLockedUntil() != null && u.getLockedUntil().isAfter(Instant.now())) {
    return R.fail(1002, "账号已锁定, 请 15 分钟后重试");
}
try {
    authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(...));
} catch (BadCredentialsException e) {
    // 防爆破: 连续失败 5 次锁定 15 分钟
    u.setFailedAttempts(u.getFailedAttempts() + 1);
    if (u.getFailedAttempts() >= MAX_FAILED) {
        u.setLockedUntil(Instant.now().plus(LOCK_DURATION));
        u.setFailedAttempts(0);
    }
    return R.fail(1001, "用户名或密码错误");
}
```

三个可琢磨的点：其一，密码错误统一提示“用户名或密码错误”，不区分“用户不存在”与“密码不对”——防用户名枚举的标准做法；但锁定提示（1002）只对存在的账号出现，仍留一丝可探测性——不可枚举与友好提示存在张力，HIP 取了后者。其二，计数持久化在数据库而非内存，重启不清零、多实例一致。其三，锁定期满后攻击者又有 5 次机会——等效于把爆破速率压到每 15 分钟 5 次：防爆破的目标不是绝对挡住，而是把暴力尝试的成本抬到不可行。

最后诚实交代 JWT 路线遗留的口子：停用账号不能即时踢出在线令牌——已签发令牌在有效期内不因停用而失效。对“离职即刻禁用”的诉求，纯无状态方案只能靠缩短有效期或引入服务端黑名单

单（那就不再纯无状态）逼近。系统设计没有免费午餐，重要的是知道自己选了什么、放弃了什么（习题3展开）。

5.3 组织机构与主数据

5.3.1 组织建模的通用难题

“科室”在医院里至少有四重身份：行政编制科室（人事口径）、门诊出诊科室（挂号口径）、病区/护理单元（床位口径）、成本中心（财务口径）。四个口径的树形结构并不重合——一个行政科室可能对应两个病区，一个病区可能混住两个科的患者。业界建模从重到轻：多套组织树各自维护并建映射（大型HRP的做法）、单树加维度标注、乃至一张平表。

HIP的选择是单树+类型标注：一张 `sys_dept` 表，`parent_id` 挂树，`type` 列标注 `CLINICAL` 临床 / `MEDTECH` 医技 / `ADMIN` 行政 / `NURSING` 护理单元 四类（`SysDept.java`）。病区不是独立实体，而是 `type='NURSING'` 的科室——床位表 `inp_bed.ward_id` 直接外键 `sys_dept`，入院记录同时记 `dept_id`（收治科室）与 `ward_id`（所在病区），两列都指向同一张表。这个“一表多角色”的设计对县市级医院是够用的：跨科混住可用 `dept_id ≠ ward_id` 表达；不够用的场景（多院区、复杂核算树）出现时，扩展方向是加维度列或映射表，而非推翻单表。

`code` 列（院内唯一）是一个专门留给集成与实施的锚点：对外接口按科室编码对照、初始化工具按编码幂等 `upsert`，都锚在 `code` 而不是数据库 `id` 上——自增 `id` 是库内私有的，编码才是跨系统的公共标识，这条规则同样适用于药品、收费项目与患者（5.5 的 `patient_no`）。

5.3.2 人员与床位

人员即 `sys_user`：`dept_id` 挂科室、`title` 记岗位。HIP没有把“账号”与“员工”拆成两张表——中小医院账号即员工档案的最小子集，拆表是过度设计；接了人事模块后人事档案另有其表，`sys_user` 保持“能登录的账号”语义。

床位 `inp_bed` 以 (`ward_id, bed_no`) 唯一，状态 `FREE/OCCUPIED` 加 `admission_id` 反指占用者。原子占床的并发控制是第8章的主题，底座层面只需记住：床位属于病区、病区是科室的一种、科室是一棵树——三层落在 V1+V8 两个迁移里，此后未再动过。

5.4 字典与术语管理

5.4.1 两层字典：院内运行与对外互操作

“字典”一词在医院信息化里混用了两个层次的东西，必须分开：

- 基础字典（院内运行用）：药品目录、收费项目、ICD 简表、科室、班次……业务流程直接引用它们的编码与属性，改一条价格立刻影响收费。
- 标准术语（对外互操作用）：WS 363 数据元、值域代码、医保贯标编码……院内编码与国家/行业标准编码之间的映射，供上报、互联互通测评、区域平台交换使用（第3章）。

前者是业务的一部分，后者是治理的一部分。HIP的基础字典三张主表建在 V4（`V4_masterdata.sql`）：`md_drug`（药品，含 `antibiotic` 抗菌药标记供分级管理用）、`md_charge_item`（收费项目，`category` 区分 `LAB/EXAM/TREAT`，`exec_dept_id` 挂执行科室）、`md_icd10`（诊断简表，带拼音码首字母检索）。一个值得讨论的取舍：`md_drug` 把 `price` 和 `stock` 直接长在字典上——严格的主数据管理（MDM）会说库存是事务数据不该进主数据表；但对单库存点的门诊药房，一列 `stock` 配上原子扣减（第7章）就是全部所需，独立的多库存点模型（V7 的 `inventory` 体系）是业务长出来之后的事。字典表长什么样，取决于消费它的业务，而不是教科书上的 MDM 分层。

术语映射落在 `dg_term` 表：`category` 分类 + `local_name` 院内名称 → `std_code / std_name / std_system` 标准编码三元组，配套 `dg_data_element` 承载 WS 363 数据元定义，维护接口在数据治理模块（`DataStdController.java`，唯一键防重，重复映射返回 4631）。注意本章只讲机制——表怎么设计、映射怎么存；映射率怎么管、质量规则怎么跑、上报怎么用，属数据治理的应用，归第 12 章。

5.4.2 实施可替换性：字典是“可整表换血”的

字典与其他底座表的最大区别是：结构属平台，内容属医院。示范库里的 10 条药品、13 条收费项目只是“最小可运行样例”，实施时必须换成院方的真实目录。HIP 的种子数据三类分层（种子数据分层清单.md，第 14 章展开）把这件事制度化：

- **A 平台必需**（菜单树、角色、配置键、数据元框架）随 Flyway 装机即有，实施不动；
- **B 行业基线**（药品/收费/ICD/医保对照等字典）迁移带默认值，实施用 `init-hospital` 工具的 CSV 模板按编码 `upsert` 整表覆盖——同码更新、新码插入、样例可停用；
- **C 演示数据**（测试患者、演示账号）只走脚本、永不进迁移，生产库禁止出现。

“按编码 `upsert`”呼应 5.3 的锚点原则：业务键是 `code` 而非 `id`，字典可反复导入而幂等，实施工程师拿院方 Excel 转 CSV 一遍遍试到对为止——可替换性不是一个功能，而是“业务键 + 幂等导入 + 分层清单”共同保障的性质。

5.5 EMPI 患者主索引

5.5.1 患者身份问题与判重算法谱系

EMPI（Enterprise Master Patient Index，患者主索引）回答一个看似简单的问题：眼前这个患者，是不是档案里的那个人？答错的两个方向代价悬殊：漏配（一人两档）造成病史割裂、重复检查；误合并（两人一档）会把别人的过敏史、血型安到患者头上——直接的医疗安全事故。行业共识因此是宁可重复建档，不可错误合并。

判重算法的谱系，按确定性递减排列：

1. 确定性匹配：证件号（身份证/医保卡号）完全相等即同一人。精度最高，覆盖不了无证件场景；
2. 加权概率匹配（Fellegi - Sunter 一族）：姓名、性别、出生日期、电话、地址各字段按区分度加权打分，超上阈值判同、低于下阈值判异、中间地带进人工复核队列。区域平台与大型 EMPI 产品的标配；
3. 人工合并：任何自动算法之外的最终裁决，合并操作本身要留痕可逆。

区域平台面对多机构、无统一证件的历史数据，必须上概率匹配；单家医院的新建平台，确定性匹配加保守兜底就是正确起点——概率匹配的价值要有足够体量的脏数据才能兑现，误合并的风险却从第一天就存在。

5.5.2 HIP 的建档幂等与身份证推算

HIP 的 EMPI 是独立平台模块（`platform/empi`），核心是建档接口的幂等语义：同证件号重复建档不报错、不重建，直接返回已有档案。

案例框 5-2：建档即判重——`register()` 的幂等设计

`PatientService.java` 的建档方法不到 25 行，浓缩了三个决策：

```

@Transactional
public Patient register(Patient p) {
    if (p.getIdNo() != null && !p.getIdNo().isBlank()) {
        var existing = patientRepository.findByIdTypeAndIdNo(
            p.getIdType() == null ? "ID_CARD" : p.getIdType(), p.getIdNo());
        if (existing.isPresent()) return existing.get(); // 幂等: 返回已有档案
        if ("ID_CARD".equals(p.getIdType()) && p.getIdNo().length() == 18) {
            if (p.getBirthDate() == null)
                p.setBirthDate(LocalDate.parse(p.getIdNo().substring(6, 14),
BASIC_ISO_DATE));
            if (p.getSex() == null || "U".equals(p.getSex())) {
                int seq = p.getIdNo().charAt(16) - '0';
                p.setSex(seq % 2 == 1 ? "M" : "F"); // 顺序码奇男偶女
            }
        }
    }
    Patient saved = patientRepository.save(p);
    saved.setPatientNo("P%08d".formatted(saved.getId())); // 先落库取 id, 再生成对外患者号
    return patientRepository.save(saved);
}

```

决策一：判重键是 (**id_type**, **id_no**) 复合键，不是裸证件号——身份证与医保卡号可能数字上撞车，证件类型必须进判重键。决策二：把“重复建档”做成幂等而不是报错。挂号窗口、自助机、患者端都会调建档，报错会把“已有档案”这个正常情况变成异常分支扔给每个调用方；返回已有档案则天然免疫重复提交。决策三：身份证号是自带结构的数据——18位号码的7-14位是出生日期、第17位顺序码奇数为男、偶数为女，建档时自动推算回填，把录入员少填两个框、也把“出生日期与证件号打架”这类脏数据挡在门口。推算只在字段为空时进行，录入值优先。

两个诚实的边界：代码未校验身份证第18位校验码（录错号仍会建档，可作改进项）；无证件患者（急诊无名氏、未带证件的儿童）直接新建档案，同名同生日不做自动合并——这正是5.5.1的保守共识：漏配可事后合并，误合并难以挽回。

患者号 **patient_no** (P+8位序号) 的生成用了“两段式保存”：先落库拿到自增id，再格式化回填。它是对外的公共标识（打印在就诊卡、报告单上），与库内id分离——重申5.3的锚点原则。完整判重流程见图5-3：

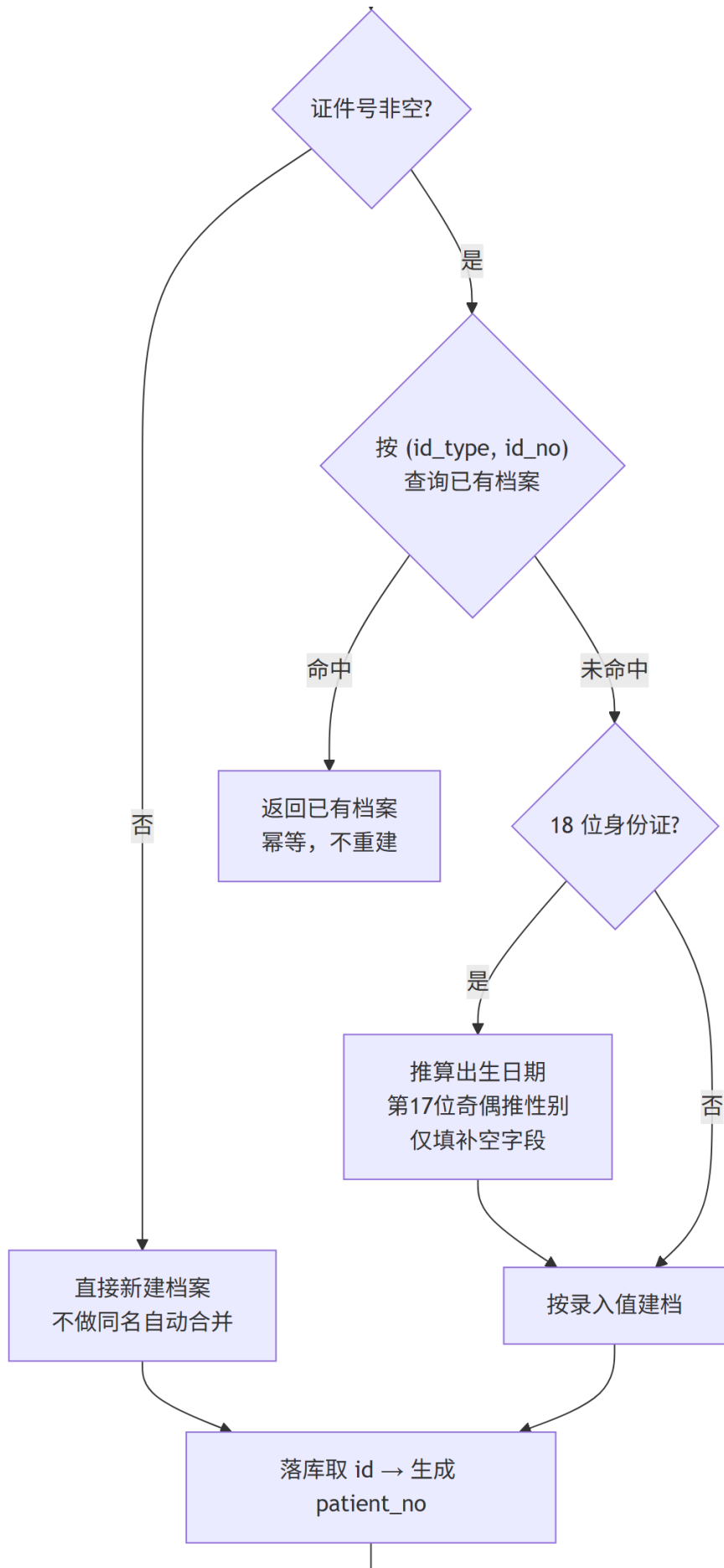


图 5-3 EMPI 建档判重流程

5.5.3 按角色脱敏的实现层次

患者的证件号、手机号属个人敏感信息，“谁能看到明文”是隐私合规的具体化。脱敏可做在四个层次，威胁模型各不相同：

层次	做法	防住谁	代价
① 存储层	落库即加密/静态脱敏	拖库者、DBA	检索困难，业务大改
② 数据访问层	数据库视图/行列权限	绕过应用直连库的人	与应用权限体系两套账
③ 服务出口层	接口出参按角色遮蔽	越权浏览的内部用户	每个出口都要覆盖
④ 展示层	前端渲染时打星号	只防肉眼——报文里仍是明文	约等于没脱敏

第④层是常见的假脱敏：接口返回明文、前端打星，按 F12 即原形毕露，测评不认可。HIP 选第③层——服务出口 **DTO** 层按角色脱敏，对准院内最现实的威胁：有登录权限的人员批量浏览患者联系方式。实现在 `PatientController.java`：列表出参时判断登录者权限，仅 **ADMIN** 返回明文，其余角色手机号保留前 3 后 4、证件号保留前 4 后 3，中段以 * 填充。两个口径细节：列表场景脱敏、经办场景按需——需核对证件的收费界面另行处理；遮蔽保头保尾是“可辨认不可复用”的平衡（能对患者确认“尾号 1234 对吗”，抄不走完整号码）。

按角色脱敏在 HIP 里不止这一处：报告分享短链走“匿名+脱敏+限时”、危急值大屏遮蔽患者全名，延伸场景与等保测评的对应在第 13 章集中讨论。底座层面立住方法论即可：先写清威胁模型，再选实现层次；层次选低了防不住人，选高了业务做不动。

5.6 系统参数中心：sys_config

医院之间的差异，绝大多数不是功能差异而是参数差异：院名、医保比例、审核阈值、买没买 DRG 模块。把这些差异从代码里抽出来集中管理的地方，就是系统参数中心。本节是全书引用最密的定义处——第 10 章的医保待遇键、第 14 章的模块开关与品牌配置，读写的都是这张表。

5.6.1 行业形态与两层配置

配置管理的演进路径：硬编码 → 配置文件 → 数据库参数表 → 独立配置中心（Nacos/Apollo 一类，多实例推送、灰度、回滚）。配置中心解决的是“成百上千实例的配置分发”；单院单体引入配置中心，是给自己添一个要高可用的新组件——数据库参数表才是这个规模下的正确答案（第 4 章“简单优先”）。

HIP 把配置切成两层（配置手册.md）：

- 部署配置：`application.yml` 与环境变量——JWT 密钥、数据库连接、MLLP 端口、外部服务地址。特征是起服务前定、改动要重启，且包含敏感值；
- 运行配置：`sys_config` 表——院名票据抬头、医保比例、审核阈值、模块开关。特征是管理页/接口随时改、即时生效。

切分判据一句话：密钥与资源位置属部署，业务参数属运行。

5.6.2 键设计：命名、类型与分类

`sys_config` 的表结构极简（V18_sys_config.sql）：`cfg_key` 主键、`cfg_value` 字符串、`remark` 注释、`updated_at`。简单表能管住 50+ 个键，靠的是三条约定：

1. 命名分域。键名用前缀表达归属：`hospital_*` 机构品牌、`yb_*` 医保待遇与审核（第 10 章）、`module.<key>.enabled` 模块开关（第 14 章）、`drg_rate / lis_allow_substitute` 业务参数、`cdr_sync_watermark` 系统内部键（任务维护，勿手改）。前缀即目录，`like 'module.%'` 一条 SQL 就能取走一族键；

- 值一律字符串，类型约定由读取方承担。`getInt` 解析失败回落默认值；布尔约定 `'1'` 为真。更重要的是语义约定进注释与手册：`0 = 不启用`——第10章分割引擎靠“起付线封顶线全0走老算法”实现新旧口径兼容，靠的正是这条约定；
- `remark` 必填。键的含义写在数据里而不是只在文档里，管理页展示注释，实施工程师不必对着裸键名猜。

配置手册与种子清单对键数的口径（24 键 / 40+ 键）随版本演进会有出入，成稿以数据库实际键集为准清点。【成稿核对：sys_config 全键数量与手册口径统一】

5.6.3 生效机制：直查不缓存

参数改了什么时候生效？这个问题在配置中心语境下要讨论推送、监听、失效——HIP 的答案只有一行：每次读取直查数据库，不缓存。

案例框 5-3: `ConfigReader`——最简单的生效机制

`ConfigReader.java` 全文不足 30 行：

```
/** sys_config 轻量读取（键不存在回落默认值）；单据前缀等低频配置直查不缓存 */
public String get(String key, String defaultValue) {
    var rows = jdbc.queryForList("select cfg_value from sys_config where cfg_key = ?",
    String.class, key);
    return rows.isEmpty() || rows.get(0) == null || rows.get(0).isBlank() ?
    defaultValue : rows.get(0);
}
```

直查的收益是把“生效机制”消灭掉了：`PUT /api/config/{key}` 落库的下一轮读取就是新值，没有缓存失效、没有通知风暴、没有“改了不生效重启试试”。代价是每次读一条主键点查 SQL——对结算时读一次比例、开单时读一次阈值这种频度，毫秒级点查完全无感。读取频度低是“直查”成立的前提；若未来出现每请求必读的热键，演进方向是加带 TTL 的进程内缓存，而那一刻必须回答“改键后最多多久生效”——秒级 TTL 通常就是可接受的答案，不必一步跳到推送。

典型消费方之一是 5.2 的 `ModuleGate` (`ModuleGate.java`)：like `'module.%.enabled'` 取回开关族，非 `'1'` 即停用，配置缺省视为启用——缺省即启用让新增模块无须回填旧库配置。菜单过滤与 API 404 双路读的是同一批键，改一个键、两路同时生效，正是“直查”带来的一致性红利。

5.6.4 修改途径与新增键纪律

运行配置的修改有三条正路：管理页、`PUT /api/config/{key}` (`SysConfigController.java`, ADMIN 专属、仅限已存在的键、写操作自动进审计)、`init-hospital` 批量初始化。接口层有一个必须知道的设计：`GET /api/config/public` 未登录可读且返回全部键值——登录页要展示院名，公开是必须的；“整表公开”反过来立下铁律：敏感值绝不允许进 `sys_config`（密钥走部署配置）。一张表的读权限模型，决定了它能装什么。

新增配置键的纪律（配置手册结尾原文）：“随迁移插入默认值 + `remark` 必填 + 本册同步补一行”。展开说：键随 `Flyway` 迁移插入（保证每个库都有默认值，代码可放心读）、注释落库、手册补行（实施逐键核对的依据）。第10章医保待遇模型升级时新增的 6 个键（V41）走的这条流程——定义处立规矩，引用处才敢依赖。

5.7 审计日志

5.7.1 审计的三问：记什么、怎么记、怎么用

网络安全等级保护对三级系统提出安全审计要求：审计覆盖到每个用户、对重要的用户行为和重要安全事件进行审计、审计记录应受保护并定期备份。【成稿核对：等保 2.0 三级安全审计条款原文及审计记录保留期限要求】落到工程上是三个问题：

- 记什么：全量记读写不现实（读请求量大且多数无审计价值），行业通行的底线是写操作全量 + 登录行为 + 敏感读按需。写操作（POST/PUT/DELETE）天然对应“改变了系统状态”，是责任追溯的主体；
- 怎么记：旁路记录，审计失败不得阻断业务——但失败要可观测（见下）；
- 怎么用：追溯（出事后查“谁在何时干了什么”）与稽核（定期抽查敏感操作）。查不动、没人看的审计等于没记。

5.7.2 HIP 的写操作全量审计

HIP 的审计写入是一个 Servlet 过滤器（AuditLogFilter.java）：`finally` 块里判断方法属 POST/PUT/DELETE 且路径以 `/api/` 开头，即向 `sys_audit_log` 落一行——用户名、方法、路径、HTTP 状态码、客户端 IP、时间（表建于 V17__phase13_16.sql）。几个设计点：

- 放在 `finally` 且捕获自身异常（“审计失败不影响业务”）：收费不能因审计表故障而中断，这是可用性对完整性的压制。但严格等保口径下有补课项：审计写失败应触发告警（第 13 章运维告警），静默吞掉会让“审计覆盖率 100%”成为无人知晓的假象；
- 登录天然在册：登录是 `POST /api/auth/login`，成功失败都留痕，锁定事件可由连续失败记录还原；
- 记状态码而非只记成功：4xx/5xx 同样入库——“谁在反复尝试越权接口”藏在失败记录里；
- 不记请求体：体积与隐私的双重考虑（请求体含患者敏感信息，审计表反成泄露面）。代价是“改了什么字段”不可考；需要字段级追溯的表应在业务层另建修改历史，而不是把审计表撑成数据湖。

查询侧支持敏感操作过滤（AuditQueryController.java，ADMIN 专属）：`sensitive=true` 时按路径 LIKE 清单筛出退费、角色授权、菜单变更、用户管理、抗菌药权限、医保目录维护、作废类操作——这份清单就是“重要用户行为”的工程定义。实现上选查询期过滤而非写入期打敏感标：清单变化无须回刷历史数据，代价是 LIKE 匹配慢——配合 `limit 200` 与 V39 的 `(username, id desc)` 索引，按人排查的主路径仍可走索引。

审计表遵守第 10 章已出现过的留痕铁律：只增不删。清理走归档与备份（每日备份任务，第 13 章），不走 delete。

5.8 数据库迁移纪律：Flyway 版本管理

5.8.1 为什么需要迁移工具

没有迁移工具的团队，数据库结构靠“上线前手工跑 SQL”维护，三个月后必然漂移：测试库与生产库不一致、新装库与老升级库不一致、没人说得清某列是谁加的。迁移工具（Flyway/Liquibase 一族）把结构变更收敛为带版本号的 SQL 文件序列：库里有一张版本记录表记录已执行到哪个版本，启动时把没跑过的迁移按序前滚，且对每个已执行文件校验校验和（`checksum`）——文件被改过则拒绝启动。数据库结构从此与代码一样有版本、可追溯、可重建。

由此推出迁移的第一纪律：只前滚，不修改历史。想改错，用一个新版本去改（改列宽出 V33__widen_appt_status、补列出 V40__yb_settle_no——第 10 章那个“结算号没地方放”的教训，最终就是以一个新迁移落地的），而不是回头编辑旧文件。

5.8.2 HIP 的版本组织与段规约

HIP 的迁移目录现有 V1 - V44，命名 `V<n>__<主题>.sql`（如 `V24__phase27_insurance`），版本号即开发史——V1 底座、V2 患者、V8 住院、V18 参数表、V40 医保结算号，一目了然。每个迁移是“建

表 + 种子 + 菜单注册”的完整单元：新功能的表结构、默认参数、菜单与角色绑定在同一文件落库，装机即得完整功能。

多医院产品化带来新的一维：实施定制（某医院独有的表）也要进迁移，但绝不能与平台迁移抢版本号。HIP 的段规约（ADR-0003，第 14 章）：

案例框 5-4：V1 - V999 平台段与 V10000+ 实施段

- **V1 - V999 平台段**：产品团队维护，随产品版本发布，所有医院一致；
- **V10000+ 实施段**：实施项目组维护，放在该院的 `impl_<hospital>` 模块内，只建 `impl_` 前缀的自有表，不改平台表。模板示例（V10000__impl_template_init.sql）：

```
-- 实施定制段示例迁移 (ADR-0003: V10000+ 专属段, 只建 impl_ 前缀自有表)
create table impl_template_note (
  id          bigserial primary key,
  content     varchar(500) not null,
  created_at  timestamptz not null default now()
);
```

中间近万个版本号的留白是刻意的隔离带：平台段还能再发布几百个版本而永不追上实施段，Flyway 按数值序执行时两段互不穿插。配上“只建 `impl_` 前缀表”的命名约束，平台升级（新平台迁移前滚）与实施定制（该院自有表）在同一个库里各行其道——版本号段 + 表名前缀，两条命名约定就实现了产品与定制的数据库级隔离，不需要任何额外机制。

5.8.3 历史迁移不改：多医院部署下的论证

“历史迁移不改”在单库开发时像洁癖，在多医院部署时是生死线。产品装到 N 家医院，各院升级节奏不同——甲院在 1.0（跑到 V40）、乙院刚装 1.1（一口气跑到 V44）。若 1.1 里修改了 V4：甲院升级时 Flyway 校验和不匹配、启动失败；即使强行对齐校验和，甲院的 V4 是旧版执行的、乙院是新版执行的——两家医院的“同版本数据库”从此不再相同，此后每一个假设“库结构一致”的迁移、每一次远程排障，都踩在流沙上。

所以纪律是绝对的：已发布的迁移一个字节都不改；种子数据分层（5.4.2）也因此不靠改迁移实现，而是“迁移照跑 + init 工具覆盖 B 类”的叠加路径。发布工艺侧的配套——升级即“停服→备份→换包→Flyway 自动前滚”、N-1→N 升级演练进 E2E——在第 14 章闭环。

回望本章，同一个词反复出现：约定——判重键、`0=不启用`、键名前缀、版本段。底座代码量不大，真正沉淀的是这些让各模块得以互相信任的约定；而约定只有落成迁移、注释、手册与 ADR 里的文字，才约束得住三年后的另一批人。

本章小结

- 底座先行由依赖方向推出：业务表外键单向指向底座表，被依赖者必须先建；变更成本与依赖数成正比，故底座求稳——稳的是模型骨架（HIP 五张权限表 V1 至今未改结构），内容（菜单、角色）随版本生长。
- 认证选 JWT 换多端统一与无状态扩展，短有效期缓解不可吊销；患者端令牌以 `portal:` 前缀承载，与院内接口双向隔离。授权是 RBAC0 + 菜单粒度，要害接口方法级注解二次把门；ADMIN 授权不可改、内置管理员不可停用——防自锁焊死在机制里。

- 口令四环：BCrypt 存储、强度校验、失败 5 次锁 15 分钟（计数持久化）、90 天更换提醒；防爆破的目标是抬高成本而非绝对阻断。
- 组织机构用单树 + 类型标注：病区是 `NURSING` 型科室，床位挂病区；`code` 编码而非自增 `id` 才是跨系统锚点。
- 字典分两层：基础字典（业务直接消费）与术语映射（对外互操作，机制在本章、治理在第 12 章）；实施可替换性 = 业务键 + 幂等 `upsert` 导入 + A/B/C 种子分层。
- EMPI 以 (`id_type`, `id_no`) 确定性匹配做建档幂等，身份证推算生日性别，同名不自动合并——宁可重复建档，不可错误合并。脱敏有存储/数据访问/服务出口/展示四层，HIP 选服务出口 `DTO` 层按角色遮蔽，对准“内部越权浏览”的威胁模型；前端打星不是脱敏。
- `sys_config` 靠三条约定管住 50+ 键：前缀分域命名、字符串值 + 读取方定型（0=不启用）、`remark` 必填；生效机制是直查不缓存——改即生效，低频读是前提。公开接口整表可读，故敏感值绝不入表；新键纪律：迁移插默认值 + 注释 + 手册同步。
- 审计写操作全量（方法 + 路径 + 状态码 + IP），旁路记录不阻断业务但失败应告警；不记请求体是隐私与体积的取舍；敏感操作查询期路径清单过滤；留痕只增不删。
- Flyway 纪律：只前滚、改错用新迁移；`V1 - V999` 平台段 / `V10000+` 实施段加 `impl_` 表名前缀实现产品与定制的库级隔离；历史迁移不改在多医院部署下是生死线——改一次，全部医院的库一致性从此存疑。

思考与练习

1. 概念：用“依赖方向”解释为什么患者主索引在 V2 而住院模块在 V8；若开发顺序倒置（先做住院、患者信息暂存字符串列），补建 EMPI 时需要做哪些返工？逐项列出。
2. 设计：HIP 的 JWT 有效期为 12 小时。分别站在安全（令牌被窃、员工离职）与可用（夜班中途重登、自助机长连）立场论证这个值，并给出你认为更优的期限与理由。
3. 代码审阅：阅读 `JwtAuthenticationFilter` 与 `DbUserDetailsService`，分析“管理员停用某账号”后，该账号已签发令牌在有效期内的实际行为；设计一个改动最小的方案让停用即时生效，并说明它对“无状态”的破坏程度。
4. 设计：新需求——“质控角色（QUALITY）在病案检查场景需要查看完整证件号”。基于 5.5.3 的四层模型，给出至少两种实现位置，比较其安全性与改动面，并说明你会如何防止这个口子被逐步扩大成“人人可见明文”。
5. 排障：某医院从 1.0 升级 1.1 时后端启动失败，日志报 Flyway 校验和不匹配（V4）。列出可能原因、定位步骤与两种处置方案（含各自风险）；说明为什么“直接更新 `flyway_schema_history` 里的校验和”是危险动作。
6. 论述：5.6.3 的“直查不缓存”策略假设配置读取低频。若医保结算量增长使 `yb_ratio_*` 键成为每秒数十次的热键，给出演进方案（缓存位置、TTL 取值、失效策略），并论述为什么“引入独立配置中心”对单院部署仍是过度设计。
7. 实践（配套实训环境）：为 HIP 新增运行配置键 `emr_lock_hours`（病历归档后锁定编辑的小时数，默认 72）。按 5.6.4 的新增键纪律完成全流程：编写新 Flyway 迁移插入键与注释、在业务代码用 `ConfigReader.getInt` 读取、补配置手册行，并验证 `PUT /api/config/emr_lock_hours` 修改后立即生效且操作进入审计日志。
8. 综合：审计表不记请求体，“退费金额是多少”无法从审计还原。设计一个敏感接口请求体摘要方案：哪些路径记、记什么（原文还是脱敏摘要）、如何防止审计表本身成为泄露面。

第 6 章 集成平台

本章属第二篇「架构篇」。医院可能是异构系统密度最高的信息环境：检验仪器说 HL7，医保说省级 SDK，支付商说 HTTPS 回调，CA 厂商说自己的动态库——而它们都要和同一个 HIS 对话。集成平台的职责，就是把这些方言收敛到可管理、可留痕、可替换的边界上。本章从医院集成的问题域讲起，依次展开 集成引擎选型、HL7 V2/MLLP 实战、适配器 SPI 设计、内容路由、事件驱动的跨模块通信、危急值闭环，最后讨论“外部条件”这一医院项目特有约束的工程化管理方法。

学习目标

学完本章，你应当能够：

1. 画出一家县级综合医院的对接图谱，按对接对象类型说出主流接入方式（HL7/MLLP、SDK、HTTP API、文件、数据库）及各自的失败影响面；
2. 比较商用 ESB、开源集成引擎与轻量自研三条技术路线的成本结构与适用场景，说明中小医院场景下的选型逻辑；
3. 读懂一条 HL7 V2 的 ORU^R01 报文，说出 MSH/PID/OBR/OBX 各段的职责、MLLP 帧格式与 ACK 应答语义；
4. 解释“出入站全量留痕”的三重价值（排障、对账、审计），以及留痕为什么必须落在独立事务里；
5. 掌握适配器 SPI 的抽象方法：最小接口面、统一结果对象、Mock 先行、行为契约书面化；
6. 说明事件驱动在模块化单体中的落地形态，比较同事务事件、事务发件箱、消息中间件三级方案的取舍；
7. 描述危急值管理闭环的状态流转与留痕要素，并能用“接入点先行”方法管理一个依赖外部条件的对接任务。

6.1 医院集成的问题域

6.1.1 对接图谱：四类对象

讨论集成方案之前，先把“要和谁打交道”数清楚。一家县级综合医院的对接对象大体分四类：

对象类型	典型代表	常见协议/形态	数据方向	失败影响
院内设备	检验仪器、心电/超声工作站、影像设备	HL7 V2/MLLP、ASTM、私有串口协议、DICOM	以进站为主(结果回传)	结果回不来,人工补录
行政机构	医保经办、区域全民健康信息平台、传染病网络直报	省级 SDK、WebService、前置库/文件	以出站为主(上传上报)	结算不了、上报逾期
商业服务	聚合支付、CA 电子签名、短信、商用审核规则库	HTTPS API、厂商 SDK	双向(调用+回调)	支付/签名功能降级
兄弟系统	遗留 HIS 组件、第三方专科系统	数据库直读、文件交换、接口	双向	数据孤岛、口径不一

这张图谱揭示了医院集成的三个特点。第一，协议高度异构且不由医院选择——设备厂商、医保局、支付商各说各话，医院方只能适配。第二，进出站的失败语义完全不同：进站失败（检验结果没收到）可以靠重发补救，出站失败（医保上传失败）往往必须阻断业务（第 10 章的同事务回滚）。第三，相当一部分对接依赖医院无法控制的外部行政条件：医保 SDK 要资质与专线，CA 要采购厂商产品，微信服务要主体认证——这是 6.8 节“外部条件工程化”要解决的问题。

6.1.2 集成平台的三项职责

无论用什么产品形态实现，医院集成平台的核心职责可以归纳为三条：

1. 协议归一：把外部方言在边界上转换为系统内部的统一模型，业务模块不感知 HL7 段结构或 SDK 报文格式；
2. 留痕审计：每一条进出站消息完整落库——它同时是排障依据、对账数据源和监管检查的原始凭证；
3. 故障隔离：外部系统的不可用不能拖垮院内业务；反过来，院内业务的回滚也不能丢失“报文到过”这一事实。

在医院信息互联互通标准化成熟度测评（第 1 章）的语境下，集成平台还是“标准化”的物理载体——测评要求的消息交换、文档共享能力，最终都落在这一层实现【成稿核对：互联互通测评方案对集成平台的具体技术要求条目】。

6.2 商用 ESB、开源引擎与轻量自研

6.2.1 三条路线

商用 ESB / 集成平台产品。国内主流 HIS 厂商与专业集成厂商都提供医院集成平台产品，通常打包了 ESB 引擎、术语注册、主索引、文档注册库等互联互通测评所需组件。优点是功能全、有测评案例背书；代价是许可费用高（数十万到数百万元）、引入独立的运行时与运维对象，且集成逻辑沉淀在产品私有的配置模型里。

开源集成引擎。以 Mirth Connect（现名 NextGen Connect）为代表，医疗行业使用最广：以“通道（Channel）”为单元，源连接器收报文（MLLP/HTTP/文件/DB），经 JavaScript 转换器映射后由目的连接器发出，自带留痕与重发界面。它把 HL7 工程的通用件做得很好，适合对接点多、报文转换复杂的场景；代价是引入一个需要专人掌握的独立系统，转换脚本本身也会长成一片需要治理的代码。

轻量自研。在应用内实现“够用”的集成能力：留痕表、HL7 解析、若干适配器接口。优点是业务同一个运行时、同一套事务、同一套运维；代价是通用能力（图形化配置、协议库覆盖）远不如专业引擎，对接点数量增长到几十个之后会遇到天花板。

三者不是互斥关系。一个务实的判断标准是对接点数量与转换复杂度：对接点十余个、转换以字段映射为主，轻量自研的总成本最低；对接点几十上百、报文方言繁杂，专业引擎的图形化管理开始值回票价。

6.2.2 HIP 的选择：平台模块 + 重型 ESB 归配套产品

HIP 面向县市级医院（对接点十余个、信息科运维人力有限），选择轻量自研，落地为模块化单体（第4章）中的平台层模块 `platform/integration`；同时在功能清单中明确“重型 ESB 属配套产品”——当医院确需完整 ESB 时外采专业产品，与 HIP 通过标准协议共存，而不是把 HIP 的集成模块养成一个 ESB。

案例框 6-1：一个刻意做小的集成模块

HIP 的 `platform/integration` 模块不足 20 个类，骨架是“一张表 + 一个服务 + 三个 SPI + 两个入站通道”：

- 一张表：`int_message_log` 统一留痕所有进出站消息（`IntMessageLog.java`，建表见 `V10__integration.sql`）。关键列：`direction`（IN/OUT）、`channel`（HL7_LIS/YB/PAY/WX...）、`ref_no`（业务关联号：申请单号/结算单号）、`payload`（完整报文，超长截断到 7900 字符）、`status`（OK/FAIL）、`error`。
- 一个服务：`IntegrationLogService.log(...)`（`IntegrationLogService.java`），方法上仅有的一个事务注解值得整段抄下：

```
/** 独立事务：业务回滚也保留通信痕迹 */
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void log(String direction, String channel, String refNo,
               String payload, boolean ok, String error) { ... }
```

`REQUIRES_NEW` 是留痕正确性的全部秘密：留痕挂起当前业务事务、在独立事务中提交。于是“报文到过/发过”与“业务处理成功”成为两个独立记录的事实——医保上传被拒、业务事务回滚，留痕仍然在，这正是第 10 章对账体系拿它当“通道账”的前提。如果留痕与业务同事务，失败场景（恰恰是最需要痕迹的场景）的痕迹会随回滚一起消失。- 三个 SPI + 两个通道：后文 6.3、6.4 展开。管理页只有一个只读接口（`IntegrationMonitorController.java`：按渠道查最近 100 条留痕），刻意不做重发按钮——进站重发的发起方是设备端（见 6.3.4），出站重发的语义由各业务模块自己定义。

6.3 HL7 V2/MLLP 实战

6.3.1 HL7 V2 报文通识

HL7（Health Level Seven）V2 是医疗设备与系统间事实上的消息标准，1980 年代末诞生，至今仍是设备接口的主流（新一代的 FHIR 见第 3 章）。V2 报文是管道分隔的分段文本：报文由段（Segment）组成，每段一行，段名 3 个大写字母；段内字段用 `|` 分隔，字段内组件用 `^` 分隔，转义与重复分隔符共同构成报文头声明的 `|^~\&`。

一条检验结果报文（`ORU^R01`）的通用示意：

```
MSH|^~\&|LIS|EMSH|HIP|EMSH|20260804120000||ORU^R01|MSG0001|P|2.5
PID|1||P00000002||张三
```

```
OBR|1|SQ20260804-123||C0001^血常规
OBX|1|NM|WBC^白细胞计数|1|15.2|10^9/L|3.5-9.5|H
OBX|2|NM|HGB^血红蛋白|2|45|g/L|130-175|LL
```

各段职责：**MSH**（消息头）声明收发双方、时间、消息类型（**MSH-9**：**ORU^R01**）、消息控制号（**MSH-10**，应答时原样回带）与版本（此处2.5）；**PID**患者标识；**OBR**申请信息，其中**OBR-2**（申请方单号）是回传结果与院内医嘱对上的钥匙；**OBX**每行一个结果项——**OBX-3**项目标识（代码^名称）、**OBX-5**结果值、**OBX-6**单位、**OBX-7**参考范围、**OBX-8**异常标志（N正常/H偏高/L偏低/**HH**、**LL**危急，是6.7节危急值闭环的触发源）。

字段索引有一个著名的陷阱：**MSH**段中**MSH-1**是字段分隔符|本身，**MSH-2**是其余分隔符^~\&，因此**MSH**的第N个|后的内容是**MSH-(N+1)**；而其他段的**SEG-1**就是首个|后的字段。手写解析器不处理这一偏移，**MSH-9/MSH-10**就会错位取值。

6.3.2 MLLP 传输

HL7 V2 只定义报文，不定义传输。设备接口的标准承载是**MLLP**（Minimal Lower Layer Protocol）：在TCP长连接上用控制字符定帧——**0x0B** + 报文 + **0x1C 0x0D**。接收方处理后回**ACK**应答报文：**MSA|AA|<控制号>**表示接受（Application Accept），**MSA|AE**（Application Error）表示出错，发送方据此决定重发。

6.3.3 HIP 的实现：解析器、监听器与统一处理路径

HIP 的 HL7 进站链路由三个类组成，报文从设备到业务落库的完整流转见图 6-1。

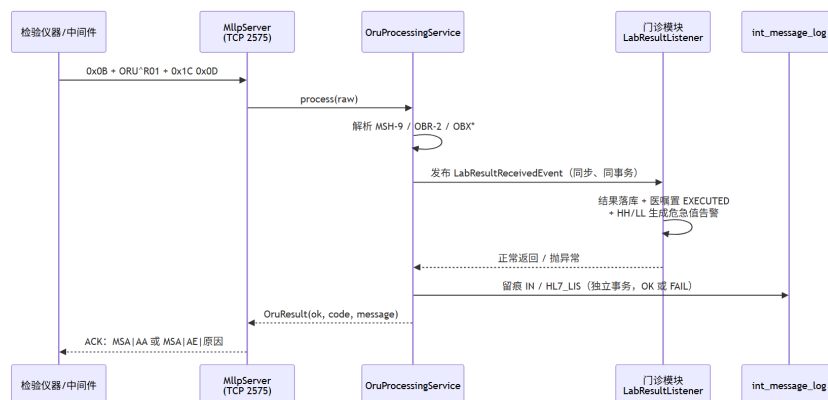


图 6-1 HL7/MLLP 检验结果报文流转时序

案例框 6-2：60 行的 HL7 解析器

HIP 没有引入 HAPI 等完整 HL7 类库，而是自写了轻量解析器 `Hl7V2Message.java`。这个决定的依据是消费方式：HIP 只需从 ORU 报文取三样东西（消息类型、OBR-2、OBX 列表），完整类库的强类型段模型用不上，而设备报文的“方言”（缺段、字段挪用）反而需要宽容取值——解析器把段建模为 `record Segment(String name, List<String> fields)`，`field(int idx)` 按 HL7 的 1 基索引取值、越界返回空串而非抛异常。MSH 偏移陷阱的处理原文如下：

```
if ("MSH".equals(name)) {
    // MSH|^~\&|... : MSH-1 是 |, MSH-2 是 ^~\&
    fields.add("MSH");
    fields.add("|");
}
```

```
fields.addAll(Arrays.asList(line.substring(4).split("\\|", -1)));
}
```

预填两个元素后，`field(9)` 恰好落在 MSH-9。配套单测（`HI7V2MessageTest.java`）逐条固定这些约定：MSH 索引对齐、OBX 顺序保持、越界不抛错、`\n / \r\n` 行尾兼容——解析器的行为契约全部用测试钉死，这比注释可靠。

需要说明：这是“消费三个字段”场景下的合理裁剪；若要做全消息类型的双向转换（ADT、ORM、计费报文），完整类库或集成引擎（6.2）是更对的工具。轻量的前提是想清楚自己只需要什么。

`MllpServer.java` 实现 MLLP 监听：默认端口 2575（`hip.integration.mllp-port`），可整体关闭（`hip.integration.mllp-enabled`）；每连接一个处理线程，30 秒读超时；按 `0x0B ... 0x1C 0x0D` 逐帧读取，处理后回 ACK——控制号从来报 MSH-10 提取原样回带，成功 `MSA|AA`，失败 `MSA|AE|<原因>`；报文连 MSH 都解析不出时仍要应答 AE，让设备端知道“收到了但不认识”。

真正的处理逻辑在 `OruProcessingService.java`，它同时被 MLLP 监听与 HTTP 端点（`HI7InboundController.java`，`POST /api/integration/h17/oru`）调用——双通道、单处理路径。有些设备中间件只会发 TCP，有些第三方系统更方便发 HTTP，承载协议是外部现实，业务语义不该有两份实现。处理步骤与错误码：非 ORU 类型 7001、缺 OBR-2 申请单号 7002、无 OBX 结果行 7003、其余异常 7000；每个分支都先留痕（成功 OK、失败 FAIL 带原因）再返回。

6.3.4 异常与重发

进站链路的重发模型是设备端驱动的：HIP 对每帧报文明确应答 AA/AE，AE 即业务性否定应答，设备/中间件按自身策略重发。这要求进站处理幂等——同一份结果重复回传不能落两份。HIP 的处理是让状态机天然免疫：`LabResultListener`（6.6）只接受状态为 CHARGED/EXECUTED 的检验医嘱，结果落库后医嘱置 EXECUTED，重复报文会再次落结果行但不会重复触发执行动作；找不到可回传医嘱时抛异常→事务回滚→ACK AE，报文不会被“静默吞掉”。留痕表中的 FAIL 记录则是人工介入的线索：集成监控页按渠道倒序展示最近报文，实施期排设备接口问题基本靠它。

6.4 适配器与 SPI 设计模式

6.4.1 模式识记：把“变”关进接口

SPI（Service Provider Interface，服务提供方接口）是“调用方定义接口、提供方交付实现”的模式：与普通接口的方向相反，接口的所有权在平台/调用方，实现方（往往是第三方或实施团队）按契约填空。它在 Java 世界的血统可追溯到 `ServiceLoader`，在 Spring 应用中的落地形态就是“接口 + 多个实现 Bean + 装配规则”；在架构语汇里，它对应六边形架构（Ports & Adapters）中的出站端口——业务核心依赖端口，外部世界的差异被挡在适配器里。

对医院平台产品，SPI 解决的具体问题是：同一个业务动作，在不同医院对接不同的外部服务商。医保各省 SDK 不同、支付服务商不同、CA 厂商不同，但门诊收费、扫码支付、病历签名的业务逻辑全国一样。没有 SPI，产品就会按省、按服务商裂变出代码分支——这是产品化的头号天敌（第 14 章“禁止分叉”原则）。

6.4.2 HIP 的三个适配器：抽象过程

HIP 的集成模块定义了三个 SPI，覆盖三类最典型的外部条件：

SPI 接口	业务动作	方法	结果对象	Mock 实现	真实形态
InsuranceAdapter	医保结算上传/ 退费冲正	2 个	InsuranceResult(ok, settleNo, message)	直接成功+留痕	省医保 SDK
PayAdapter	扫码支付下单/ 退款	2 个	PayResult(ok, qrContent, message)	本地生成二维 码内容	聚合支付服务 商
SignatureAdapter	病历内容签名/ 验签	2 个	SignResult(ok, signature, message)	SHA-256 摘要 模拟	CA 厂商 SDK

三个接口是从三段真实业务里“抽”出来的，抽象过程遵循同一套方法：

1. 找最小动作集。不是照抄外部服务商的能力清单，而是问业务真正依赖哪几个动作。门诊收费只依赖“上传结算、冲正退费”两个动作，于是 `InsuranceAdapter` 只有两个方法——SDK 的签到签退、目录下载、文件对账都是实现包内部的事，不进平台接口（详见第 10 章）。支付同理：出码与退款进接口，回调验签是服务商协议细节，由实现方在自己的回调端点内消化。
2. 统一结果对象。三个接口的返回值都是 `record(ok, 业务载荷, message)` 三元组：布尔结果 + 一个业务真正要的载荷（医保结算号/二维码内容/签名值）+ 人类可读的原因。不抛受检异常、不返回裸字符串——`ok=false` 时 `message` 必须可直接展示给窗口操作员（第 10 章错误码设计的上游约定）。
3. **Mock** 先行。每个接口发布时同步交付 Mock 实现，且 Mock 不是空壳：`MockInsuranceAdapter` 生成结算号并按渠道 YB 留痕，`MockPayAdapter` 生成 `hippay://` 协议的二维码内容并按渠道 PAY 留痕——**Mock** 也履行留痕契约，因此对账、集成监控、E2E 在没有任何真实外部服务的环境里同样成立。这是整个“外部条件工程化”（6.8）的技术基础。
4. 行为契约书面化。接口签名管不住的约定（留痕格式、失败语义、幂等、超时）写成 ADR——ADR-0002 是范本（详见第 10 章）。

装配结构见图 6-2：业务模块只依赖接口；开发/演示环境装配 Mock，实施期以 `@Primary` 或条件属性（如 `hip.insurance.adapter=<region>-sdk`）替换为地区实现包，业务代码零改动。

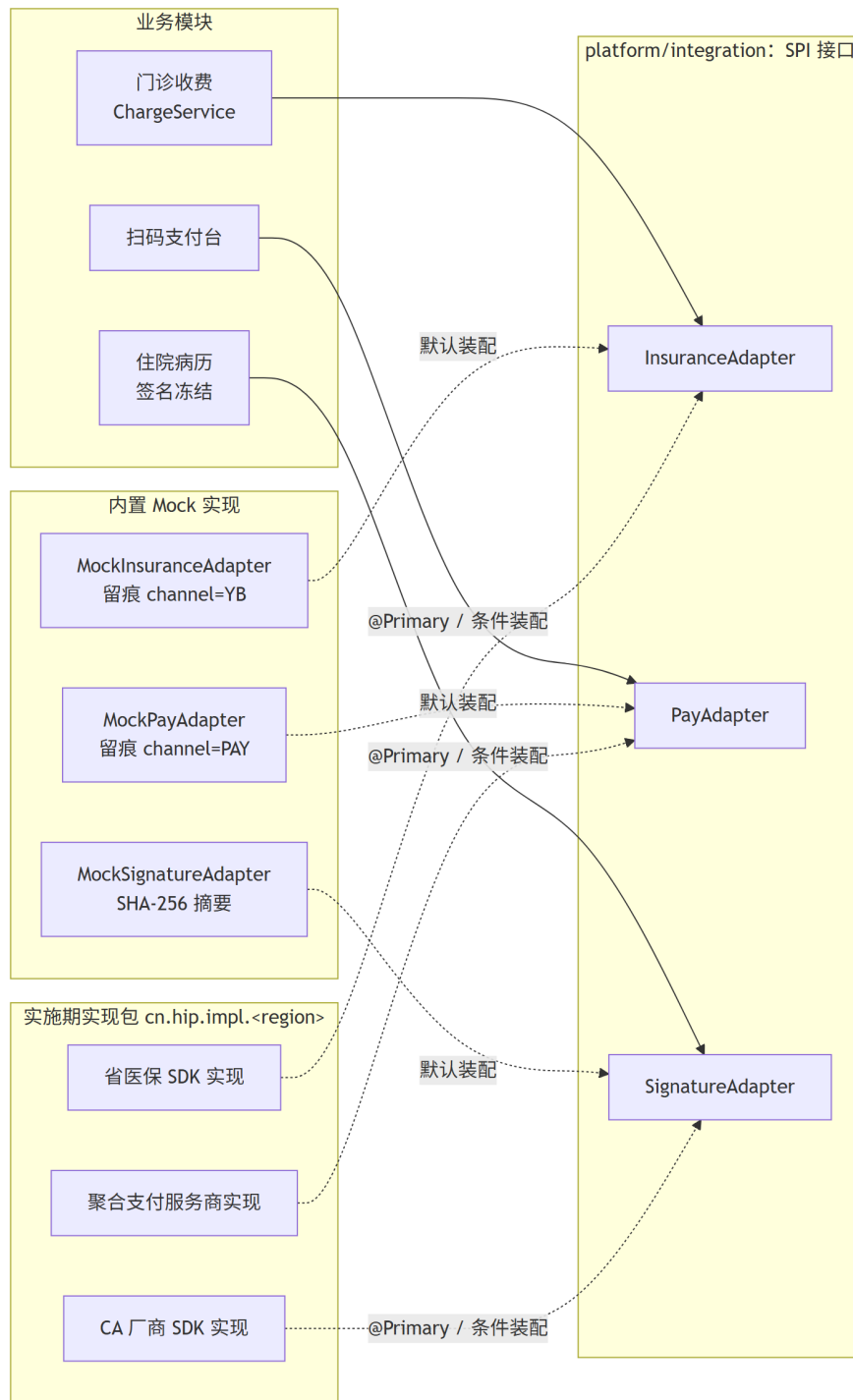


图 6-2 适配器 SPI 装配结构：业务依赖接口，实现按环境替换

案例框 6-3：诚实的 Mock——SignatureAdapter 的边界注释

Mock 最大的风险不是假，而是假得让人忘了它是假的（第 10 章案例框 10-3 的教训正源于此）。MockSignatureAdapter.java 的类注释值得作为范式：

```
/** 签名 Mock: SHA-256 摘要模拟签名值
    (内容防篡改可验, 无法证明签名人身份—生产须换 CA SDK) */
```

它精确声明了 Mock 与真实实现的能力边界：摘要能验证病历冻结后未被篡改（演示与测试足够），但不具备数字证书对签名人身份的法律证明力。一句边界注释，防住的是“Mock 用着用着就上了生产”的实施风险——每个 Mock 都应当写清楚自己“像”到什么程度、从哪里开始“不像”。

InsuranceAdapter 的完整契约（结算号回传、留痕关键字、失败回滚语义、同步超时边界）与假适配器验证法，第 10 章已详细展开，本章不再重复；这里只强调一点分工：**SPI** 模式本身属于集成平台层的设计资产，各接口的领域契约属于各业务章——这也是模块边界在教材目录上的投影。

6.5 文件/DB 适配器注册与内容路由

设备与系统之外，医院还有一类“低频但杂”的对接需求：把某类报文落到共享目录给第三方取、把某类数据写到中间库。对这类需求，HIP 提供了适配器注册 + 内容路由的雏形能力，思想来自企业集成模式（EIP）中的 Content-Based Router：按报文内容决定投递目的地。

实现由两张表与一个管理接口构成（建表见 V32__phase34_outp_pharmacy.sql，接口见 AdapterController.java）：

- `int_adapter`：适配器注册表——`name` 唯一，`type` 仅允许 FILE/DB 两类，`config` 存目的地（文件目录 / 数据源标识），`enabled` 支持一键停用。注册接口用 `INSERT ... ON CONFLICT (name) DO UPDATE` 实现幂等的“注册即更新”；
- `int_route_rule`：路由规则表——`keyword` + `adapter_id`，报文内容含关键字即路由到对应适配器，按规则 `id` 顺序首条命中。

种子数据给出了用法示范：注册“本地文件落地”（FILE，目录 `outbox/`）与“备用库直写”（DB，目标 `int_message_log`）两个适配器，并配一条规则“报文含 `ORU` → 走文件落地”。管理页（适配器路由，菜单挂在集成平台目录下）提供 `route-test` 接口：贴一段报文内容，返回命中的适配器与目的地，并按渠道 `ADAPTER` 留痕模拟投递；命中已停用的适配器则明确报错（4712），而不是静默跳过——把“配置对不对”做成使用者可自助验证的功能，是所有规则类配置（路由、审核规则、CDSS 规则）共同的可用性要点。

要诚实标注这套能力的定位：它是雏形（源码注释原话：“重型 ESB 属配套产品”）——单关键字匹配、首条命中、两类适配器，覆盖的是轻量落地需求；正则匹配、优先级、转换链、可视化编排属于专业集成引擎的领地（6.2）。产品文档把边界写明，比功能本身更重要。

6.6 事务发件箱与事件驱动：不引入消息中间件的跨模块通信

6.6.1 三级方案与双写问题

集成层收到检验结果后，需要通知业务模块落库；医嘱执行后，需要通知数据中心同步——跨模块通信是所有多模块系统的必答题。教科书答案是消息中间件，但引入 Kafka/RabbitMQ 意味着新的运行时、新的运维对象、新的失败模式（消息丢失/重复/乱序），对县市级医院的运维现实（第 4 章）是沉重负担。按可靠性与复杂度排列，实际存在三级方案：

1. 同进程同事务事件：发布方与订阅方在同一进程，事件同步派发，订阅方的处理加入发布方事务——要么一起提交，要么一起回滚。零中间件，强一致，代价是同步耦合（订阅方慢会拖住发布方）且只能进程内；
2. 事务发件箱（Transactional Outbox）：业务事务内把“待发事件”写入本地发件箱表（与业务数据同库同事务），独立的投递器轮询（或借助 PostgreSQL LISTEN/NOTIFY 唤醒）读表外发、成功后标记。它解决的是双写问题——“改业务库”与“发消息”是两个资源，直接双写必有一致性窗口；发件箱把两次写合并为一次本地事务，投递侧用“至少一次+消费幂等”补齐；
3. 消息中间件：跨进程、削峰、多订阅者、重放，代价如前述。

6.6.2 HIP 的现状与演进路径

HIP 处在第一级，且是有意为之。模块化单体让所有模块共享一个进程与一个数据库，Spring 的 `ApplicationEvent` 就是现成的进程内总线：集成层解析完 ORU 报文后发布 `LabResultReceivedEvent`（`LabResultReceivedEvent.java`），门诊模块的 `LabResultListener.java` 以 `@EventListener + @Transactional` 同步消费——解析、落库、医嘱执行、危急值告警在同一个事务里完成，任何一步失败整体回滚，MLLP 层随即应答 AE 让设备重发。消息可靠性问题被整体转化为数据库事务问题，而后者是关系库四十年的成熟领地。

这里有一个精心安排的例外：留痕不参与这个事务（6.2 案例框 6-1 的 `REQUIRES_NEW`）。业务回滚时，“收到过这条报文且处理失败”的事实必须保留——事务边界的划定，本质是在回答“哪些事实同生共死、哪些事实独立成立”。

事件模型的另一收益是解耦了事件的生产入口：检验结果既可能来自仪器直连（MLLP），也可能来自检验技师手工录入后审核发布——LIS 工作台的发布接口（`MedTechController.java` 中 `publish`，注释原话“复用 LIS 事件链路”）把手工结果组装成同一个 `LabResultReceivedEvent` 发布，下游的落库、医嘱执行、危急值逻辑一行不改。事件是入口的汇聚点——这是它相对于“服务直调”的架构价值，即使在同步同事务的最简形态下也成立。

演进路径在总体规划中预留：数据量增长或出现真正的异步场景（向区域平台推送、短信通知）时，按“PostgreSQL LISTEN/NOTIFY + 事务发件箱，量大后上 RabbitMQ”的顺序升级。需要向读者诚实说明：发件箱表在 HIP 当前版本尚未实体化，它是规划中的下一级台阶——升级的触发条件是“出现跨进程消费者”，而不是架构美学。第 10 章医保上传选择同事务同步调用、把异步方案留给被数据证明的瓶颈，与本节是同一个决策模式的两次应用。

6.7 危急值管理闭环

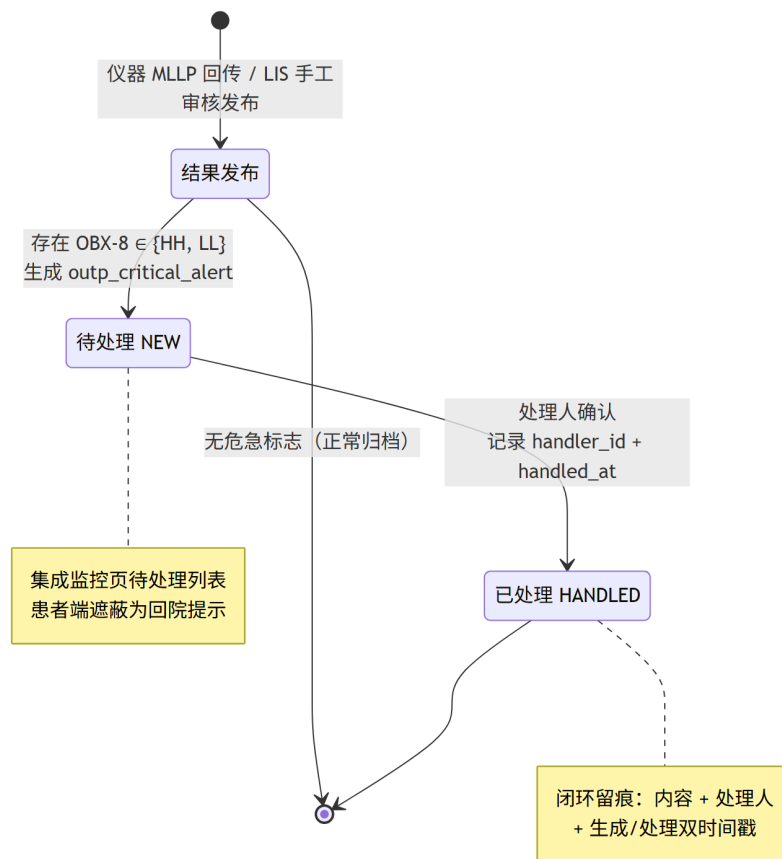
6.7.1 制度要求

危急值（Critical Value）指提示患者可能处于生命危险边缘、须立即干预的检验检查结果（如血钾 >6.5 mmol/L、血小板 $<30 \times 10^9/L$ ）。危急值报告制度是国家卫生健康行政部门发布的医疗质量安全核心制度之一【成稿核对：《医疗质量安全核心制度要点》发文年份与文号】，要求明确：危急值发现后须及时报告至临床、临床确认接收并处置、全过程记录可追溯（谁报告、谁接收、何时处置）。电子病历分级评价与等级医院评审都会核查危急值闭环的信息化留痕。

对信息系统，“闭环”意味着一条不许断的链：发现（仪器/审核）→生成告警→送达临床→确认处理→留痕（人+时间）。任何一环靠口头电话而无系统记录，检查时都视为断链。

6.7.2 HIP 的实现：从 OBX-8 到处理留痕

HIP 的危急值链路直接生长在 6.3 - 6.6 的集成链路上，状态流转见图 6-3。



案例框 6-4：危急值告警的生成与闭环留痕

触发点在 `LabResultListener` 的落库循环里 (`LabResultListener.java`)：结果逐行入库时比对异常标志集合 `Set.of("HH", "LL")`，命中即拼装告警内容并落 `outp_critical_alert`：

```
if (item.abnormalFlag() != null && CRITICAL_FLAGS.contains(item.abnormalFlag())) {
    critical.append("%s=%s%s(%s)".formatted(
        item.name(), item.value(), item.unit() == null ? "" : item.unit(),
        item.abnormalFlag()));
}
...
alert.setContent("【危急值】%s: %s".formatted(order.getItemName(),
    critical.toString().trim()));
```

告警表 (`OutpCriticalAlert.java`) 的列就是闭环的证据结构：`order_id / registration_id` 双外键锚定“哪份申请、哪次就诊”，`content` 快照告警内容，`status` (`NEW/HANDLED`) 承载状态机，`handler_id + handled_at + created_at` 回答“谁处理、何时报、何时处置”。处理入口 (`LabResultController.java`) 从登录态取处理人 `id` 写入——处理人来自会话而非前端参数，留痕才可信。

两个设计点值得剖析。其一，告警生成与结果落库同事务：不存在“结果入了库、告警没生成”的缝隙——这是 6.6 同事务事件模型送的礼物。其二，判定标准挂在 **OBX-8** 上意味着危急值的裁量

权在结果的发布方（仪器规则或审核技师），系统信任上游标志。这与部分医院“院内统一危急值界值表、系统按值判定”的做法不同，各有适用场景：前者尊重设备与专业判断、实现简单；后者全院口径统一、但要维护一张随检验方法学变化的界值表。产品可以两者兼容——按标志判定兜底，界值表判定作为可配置增强。

送达侧，当前是“拉”模式：集成监控页顶部常驻危急值待处理列表（`MonitorView.vue`），工作台首页有提醒入口；患者端刻意遮蔽危急值——报告页不向患者展示 HH/LL 结果本身，只提示尽快回院（功能清单：「危急值遮蔽为回院提示」），因为危急值的第一告知人应当是医生而不是 H5 页面。对照 6.7.1 的制度要求，“拉”模式是闭环的最低成立形态；短信/电话双通道通知、超时未处理自动升级（通知科主任）是明确的增强方向，也是等级评审现场最常被问到的两个点。

6.8 外部条件工程化

6.8.1 问题：进度不由你控制的对接

回看 6.1 的对接图谱会发现一个残酷事实：图谱里最重要的几条线——医保、CA、支付、设备直连——没有一条的开通时间由开发方控制。医保 SDK 要等资质审批与专线开通，CA 要等医院完成厂商采购，微信服务要等主体认证，设备直连要等厂商工程师到场。传统项目管理把这些做成甘特图上的“外部依赖”，然后眼睁睁看着它们把上线日期推成橡皮筋。

HIP 把这个问题工程化为一套方法，称为外部条件管理，核心是三句话：接入点先行，Mock 实现兜底，条件到位即接。

6.8.2 方法的三根支柱

支柱一：接入点先行。每个外部条件对应的技术侧工作——SPI 接口、表结构、页面、配置键——在等待期内全部完成并通过测试。“等医保 SDK”等的只是一个实现类，不是一个模块；“等 CA”等的只是换一个 Bean。HIP 功能清单的附录明确列出全部 8 项外部条件，并声明“接入点全部就绪”（功能清单.md）：省医保 SDK、CHS-DRG 官方目录、商用 CDSS/审核规则库、CA 签名产品、微信资质/电子健康卡、检验设备直连（MLLP 已通，按设备协议配置）、PACS/影像 AI、聚合支付服务商。注意每一项的表述都精确到“还差什么”——这份清单同时是给院方的承诺书与给实施团队的任务书。

支柱二：Mock 不是占位符，是可验收的替身。6.4 已经看到，HIP 的 Mock 履行真实实现的全部平台侧契约（留痕、结果对象、失败语义可注入），因此整条业务链路——含对账、监控、E2E——在零外部条件的环境里可完整演示与回归。这带来一个容易被低估的商务价值：系统验收可以与外部条件解耦。医院可以先验收业务功能，外部条件到位后做增量验收，而不是让整个项目吊死在医保局的审批流程上。

支柱三：替换工艺被彩排过。“条件到位即接”不能只是口号——替换是否真的只要换一个实现类？HIP 用两件事证明：一是 ADR 化的替换步骤（ADR-0002 的四步：新建实现类 → Bean 装配替换 → 跑 E2E 全套 → 真实环境首日人工对账），二是虚拟医院全量彩排中的假适配器验证——写一个模拟真实形态的实现包走完整个替换流程（详见第 10 章、第 14 章）。彩排过的工艺才是工艺，否则只是愿望。

6.8.3 管理形态

落到项目管理上，外部条件的正确形态是一张双线对齐表：技术线（接入点完成情况，开发方负责）与行政线（资质/采购/专线的申请状态，院方负责），逐项列出当前卡点与责任方，例会过一遍。它把“等待”变成“排队”——每个条件都知道自己排在哪儿、差什么、谁去催。第 10 章 10.7.4 的医保接口调研清单，就是这张表在医保条件下的展开页；LIS 设备、CA、支付各有对应页。

外部条件工程化还有一个隐含的产品设计约束：凡外部条件必有开关或降级。医保未接：现金/自费流程完整可用，医保模块可整体关闭（模块开关，第14章）；CA未接：Mock签名保证病历冻结流程可走，出具的是“防篡改”而非“法律效力”；支付未接：窗口现金收费不受影响。系统的可用性下限不被任何单一外部条件绑架——这既是工程韧性，也是面对医院客户时最有说服力的一句话：“哪个条件都不拦着你上线。”

本章小结

- 医院集成的问题域可归纳为四类对接对象（设备/行政机构/商业服务/兄弟系统），集成平台的三项职责：协议归一、留痕审计、故障隔离。进站失败靠重发，出站失败常须阻断业务，两者的失败语义设计完全不同。
- 集成引擎选型看对接点数量与转换复杂度：十余个对接点、字段级映射，轻量自研总成本最低；HIP以不足20个类的平台模块承载集成能力，重型ESB明确归配套产品。
- 留痕表 `int_message_log` 以 `REQUIRES_NEW` 独立事务落库，把“报文到过”与“业务成功”分离为两个事实——它同时是排障依据、对账通道账与监管凭证。
- HL7 V2 是管道分隔的分段文本，MSH-1 是分隔符本身这一索引偏移是手写解析器的经典陷阱；MLLP 以 `0x0B/0x1C 0x0D` 定帧，ACK 的 AA/AE 驱动设备端重发，进站处理须幂等。HIP 的 HTTP 与 MLLP 双通道共用同一处理路径。
- 适配器 SPI 的抽象四步：找业务依赖的最小动作集、统一 `record(ok, 载荷, message)` 结果对象、Mock 先行且履行留痕契约、行为约定书面化为 ADR。Mock 必须写明能力边界（“像”到哪里为止）。
- 跨模块通信三级方案：同事务事件 → 事务发件箱 → 消息中间件。模块化单体下，同步同事务事件把消息可靠性转化为数据库事务问题；事件同时是多入口（仪器直连/手工录入）的汇聚点。升级的触发条件是出现跨进程消费者，不是架构美学。
- 危急值闭环 = 发现（OBX-8 的 HH/LL）→ 告警生成（与结果落库同事务）→ 送达 → 确认处理 → 双时间戳留痕；处理人取自登录态而非前端参数。通知升级（短信/电话、超时上报）是普遍的增强方向。
- 外部条件工程化三支柱：接入点先行、Mock 兜底、替换工艺彩排；管理形态是技术线与行政线的双线对齐表；产品约束是“凡外部条件必有开关或降级”。

思考与练习

1. 概念：为什么检验设备直连要经过集成层（解析 + 留痕 + 事件），而不是让 LIS 模块自己监听一个 TCP 端口？从协议归一、留痕审计、故障隔离三项职责分别给出理由。
2. 报文：为患者“李四”（病历号 P00000015）手写一条 ORU^R01 报文：申请单号 SQ20260810-001，项目“血清钾”，结果 6.8 mmol/L，参考范围 3.5-5.3，危急偏高。标注 OBR-2 与 OBX-5/6/7/8 的位置，并写出 HIP 应答的 ACK 报文（说明 MSA-2 控制号的来源）。
3. 设计：为 `PayAdapter` 增加“支付状态查询”能力：给出方法签名与结果对象、Mock 实现的行为与留痕关键字。再论述：这个方法该不该进 SPI？（提示：回调丢失时的对账兜底 vs 接口面最小化，参考 6.4.2 的抽象方法。）
4. 排障：某医院设备中间件报告“MLLP 发送成功、收到 AA”，但医生站看不到结果。按图 6-1 的链路写出排查步骤与每步查询的表/接口。再考虑另一情形：中间件持续收到 AE 且留痕 `error` 为“未找到可回传的检验申请”，列出三种可能的业务原因。
5. 推演：若把 `LabResultReceivedEvent` 的消费从同事务改为事务发件箱异步投递，图 6-1 的时序如何变化？ACK 的语义从“业务处理成功”退化为什么？危急值告警的生成延迟、失败重试、死信处理分别需要补哪些机制？

6. 论述：一家 500 床医院有 30 个对接点、其中 12 个需要报文转换，信息科有 2 名开发人员。重新评估 6.2 的三条路线，给出你的选型与混合方案（哪些对接留在应用内 SPI、哪些交给集成引擎），并说明留痕与监控如何统一。
7. 实践（配套实训环境）：为内容路由增加“正则匹配 + 优先级”能力：给出 `int_route_rule` 的表结构改动（DDL）、匹配算法（优先级排序、正则与关键字共存时的语义）、`route-test` 接口的返回扩展（展示全部命中而非首条），并补一条 E2E 断言。
8. 观察（临床对照）：访谈一位检验科技师与一位临床医生，记录本院危急值从仪器报警到医生处置的真实链条（含电话环节），对照 6.7 的闭环模型标出未留痕的环节，提出信息化补齐方案并估计各环节的时限指标。

第 7 章 门急诊系统

本章属第三篇「业务篇」。门急诊是医院业务量最大、并发最高、链条最长的领域：一个患者半天之内要经过挂号、候诊、接诊、开单、缴费、检查、取药六七个环节，每个环节背后是不同科室、不同角色在同一套数据上协同。本章从号源模型讲起，沿患者动线依次展开医生站、统一订单行、合理用药、收费、聚合支付、发药审方、预约叫号，最后到急诊预检分诊与留观。全章以 HIP 平台的门诊模块为贯穿案例，重点剖析三个高价值工程主题：原子占号与原子扣库存的并发控制、退号/退费/退药的联动拦截链、审方状态机的非强制模式取舍。

学习目标

学完本章，你应当能够：

1. 说出「排班—号源—挂号」三层模型的职责划分，解释号源容量为什么是并发控制问题而不是业务校验问题；
2. 掌握防超挂的三种数据库解法（悲观行锁、条件更新、乐观锁），并说明它与医保年度额度控制（第 10 章）是同一模式的变体；
3. 解释统一订单行模型相对「处方表 + 申请单表」分立方案的优势，画出订单行的完整状态机；
4. 说出合理用药五重拦截（过敏交叉/重复用药/药物相互作用/年龄禁忌/抗菌药分级）各自的药学依据与数据来源，理解 FORBID/CAUTION 分级的临床可用性含义；
5. 设计退号、退费、退药之间的联动拦截规则，解释为什么逆向操作必须按正向链条的反序逐级解除；
6. 描述聚合支付的一般架构（出码—回调—对账）与 PayAdapter SPI 的契约要素；
7. 论述药师审方「强制模式」与「非强制模式」的合规与人力权衡，说出从非强制升级到强制需要改动的卡点。

7.1 号源与排班模型

7.1.1 号源的业务本质：容量管理

挂号是门诊一切业务的起点。它表面上是「登记一个患者」，本质上是分配一份稀缺的诊疗容量：某医生某日上午只能看 30 个病人，第 31 个必须被拒绝。因此挂号模型天然分三层：

- 排班（schedule）：管理侧决定「谁、哪天、哪个时段、出什么号、多少个」——科室/医生、日期、时段（上午/下午/全天）、号别（普通/专家）、单价、容量；
- 号源：排班的容量视角——总量与已占用量的差即余号。号源可以进一步细分为时点号（8:30 - 8:40 一个号）或时段号（上午段内排序），县市级医院多用时段号；

- 挂号记录 (registration): 患者与一个号源的绑定, 携带就诊序号 (叫号用)、费用与状态。它是后续病历、处方、收费、报告的根——门诊域几乎所有表都直接或间接挂在挂号记录之下。

预约挂号 (互联网端、自助机) 与窗口挂号只是渠道差异, 最终都收敛到同一个动作: 在某条排班上占掉一个号。渠道越多, 这个动作的并发越高。容量之外, 排班还要处理两类运营变化: 停诊 (医生临时有事, 排班置停用, 已挂号患者退号或改约) 与加号 (容量临时上调)。停诊用启用标志而非删除排班实现——已有挂号记录外键挂在排班上, 删除会破坏历史; 这也解释了 HIP 占号语句里 `enabled = true` 条件的存在: 停诊瞬间之后的占号请求自然失败, 不需要额外通知在途渠道。

HIP 的排班表 `outp_schedule` 与挂号表 `outp_registration` 的建表语句见 `V3_outpatient_registration.sql`。注意排班表的两个整型列: `capacity` (容量) 与 `booked` (已占用), 以及一条数据库检查约束 `chk_booked (booked >= 0 and booked <= capacity)`——它是防超挂的最后一道防线, 下文会回到它。

7.1.2 超挂问题: 并发控制的三种解法

「余号还有 1 个, 两个渠道同时来挂」是号源模型的经典竞态。如果代码写成「先查余号, 再判断, 再加一」(`read-check-write`), 两个事务都会读到「还剩 1 个」, 都通过判断, 最终 `booked = capacity + 1`, 超挂发生。校验写在应用层的 `if` 里, 防不住并发——这是初学者最常见的错误认知。数据库层面有三种正统解法:

方案	做法	特点
悲观行锁	事务内 <code>SELECT ... FOR UPDATE</code> 锁定排班行, 再读再判再写	通用性最强, 可承载复杂判断逻辑; 持锁时间长
条件更新	单条 <code>UPDATE ... SET booked = booked + 1, WHERE booked < capacity</code> 按受影响行数判断成败	判断与更新在一条原子语句内完成, 无竞态窗口, 代码最短
乐观锁	行上加版本号, 更新时带 <code>WHERE version = ?</code> , 失败则重试	无锁等待, 但要写重试循环, 高冲突时反而低效

选型的关键是判断逻辑的复杂度: 占号的判断只有一个条件 (还有余号), 能塞进 `UPDATE` 的 `WHERE` 子句, 条件更新是最优解; 医保年度额度 (第 10 章 10.2.3) 要「读累计值—算扣减额—回写」, 判断与计算无法压进一条 `UPDATE`, 才需要 `SELECT ... FOR UPDATE`。两者是同一模式的变体: 把「检查—修改」的原子性交给数据库, 而不是应用代码。第 8 章的住院占床、本章 7.7 的扣库存, 都在这个谱系上。

案例框 7-1: HIP 的原子占号

HIP 的占号实现在 `OutpScheduleRepository.java`, 一条 JPQL 条件更新:

```
/** 并发安全占号: 只有还有余号时才自增, 返回受影响行数 */
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("update OutpSchedule s set s.booked = s.booked + 1 " +
        "where s.id = :id and s.booked < s.capacity and s.enabled = true")
int occupySlot(@Param("id") Long id);
```

调用方 (`RegistrationService.java`) 按返回值分流: `occupySlot(scheduleId) == 0` 即「号源已满」(错误码 3003)。两个并发事务执行这条 `UPDATE` 时, 数据库对同一行的写互斥保证了只有一个能满足 `booked < capacity`。三个配套细节值得体会:

- 就诊序号复用占号结果：占号成功后重读排班，`reg.setRegNo(after.getBooked())`——第N个占到号的`患者`就是第N号，序号的唯一性由占号的原子性免费获得，不需要另一套发号器。注解里的`clearAutomatically`正是为了让这次重读穿透JPA一级缓存拿到最新值；
- 重复挂号前置检查：同一患者对同一号源已有`REGISTERED`记录则拒绝（错误码3002），防手滑双击；
- 数据库约束兜底：即使未来有人绕过`occupySlot`直接改`booked`，检查约束`chk_booked`仍会拒绝越界值。应用层原子操作管正常路径，数据库约束管所有路径——纵深防御在数据层的体现。

7.1.3 退号与号源释放

退号是占号的逆操作：挂号记录置`CANCELLED`、`releaseSlot`把`booked`减一（同样是条件更新，`booked > 0`防负数）。但退号不是无条件的——患者可能已经缴过挂号费甚至已就诊。HIP的规则：仅`REGISTERED`状态可退；退号时扫描名下订单行，未收费的挂号费订单自动作废，存在已收费订单则拒绝退号、提示先退费（错误码3006）。这是本章「联动拦截链」的第一环，完整链条在7.5.3展开。

7.2 门诊医生站

7.2.1 接诊：把「排队的号」变成「进行中的就诊」

医生站的第一个动作是接诊：挂号状态从`REGISTERED`（或已叫号`CALLED`）置为`VISITED`。这个看似平凡的状态跳转承担两个职责：一是开单前置条件——HIP规定未接诊不得开单（错误码4003），防止医生点错患者开错单；二是归属确认——按科室挂的普通号没有预定医生，首位接诊医生回填`doctorId`。已退号（`CANCELLED`）的记录拒绝接诊。实现见`DoctorStationService.java`的`startVisit`。

7.2.2 门诊电子病历

门诊病历远比住院病历轻量，但骨架同源：主诉、现病史、既往史、体格检查、处理意见五段，对应`outp_emr`表的五个文本列。两个工程要点：

- 一次就诊一份病历：`outp_emr`与挂号记录一对一，保存即「有则更新、无则创建」的幂等语义；
- 签名冻结：病历经电子签名后不可再改。HIP的`signEmr`把五段内容拼接为摘要串交给`SignatureAdapter`（签名适配器SPI，第6章）签名，签名值与时间落库；此后`saveEmr`遇到已有签名直接拒绝（错误码4008「病历已签名冻结」），重复签名同样拒绝（4010）。CA（Certificate Authority，数字证书认证机构）产品是外部条件，Mock阶段用摘要模拟，接口形态与住院病历共用一套（第8章）。

「签名冻结」的顺序值得强调：先校验内容存在，再签名，签名成功才落签名值——签名失败（CA服务不可用）抛错，病历停留在可编辑状态，而不是留下一个「签了一半」的记录。冻结之后如需修正，正规路径是新增修正记录而非解冻改写；门诊病历虽轻量，「签名即定稿」的证据链纪律与住院病历完全一致，这也是电子病历应用水平分级评价（第1章）对病历系统的基本要求。

7.2.3 诊断管理

诊断表`outp_diagnosis`按ICD-10编码记录，列表第一条即主诊断（`primaryDiag = true`），保存采用整表替换（删旧建新）——门诊诊断条数少、修改频繁，替换语义比逐条diff简单可靠。主诊断的下游消费者远比门诊本身多：医保结算与智能审核的适应症判断（第10章）、单病种自动入组（第12章）、CDSS的诊断建议（7.4）都以它为输入。门诊诊断的编码质量是一笔「上游存款」，这与第10章「DRG的数据质量问题几乎全部要回溯上游」是同一件事的门诊版。

7.3 统一订单行

7.3.1 从分立表到统一订单行：业界模型的演进

传统 HIS (Hospital Information System, 医院信息系统) 的门诊开单常常是「多套表」: 处方及处方明细一套、检查申请单一套、检验申请单一套、治疗单一套, 收费时再从各套表里抽取费用汇总。这个设计的病根在费用口径: 同一笔钱的「业务形态」(处方明细) 与「费用形态」(收费明细) 落在两处, 金额、状态要靠程序同步, 退费、对账、统计每一处都要跨表拼接, 口径漂移是迟早的事。

演进方向是统一订单行 (order line) 模型: 处方药品、检验、检查、治疗, 每一条都是一行结构同构的「订单」, 用类型字段区分, 价格、数量、金额、状态内聚在行上。收费按行结算、发药按行扣减、执行按行销账、统计按行汇总——一行数据同时是业务明细和费用明细, 费用不落两处账。药品行多出的用法用量字段 (用法/频次/单次剂量/天数) 作为可空列共存, 处方的「张」概念退化为一个分组号: 同次开立的药品共享处方号, 检查检验各自成申请单号。

7.3.2 HIP 的 outp_order 模型

HIP 的订单行实体 OutpOrder.java 完整体现了这个模型: `orderType` (DRUG 药品 / LAB 检验 / EXAM 检查 / TREAT 治疗)、`groupNo` (处方号或申请单号)、项目快照 (编码/名称/规格/单位/单价——开单时刻的快照, 此后调价不影响已开订单)、金额、药品用法组、状态、审方组。开单入口 `createOrders` 一次接收一组行: 药品行共享一个 CF 前缀处方号, 非药品行各分配 SQ 前缀申请单号; 药品从药品字典取价、其余从收费项目字典取价 (第 5 章主数据)。

案例框 7-2: 挂号费也是一条订单行

挂号费是最容易被特殊对待的费用——很多系统把它直接记在挂号表上单独收取。HIP 的做法是挂号成功即生成一条 `orderType = "REG"` 的订单行 (`itemCode = "REG"`, 分组号 GH-{挂号id}), 进入与处方、检查完全相同的收费队列 (RegistrationService.java 的 `register` 尾部)。收益立刻显现: 收费员一次结清「挂号费+处方+检查」不需要两套收银逻辑; 退号拦截只需检查订单行状态 (7.1.3), 不需要为挂号费单写一条规则; 日报表天然覆盖挂号收入。模型统一的价值, 正在于让每一个「特例」消失。

订单行是门诊全流程的中枢。图 7-1 给出以它为核心的数据流全景:

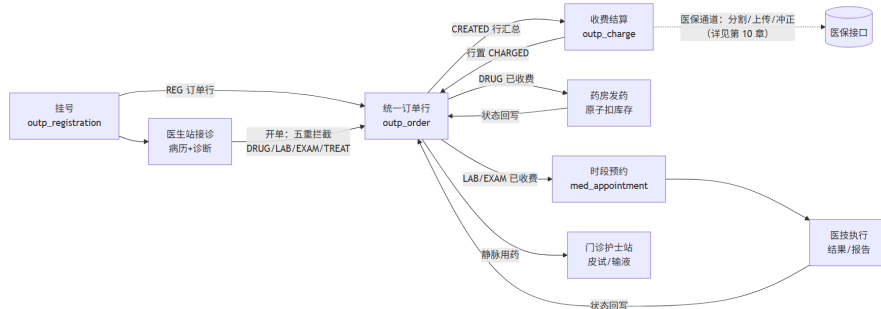


图 7-1 门诊全流程数据流：统一订单行是收费、发药、执行的共同数据源

7.3.3 订单行状态机与执行闭环

订单行的状态机是全章多个业务的交汇点 (图 7-2):

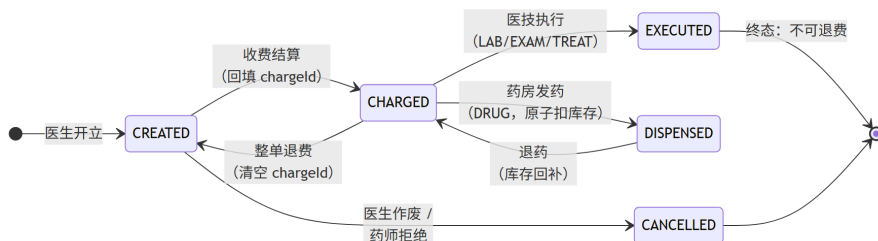


图 7-2 订单行状态机：正向沿开立→收费→发药/执行推进，逆向必须逐级回退

下游执行按类型分流：药品行进入药房发药队列（7.7）；检验/检查行收费后先预约时段（7.8.2）再由医技执行站销账——执行站校验「仅已收费项目可执行」并拒绝药品与挂号费行（ExecStationController.java），执行同时录入结果文本形成简版报告，医生站可回查；静脉用药行还会流入门诊护士站生成输液执行单，青霉素类、头孢类药物输液前须有阴性皮试结果放行（OutpNurseStationController.java）。检验结果的正式通道（LIS 标本流转、危急值回传）在第9章展开。

7.4 合理用药与 CDSS

7.4.1 拦截还是提醒：CDSS 的临床可用性问题

CDSS（Clinical Decision Support System，临床决策支持系统）在处方环节的价值排序是明确的：在开单一刻拦住错误处方，优于药师事后审出，更优于患者用药后出事。但「拦截」是把双刃剑——规则误报一次拦住正常处方，门诊高峰期的医生会立刻要求关闭整个系统。国际上对此有专门的名词叫警报疲劳（alert fatigue）：提示越多越滥，医生越倾向于不看内容直接点掉，真正致命的警告反而被淹没。文献中临床系统的用药警告忽略率长期居高不下【成稿核对：如引用具体比例数字需核对文献出处】，其教训可以概括为一条设计准则：每弹出一一次不该弹的提示，都在透支下一次该拦截时的注意力。

因此成熟的 CDSS 都做分级干预：确定性高、后果严重的规则做硬拦截（FORBID，禁止提交），其余做提醒留痕（CAUTION，弹出警示但放行，记录用于事后质控）；上线新规则先以留痕级试运行，观察命中与误报，再决定是否升级为拦截。这与第10章医保审核的 WARN→BLOCK 演进路径同构：分级不是技术设计，是与临床的信任契约。

7.4.2 五重拦截的药学依据与数据来源

HIP 在开单链路上串联了五重拦截，每一重的知识来源与实现难度不同：

拦截	药学/管理依据	数据来源	级别
过敏交叉	同类结构药物交叉过敏（青霉素类共享 β-内酰胺环侧链、阿司匹林与水杨酸类交叉）	患者过敏史（EMPI，第5章）× 交叉映射	拦截
重复用药	同次就诊重复开具同一药品，重复给药风险	本次就诊订单行	拦截
药物相互作用（DDI, Drug - Drug Interaction）	如头孢类与含乙醇制剂联用致双硫仑样反应	规则表 <code>cdss_ddi_rule</code>	分级
年龄禁忌	如 18 岁以下禁用喹诺酮类（影响软骨发育）	患者出生日期 × 规则表 <code>cdss_age_rule</code>	拦截
抗菌药分级	抗菌药物按非限制/限制/特殊三级管理，处方权与医师层级挂钩【成稿核对：《抗菌药物临床应用管理办法》文号与分级条款】	药品分级属性 × 医师授权表 <code>med_abx_privilege</code>	拦截

疗程超限（如解热镇痛药连用不超过 5 天）作为第六类以 CAUTION 提醒存在——疗程判断个体差异大，不适合硬拦。

案例框 7-3：HIP 五重拦截的实现剖析

拦截分两段实现。前两重内聚在开单服务（DoctorStationService.java）：过敏交叉用一张常量映射把过敏史关键词展开为药名特征——

```
private static final Map<String, List<String>> ALLERGY_CROSS = Map.of(
    "青霉素", List.of("西林", "青霉素"),
    "头孢", List.of("头孢"),
    "磺胺", List.of("磺胺"),
    "阿司匹林", List.of("阿司匹林", "水杨酸"));
```

患者过敏史含「青霉素」，开「阿莫西林」（命中「西林」）即抛错误码 4012。后三重在 CdssService.java，全部表驱动（V25__phase28_drg_cdss.sql）：DDI 规则表带 severity 列，FORBID 抛 4015 拦截、CAUTION 落 cdss_alert 表留痕放行；DDI 核对的范围是「本次新开药×同诊全部在用药」的两两组合——只查新开药之间是不够的，上一张处方里的头孢与这一张里的藿香正气水同样构成双硫仑风险。年龄规则按出生日期算实足年龄比对上下限，命中抛 4017。抗菌分级则读药品字典的 abxLevel 与医师授权表比较，缺省处方权为 1 级（仅非限制级）——宁可让未配置授权的医师开不出限制级抗菌药，不可默认放行（错误码 4014）。

注意抛错的位置：五重拦截全部发生在订单行落库之前，拦截即整组开单失败。这利用了事务原子性——不存在「一组医嘱开了一半被拦」的中间态。

7.4.3 规则库的可维护性

五重拦截中，交叉过敏映射是硬编码常量，DDI/疗程/年龄是数据库规则表——这个差异是有意的分层：交叉过敏是稳定的药学常识，变更频率以年计；DDI 与疗程规则则需要药剂科持续维护（新药引进、依据更新），必须表化并配管理页面。表驱动还带来实施可替换性：换一家医院，规则内容可整表替换而代码不动（第 14 章「配置优先」）。再往前一步是接入商用合理用药知识库（数十万条规则，外部条件）——与第 10 章医保审核「管道自建、内容外采」是同一个结论：中小医院自研规则内容不现实，但规则引擎、留痕表、分级机制这套管道必须自建，商用库到位即灌入。

CDSS 的另一翼是正向建议：cdss_suggestion 表按诊断 ICD 前缀给出检查与处置推荐（如 I21 急性心肌梗死提示启动胸痛绿色通道、D2B ≤ 90 分钟），在医生站随诊断展示。拦截防错、建议提质，两者共用「规则表 + 触点嵌入」的实现骨架。

7.5 门诊收费

7.5.1 结算模型：一次结清与结算单

门诊收费的通用模型是结算单（settlement）：收费员选定患者，系统汇总名下全部待收费订单行，一次结清生成一张结算单，订单行回填结算单号并置为已收费。HIP 的实现（ChargeService.java）：取该挂号下全部 CREATED 行求和，生成 outp_charge 结算单（单号 SJ日期-序号，先落库取自增 id 再拼单号，保证唯一且可读），逐行置 CHARGED 并回填 chargeId。支付方式四选一：现金/微信/支付宝/医保。

医保单（payMethod = "YB"）在同一事务内追加三步：费用分割（订单行装配为 YbLine 列表调用分割引擎）、医保上传（失败抛 5006 整体回滚）、医保结算号回填 ybSettleNo。这条链路的设计论证——为什么同步同事务、结算号为什么必须持久化、失败语义与错误码——医保通道详见第 10 章（10.3），此处只需看见：门诊收费为医保留好了挂载点，本地结算逻辑本身不感知医保细节。

结算单还有一个容易被低估的下游：收费员日结。每个收费员班末按「现金/扫码/医保」分渠道汇总当班结算与退费，与实收现金、通道流水核对后交款——结算单上的 cashierId、payMethod、状态三列就是日结报表的全部分组键。日结与运营报表在第 12 章展开，这里的教学点是：收费表的每一列在设计时就要想清楚它的报表消费者，事后补列往往意味着历史数据无法回溯。

7.5.2 退费与重收

HIP 的退费是整单退费：结算单置 `REFUNDED`，名下订单行全部退回 `CREATED` 并清空 `chargeId`——一行回到「已开立」状态即可被再次结算，重收不需要任何专门代码，这是状态机闭环的红利。医保单退费伴随冲正与分割回退，冲正失败整体回滚（错误码 5007，与第 10 章教训案例呼应）。

「整单退」与「部分退」是收费模块的经典取舍：部分退费（只退结算单中某几行）对患者更友好，但会派生「一张结算单部分有效」的中间态，医保冲正、发票冲红、日结统计全部要适配行级口径，复杂度陡增。HIP 选整单退——要退某一行，先整单退再重收其余行，用两次简单操作替代一种复杂状态。对窗口收费场景，这个折中是务实的。

7.5.3 联动拦截链：逆向操作的反序原则

正向链条是 挂号 → 开单 → 收费 → 发药/执行，逆向操作必须按反序逐级解除，每一级只负责检查自己的直接下游：

案例框 7-4：三层联动拦截的代码分布

- 退号拦截 (`RegistrationService.java cancel`)：名下存在 `CHARGED` 订单行 → 3006「存在已收费项目，请先退费再退号」；未收费行自动作废，号源释放；
- 退费拦截 (`ChargeService.java refund`)：单内存在 `DISPENSED` 行 → 5004「已发药项目需先退药」；存在 `EXECUTED` 行 → 5005「已执行项目不可退费」——检查已经做了、药已经输了，服务无法逆转，费用自然不可退；
- 退药回退 (`DispenseService.java returnDrug`)：仅 `DISPENSED` 的药品行可退药，库存回补、流水留痕，状态退回 `CHARGED`——此后整单退费的路才被打通。

于是「已发药的处方要退钱」的完整路径是：退药 (`DISPENSED`→`CHARGED`) → 退费 (`CHARGED`→`CREATED`) → (如需) 退号。每一步的拦截条件都只看相邻状态，没有任何一处代码需要通晓全链条——状态机建对了，复杂的业务规则退化为一组局部检查。反例是常见的「退号时联查处方表、发药表、执行表」写法：每接入一个新下游都要回头改退号代码，漏改一处就是资损。

对账视角的补充：结算单状态只有 `PAID/REFUNDED` 两个，退费不删结算单、不删订单行的收费痕迹 (`chargeId` 清空但结算单保留)，日结报表按状态过滤——与第 10 章「留痕只增不减、汇总显式声明状态口径」同一原则。

7.6 聚合支付

7.6.1 聚合支付的一般架构

窗口现金之外，扫码支付已是门诊主流。医院直连微信、支付宝各自的商户接口意味着两套证书、两套回调、两套对账文件，多院区多商户号时组合爆炸，因此行业普遍引入聚合支付层（自建或第三方服务商），对医院侧收敛为一套接口。其通用交互是三段式：

1. 出码：按待收金额向通道下单，取回二维码内容（Native 模式），收银台展示，患者扫码；
2. 回调：支付成功后通道异步通知医院侧，医院验签后完成入账——注意方向反转，医院从调用方变为被调方，必须处理回调乱序、重复通知、伪造报文三类问题，幂等与验签是回调处理的铁律；
3. 对账：次日下载通道对账单与本地支付流水核对——支付是资金业务，与医保一样适用「对账是最后防线」（第 10 章 10.5）。

7.6.2 PayAdapter SPI 与出码流程

与医保适配器同理，支付通道差异收敛到 SPI（Service Provider Interface，服务提供者接口）之后。HIP 的 PayAdapter.java 只有两个方法：

```
public interface PayAdapter {
    record PayResult(boolean ok, String qrContent, String message) {}
    PayResult createPayment(String payNo, BigDecimal amount, String channel); // 出码
    PayResult refund(String payNo); // 退款冲正
}
```

真实聚合支付商实现（下单/回调验签/退款）属外部条件，Mock 实现生成模拟二维码并按第 6 章约定落集成留痕（channel='PAY'，payload 含 pay.create 关键字——对账匹配约定与医保同构）。

案例框 7-5：防重出码与「确认即结算」

扫码支付台的服务层（PayService.java）有两个值得剖析的决策：

防重出码。同一患者已有 PENDING 支付单时拒绝再次出码（错误码 5102「已有待支付二维码，请先支付或作废」）。否则收银台刷新页面就生成一张新码，患者扫了旧码、系统等着新码，资金与单据立刻对不上。作废接口（PENDING → CANCELLED 条件更新）是这条规则的配套出口。

确认即结算，收银逻辑单一入口。支付确认（回调或 Mock 确认）不自己写入账逻辑，而是调用 7.5 的 ChargeService.settle 完成结算，再把支付单置 SUCCESS 并关联结算单号。扫码支付与窗口收费从此共享同一套结算代码——订单汇总、行状态推进、结算单生成、未来的医保联动，只有一份实现。反面教材是支付模块自己再写一遍「汇总订单、改状态、生成单据」：两套收银逻辑的口径漂移只是时间问题。

支付单表 pay_order（V27__phase29_patient_pay.sql）的 charge_id 列在支付成功后回填，形成「支付单 ↔ 结算单」的双向可追溯——这是支付对账（按支付单核通道账）与业务对账（按结算单核日结）能各自独立进行的结构基础。

7.6.3 支付对账

支付对账的口径：本地 pay_order 中 SUCCESS 的支付单 ↔ 通道对账单逐笔核对（单号、金额、时间）。三类典型差异及其含义：

差异形态	常见原因	风险与处置
本地成功、通道无账	回调被伪造、确认接口被绕过验签调用	最危险：医院发了药收了「假钱」。立即冻结相关单据，排查验签实现
通道成功、本地无账	回调丢失或处理异常（掉单）	患者付了钱、单据未结，窗口投诉高发。按通道单补发确认（幂等），掉单率是通道健康度指标
金额不符	部分退款、通道手续费口径混入	核对退款流水，统一「应收金额」口径

处置流程与第 10 章医保对账完全同构：每日自动核对、差异开工单、留痕即依据。真实通道接入后，对账文件的自动下载解析与差异工单化是实施验收项；在此之前，Mock 通道下的对账代码路径同样应当彩排——这是第 10 章「假适配器验证法」在支付域的应用。

7.7 药房发药与审方

7.7.1 发药：原子扣库存

发药的并发问题与占号同构：库存只剩 10 支，两个窗口同时发 8 支，read - check - write 写法会把库存扣成负数。解法同样是条件更新，HIP 的 DrugItemRepository.java：

```
/** 并发安全扣库存：库存足够才扣，返回受影响行数 */
@Query("update DrugItem d set d.stock = d.stock - :qty where d.id = :id and d.stock >= :qty")
int deductStock(@Param("id") Long id, @Param("qty") int qty);
```

案例框 7-6：整方发药的事务语义

发药服务 (DispenseService.java) 按挂号取全部「已收费药品行」，在一个事务内逐行扣库存：任何一行扣减失败（返回 0）即抛 6002「库存不足」，整个事务回滚——已扣掉的前几行自动恢复。这个语义值得强调：处方是临床整体（三联用药缺一不可），「发了两种、第三种没药」的部分发药状态对患者是危险的，对账务是混乱的。事务原子性 + 行级条件更新的组合，让「整方要么全发、要么全不发」不需要任何补偿代码。

每行扣减成功后落库存流水（logOut，带处方号），发药与库存的可追溯由流水而非余额提供——余额只是快照，流水才是账。退药（returnDrug）反向回补库存并落 logReturn 流水。细心的读者会注意到退药的库存回补写的是「读余额—加数量—保存」而非条件更新：回补方向不会把库存变负，超卖风险不存在，但并发退药理论上丢失更新的窗口——这是一个刻意接受的简化，也是本章习题 3 的素材。

7.7.2 审方：状态机与非强制模式的取舍

按《医疗机构处方审核规范》，处方应当经药师审核通过后方可进入收费和调配环节【成稿核对：国卫办医发（2018）14 号及「所有处方均应经审核」的表述】。这在系统上对应强制审方：处方开立后进入待审状态，审核通过才允许收费。但强制模式有现实前提——门诊高峰每分钟数张处方，必须有足够的审方药师坐席（或前置机审软件兜底），否则审方队列会把收费窗口堵死。县市级医院药师人力普遍紧张，一步到位的强制模式常常在上线一周后被行政命令「先关掉」。

HIP 因此实现为非强制模式：审方工作台 (ReviewController.java) 从订单行推导待审队列（DRUG 且 CREATED 且 reviewStatus is null），药师通过/拒绝与收费并行推进，互不阻塞（图 7-3）：

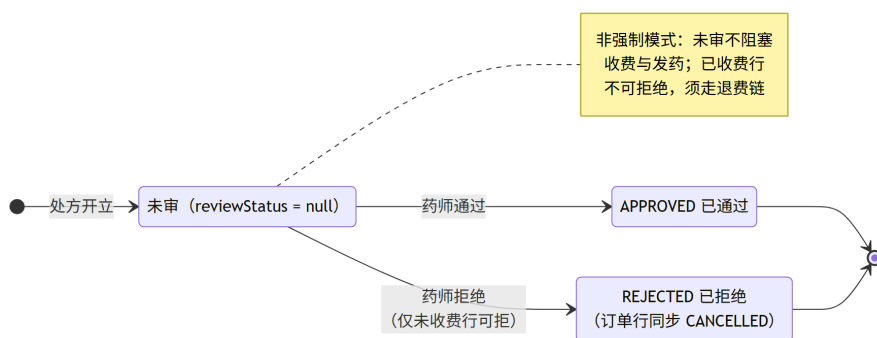


图 7-3 审方状态机（非强制模式）：审方与收费并行，冲突用联动拦截链化解

非强制模式的两个关键闭合点：拒绝即作废——拒绝的处方行直接置 CANCELLED（带拒绝理由与药师 id），不会再被收费；已收费不可拒——行已 CHARGED 时药师拒绝被系统拒绝（4103「已收费处方不能拒绝，请走退费」），冲突交给 7.5.3 的联动拦截链按反序解除，而不是让审方直接改写收费结果。审方状态永远随行保留，事后质控可统计审方率、拒绝率。

从非强制升级到强制，改动点被状态机收敛得很小：收费入口的取行条件从 status = 'CREATED' 收紧为「CREATED 且 reviewStatus = 'APPROVED」（或该行非药品）」，再配一个开关走系统参数（第 5

章 `sys_config`)。把合规模式做成配置而不是分叉，医院药师坐席补齐之日，一个参数完成升级——这是「配置优先」(第14章)在合规演进上的应用。前置的五重拦截(7.4)在两种模式下都在运行，它是审方人力不足时的自动化底线。

7.8 预约与叫号体系

7.8.1 诊间叫号：过号顺延与语音播报

叫号系统的架构选择有两条路：独立叫号系统(专用队列服务器+硬件屏+语音盒，与HIS接口对接)与HIS内建叫号(队列从业务数据推导，大屏就是一个网页)。前者是三甲医院多院区大流量下的常态，后者对县市级医院性价比压倒性——没有第二套系统要同步，没有接口要维护。HIP走内建路线：候诊队列直接从挂号记录推导，今日该科室 `REGISTERED` 的记录按就诊序号(`regNo`，即7.1占号时的序号)升序即候诊队列。叫号(`QueueController.java`)取最小号序置 `CALLED` 并落叫号流水 `outp_call_log`；患者未到则过号(`CALLED` → `PASSED`)移入过号队列——大屏单列显示、可随时重呼(`PASSED` → `CALLED`)。过号语义的要点是「顺延不插队」：过号患者不占候诊队首，重呼由诊台主动发起，避免「叫了三次没人应，后面的病人干等」。大屏轮询队列接口展示候诊/已叫/就诊中/过号四栏；语音播报用浏览器原生 `Web Speech API` 合成「请X号XXX到X诊室就诊」，零外部依赖(`QueueBoardView.vue`)——对县市级医院，「不引入语音盒子等专用硬件」本身就是选型理由。

7.8.2 检查检验时段预约

大型设备(CT、超声、胃镜)是与号源同构的稀缺容量，检查检验申请单收费后应当预约时段而非现场排大队。HIP的 `med_appointment` (`AppointmentController.java`)按「日期+上/下午」分池，池内顺序号自动取当前最大值加一；可预约的前提是「已收费且未预约」(申请单与预约一对一)。改约仅限未叫号(`BOOKED`)的预约，改入新时段排队尾(重取顺序号)；设备故障等场景用批量取消释放整池。时段池没有容量上限校验，是当前实现的一个简化——号源式的 `capacity` 控制是它的自然演进方向。

7.8.3 三队列泛化：队列即视图

案例框 7-7：一个队列引擎服务三个场景

药房取药、检查、检验三个叫号场景由同一个队列中心承载(`QueueCenterController.java`)，其核心设计是队列即视图：系统里不存在「队列表」，等待队列是从业务数据实时推导的查询——

- 药房队列： `outp_order` 中已收费未发药的药品行按处方号聚合(一张处方叫一次号，明细拼接展示)，排除今日已叫过号的处方；
- 检查/检验队列： `med_appointment` 中今日 `BOOKED` 的预约按「时段+顺序号」排序。

三个场景共用一张叫号流水 `queue_call_log` (`biz_type` 区分、 `biz_key` 承载处方号或预约id)，叫号动作=插一条流水(+预约类置 `CALLED`)。队列即视图的收益是零同步成本：收费一笔药费，药房队列「自动」多一个人；退费了，队列「自动」少一个人——因为队列本来就是订单行状态的查询结果，不存在队列表与业务表不一致的可能。代价是每次刷新都是多表联查，队列长度以百计的门诊场景完全可承受；真到了要物化队列表的量级，那是用一致性风险换性能的另一个决策。

7.8.4 皮试与输液：护士站的执行队列

门诊护士站沿同一思路从订单行推导输液执行队列(已收费/已发药且用法含「静」的药品行)，输液单状态 `PENDING` → `RUNNING` → `DONE`，巡视记录逐条留痕；青霉素类、头孢类输液前校验存在阴性皮试记录方可执行。皮试结果一经录入(NEG/POS)不可改(条件更新 `result = 'PENDING'` 才允许写入)——医疗记录的「只写一次」纪律在最小的表上同样成立。

7.9 急诊预检分诊与留观

7.9.1 预检分诊：先分级，后挂号

急诊的第一原则是病情优先于流程：患者到达先由分诊护士评估分级，危重者直接进抢救，挂号收费都可以事后补办。国内急诊预检普遍采用四级分级（I级濒危、II级危重、III级急症、IV级非急症），依据主诉、生命体征与高危指征判定【成稿核对：急诊预检分诊分级的现行行业标准/专家共识名称】。信息系统对应两个特殊设计：

- 弱依赖建档：分诊记录的 `patient_id` 可空、姓名必填但接受「无名氏编号」——昏迷无名患者先分诊救治，身份后补（`TriageController.java` 只强制姓名、1-4级、主诉三项，数据库另有 `chk_triage_level` 约束兜底）；
- 分级队列：分诊列表按级别升序、时间降序排列（`findTop100ByOrderByLevelAscIdDesc`）——I级永远置顶，同级内新到优先展示。这与门诊按号序的公平队列形成有意的对比：门诊排队讲先来后到，急诊排队讲病情轻重，队列的排序键就是业务价值观。

分诊记录带全套生命体征列（体温/脉搏/呼吸/血压/血氧），状态沿 `TRIAGED → IN_TREATMENT → ADMITTED/DISCHARGED` 推进，去向可以是收入院（衔接第8章）或离院。分级不仅决定排序，还对应响应时限要求（如I级即刻处置、II级数分钟内、III/IV级按序候诊【成稿核对：各级具体时限的标准表述】）——分诊时间在系统中自动留痕，「分诊到接诊」的时间差因此可统计，是急诊质控（第12章）的核心指标之一；胸痛、卒中等绿色通道的D2B/DNT时间轴同样以分诊时间为起点。

7.9.2 急诊留观

介于「看完就走」与「收入院」之间的是留观：患者占用留观床持续观察数小时至数十小时。HIP的留观（`ErObservationController.java`）以分诊记录为根：入观校验三件事——分诊记录存在、该患者未在观、留观床未被占用；在观期间护士逐条追加观察记录（仅 `IN` 状态可写）；离观必须登记去向（离院/入院/转院三选一），置 `OUT` 并记时间。

留观占床的实现是一个很好的并发控制反面对照：它用的是「count 查询检查床位空闲，再 insert」两步——与7.1批判过的 `read-check-write` 同构，两个患者并发抢同一张留观床理论上都能通过检查。为什么这里可以接受？急诊留观床十余张、入观由护士站单点操作，冲突概率与后果都远小于全渠道并发的挂号。并发控制的强度应当与竞争激烈度匹配，不加区分地处处上锁与处处不设防同样是错误。当然，更稳的做法成本也不高：给 `er_observation` 建一个 `where status = 'IN'` 的部分唯一索引（PostgreSQL `partial unique index on bed_no`），把约束沉到数据库——这是习题7的起点。住院正式床位的占用则是高价值强竞争资源，第8章会看到它按占号模式做了原子占床。

本章小结

- 挂号的本质是容量分配。「排班—号源—挂号记录」三层模型下，防超挂是并发控制问题：判断简单的用条件更新（`UPDATE ... WHERE booked < capacity`，占号、扣库存），判断复杂的用 `SELECT FOR UPDATE` 行锁（医保年度额度，第10章）——同一模式的两个变体，数据库检查约束作最后防线。
- 统一订单行把处方/检查/检验/治疗（乃至挂号费）收敛为同构的行，一行同时是业务明细与费用明细；收费、发药、执行、叫号全部是订单行状态机上的推进，队列全部是订单行的视图。
- 合理用药五重拦截（过敏交叉/重复/DDI/年龄/抗菌分级）在开单落库前完成，`FORBID/CAUTION` 分级是与临床的信任契约；规则表化保证药剂科可维护、实施可替换，商用知识库到位即灌入——管道自建、内容外采。

- 收费的结算/退费构成状态闭环：退费把行退回 `CREATED`，重收零成本。逆向操作按正向链条反序逐级解除（退药→退费→退号），每级只检查相邻状态，联动拦截自然涌现。
- 聚合支付三段式（出码/回调/对账）收敛到 `PayAdapter SPI` 之后；防重出码、确认即结算（复用统一收银入口）是扫码台正确性的两个支点。
- 发药在一个事务内逐行原子扣库存，整方要么全发要么全不发；审方取非强制模式——审方与收费并行、拒绝即作废、已收费不可拒，升级强制模式只需收紧收费取行条件加一个配置开关。
- 急诊分诊「先分级后挂号」：弱依赖建档容纳无名氏，队列按级别优先排序；留观占床采用与竞争烈度匹配的轻量检查。并发控制的强度是设计决策，不是教条。

思考与练习

1. 概念：解释「排班」「号源」「挂号记录」三者的关系。号源余量校验写在应用层 `if (booked < capacity)` 里为什么防不住超挂？条件更新方案中，数据库靠什么机制保证两个并发事务只有一个成功？
2. 对比：7.1 的占号用条件更新，第 10 章的年度额度用 `SELECT ... FOR UPDATE`。分析两个场景的判断逻辑差异，说明为什么占号不需要行锁、额度累计却需要；再给出一个必须用行锁而条件更新无法胜任的门诊场景。
3. 并发：案例框 7-6 指出退药的库存回补是「读余额—加数量—保存」，存在丢失更新窗口。构造一个两笔并发退药导致库存少补的时序图；分别用条件更新和 JPA 乐观锁给出修复方案，并比较改动量。
4. 设计：HIP 采用整单退费。若医院要求支持部分退费（退结算单中指定行），列出需要改动的模块与新增的中间状态，重点分析医保冲正（第 10 章）与日结报表的口径如何适配。
5. 推演：某医师（抗菌药处方权 1 级）为一名 16 岁、过敏史含「青霉素」的患者开具：阿莫西林胶囊、左氧氟沙星片、布洛芬缓释胶囊 7 天。按 7.4 的五重拦截与疗程规则，写出系统逐项的拦截/提醒结果（含错误码级别），并说明拦截发生时该组医嘱的落库结果。
6. 论述：药师审方的强制模式与非强制模式各自的合规风险与运营风险是什么？某院药师坐席已补齐，计划切换强制模式，请给出切换方案：改动点、灰度策略（按科室？按药品类别？）、回退预案，以及切换后审方队列积压的监控指标。
7. 实践（配套实训环境）：为急诊留观床加上数据库级防重占：编写 Flyway 迁移为 `er_observation` 创建 `bed_no` 上 `where status = 'IN'` 的部分唯一索引，改造入观接口将唯一约束冲突翻译为业务错误码 4562，并写一个两线程并发入观同一床位的测试证明修复有效。
8. 综合：参照图 7-1，画出你所在（或熟悉）医院门诊流程的数据流图，标出哪些环节存在「同一笔费用落两处账」或「队列表与业务表需要同步」的设计，并按本章的模型给出收敛方案。

第 8 章 住院与护理

本章属第三篇「业务篇」。住院是医院资源密度最高、执行链条最长的业务域：一张床位从占用到释放，中间穿过医嘱、护理执行、病历文书、风险评估与出院结算，任何一环脱扣都会变成费用差错或医疗安全事件。本章从床位这个“第一资源”讲起，依次展开记账式医嘱、电子病历与 CA 签名、护理执行闭环、风险评估量表、出院结算的完整性校验，最后讨论移动形态与 ICU/临床路径两个专科场景。全章以 HIP 平台的住院模块为贯穿案例。

学习目标

学完本章，你应当能够：

1. 解释床位为什么必须建模为独占资源，用条件 UPDATE 写出原子占床语句，并说明它与号源占用、库存扣减同属一个并发控制家族；
2. 区分长期医嘱与临时医嘱，说出医嘱计费时点的三种行业方案（开立计费/执行计费/发药计费）及各自的差错形态；
3. 论证“记账式医嘱行 = 费用行”这一模型选择如何成为医保行级分割（第 10 章）的上游前提；
4. 描述住院病历的文书体系与时限要求，解释 CA 电子签名的“冻结”语义，说出签名适配器 SPI 的接口契约；
5. 画出“开立→执行→出院拦截→结算”的医嘱生命周期状态机，解释未执行医嘱为什么必须拦截出院；
6. 说出 Braden、Morse 两个护理风险评估量表的评估对象、分值区间与分级阈值，设计评估趋势与高危预警的数据结构；
7. 比较原生 App、PDA 专用程序与 H5 三种移动护理形态的取舍，说明“复用同一套 API”对移动端建设成本的意义。

8.1 住院登记与床位管理

8.1.1 床位：住院业务的第一资源

门诊的核心资源是号源，住院的核心资源是床位。两者的共性是独占性：一个号只能挂给一个人，一张床同一时刻只能住一个患者。差别在生命周期——号源以“天×医生”为单位批量生成、当天作废；床位是长期存在的实体资产，状态随入院、转科、出院反复翻转，还要挂靠病区（护理单元）的组织结构。

床位建模的通用要点：

1. 床位挂病区，不挂科室。住院患者有两个组织归属：科室（医疗归属，决定主管医生和费用归口）与病区（护理归属，决定哪个护理单元管这张床）。小医院常常一科一区，二者合一；但模型上必须分开——外科病区加床收内科患者（跨科收治）是真实存在的场景，合并建模的系统遇到它只能造假数据。
2. 床位状态机要小。空闲/占用是必须的，扩展态（预约、消毒、维修）按需加，不要预设一个庞大状态机——每多一个状态，所有读写床位的代码都多一个分支。
3. 占床必须防并发。两个住院处窗口同时把患者办进同一张空床，是真实会发生的竞态。

第三点是本节的重心。第7章的号源占用用“条件 UPDATE + 受影响行数判定”解决了防超挂，床位是同一问题的变体：把“还有余号才自增”换成“是空床才占用”。

案例框 8-1: HIP 的原子占床

HIP 的床位表 `inp_bed` 只有五个字段：病区、床号、状态、当前住院 `id`，加一条 `unique (ward_id, bed_no)` 约束（V8_inpatient.sql）。占床是仓储层的一条条件 UPDATE（BedRepo.java）：

```
/** 原子占床：只有空床才占 */
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("update InpBed b set b.status = 'OCCUPIED', b.admissionId = :admissionId " +
        "where b.id = :bedId and b.status = 'FREE'")
int occupy(@Param("bedId") Long bedId, @Param("admissionId") Long admissionId);
```

并发安全完全由数据库保证：两个事务同时执行这条 UPDATE，行锁使它们串行化，后到者的 WHERE 条件（`status = 'FREE'`）不再成立，返回受影响行数 0。调用方（`InpatientService.admit`）据此抛业务异常“床位已被占用”，整个入院事务回滚——先落库的住院档案记录也一起消失。不需要悲观锁、不需要分布式锁、不需要重试，一条 SQL 的语义就是完整的并发控制。

这与第7章号源占用（`booked < capacity` 才自增）、第7章药房发药和本章 8.4 的库存扣减（`stock >= qty` 才减）是同一个模式的三次实例化。值得把它总结成一条设计定式：独占或限量资源的获取，一律写成“条件更新 + 判定受影响行数”，把竞态裁决权交给数据库行锁。反例是“先 SELECT 检查再 UPDATE”的两步写法——检查与更新之间的窗口期就是超卖事故的温床。

入院登记（`admit`）的编排顺序也有讲究：先保存住院档案拿到自增 `id`，用 `id` 生成住院号（`ZY + 日期 + 序号`），再占床——占床失败时档案随事务回滚，不留孤儿记录；反过来先占床再建档，建档失败就需要补偿性放床，多出一条失败路径。

8.1.2 押金：后付制下的信用垫

门诊是“先缴费后执行”，住院是后付制——费用在住院期间持续发生，出院时一次结清。后付制的信用风险由预交金（押金）缓冲：入院时收首笔，住院中费用逼近押金余额时催缴续交。押金在会计上是负债（预收款），不是收入，出院结算时冲抵费用、多退少补。

HIP 的 `inp_deposit` 表按笔记录押金（金额、支付方式、操作员），只增不改——每笔押金是一条独立流水，退补发生在结算环节而不是修改历史记录。这是财务类数据的通用纪律：流水永远只追加，余额永远由汇总计算（与第10章“留痕只增不减”同源）。

8.1.3 转科换床：先占后放

转科是住院期间的组织归属变更，涉及床位交换。HIP 的 `transfer` 实现顺序值得注意：先原子占新床，成功后才释放旧床，二者在同一事务内。反过来先放旧床再占新床，一旦新床被抢，患者就“悬

空”了——没有任何床位归属。“先取得新资源、再释放旧资源”是资源交换的通用顺序（对比：数据库迁移先建新列再删旧列，发布工艺先起新实例再下线旧实例）。

转科全程落 `inp_transfer_log`（从科室/到科室/从床/到床/操作员），这张留痕表既是床日统计的数据源，也是费用归属争议时的仲裁依据。

8.2 记账式医嘱体系

8.2.1 长期医嘱与临时医嘱

医嘱（Physician Order）是住院诊疗的驱动核心。临床上分两类：

- 长期医嘱：有效期跨天，按频次反复执行（“头孢呋辛 1.5g 静滴 bid”），直到停止医嘱下达；
- 临时医嘱：一次性执行（“急查血常规”“明晨禁食”），执行完即结束。

两类医嘱的信息化含义完全不同：临时医嘱是“一条医嘱一次执行”，模型简单；长期医嘱是“一条医嘱 N 次执行”，需要医嘱拆分机制——按频次把长期医嘱展开为每日执行条目（业内称“转抄”或“摆药单生成”），执行记录与医嘱是一对多。长期医嘱还有配套的停嘱流程：停止时间之后的未执行条目要作废。

医嘱的另一个结构概念是成组：几种药品混在一组液体里静滴，临床是一个动作，医嘱是多行记录，用组号绑定——执行、停嘱、退费都以组为单位。

8.2.2 医嘱计费时点：行业的三种选择

医嘱行何时变成费用行，行业有三种方案，差错形态各不相同：

方案	计费时点	优点	风险
开立计费（记账式）	医生开立医嘱即产生费用	费用最完整，不漏记	开而未执行会多收——须有拦截或冲销机制
执行计费	护士执行时产生费用	费用与实际执行一致	执行漏记账即漏费，追补困难
发药计费	药房摆药/发药时计费	与药品实物流一致	只覆盖药品，非药品项目仍需前两者之一

没有无代价的选项，选择的实质是把差错押在哪一侧：开立计费押“多收”（可通过拦截未执行医嘱消除），执行计费押“漏收”（漏掉就很难发现）。多收有患者和医保盯着，漏收只有年底财务对不上账时才浮现——可被外部发现的差错优于静默差错，这是多数 HIS 采用记账式的深层理由。

8.2.3 记账式医嘱行：费用的最小粒度

记账式的关键设计是让医嘱行本身携带完整的计费信息——项目编码、名称、单价、数量、金额，医嘱行就是费用行，不再另设计费表。这个选择的价值要到下游才充分显现：医保行级分割（第 10 章 10.2.4）要求住院费用逐行归类，记账式医嘱天然满足——出院结算时把已执行医嘱行直接装配成分割入参即可。上游模型多存几个冗余字段，下游整条业务线不用返工；反之，医嘱只存 `item_id`、费用另算总额的系统，接 DRG 时要伤筋动骨地补行级数据。

案例框 8-2：HIP 的医嘱行快照

HIP 的 `inp_order` 表（`V8_inpatient.sql`）每行冗余存储 `item_code / item_name / spec / unit / unit_price / amount`，开立时从药品或收费项目字典快照进来（`InpatientService.java` 的 `createOrders`）：

```

if ("DRUG".equals(line.orderType())) {
    DrugItem drug = drugRepository.findById(line.itemId()).orElseThrow(...);
    o.setItemCode(drug.getCode());
    o.setItemName(drug.getName());
    o.setUnitPrice(drug.getPrice());
    o.setUsageRoute(line.usageRoute()); // 药品医嘱特有：途径/频次/单次剂量
    o.setFrequency(line.frequency());
    o.setDosePerTime(line.dosePerTime());
} else { ... /* 收费项目同理 */ }
o.setAmount(o.getUnitPrice().multiply(BigDecimal.valueOf(o.getQty())));

```

快照而非外键引用当前价，理由是历史不随字典漂移：药品明天调价，昨天开的医嘱金额不能变——费用、发票、医保上传都已按旧价发生。字典表存“现在的真相”，业务行存“当时的真相”，这是所有交易类系统的通用纪律。

每条医嘱带 `group_no`（YZ + 日期 + 序号）承载成组语义；库存出库留痕（8.4）也以组号为关联键。需要说明边界：HIP 当前的医嘱模型是临时医嘱形态（一条医嘱一次执行），长期医嘱的按频次拆分尚未实现——对以静滴为主的县级医院演示场景够用，但实施于真实病房前，长期医嘱拆分是必须补齐的能力，这也是本书刻意保留的“模型如何生长”的观察点。

医嘱的生命周期状态机如图 8-1。注意它比门诊订单（第 7 章：开立→收费→执行）少一个收费态——记账式下开立即记账，不存在“待收费”中间态；执行的把关移到了出院结算的拦截（8.7）。

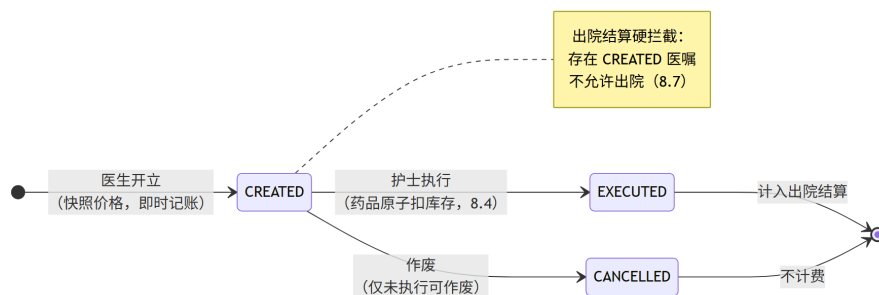


图 8-1 住院医嘱生命周期状态机

8.3 住院电子病历与 CA 签名冻结

8.3.1 病历文书体系与时限

住院病历不是一份文档，而是一个文书家族：入院记录、首次病程记录、日常病程、上级医师查房记录、出院小结（死亡记录）等，各有法定书写时限——按《病历书写基本规范》，入院记录须于入院后 24 小时内完成，首次病程记录须于入院后 8 小时内完成【成稿核对：卫医政发〔2010〕11 号具体时限条款】。时限不是建议而是质控红线，电子病历系统应把它做成可自动核查的规则而非依赖人工抽查。

HIP 的 `inp_medical_record` 用 `record_type`（ADMISSION/PROGRESS/DISCHARGE）区分文书类型（`InpEmrController.java`）；时限质控接口据此自动清点“入院超 24 小时无入院记录”“已出院无出院小结”的缺陷清单（第 12 章将展开质控体系）。文书类型字段的价值正在于此：病历若只是一堆无类型的文本，时限质控就无从下手——结构化的第一步不是把内容拆成字段，而是把文书本身分类。

8.3.2 电子签名：从“能改”到“冻结”

纸质病历靠手写签名固定责任，电子病历的对应物是电子签名。《中华人民共和国电子签名法》确立了基本框架：可靠的电子签名与手写签名或盖章具有同等的法律效力，“可靠”的要件包括签名制作数据由签名人专有控制、签署后对签名及数据电文的任何改动能够被发现【成稿核对：电子签名法第十三、十四条表述】。落到医院场景，行业通行做法是接入 CA（Certificate Authority，数字证书认证机构）：医生持数字证书（USB Key 或手机协同证书），对病历内容摘要做非对称加密签名，第三方可用公钥验签。

对系统设计者，电子签名的核心不是密码学，而是冻结语义：

1. 签名对象是内容快照——对参与签名的字段拼接摘要，签的是“此刻的这份内容”；
2. 签名后禁止修改——已签名病历的任何编辑请求都必须拒绝，修改须走“废弃重签”或版本化的修订流程（保留原版本与修订痕迹）；
3. 验签可追溯——存签名值与签名时间，事后可证明“内容未被篡改”。

没有冻结的签名是自欺：签完还能改内容，签名值与内容脱钩，法律效力无从谈起。

案例框 8-3: SignatureAdapter SPI 与 Mock 的诚实边界

CA 产品是外部条件（须采购厂商服务，如 CFCA/BJCA），HIP 按第 6 章的适配器模式将其收敛为两个方法的 SPI（SignatureAdapter.java）：

```
public interface SignatureAdapter {
    record SignResult(boolean ok, String signature, String message) {}
    SignResult sign(String content, Long signerId); // 对内容摘要签名，返回 Base64 签名值
    boolean verify(String content, String signature); // 验签
}
```

签名动作的业务侧编排（DoctorStationService.signEmr，DoctorStationService.java）完整体现冻结语义：已签名者拒绝重签（“病历已签名”）；把主诉、现病史、既往史、查体、处置意见按固定顺序拼接为签名内容；签名成功才写入 signature 与 signed_at；此后一切编辑入口检查签名列非空即拒绝。

Mock 实现（MockSignatureAdapter.java）用 SHA-256 摘要模拟签名值，其类注释值得抄录：“内容防篡改可验，无法证明签名人身份——生产须换 CASDK”。这句话是 Mock 工程的样板：摘要能提供完整性（内容改动可被发现），但人人都能算摘要，不具备“签名人专有控制”要件，因此不构成法律意义的电子签名。把 Mock 能做到什么、做不到什么写进代码注释，实施时才不会把演示能力当成合规能力交付。

一个对读者有教学价值的诚实披露：HIP 当前的签名冻结闭环落在门诊病历（outp_emr 的 signature/signed_at 列，V12 迁移）；住院病历表 inp_medical_record 尚未挂签名列，属同一 SPI 下的待接扩展点——住院文书是多类型、多份记录，逐份签名的交互与“归档时批量补签”的流程设计比门诊单份病历复杂，这正是它排在后面的原因。功能规划与代码现状的这类差距，在真实项目中普遍存在，管理它的方法（功能清单标注外部条件、验收时逐条核对）见第 14 章。

8.4 护理执行闭环

8.4.1 从医嘱到床旁：闭环管理

“医嘱闭环管理”是电子病历评级（第1章）的高频考点，含义是医嘱从开立到执行的每个环节都有记录、可追溯：开立→审核→（转抄/摆药）→执行→复核。给药场景还叠加“三查七对”与给药安全的5R原则（正确的患者、药品、剂量、途径、时间）。信息系统对闭环的贡献在两点：每个环节留下执行人与时间戳；用系统校验替代部分人工核对（如扫码核对腕带与药袋）。

8.4.2 执行动作的三重联动

护士点击“执行”的瞬间，系统要同时完成三件事：医嘱状态翻转、库存扣减、出库留痕——三者必须在同一事务内，任何一个失败全部回滚。

案例框 8-4：执行即扣库存

HIP 的执行实现（InpatientService.java 的 execute）：

```
@Transactional
public InpOrder execute(Long orderId, Long executorId) {
    InpOrder o = orderRepo.findById(orderId).orElseThrow(...);
    if (!"CREATED".equals(o.getStatus())) {
        throw new InpException(9009, "仅未执行医嘱可执行");
    }
    if ("DRUG".equals(o.getOrderType())) {
        if (drugRepository.deductStock(o.getItemId(), o.getQty()) == 0) {
            throw new InpException(9010, "库存不足: " + o.getItemName());
        }
        inventoryService.logOut(o.getItemId(), o.getQty(), o.getGroupNo(), executorId);
    }
    o.setStatus("EXECUTED");
    o.setExecutorId(executorId);
    o.setExecutedAt(Instant.now());
    return orderRepo.save(o);
}
```

deductStock 是又一条“条件更新 + 判定行数”（stock >= qty 才减，见 DrugItemRepository.java），与 8.1 的占床共用同一并发定式——两个护士同时执行最后一支药，只有一人成功。出库留痕以医嘱组号为业务关联键，进销存对账（第2章药事流程）由此可回溯到“哪条医嘱消耗了这支药”。

两处状态防线也值得注意：入口的 CREATED 校验防重复执行（重复扣库存、重复计费）；executorId 与 executedAt 使每次执行可追责——闭环管理的“环”就是由这些执行人+时间戳串起来的。

执行闭环全景如图 8-2：

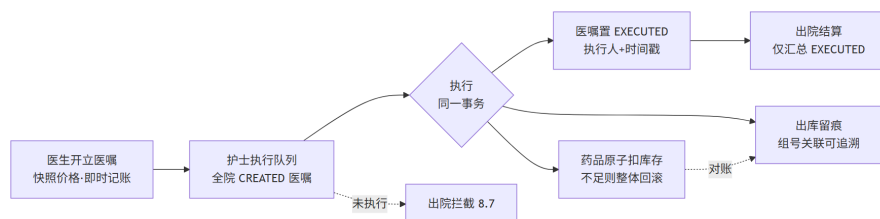


图 8-2 护理执行闭环：一次执行动作的三重联动

8.4.3 生命体征与体温单

生命体征（体温、脉搏、呼吸、血压、血氧）是护理数据中最规整的时间序列。纸质时代的载体是三测单（体温单）——以折线呈现体温脉搏走势的经典图表，任何护士看一眼就能读出趋势。电子化的正确做法不是发明新展现，而是复刻这张临床人人会读的图：HIP 的 `inp_vital_sign` 表按次存储六项体征（V9__inp_emr_vitals.sql），前端按测量时间正序取数画体温单折线。模型上的要点只有一个：测量时间（`measured_at`）与录入时间分离——补录昨晚的体温，曲线上的点必须落在昨晚。

8.5 护理风险评估

8.5.1 量表：把临床判断变成可比的数字

护理风险评估用标准化量表把“这个患者危不危险”变成分数，两个最常用的：

- **Braden** 量表评估压力性损伤（压疮）风险：从感知能力、潮湿、活动能力、移动能力、营养、摩擦与剪切力六个维度打分，总分 6 - 23，分数越低风险越高；
- **Morse** 量表评估跌倒风险：跌倒史、多重诊断、行走辅助、静脉输液、步态、认知六项，总分 0 - 125，分数越高风险越高。

两个量表方向相反（一个低分危险、一个高分危险），是系统实现时最容易写反的地方。评估不是一次性动作：入院、转科、手术后、病情变化都应重评，因此数据结构必须支持同一患者多次评估构成趋势；高危患者要触发护理干预（防压疮气垫、防跌倒标识），系统侧的对应物是高危预警清单。

8.5.2 阈值分级与预警清单

案例框 8-5：HIP 的风险评估实现

HIP 的实现（`RiskAssessController.java`）把分级阈值写在服务端：**Braden** ≤ 12 高危、13 - 14 中危、 ≥ 15 低危；**Morse** ≥ 45 高危、25 - 44 中危、 < 25 低危【成稿核对：阈值与文献通行分档的一致性】。三个工程细节：

- 范围校验在服务端（**Braden** 6 - 23、**Morse** 0 - 125 之外直接拒绝）——前端校验挡不住接口直调与集成调用，凡是有临床含义的取值范围，服务端必须复核；
- 只收总分不收分项是一个明确的取舍：分项得分对护理质控有价值（哪个维度拉低了分数），但录入负担翻倍。HIP 选择先落总分把闭环跑通，分项留给量表结构化的后续迭代——量表种类繁多（还有 GCS、疼痛评分、VTE 评估等），逐一结构化应由实施需求驱动而非预先大而全；
- 高危预警清单用 PostgreSQL 的 `distinct on` 取每个在院患者每类量表的最近一次评估，过滤出高危者，联出床位与科室——“最近一次”是语义要点：患者三天前高危、昨天复评已降级，就不该再出现在预警清单上。评估历史全量保留（趋势接口按时间正序输出，供前端画风险变化曲线），清单只看最新，这又是“留痕全量、口径过滤”原则的一次出场。

8.6 护理白板与交接班

护理白板的原型是病区走廊里那块真实的白板：床号、姓名、护理级别、过敏、注意事项。它的信息化价值不在存储而在聚合——把散在各表的在院信息拼成护士站一眼可扫的总览。HIP 的白板接口（`NursingQualityController.java`）用一条 SQL 联五张表：住院档案、床位、病区、科室、患者，每行附带一个关键的子查询——该患者未执行医嘱数。这个数字让白板从“静态名单”变成“工作驱动器”：哪张床还有活没干，一眼可见。

护理级别（特级、一级、二级、三级）由医嘱确定，决定巡视频次与护理投入【成稿核对：《综合医院分级护理指导原则》及 WS/T 431 现行版本】。HIP 把它存为 `inp_admission.care_level`，修改

接口做白名单校验（只接受四个合法值）——枚举类临床属性宁可硬校验，也不要留自由文本，否则统计时会收获“I级/一级/1级”三种写法。

交接班是护理连续性的制度保障，行业推荐 SBAR 结构（现状 Situation、背景 Background、评估 Assessment、建议 Recommendation）。HIP 的 `shift_handover` 表做了最小建模：科室、班次类型、交班摘要、待办事项、记录人——把“待办”从摘要中单列出来，是让接班者第一眼看到“我要做什么”，这一列的存在比任何格式规范都更接近 SBAR 的本意。

8.7 出院结算

8.7.1 结算的完整性问题

出院结算是住院费用的总闸门：汇总全程费用、冲抵押金、多退少补。它比门诊收费多一个独有的难题——完整性。门诊是先缴费后执行，收费那一刻费用天然完整；住院费用持续发生，结算时刻系统里可能还躺着开立未执行的医嘱。这时结算面临两难：把未执行医嘱计入费用，是收了没提供的服务（违规）；不计入，若护士随后补执行，费用就漏掉了（漏收），且医保分割已按旧口径上传，无法追补。

正确答案是两者都不选：拦截。存在未执行医嘱就不允许结算，强制临床先把医嘱处理干净——该执行的执行、不做了的作废。拦截把“结算时刻费用集合已封闭”变成一个系统保证的不变式，后续一切计算（费用汇总、押金冲抵、医保分割）都站在这个不变式上。

案例框 8-6: HIP 的出院结算编排

`discharge` 方法（`InpatientService.java`）的关键序列：

```
List<InpOrder> orders = orderRepo.findByAdmissionIdOrderByIdAsc(admissionId);
boolean hasPending = orders.stream().anyMatch(o -> "CREATED".equals(o.getStatus()));
if (hasPending) {
    throw new InpException(9012, "存在未执行医嘱，请先执行或作废");
}
BigDecimal total = orders.stream()
    .filter(o -> "EXECUTED".equals(o.getStatus()))
    .map(InpOrder::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

拦截之后，汇总仍显式只取 `EXECUTED`（作废医嘱不计费），押金按 `inp_deposit` 流水求和，`balance = 押金 - 总费用`（正数退、负数补）。医保患者（`payMethod=YB`）在同一事务内追加两步：把已执行医嘱行装配为 `YbLine` 调用统一分割引擎（`splitAndAudit("INP", ...)`——8.2.3 铺垫的行级前提在此兑现），再经 `InsuranceAdapter` 上传结算，失败即抛错、整个出院回滚；成功则回填 `yb_settle_no`（第10章那个“教训案例”的修复成果，此处是它的住院侧落点）。最后才翻转住院状态为 `DISCHARGED` 并释放床位。

两道容易被忽略的防线：`inp_settlement` 表对 `admission_id` 建了 `unique` 约束——重复结算在数据库层面被封死，即使代码防线失守（并发双击、重放请求），第二条结算单也插不进去；结算单号 `CY + 日期 + 住院 id` 具备幂等友好性。校验在代码、约束在数据库，双层防线缺一不可：代码防线给出友好报错，数据库约束保证最坏情况下数据不烂。

结算流程全景如图 8-3:

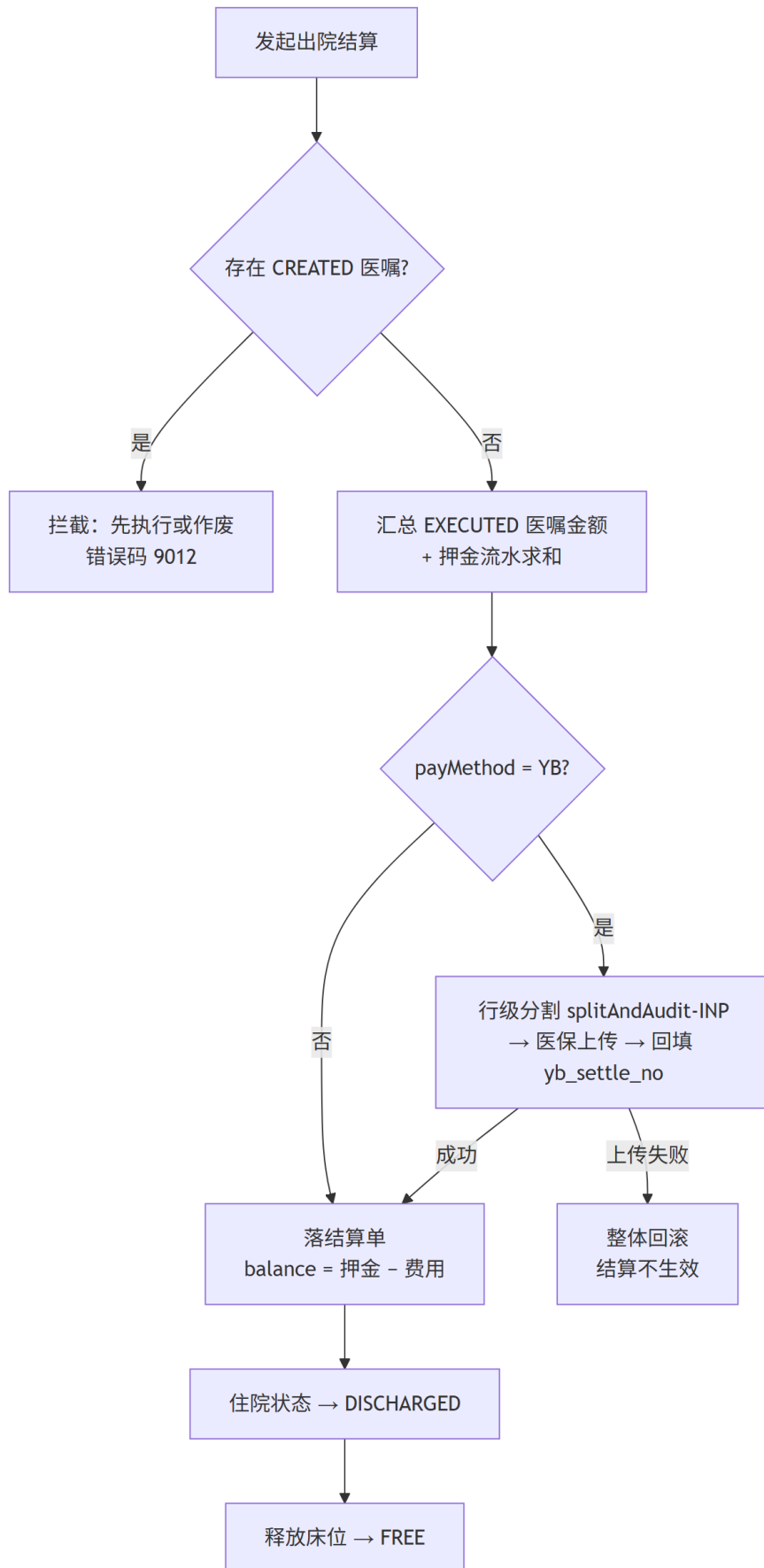


图 8-3 出院结算流程：拦截、汇总、医保联动与资源释放

8.7.2 与医保的接缝

出院结算是第8章与第10章的交汇点，三条依赖在此闭合：行级医嘱（8.2）供给行级分割（10.2.4）；执行拦截（8.7.1）保证分割输入的费用集合封闭；同事务上传+结算号回填沿用门诊结算的强一致语义（10.3）。读者可以体会“模块边界”与“业务链条”的关系：模块按领域切分（住院/医保），但正确性是沿链条整体成立的——上游少一个字段，下游多一次返工。

8.8 移动护理与移动查房

移动护理的经典形态是 **PDA+扫码**：护士持专用手持机，扫患者腕带与药袋条码完成身份核对、床旁执行、体征录入。它是给药安全闭环的完全体，但成本不菲：专用硬件采购与维护、原生应用的开发与升级分发、病房无线网络的全覆盖改造。

县级医院起步阶段的务实替代是 **H5 移动页**：手机浏览器直接访问，零安装、零分发、随系统同步升级。代价是拿不到扫码硬件能力（浏览器调用摄像头识码的体验与可靠性远逊于专业扫码头），身份核对退回人工。

HIP 选择了 H5 形态（`MobileWorkView.vue`），其实现方式比形态本身更值得记录：移动工作台没有一个专属后端接口。移动护理的体征录入调用的是 8.4 的 `/vitals` 接口，移动查房写病程调用的是 8.3 的 `/records` 接口（`recordType=PROGRESS`），在院列表复用住院列表接口——移动端只是同一套 API 的另一个视图（单列卡片、抽屉式详情、480px 宽度约束）。这带来两个直接收益：移动端永远不会与桌面端数据不一致（同一接口无所谓一致性问题）；后端零增量，移动端的全部成本就是一个 Vue 页面。“移动优先”对中小医院的正确解读不是另建一套移动系统，而是让既有 API 天然可被移动端消费。当然也要诚实标注边界：无扫码核对的 H5 移动护理，承载的是“录入便利”而非“给药安全闭环”，二者不可混称——向院方汇报能力清单时，这类措辞的分寸就是信任的分寸。

8.9 专科场景：ICU 与临床路径

8.9.1 ICU：同一类数据的另一种密度

ICU 的护理记录与普通病房在数据模型上同构、在密度与维度上不同：体征从每日两三测变成每小时甚至每 15 分钟一记；在常规六项之上增加 **GCS**（Glasgow Coma Scale，格拉斯哥昏迷评分，睁眼+语言+运动三项合计 3-15 分，越低越危重）、**24 小时出入量**（入量：输液/饮水；出量：尿量/引流，出入平衡是循环与肾功能管理的核心指标）与呼吸机标记。

HIP 为此单建 `inp_icu_record` 表（`V21_phase24_specialty.sql`、`IcuRecordController.java`）而不是往 `inp_vital_sign` 上加列——普通病房的体征录入界面不该被 GCS、出入量这些用不到的字段拖累，按使用场景分表比按数据相似性合表更利于两边各自演进。表上有两个可抄的细节：**GCS** 的取值范围同时做了服务端校验与数据库 CHECK 约束（`chk_icu_gcs`，3-15），又一次双层防线；**24 小时出入量**汇总用 `recorded_at > now() - interval '24 hours'` 的滚动窗口而非自然日——凌晨 3 点查房间的是“过去 24 小时”，不是“今天零点以来”。

8.9.2 临床路径：把诊疗过程模板化

临床路径（Clinical Pathway）是对特定病种制定的标准化诊疗流程：第几天做什么检查、用什么药、达到什么标准出院。它的管理价值在变异分析——实际诊疗偏离路径（超期、退出）的记录与统计，既是医疗质量指标，也是 DRG 时代成本管控的抓手（路径内的病例费用可预期）。

案例框 8-7：入径即开单，变异即留痕

HIP 的路径实现 (PatientCareController.java) 由三张表支撑: 模板 cp_pathway_template (病种名、ICD 前缀、医嘱项列表存为 JSON——复用 8.2 的 OrderLine 结构)、入径 cp_enrollment、变异 cp_variance (V28_phase30_medrecord_clinical.sql)。

入径动作 (applyPathway) 的实质是按模板批量调用 createOrders——路径不是独立于医嘱的另一套体系, 而是医嘱的“预制菜”: 模板里存的就是医嘱行参数, 入径展开成真实医嘱后, 执行、计费、拦截全部走既有闭环, 路径模块自身不需要任何执行逻辑。cp_enrollment 的 unique (admission_id, template_id) 防重复入径。

变异登记分三型 (DELAY 超期 / EXIT 退出 / OTHER), 其中 EXIT 型变异同步把入径状态翻转为 EXITED——退出路径必有原因, 所以“退出”不做成独立操作, 而是变异登记的副作用, 保证退出必留因。统计接口输出各路径的完成率、退出率、变异率 (有变异记录的入径占比), 这三个比率就是路径管理向医务科汇报的核心口径。

模板 JSON 化是又一处“简化结构而不简化语义”: 正式的路径定义有按天分阶段、可选项、评估节点, HIP 的平铺医嘱列表是它的最小可用子集——模板结构可以后续升级, 而入径留痕、变异分析这些管理闭环从第一天就是完整的。

住院域还有两个专科流程——手术麻醉 (申请/麻醉记录单/PACU 苏醒评分) 与用血管理 (申请→审批→发血→输血的全状态机, blood_apply 表)——从科室视角归入医技篇, 见第 9 章。

本章小结

- 床位是独占资源, 原子占床用“条件 UPDATE + 判定受影响行数”实现, 与号源占用、库存扣减同属一个并发定式: 把竞态裁决权交给数据库行锁。入院编排先建档后占床, 转科先占新床再放旧床, 押金流水只增不改。
- 医嘱计费时点三方案 (开立/执行/发药计费) 押的是不同差错: 记账式押“多收”但可被拦截消除, 执行计费押“漏收”且静默。记账式医嘱行自带编码与金额快照, 医嘱行即费用行——这是医保分级分割 (第 10 章) 的上游前提; 快照而非引用, 让历史不随字典调价漂移。
- 电子签名的核心是冻结语义: 签内容快照、签后拒改、验签可追溯。CA 能力收敛为两方法的 SignatureAdapter SPI; Mock 用摘要模拟须诚实标注“可验完整性、不能证明身份”。文书按类型建模是时限质控自动化的前提。
- 护理执行是一个事务三重联动: 状态翻转、原子扣库存、出库留痕; 入口状态校验防重复执行, 执行人+时间戳构成闭环追溯。生命体征按测量时间建模, 电子体温单复刻临床熟悉的三测单。
- Braden (6 - 23, 低分危险) 与 Morse (0 - 125, 高分危险) 方向相反; 范围校验放服务端, 评估留全量趋势, 预警清单只看最近一次高危。护理白板的价值在聚合, “未执行医嘱数”让它成为工作驱动器。
- 出院结算的独有难题是完整性: 未执行医嘱既不能计费也不能放过, 唯一正解是拦截, 使“结算时费用集合封闭”成为系统不变式。医保分割与上传在同一事务, 结算号回填; unique 约束封死重复结算——校验在代码、约束在数据库。
- H5 移动工作台复用同一套 API, 零后端增量、天然数据一致; 但无扫码核对的 H5 承载的是录入便利而非给药安全闭环, 能力表述要有分寸。
- ICU 按场景单建密集记录表 (GCS 双层校验、24 小时滚动窗口出入量); 临床路径是医嘱的模板化——入径即批量开单, 复用全部执行闭环, EXIT 变异同步退出保证“退出必留因”。

思考与练习

1. 概念：解释“记账式医嘱”与“执行计费”两种方案各自把差错押在哪一侧，并论述“可被外部发现的差错优于静默差错”这一判断在两个方案对比中的含义。
2. 设计：HIP 当前的医嘱模型是临时医嘱形态。为其增加长期医嘱支持：给出 `inp_order` 需要的字段改动（或新增拆分表）、按频次（`bid/tid/qd`）生成每日执行条目的规则、停嘱时对未来条目的处理，并说明拆分后 8.7 的出院拦截条件要如何改写。
3. 并发：8.1 的原子占床与第 10 章 10.2.3 的年度累计行锁（`SELECT FOR UPDATE`）都是并发控制，但用了不同手段。分析两个场景的差别（一次性判定 vs 读改写累计），说明为什么占床不需要 `FOR UPDATE`，而累计更新需要。
4. 排障：某病房反馈“患者明明已办出院，床位仍显示占用”。结合 `discharge` 的事务编排（图 8-3），列出可能的原因与排查顺序（提示：医保上传失败回滚时前端如何提示、`release` 是否与状态翻转同事务、是否有人绕过接口改库）。
5. 计算与判断：患者 Braden 评分 13 分、Morse 评分 50 分，三天后复评 Braden 16 分、Morse 40 分。给出两次评估的风险分级；说明此时该患者是否应出现在高危预警清单，为什么“清单看最新、趋势看全量”必须并存。
6. 论述：8.3 披露了“功能清单标注 CA 签名，住院病历实际未挂签名列”的差距。从产品管理与验收的角度，论述这类差距应如何呈现给院方（隐瞒、模糊、还是标注外部条件与计划），并结合电子签名法的“可靠电子签名”要件，说明用 Mock 摘要冒充 CA 签名上线的法律与合规风险。
7. 实践（配套实训环境）：为移动工作台补一个“待执行医嘱”页签：复用 `/api/inpatient/orders/pending` 接口按当前病区过滤，执行按钮调用既有 `execute` 接口。写出前端改动要点，并设计一个 E2E 用例验证“移动端执行后，桌面端护士执行队列同步减少”。

第 9 章 医技系统

本章属第三篇「业务篇」。医技科室——检验、影像、病理、手术麻醉、输血——是医院的「服务生产车间」：临床开出申请，医技交付结果。这些系统看似彼此无关（LIS 管试管、PACS 管影像、血库管血袋），本质上却在解同一道题：一件实物（标本/胶片/血袋）与一条信息（申请/报告/记录）如何始终对齐。本章的贯穿视角是：医技各系统都是「状态机 + 闭环」——实物每移动一步，信息状态推进一格，任何一侧单独前进都是事故的开端。全章依次展开 LIS 标本流转、设备直连与危急值、RIS 报告、PACS 与 DICOM、病理、手术麻醉、用血管理与时段预约，以 HIP 平台的医技模块为贯穿案例。

学习目标

学完本章，你应当能够：

- 1. 说出检验前、中、后三个阶段各自的质量风险，解释 TAT 指标的定义与分段测量方法；
- 2. 画出 LIS 标本从采集到发布的完整状态机，解释「条码 + 唯一约束」如何保证标本与申请一一对应；
- 3. 描述检验结果从仪器回传到临床告警的完整链路，说明设备直连与手工录入为什么应当汇聚到同一处理路径；
- 4. 解释 RIS 报告「书写—审核」双签流程的质量含义，说出 DICOM 的 Study/Series/Instance 三层模型与 PACS 的基本部署形态；
- 5. 说明病理标本流转与检验标本流转的异同，理解「复用状态机模式」的工程价值；
- 6. 描述 PACU Steward 苏醒评分的构成与出室判定逻辑，画出用血申请到输血记录的全状态机，说出临床用血分级审批的法规背景；
- 7. 用「实物流与信息流对齐」的统一视角，分析一个新医技场景（如消毒供应追溯）应当如何建模。

9.1 LIS 检验流程：标本流转状态机

9.1.1 检验前、中、后：质量问题大多不在实验室里

LIS（Laboratory Information System，实验室信息系统）管理的对象是检验全过程，业界按 ISO 15189 医学实验室质量管理体系的习惯把它分为三段：

阶段	环节	典型质量风险
检验前（pre-analytical）	开单→采样→运送→核收	贴错条码（张冠李戴）、采错管、溶血、送检超时
检验中（analytical）	上机→检测→室内质控	仪器漂移、试剂批号、质控失控
检验后（post-analytical）	审核→发布→临床解读	漏审、危急值未通知、报告发错人

多项研究表明，检验差错的大头发生在检验前阶段——也就是实验室墙外、信息系统最能发力的那一段。标本流转信息化的核心价值不是「让技师少敲键盘」，而是把检验前的每一次实物交接都变成可校验、可追溯的状态跳转：护士扫申请条码采样，标本上就多了一个与申请——对应的身份；实验室扫码核收，「标本确实到了」才成为系统事实。

衡量这条链路效率的通用指标是 **TAT**（Turnaround Time，周转时间）。TAT 的口径必须分段定义——「开单到发布」「采样到发布」「核收到发布」是三个不同的数字，分别考核不同环节的责任人。分段测量的前提是每个状态跳转都带时间戳——这就是状态机表设计里那一串 `collected_at / received_at / published_at` 列的用途：状态机的时间戳列，天然就是 **TAT** 的原始数据。

TAT 的管理用法有两层。日常层面看分位数而非平均数：平均 TAT 被大量常规标本稀释，管理者真正要盯的是 **P90**——最慢的那一成标本卡在哪一段；急诊检验通常单列口径并设定时限承诺（如急诊生化 1 小时）。评审层面，检验报告时限是医疗质量控制指标与等级评审的常规核查项（第 12 章时限质控的一个数据源就是本章的时间戳列）。一个设计推论值得记住：如果某个环节没有对应的状态跳转，它的耗时就永远测不出来——比如标本运送时长，在「采集→核收」两态模型里与病区滞留时长混在一起，要单独考核运送，就得加一个「已送出」状态。状态机的粒度，本质上是管理粒度的选择。

9.1.2 状态机建模：实物流与信息流对齐

把标本流转画成状态机（图 9-1）：

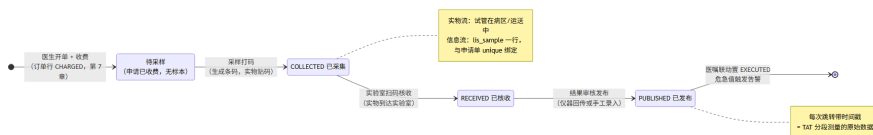


图 9-1 LIS 标本流转状态机：每个状态跳转对应一次实物交接

注意「待采样」不是标本表里的状态——标本尚不存在，它是申请单已收费而标本表无记录这个查询条件推导出的虚拟状态。这与第 7 章「队列即视图」是同一个思想：能从业务数据推导的状态不要冗余存储。

案例框 9-1：HIP 的标本表与幂等采样

HIP 的标本表 `lis_sample` (`V17_phase13_16.sql`) 核心列：`order_id` (**unique** 外键指向订单行)、`barcode` (**unique**，条码号)、`status` (`COLLECTED/RECEIVED/PUBLISHED`)、三个时间戳与审核人 `verifier_id`。两个 **unique** 约束是对齐关系的数据库表达：一份申请至多一管标本，一个条码至多贴一管。

采样接口 (`MedTechController.java collect`) 用一条条件插入同时完成校验与写入：

```

insert into lis_sample(order_id, barcode, substitute, substitute_name)
select ?, ?, ?, ?
where exists (select 1 from outp_order o where o.id = ?
              and o.order_type = 'LAB' and o.status = 'CHARGED')
and not exists (select 1 from lis_sample s where s.order_id = ?)
  
```

影响行数为 0 即返回「申请不存在/未收费/已采样」（错误码 9940）。这个写法把「先查再插」两步并成原子一步，配合 **unique** 约束兜底，并发重复扫码也只会产生一管标本——与第 7 章原子占号是同一模式。

后续每一步推进都是带状态前置条件的 **UPDATE**：核收 `where barcode = ? and status = 'COLLECTED'`，影响 0 行即报「标本不存在或状态不符」(9941)。整个状态机没有一行显式的「if 状态判断」Java 代码——前置条件长在 SQL 的 **WHERE** 里，数据库的原子性就是状态机的原子性。这是 HIP 医技四个状态机（标本/报告/病理/血袋）共用的实现风格。

表上还有一对「政策列」：`substitute / substitute_name`（替检标识与替检人）。替检——他人代替患者留取标本——在体检等场景屡禁不止，各院管理口径不同，HIP 把它做成参数：`sys_config` 键 `lis_allow_substitute` 为 `false` 时替检采样直接拦截 (4770)，为 `true` 时放行但留替检人标识，分检界面醒目提示 (`V37__phase39_assess_substitute.sql`)。管理争议不做死在代码里，交给参数——「配置优先」(第 14 章) 的又一实例。

9.1.3 发布：状态机的出口连着三件事

「发布」是标本状态机的终态，也是三条下游链路的起点：结果落库（患者报告可查）、医嘱联动执行（订单行 `CHARGED`→`EXECUTED`，闭环回第 7 章的统一订单行）、危急值判定 (9.2)。HIP 的发布接口把这三件事收敛为发布一个领域事件——无论结果来自仪器还是手工录入，下游处理完全一致，下一节展开。

9.2 检验设备直连与危急值触发

9.2.1 双通道，单语义

检验结果进入系统有两条通道：设备直连（生化、血球等仪器经中间件以 HL7/MLLP 协议回传，通信机制、报文解析与 ACK 应答详见第 6 章）与手工录入（无接口的项目由技师在 LIS 工作台录入并审核发布）。两条通道的传输形态天差地别，但业务语义完全相同：「某申请单的一组结果项，已经审核成立」。

语义相同就应当只有一份处理代码。HIP 的做法是让两条通道汇聚到同一个领域事件 `LabResultReceivedEvent` (`LabResultReceivedEvent.java`)：MLLP 进站报文经 `OruProcessingService` 解析后发布该事件；LIS 工作台的手工发布接口 (`/api/lis/samples/{barcode}/publish`) 把录入的结果行装配成同一事件发布。消费方 `LabResultListener.java` 对来源无感知：按申请单号定位医嘱→结果逐行落库→医嘱置 `EXECUTED`→比对危急标志生成告警，全程同一事务。

这个设计的价值在演进时兑现：接入第二台仪器、第三种中间件，甚至未来的区域检验中心回传，都只是「多一个事件发布方」，落库、执行联动、危急值三件事的正确性不需要重新验证。通道是外部现实，语义才是系统资产。

9.2.2 危急值：从仪器标志到临床处置

危急值管理制度的要求与 HIP 告警闭环（生成→送达→确认处理→留痕）的实现已在第 6 章完整展开，本节从医技业务视角补两点。

其一，判定权在发布方。HIP 按结果行的异常标志 (HL7 OBX-8 的 HH/LL) 触发告警，意味着「是否危急」由仪器规则或审核技师裁量，系统信任上游。对照的另一派做法是院内统一危急值界值表、系统按数值判定——全院口径统一，但要维护一张随检验方法学变化的表。第 6 章已讨论两者取舍，从医技科室的立场看还有一层：危急值目录本身是检验科与临床科室协商的管理产物（哪些项目、什么界值、通知到谁），信息系统只是执行者，上线前的目录评审会比代码更决定这个闭环的质量。

其二，发布即触发，无人值守也不漏。危急值告警生成与结果落库同事务 (图 9-2)，仪器深夜回传的危急结果同样立即生成待处理告警——这正是设备直连相对「技师看到异常打电话」的本质改进：闭环的第一环不再依赖人的在场。

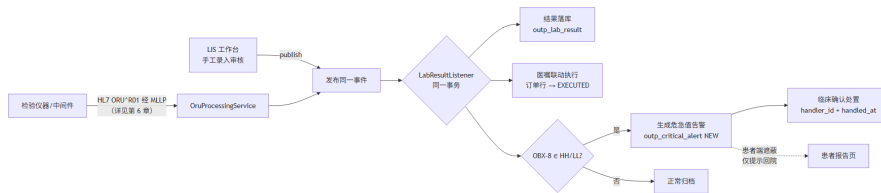


图 9-2 检验结果双通道汇聚与危急值闭环：通道不同，语义与下游处理只有一份

9.3 RIS 检查报告

9.3.1 报告双签：书写与审核分离

RIS (Radiology Information System, 放射信息系统) 在通用语境下已泛化为「检查报告系统」, 覆盖放射之外的心电、超声、内镜等模态。检查报告与检验报告的关键差异在于结果是文本判断而非数值: 检验结果可由仪器直出、按参考范围自动标记异常, 检查报告则是医生对影像/图像的专业描述 (所见, findings) 与结论 (意见, impression), 主观成分高, 因此质量控制依赖双签流程——报告医生书写、上级医生审核, 审核发布前临床不可见。电子病历分级评价对检查报告的审核环节有明确留痕要求 (报告人、审核人、双时间戳)。

9.3.2 HIP 的实现：惰性登记与三态流转

HIP 的检查报告表 `ris_exam` 状态机为 REGISTERED (已登记) → REPORTED (已书写) → VERIFIED (已审核发布), 配 `reporter_id / verifier_id` 与双时间戳。两个实现细节值得剖析:

惰性自动登记。传统 RIS 需要登记环节 (患者到检、技师登记建档)。HIP 简化为: 工作列表接口每次查询时, 把「已收费、尚无报告记录」的检查申请批量 INSERT 进 `ris_exam` (`insert ... select ... where not exists`), 登记环节消失, 申请单收费即出现在待报告队列。同一条 SQL 顺手完成模态归类: 按项目名称关键字把申请分入 ECG (心电)/ENDO (内镜)/US (超声)/GENERAL (普放) 四类, 报告工作台按模态过滤——心电室医生只看心电队列。按名称关键字归类是实施上的务实起点 (不依赖字典改造), 其规范形态是收费项目字典维护模态属性, 两者可以共存: 字典有值用字典, 无值落关键字兜底。

审核联动执行。审核发布 (`verify`) 在置 VERIFIED 的同时把订单行置 EXECUTED——与 LIS 发布、病理诊断的联动完全同构。前置条件同样长在 WHERE 里: `where id = ? and status = 'REPORTED'`, 未书写报告直接审核会得到 9945「须先书写报告」; 书写接口允许 REGISTERED / REPORTED 两个状态进入 (报告可反复修改), 一旦 VERIFIED 即不可再写——审核后修改必须走废止重发流程, 这是报告法律效力的要求 (HIP 当前未实现废止流程, 属明确的演进项)。

9.3.3 报告的最后一公里：对外分享

报告发布后还有一段常被忽视的旅程: 患者要拿着它去外院会诊、给家属看、上传保险理赔。传统答案是打印胶片与纸质报告; 互联网+时代的答案是云胶片/电子报告分享——但「把报告变成一个可转发的链接」立刻触碰隐私红线: 链接会被转发到不可控的范围, 而报告是敏感医疗数据。

案例框 9-2: 分享短链的三道防线

HIP 的报告分享 (ReportShareController.java) 为「可转发」与「可控」的矛盾给出一个最小设计, 三道防线依次是:

- 只分享已审核发布的报告: 生成短链前校验 `status = 'VERIFIED'` (4660)——未定稿的报告连患者本人都不该看到, 遑论外传;

- 限时失效：短链 token（UUID 去连字符）默认 72 小时过期，落 report_share 表的 expire_at，访问时 expire_at < now() 即拒绝（4662）。链接的生命周期与分享场景（一次会诊、一次理赔）匹配，而不是永久有效；
- 匿名访问 + 脱敏：短链是 permitAll 的匿名端点（拿到链接即可看，这是「可转发」的本意），因此返回内容按最小必要裁剪——患者姓名脱敏为「首字 + **」，不返回患者号、身份证等任何标识字段，报告内容（所见/意见/审核时间）完整呈现。

```
return R.ok(Map.of(
    "patientName", name.charAt(0) + "**", // 姓名脱敏
    "itemName", row.get("item_name"),
    "findings", row.get("findings"),
    "impression", row.get("impression"), ...));
```

每次访问还会累加 access_cnt ——访问计数不是功能，是审计线索：一条 72 小时被访问几百次的分享链接，值得安全管理员看一眼。这个「匿名脱敏限时」三件套与第 13 章按角色脱敏、危急值遮蔽同属一个隐私设计谱系：数据出院墙的每一条通道，都要回答「给谁看、看多少、看多久」三个问题。

9.4 PACS 与影像

9.4.1 DICOM 通识：三层模型与一套协议

PACS（Picture Archiving and Communication System，影像归档与通信系统）负责医学影像的采集、存储与调阅，其地基是 DICOM（Digital Imaging and Communications in Medicine）标准。理解 DICOM 只需抓住两点：

数据模型是三层树：一次检查（Study）包含若干序列（Series，如 CT 的平扫序列、增强序列），序列包含若干实例（Instance，通常一帧图像一个文件）。三层各有全局唯一的 UID，患者信息（姓名、患者号）、检查信息（模态、部位、时间）以标签（Tag，如 (0010,0010) 患者姓名）形式内嵌在每个 DICOM 文件头里——影像文件是自描述的，这是 PACS 能按患者号检索影像的基础，也是对接时最常见的坑源：设备 worklist 未对接时技师手工输入患者号，输错一位，影像就「丢」了——文件还在，只是挂在了不存在的患者名下。

通信协议自成体系：经典 DICOM 网络服务包括 C-STORE（影像推送入库）、C-FIND（查询）、C-MOVE（取回），新一代 DICOMweb（WADO-RS/QIDO-RS/STOW-RS）把同样的语义搬到 HTTP/REST 上，Web 端调阅因此不再需要专用客户端。

9.4.2 HIP 的形态：开源 PACS + 调阅入口 + AI 接入点

县市级医院自研 PACS 存储引擎没有意义——这是成熟开源件的领地。HIP 的选择是 Orthanc（轻量开源 DICOM 服务器，自带 Web 查看器与 REST/DICOMweb 接口）：docker-compose 全量部署清单中作为独立服务编排（docker-compose.full.yml），影像数据独立卷存储；平台侧只做两件事——配置键 hip.integration.pacs-url 指向 Orthanc 地址，RIS 报告页据 /api/ris/pacs-info 返回的地址拼装查看器链接，实现「报告旁一键调影像」。仓库根目录的 demo.dcm 是一份样例 DICOM 文件，用于部署验证：上传 Orthanc、按患者号调阅走通，即证明存储与调阅链路成立。

平台与 PACS 之间刻意保持松耦合：不同步影像数据、不解析 DICOM 文件，只持有一个 URL。这意味着实施时替换为医院已有的商用 PACS 只需改一个配置键——代价是无法做深度联动（如报告界面内嵌关键帧），这层增强留给确有需求的医院按第 14 章的 SPI 扩展层实现。

影像 AI（辅助读片、病灶标注）是采购热点，HIP 按「外部条件」形态预留接入点：独立 AI 子服务（`ai-service`，FastAPI）与主平台以 HTTP 通信，主平台设置短超时并优雅降级——子服务不可达时返回「AI 子服务不可用，请人工判读」而非报错（`MedTechController.labAdvice`，当前以检验智能解读先行验证该形态）。这个模式的要点是：AI 输出永远是建议，不阻断业务，不可用不致灾——AI 服务的可用性等级远低于 HIS 主链路，架构上必须隔离其故障域。

9.5 病理系统

9.5.1 病理流程的特殊性

病理被称为诊断的「金标准」，其流程与检验同为标本驱动，但有三点显著不同：周期长（取材、固定、脱水、包埋、切片、染色、阅片，常规报告以天计，与检验的小时级 TAT 完全不同）；标本不可再生（血可以重抽，切下来的组织只有一份，标本差错的后果重于检验）；报告是诊断文书（大体所见、镜下所见、病理诊断三段式，直接决定手术方案与后续治疗）。因此病理信息化的重心更偏向标本安全（每一次交接留痕）与报告严肃性（诊断必填、发布留痕）。

9.5.2 HIP 的实现：复用标本流转模式

HIP 的病理模块（`PathologyController.java`）是 LIS 标本模式的第二次实例化：`path_specimen` 表同样以 `order_id` unique 绑定申请、`barcode` unique 标识实物（前缀 PB 与检验条码 BC 区分），状态机为 COLLECTED（取材）→ RECEIVED（核收）→ DIAGNOSED（诊断发布）；取材登记同样用条件插入保证幂等（4550），诊断发布同样联动医嘱执行。差异只在领域字段：`specimen_desc`（标本描述）、`gross_finding` / `micro_finding` / `diagnosis`（三段式报告）、`pathologist_id`（诊断医师），且诊断必填（4552「诊断不能为空」）——报告严肃性的最低系统表达。

值得体会的是「复用模式」而非「复用代码」：病理没有去调用 LIS 的接口或继承它的类，而是用同一套设计范式（unique 对齐 + 条件插入 + WHERE 状态守卫 + 发布联动）独立实现。两个业务的状态语义、字段、节奏都不同，强行抽象一个「通用标本基类」收益甚微，而模式层面的一致让维护者看懂一个就看懂全部。判断抽象与否的标尺是变化方向：LIS 与病理各自随本科室的业务演进，同构只是此刻的巧合——为巧合建抽象，是过度设计的常见形态。

当前实现的简化也应指出：病理待处理队列按项目名称含「病理/活检」筛选（与 RIS 模态归类同一务实手法）；制片环节（包埋、切片、染色的工序管理）未建模，状态机从核收直跳诊断——对病理科规模较大的医院，工序级追溯是首先要补的扩展。

写到这里，本章已出现三个标本/单据状态机，连同下一节的血袋，把它们并排放在一张表里，「同一范式、不同领域」便一目了然：

	LIS 标本	RIS 报告	病理标本	用血申请
表	<code>lis_sample</code>	<code>ris_exam</code>	<code>path_specimen</code>	<code>blood_apply</code>
对齐约束	<code>order_id</code> unique + 条码 BC	<code>order_id</code> unique	<code>order_id</code> unique + 条码 PB	<code>admission_id</code> 外键
状态链	采集→核收→发布	登记→书写→审核	取材→核收→诊断	申请→审批→发血→输血
建档方式	扫码条件插入	查询时惰性登记	扫码条件插入	医师显式申请
终态联动	医嘱执行+危急值	医嘱执行	医嘱执行	—（记账走医嘱）
领域特有	替检参数、审核人	双签、模态归类	三段式报告、诊断必填	法定审批、反应处置字典

四列的差异恰好都落在「领域知识」行——状态机骨架是工程范式，骨架上的血肉才是各科室的业务本身。读懂这张表，遇到第五个同构场景（习题8的消毒供应）时，你应当能在一小时内画出它的表结构。

9.6 手术麻醉

9.6.1 手术流程与麻醉记录

手术麻醉系统（手麻系统）覆盖「申请—排程—术中—复苏」四段。HIP的手术主表 `inp_surgery` 挂在住院记录之下（`admission_id`，第8章），状态机为 REQUESTED（已申请）→ SCHEDULED（已排程）→ DONE（完成），申请时给定手术时间即直接进入 SCHEDULED；完成时填写手术记录 `op_note` 与麻醉小结 `anes_note`。排程在 HIP 中是一个时间字段而非独立的手术室资源模型——手术间/台次/接台的容量管理与第7章号源同构，是该模块的自然演进方向。

麻醉记录单是术中文书的核心：麻醉医生按固定间隔（常规5分钟）记录生命体征，形成时间轴曲线。HIP的 `anes_record` 表（`V28__phase30_medrecord_clinical.sql`）每行一个时点：`phase`（INTRA 术中/PACU 复苏）、心率/收缩压/舒张压/血氧四项体征、`steward_score` 与备注，前端按 `recorded_at` 排序绘制时间轴（`AnesController.java`）。真实手麻系统的记录来自监护仪自动采集（每5秒一组，走设备网关），HIP当前为手工录入形态——但表结构对采集来源中立，接入监护仪网关时只是写入频率变化。

9.6.2 PACU 与 Steward 出室评分

患者术后先进 PACU（Post-Anesthesia Care Unit，麻醉后恢复室）观察，达标方可返回病房——「达标」必须客观化，否则出室时机取决于谁在值班。通行工具是量表评分，Steward 苏醒评分是其中经典之一：清醒程度、呼吸道通畅程度、肢体活动度三项各0-2分，总分6分，≥4分方可离开恢复室（另一常用量表为 Aldrete 评分，五项十分制，判定思想相同）。

案例框 9-3：把出室判定做成接口而不是习惯

HIP 的 PACU 逻辑不足 20 行，但完成了一次典型的「临床规则系统化」：

```
/** 出复苏室判定：最近一次 PACU 记录 Steward ≥ 4 可出室 */
@GetMapping("/records/pacu-status")
public R<Map<String, Object>> pacuStatus(@RequestParam Long surgeryId) {
    var rows = jdbc.queryForList("""
        select steward_score from anes_record
        where surgery_id = ? and phase = 'PACU' and steward_score is not null
        order by recorded_at desc limit 1
        """, surgeryId);

    Integer score = rows.isEmpty() ? null : ((Number)
rows.get(0).get("steward_score")).intValue();
    return R.ok(Map.of("latestScore", score == null ? -1 : score,
        "canLeave", score != null && score >= 4));
}
```

三处设计意图：取最近一次而非最高分——评分会波动，出室依据只能是当前状态；无 PACU 评分即不可出室（`canLeave=false`）——没记录等于没评估，宁可拦住；录入侧配数据库 CHECK 约束（`steward_score between 0 and 6`）与接口校验（4672）双保险——量表的值域是医学定义，脏数

据进来，评分曲线与出室判定都失真。写入侧还有一个外键存在性校验（4671「手术记录不存在」）：麻醉记录必须挂在真实手术之下，孤儿记录在文书检查时无从解释。

这段代码回答了一个常被问到的问题：临床评分口算就行，为什么要系统做？答案在留痕——「出室时 Steward 为 5 分」从口头判断变成带时间戳的记录，等级评审与医疗纠纷回溯时，这行数据就是证据。评分量表系统化的价值一半在计算，一半在举证。

手麻在 HIP 的模块开关（ModuleGate 六域之一，第 14 章）中标记为「可整体关闭」：无手术室的乡镇卫生院、专科门诊部关掉即隐藏菜单并封禁 API——功能边界按医院形态伸缩，而不是所有医院背同一套全功能。

9.7 用血管理

9.7.1 法规背景：被审批的稀缺资源

临床用血在所有医技资源中受最严格的法定管制。《医疗机构临床用血管理办法》【成稿核对：卫生部令第 85 号，2012 年施行，确认现行有效版本】与《临床输血技术规范》【成稿核对：卫医发（2000）184 号，确认是否有新版替代】确立的核心制度包括：用血申请由医师提出并逐级审批（大量用血须更高级别审批与科主任签字，各院阈值按规范执行）；输血前完成血型鉴定与交叉配血；输血过程记录入病历；发生输血不良反应须记录处置并上报。对信息系统，这意味着用血天然是一个审批流 + 状态机 + 全程留痕的业务——比其他医技流程多出一个不可跳过的审批环节。

9.7.2 全状态机：申请→审批→发血→输血

HIP 的用血模块（BloodController.java，表 blood_apply）状态机如图 9-3：

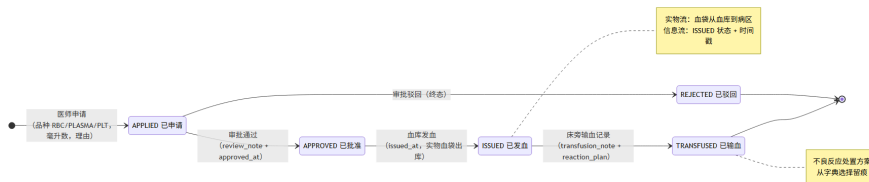


图 9-3 用血全状态机：审批环节是法定的，跳过审批的发血在系统中不可表达

每一步推进都是 WHERE 状态守卫的 UPDATE：发血要求 `status = 'APPROVED'`（4532「须审批通过后方可发血」），输血记录要求 `status = 'ISSUED'`（4533「须发血后方可记录输血」）。「未批先发」在这个系统里不是被禁止，而是不可表达——没有任何一条代码路径能让 APPLIED 直接变成 ISSUED。这是状态机建模相对「表单 + 标志位」的根本优势：合规不靠自觉，靠可达性。

申请入口的校验同样体现法规意识：血制品限定枚举（RBC 红细胞/PLASMA 血浆/PLT 血小板）、用量必须为正、患者必须在院（4530）。输血记录环节在 1.0.1 版本补充了不良反应处置方案字典（`blood_reaction_plan`，V44_quickwins_101.sql）：「立即停止输血保持静脉通路」「保留血袋与输血器送血库」等处置项从字典选择留痕，而非自由文本——检查者要的是「按规范处置的证据」，字典化让证据可统计、可核查。

当前实现与全功能血库系统的差距应当明示：血型鉴定与交叉配血结果未建模（真实流程中是发血的前置条件）、血袋号与冷链未追溯、与血站信息系统未对接。HIP 的用血模块定位是临床侧用血流程的合规闭环，血库侧的库存与检测管理属于外延——这也是它与手麻并列为可关闭模块的原因：不设血库的医疗机构直接关闭。

9.8 检查检验时段预约

预约的入口形态（患者/护士站选时段）与叫号大屏已在第7章预约叫号体系中介绍，本节从医技科室的运营视角补齐这个闭环。

时段池模型。大型设备是稀缺容量，`med_appointment` 按「日期+上午/下午」分池，池内顺序号取当前最大值加一；申请单与预约一对一（`order_id` unique），前提是已收费——先付费后占用稀缺资源，与挂号同理。顺序号的 `max+1` 写法在并发下存在重号窗口，医技预约的并发远低于挂号，HIP 接受了这个简化；若要收紧，第7章原子占号的 UPDATE 自增模式可以平移过来。

改约与批量取消。改约仅限未叫号（BOOKED）的预约——已叫号说明患者已在现场，改约语义不再成立（4513）；改约进入新时段的队尾（重取顺序号），不插队。设备故障停机的运营场景由批量取消承接：整池预约一键置 CANCELLED，配合通知患者改期。一个真实的教训藏在迁移脚本里：状态列最初定义为 `varchar(8)`，放不下 CANCELLED 九个字符，上线后追加迁移拓宽（V33_widen_appt_status.sql）——枚举列的宽度要按「未来最长的状态词」而不是「现在最长的」预留，小坑，但每个团队都踩过。

与叫号联动。医技端到检叫号复用第7章的统一队列中心：叫号即把预约置 CALLED 并落 `queue_call_log`（`biz_type` 为 EXAM/LAB），大屏与语音播报与药房共用一套机制；执行完成置 DONE，从队列消失。至此患者动线闭环：开单→收费→预约→叫号→执行→报告——每一步都是本篇某一章的状态机，彼此以状态跳转衔接。

本章小结

- 医技各系统共享同一个本质：实物流与信息流对齐。标本、报告、血袋的每一次实物交接对应一次状态跳转，跳转带时间戳（TAT 的原始数据）、带操作人（留痕举证）。
- 状态机的实现范式贯穿全章：**unique** 约束表达一一对应（申请↔标本/报告/预约）、条件插入保证建档幂等、**WHERE** 状态前置守卫推进——合规规则「不可表达」优于「被禁止」，用血的「未批先发」即为例证。
- 检验质量的大头在检验前阶段，标本流转信息化的价值是把实验室墙外的交接变成可校验的状态跳转；替检等管理争议做成参数而非硬编码。
- 检验结果双通道（设备直连/手工录入）汇聚单事件，落库、医嘱联动、危急值判定只有一份实现：危急值发布即触发、与落库同事务，闭环第一环不依赖人的在场（通信机制详见第6章）。
- RIS 报告走书写一审核双签，审核联动医嘱执行；惰性登记消灭登记环节，模态按项目名归类是实施务实起点。报告对外分享用「匿名脱敏限时」三件套（仅限已审核、短链 72 小时失效、姓名脱敏+访问计数）回答「给谁看、看多少、看多久」。
- PACS 走「开源件（Orthanc）+ 松耦合调阅入口」路线，DICOM 三层模型（Study/Series/Instance）与自描述文件头是理解影像系统的钥匙；影像 AI 按外部条件形态预留接入点，优雅降级、不阻断业务。
- 病理复用标本流转模式而非复用代码——同构是此刻的巧合，为巧合建抽象是过度设计。
- 手麻的核心文书是术中时间轴与 PACU 评分，Steward≥4 出室判定系统化的价值一半在计算、一半在举证；手麻与用血按模块开关可整体关闭，功能边界随医院形态伸缩。
- 用血是法定审批流：申请→审批→发血→输血全状态机，不良反应处置字典化留痕；时段预约以「日期+时段」分池，改约不插队、批量取消应对设备停机，与统一叫号中心联动收口患者动线。

思考与练习

1. 概念：解释检验前、中、后三阶段的划分，各举一个信息系统能够拦截的差错场景；说明「采样到发布」与「核收到发布」两个 TAT 口径分别考核谁的责任。

2. 建模：图 9-1 的「待采样」是查询推导的虚拟状态而非表中状态。讨论把它实体化（标本表增加 PENDING 状态、开单即建行）的利弊，并说明哪种场景下实体化是必要的（提示：采样前的标本容器准备与打印批量）。
3. 设计：HIP 的标本、报告、病理、血袋四个状态机都用「WHERE 状态守卫 + 影响行数判断」实现。写出这种写法相对「先 SELECT 判断状态再 UPDATE」的两点优势与一点局限（提示：并发、代码量、失败原因的区分度——影响 0 行时如何告诉用户是「不存在」还是「状态不符」?）。
4. 排障：某医院上线设备直连后，偶发「仪器已传结果但临床查不到报告」。已知回传报文的 OBR-2 由中间件从仪器工单号转换而来。按 9.2 与第 6 章知识列出至少三个可能断点与对应排查手段（提示：留痕表的 FAIL 记录、申请单号映射、医嘱状态前置条件）。
5. 推演：患者 PACU 记录依次为 Steward 3 分（14:00）、5 分（14:20）、4 分（14:35）。按 HIP 的判定逻辑，14:30 与 14:40 两个时刻查询出室状态各返回什么？若把实现从「最近一次」改为「历史最高分」，会产生什么临床风险？
6. 论述：9.7 指出「未批先发在系统中不可表达」。结合第 7 章退费联动拦截与第 10 章医保状态机，论述「用状态可达性表达业务规则」相对「用 if 校验表达业务规则」的工程优势，并讨论其边界——什么样的规则不适合编入状态机（提示：紧急抢救用血的「先用后补」例外流程）。
7. 设计：为 HIP 补齐交叉配血环节：在 APPROVED 与 ISSUED 之间插入 CROSSMATCHED 状态，写出 blood_apply 的表结构改动、新接口的前置条件 SQL，并说明历史数据迁移策略（存量 ISSUED/TRANSFUSED 单据如何处理）。
8. 实践（配套实训环境）：消毒供应中心的器械包追溯（回收→清洗→打包→灭菌→发放→使用）与本章的标本流转同构。按 9.1 的范式设计 cssd_package 表与状态机，指出哪一步交接应当双人复核，并说明灭菌批次（一锅多包）与本章「一申请一标本」模型的结构差异如何处理。

第 10 章 医保接口与 DRG

本章属第三篇「业务篇」。医保是医院收入的最大来源，也是医院信息系统中“算错一分钱都不行”的领域。本章从医保业务模型讲起，依次展开费用分割、结算与冲正、智能审核、对账、DRG 入组与支付模拟，最后讨论省级 SDK 适配的实施工艺。全章以 HIP 平台的医保模块为贯穿案例，包括一个真实的返工教训。

学习目标

学完本章，你应当能够：

1. 说出医保三大目录的构成，解释甲/乙/丙类费用的报销差异，理解目录对照为什么是实施工作的重头；
2. 独立设计一个支持起付线、封顶线、分比例待遇模型的费用分割引擎，并说明金额舍入与并发累计的处理要点；
3. 画出“结算—上传—冲正—对账”的完整状态流转图，解释医保结算号为什么必须持久化；
4. 理解事前提醒式智能审核的规则建模方法，以及从 WARN 到 BLOCK 的演进路径；
5. 设计业务账与通道账的对账方案，解释“留痕即对账依据”的工程含义；
6. 描述 CHS-DRG 的分组层次（MDC→ADRG→DRG），手工完成一个病例的入组推演，解释 CMI、费用消耗指数、时间消耗指数的计算与含义；
7. 说出医保适配器 SPI 的接口契约要素，理解“假适配器验证法”在实施验收中的作用。

10.1 医保业务模型：三大目录与目录对照

10.1.1 基本医疗保险的支付框架

我国基本医疗保险（下称“医保”）覆盖职工医保与城乡居民医保两大制度。从医院信息系统的视角看，医保决定了两件事：患者结算时个人付多少、统筹基金付多少（费用分割），以及医保经办机构事后如何与医院结算（基金支付，如今主要按 DRG/DIP 付费）。前者发生在收费窗口的每一笔交易里，后者发生在月度、年度的机构清算里。这两个层面对应本章的前后两半。

医保能否支付、支付多少，首先取决于费用项目是否在医保目录内。医保目录由三部分构成，业内合称“三大目录”：

目录	内容	对应医院数据
药品目录	可报销的西药、中成药、中药饮片	药品字典
诊疗项目目录	可报销的检查、检验、治疗、手术项目	收费项目字典
医疗服务设施目录	床位费等服务设施标准	收费项目字典（床位类）

目录内项目按报销规则分为两类：甲类项目全额纳入报销范围，按待遇比例报销；乙类项目需个人先行自付一定比例（各省、各项目不同，常见 5% - 30%），剩余部分再纳入报销范围。目录外项目全部自费，业内习惯称为丙类（注意“丙类”并非国家目录的正式分类，而是行业沿用的口径——目录里只有甲、乙两类，“丙类”就是“不在目录里”）。

10.1.2 目录对照：实施工作的第一座山

医院有自己的药品字典和收费项目字典，医保局有自己的目录编码（国家医保局自 2019 年起推行“贯标”，统一了医保药品、诊疗项目等 15 项编码）。两套编码体系必须建立映射，患者结算时系统才知道每一行费用是甲类、乙类还是自费——这项工作叫目录对照。

目录对照的工程要点：

1. 数据量决定工具形态。一家县级医院的药品目录常有三五千条，收费项目上千条。逐条手工维护不可实施，必须提供批量导入工具（CSV 模板 + 校验 + 对照率统计）。对照率（已对照条数 / 全部条数）是实施进度的核心指标，也是上线门槛——对照率不足就上线，意味着大量项目按自费口径结算，患者窗口投诉会立刻教育你。
2. 对照是持续工作，不是一次性工作。医保目录每年调整，医院新进药品也要随时补对照。系统要留出“未对照”状态的处理策略。
3. 未对照项目的保守口径。结算时遇到未对照项目怎么办？两种选择：拒绝结算（业务卡死，不可取），或按自费处理（患者可能多付，但基金不多付）。行业通行做法是后者——宁可让患者事后退差价，不可让基金多支付，因为向患者退钱是院内流程，向基金退钱是违规使用基金的稽核问题。

案例框 10-1: HIP 的目录对照表设计

HIP 用一张 `yb_catalog_map` 表承载对照关系，关键列：`item_type`（DRUG 走药品目录 / ITEM 走诊疗目录）、`item_code`（院内编码）、`charge_class`（A/B/C）、`self_ratio`（乙类先行自付比例）。配套三件东西：- CSV 批量导入接口（并入医院初始化工具 `init-hospital.py` 的字典模板家族）；- 对照率统计（医保管理页展示，实施验收指标）；- 分割引擎的保守兜底：查不到对照记录时按 `c` 类（自费）处理，`self_ratio` 取 0。

值得注意的是“按 `item_type` 分列”这个设计：药品和诊疗项目在医保侧是两个目录、两套编码规则，混在一张映射表里用类型字段区分，比拆两张表更利于统一的导入工具和对照率口径。

10.1.3 患者的医保身份

分割计算的另一个输入是患者的参保类型：职工医保、居民医保待遇不同（比例、起付线、封顶线都不同），自费患者则完全不走分割。参保类型属于患者主索引（EMPI）的属性，但要注意它的时效性——患者可能断保、可能身份变化，真实环境下应在挂号或结算时通过医保接口做参保信息查询（人员待遇核查），而不是只信本地档案。HIP 在 Mock 阶段将参保类型存于 `empi_patient.insurance_type`，接口就绪后此处即是替换点。

10.2 费用分割引擎

10.2.1 什么是费用分割

费用分割（也称费用分解）是把一张结算单的总金额，按医保规则拆成若干部分的计算过程。最核心的输出是两个数：统筹基金支付（医保付）与个人支付（患者付）。展开来，一张分割结果单通常包含：

- 甲类金额、乙类金额、自费（丙类）金额——按目录分类的汇总；
- 乙类先行自付金额——乙类项目中患者必须先掏的部分；
- 起付线金额——进入统筹之前患者自付的门槛部分；
- 统筹基金支付——医保基金实付；
- 个人支付——以上各项自付的合计（真实环境还会再分个人账户支付与现金支付）。

在展开计算之前，先补全“个人支付”的内部结构。患者最终自付的钱可能来自两个池子：个人账户（职工医保按月划入的账户余额，可用于支付自付部分）与现金（含移动支付）。此外，统筹之上还有两层保障：大病保险对高额自付部分二次报销，医疗救助对困难人群兜底。完整的支付构成因此是一个四层瀑布：统筹基金 → 大病保险 → 医疗救助 → 个人（个账/现金）。对医院端系统，大病与救助通常在医保侧结算接口内一并算好返回，医院只需正确展示与入账；但分割结果表的字段设计要预留这些层次，否则接真实接口时会发现返回报文里的金额没地方放——这与 10.3 的“结算号没地方放”是同一类教训。

分割的计算顺序是理解一切医保待遇的钥匙，值得用一个完整例子走一遍。

例：某职工医保患者门诊结算 1000 元，其中甲类 600 元、乙类 300 元（先行自付比例 10%）、自费 100 元。统筹比例 70%，年度起付线 200 元（尚未使用），封顶线未触及。

1. 可报销基数 = 甲类全额 + 乙类扣除先行自付后的部分 = $600 + 300 \times (1 - 10\%) = 870$ 元；
2. 扣起付线： $870 - 200 = 670$ 元进入统筹计算；
3. 统筹支付 = $670 \times 70\% = 469$ 元；
4. 个人支付 = 总额 - 统筹 = $1000 - 469 = 531$ 元（其中自费 100、乙类先行自付 30、起付线 200、共付段个人比例 201）。

这个例子隐含了几个真实系统必须处理、教科书却常略过的问题：舍入、年度累计、并发。下面逐一展开。

10.2.2 金额舍入：逐行还是账单级

医保金额按“分”结算，计算中出现的比例乘法必然产生舍入。舍入发生在哪一层，结果不同：逐行舍入求和与账单级一次舍入可能相差几分钱。几分钱在会计上不是小事——分割结果要与医保上传报文一致、与对账结果一致，口径不统一就会出现永远对不平的“幽灵差异”。

工程结论是：选定一种口径并在全链路保持一致，且在待遇模型变化时保证向后兼容。

案例框 10-2：HIP 分割引擎的双口径兼容

HIP 的分割引擎（`InsuranceSplitService`，`InsuranceSplitService.java`）经历过一次待遇模型升级：早期版本没有起付线/封顶线，统筹金额按逐行“可报销金额×比例”舍入后求和；升级引入年度待遇后，统筹改为账单级“（可报销基数-起付线扣除）×比例，封顶截断”。

升级时的兼容处理很有代表性：待遇参数（起付线、封顶线）全部走 `sys_config`，且约定 0 = 不启用。当两个参数都为 0 时，引擎显式走老算法分支（逐行舍入求和），保证与历史版本逐分一致——已上线医院不配置新参数，行为完全不变；配置了参数的医院才启用新模型。这是“配置优先”原则（第 14 章）在算法演进上的应用：新算法以配置开关的形式并存，而不是替换。

```

if (deductible.signum() == 0 && cap.signum() == 0) {
    fund = lineFundSum;    // 待遇模型未启用：与既有逐行舍入算法逐分一致
} else {
    BigDecimal dedRemain = deductible.subtract(annualUsed).max(ZERO);
    dedTake = eligibleTotal.min(dedRemain);
    fund = eligibleTotal.subtract(dedTake).max(ZERO).multiply(ratio).setScale(2,
    HALF_UP);
    if (cap.signum() > 0) fund = fund.min(capRemain);    // 封顶截断
}

```

10.2.3 年度待遇累计与并发控制

起付线和封顶线都是年度累计概念：患者本年已累计自付满起付线后，后续结算直接进统筹；统筹累计达到封顶线后，超出部分基金不再支付。这要求系统维护一张患者年度累计表（HIP 中为 `yb_patient_annual`，按 `patient_id + year` 唯一），每次结算读取并更新“已用起付线额度”“已用统筹额度”。

累计表带来一个容易被忽略的并发问题：同一患者的两笔结算并发进行（家属在自助机缴费、本人在窗口缴费），如果两个事务都读到“起付线还剩 200 元”，就会重复扣减，导致基金多付。这是典型的“读一改一写”竞态。

解法是数据库行级锁：在事务内以 `SELECT ... FOR UPDATE` 锁定该患者的年度累计行，第二个事务会等待第一个提交后才能读到最新累计值。HIP 的实现还有一个细节——累计行用 `INSERT ... ON CONFLICT DO NOTHING` 惰性创建，保证首笔结算也能锁到行。这一整套（惰性建行 + 行锁 + 事务内更新）是有状态额度类业务的通用模式，第 7 章的号源占用、第 8 章的床位占用是同一模式的变体。

同样成对出现的是退费时的额度回退：结算占用了起付线和统筹额度，退费就必须归还，否则患者年度待遇被“吃掉”。HIP 的 `reverse()` 方法按分割留痕回退年度累计（用 `GREATEST(x - ?, 0)` 防止负数），而分割明细记录本身保留不删——留痕永远只增不减，汇总口径通过单据状态过滤（10.3.4 将再次遇到这个原则）。

10.2.4 行级分割：门诊与住院的统一

门诊结算按处方/订单行分割，住院结算历史上不少系统按“总额”粗分——出院时把住院总费用一把算。粗分的问题在 DRG 时代被放大：医保结算清单要求费用行级归类明细，总额口径无法生成合规清单，智能审核也无从下手。

正确方向是门诊与住院共用同一个行级分割引擎。这在工程上是一次典型的“泛化下沉”：分割逻辑最初长在门诊模块里，住院需要时不是复制一份，而是把它抽到独立的医保模块，入参泛化为 `bizType (OUTP/INP) + 单号 + 订单行列表`。住院的“订单行”来自记账式医嘱（每条医嘱执行记账都带项目编码与金额，见第 8 章），天然满足行级输入的要求——上游模型设计对了，下游归并就是水到渠成；反过来，如果住院记账只存总额，行级分割就要伤筋动骨地补数据。

HIP 的 `YbLine` 记录类型（`orderType/itemCode/itemName/amount/qty`）就是这个统一入参：门诊收费和住院出院结算各自把自己的费用行装配成 `YbLine` 列表调用同一个 `splitAndAudit`。分割比例按 `bizType` 取不同配置键（`yb_ratio_staff` 与 `yb_ratio_staff_inp`），支持门诊/住院分比例——这也是各地待遇政策的普遍形态。

10.3 结算流程：上传、冲正与一个真实教训

10.3.1 医保结算的完整链路

一笔医保患者的门诊结算，在系统内部的完整链路是：

收费确认（事务开始）

- ├ 1. 费用分割 (splitAndAudit: 算统筹/自付, 写分割留痕, 跑审核规则)
- ├ 2. 本地结算落库 (outp_charge 置 PAID, pay_method='YB')
- ├ 3. 医保上传 (InsuranceAdapter.uploadSettlement → 医保侧入账)
- | └ 成功: 回填医保结算号; 失败: 抛错回滚整个事务
- └ 4. 出站留痕 (int_message_log: channel='YB', 报文与结果落库)

（事务提交）

链路失败时的错误码设计值得单独一提。医保上传失败对收费员是高频可见的异常，提示语义必须精确：HIP 为医保通道定义了专用错误码段（如 5006 结算上传失败、5007 冲正失败、9013 通道不可用），前端据码给出差异化指引——“医保拒绝：{医保侧原因}，请核实参保状态”与“医保通道超时，请稍后重试”的处置动作完全不同。把适配器返回的 message 原样透传到窗口，是排障效率与窗口体验的双重保障；吞掉原因只报“结算失败”，收费员只能打电话给信息科，信息科只能翻日志。

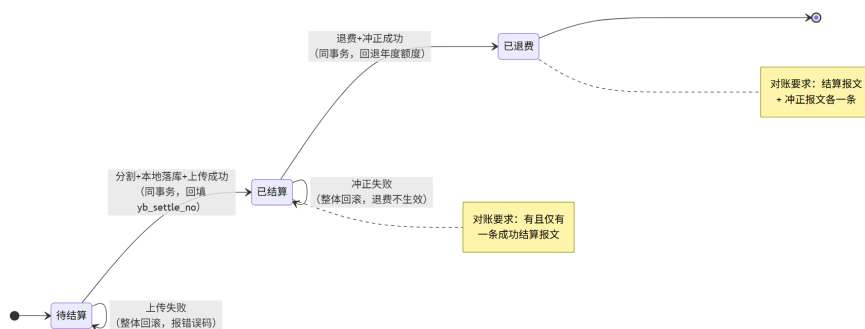
关键设计决策是第 3 步在收费事务内同步执行。同步的代价是医保接口的延迟直接叠加在窗口收费的响应时间上（所以适配器必须设置合理超时）；同步的收益是强一致——医保上传失败，本地结算一起回滚，不会出现“本地已收费、医保未入账”的悬挂状态。对中小医院，一致性的价值远大于那几百毫秒的延迟；日结量大到同步扛不住的大型医院，才需要考虑异步补偿方案，而那要配套一整套悬挂单据的对账与冲销机制，复杂度完全不在一个量级。先选简单且正确的方案，把复杂方案留给被数据证明的瓶颈——这是贯穿本书的架构态度（第 4 章）。

10.3.2 冲正：结算的逆操作

患者退费时，本地退费必须伴随医保冲正（撤销医保侧的入账），否则医保基金为一笔已退掉的费用买了单——这在基金监管口径下是要被追回甚至处罚的。冲正链路和结算对称：本地退费 + uploadRefund 冲正 + 年度额度回退（10.2.3）+ 出站留痕。

冲正失败怎么办？与结算同理：与本地退费同事务、失败同回滚。宁可告诉窗口“退费暂时办不了，稍后重试”，也不能让本地退了、医保没冲。

把结算与冲正合起来，一张医保单据的完整生命周期如下（图 10-1）：



图中两条“自环”（失败回滚回原状态）正是强一致设计的可视化：任何状态下，本地账与医保账要么一起前进，要么一起停在原地——不存在中间状态，这也是 10.5 对账规则能写得干净的前提。

10.3.3 教训案例：被丢弃的返回值

案例框 10-3：两个真实的正确性缺陷（HIP 医保完善专项，2026-08）

HIP 医保模块在一次代码级审阅中发现两处正确性问题，都属于“演示能跑、真实环境必翻车”的类型，值得逐字读：

缺陷一：退费冲正的结果未校验。`ChargeService` 调用 `uploadRefund` 后丢弃了返回值——冲正失败时本地退费照样提交。演示环境的 `Mock` 适配器永远成功，问题被完美掩盖；真实环境下医保接口有网络抖动、有业务拒绝，一旦失败，唯一的发现渠道是日后对账，而那时钱已经退给患者了。修复：冲正失败与结算失败同语义——抛出专用错误码回滚本地事务。

缺陷二：医保结算号未持久化。`uploadSettlement` 返回的 `settleNo`（医保侧结算流水号）被拿到后只随报文留痕存进了 `payload`，没有回填业务表。而真实省级 SDK 的冲正、线下对账，都要求按医保结算号发起——不是按医院单号。等到要接真 SDK 时才发现：冲正接口的必填参数，系统里根本没有结构化存储。修复：新增迁移为 `outp_charge / inp_settlement` 增加 `yb_settle_no` 列，结算成功即回填。

两个缺陷指向同一条教训：**Mock** 适配器的“永远成功”会系统性掩盖失败路径与回传数据的处理缺陷。对策有二：一是代码纪律——凡外部调用，返回值必须消费，失败语义必须显式处理，“丢弃返回值”应当在代码评审清单上；二是测试设计——**Mock** 必须能模拟失败（HIP 后续在 E2E 中加入了冲正失败注入的断言）。第 10.7 节的“假适配器验证法”是这条教训的制度化产物。

10.3.4 留痕与汇总口径

医保链路的每一次出站调用都落 `int_message_log`（集成留痕表，第 6 章）：渠道、方向、关联单号、完整报文、成功与否。留痕有三重身份：排障依据、对账数据源（10.5）、监管检查时的原始凭证。

留痕之外还有汇总口径问题——同一批数据，不同页面统计出不同数字，是医保模块最伤信任的 bug 类型。HIP 曾有一例：医保管理页的“今日分割”汇总把已退费单据也计入统筹合计，导致统筹/自付合计高估。根因是分割留痕设计为“只增不减”（退费不删分割记录，见 10.2.3），而汇总 SQL 忘了按单据状态过滤。教训成了一条设计规则：留痕表永不物理删除，一切汇总必须显式声明状态过滤条件——把“哪些状态计入”写进口径注释，而不是留给每个写 SQL 的人现场发挥。

10.4 智能审核

10.4.1 事前、事中、事后

医保基金监管有“事前提醒、事中审核、事后稽核”三道防线。对医院端系统而言：

- 事前：开单/结算时实时提示可疑项（超量开药、重复收费、自费占比过高），医生或收费员当场处理；
- 事中：结算上传时医保侧规则引擎拦截（两定接口返回拒绝）；
- 事后：医保局大数据筛查、飞行检查，问题资金追回并处罚。

医院端能主动建设的是事前审核——在费用发生的源头拦住问题，远好于事后被追回。事后被动应对的成本（资金追回、协议处理、信用评价扣分）是事前建设成本的数量级倍数。

10.4.2 规则建模：从 WARN 起步

事前审核规则库的建设常见误区是一步到位追求“强拦截”。强拦截（**BLOCK**：违规即禁止提交）需要规则精度非常高，误拦一次门诊高峰期的正常处方，临床的反弹会让整个规则库被要求关闭。稳妥的演进路径是：

1. **WARN** 阶段：规则命中只记录、只提示，不阻断业务。积累命中数据，人工复盘误报率；
2. 调参阶段：阈值参数化（走系统配置而非硬编码），按院、按规则调整到误报可接受；
3. **BLOCK** 升级：精度达标的规则逐条升级为拦截，保留 **WARN** 通道给新规则试运行。

动手建规则前，先建立规则的类型学——绝大多数医保审核规则落在以下几类，每类的数据需求与实现难度差异很大：

规则类型	示例	数据需求	实现难度
数量/剂量限制	单品种单次数量上限、日剂量上限	当前单据	低
频次限制	同项目 30 天内限做 1 次	患者历史单据	中
重复/互斥	同单重复项目、互斥项目并开	当前单据 + 互斥对照表	低 - 中
适应症限定	某药限某诊断使用(“限二线用药”)	单据 + 诊断 + 限定条件库	高
人群限定	性别、年龄与项目不符	单据 + 患者属性	低
费用结构	自费占比、次均费用异常	单据汇总	低
行为模式	分解处方、分解住院	跨单据、跨就诊的时序分析	高

前两列低难度规则用院内数据即可自建，是 WARN 阶段的合理起点；适应症限定类依赖大规模临床知识库（药品说明书、医保限定支付条件的结构化），这正是商用规则库的价值所在；行为模式类则更多出现在监管侧的事后筛查（10.8）。自建规则的选型标准：数据在手、语义明确、误报可控。

案例框 10-4：HIP 的三条雏形规则

HIP 的审核雏形内置三条规则，阈值全部走 `sys_config`：- **R001** 单品种数量超限：药品行数量 $> yb_audit_qty_limit$ ，提示核实是否超量开药；- **R002** 同单重复项目：同一结算单内同名项目出现多次，提示核实重复收费；- **R003** 自费占比过高：丙类金额占比 $> yb_audit_self_ratio_limit$ （默认 50%），提示与参保人确认——这条对应医保协议中“自费项目须参保人知情同意”的普遍要求。

命中统一落 `yb_audit_log`（单号、规则码、级别 WARN、消息），医保管理页可查。设计上预留了两个演进点：规则表化（从代码枚举改为规则表驱动）是 BLOCK 升级的前提；接入商用审核规则库（数万条临床规则，属外部条件）时，`yb_audit_log` 同表承接，级别字段天然支持 WARN/BLOCK 分流。三条雏形规则的真正价值不在拦截能力，而在把审核的管道铺通——落痕表、配置键、页面、演进路径都就位，规则内容随时可换可扩。

这个“雏形规则 + 商用规则库接入点”的组合，是中小医院务实的路径：自研数万条临床审核规则不现实，但管道自建、内容外采，两头的成本都可控。

10.5 对账体系

10.5.1 为什么必须对账

即使结算与冲正都做了同事务强一致（10.3），对账仍然不可省略。因为一致性只覆盖了“系统按设计运行”的情形，而对账防的是设计之外的世界：适配器实现的缺陷、医保侧的单边账（医院认为失败已回滚、医保侧实际入账成功）、人工干预留下的数据、程序缺陷造成的漏传。对账是医保资金安全的最后一道系统性防线——它不阻止错误发生，但保证错误在一天内被发现，而不是在年度稽核时被医保局发现。

10.5.2 业务账与通道账

对账的本质是拿两本独立记录的账逐单核对：

- 业务账：本地业务表中的医保结算单（门诊 `outp_charge` 中 `pay_method='YB'` 的单据、住院 `inp_settlement` 同口径）；

- 通道账：集成留痕表 `int_message_log` 中医保渠道的成功报文——它是“我确实向医保发过这笔、且医保确认成功”的证据。

核对规则按单据状态展开：正常结算单应有且仅有一条成功的结算报文；已退费单应同时有结算报文与冲正报文。任一侧缺失即为差异。

案例框 10-5：HIP 的对账实现与匹配约定

HIP 的 `InsuranceReconService` 按日对账：取当日业务账逐单点查通道账，匹配条件为 `channel='YB' AND status='OK' AND ref_no=单号 AND payload LIKE '%api关键字%'`（门诊结算 `outpatient.settle`、冲正 `outpatient.refund`、住院含 `settle`）。核对结果落 `yb_recon_batch` 批次表（总数/一致数/差异数/差异单号清单）。

这里有一个微妙但重要的工程决策：对账匹配依赖报文 `payload` 中的 `api` 关键字，这实际上给适配器实现方立了一条约定——真实 SDK 实现的留痕报文必须保持关键字进 `payload`，否则对账会把正常单据全部判为差异。这条约定被明文写进了 ADR-0002（10.7）：当一个模块的正确性建立在另一个模块的实现细节上时，这个细节就不再是细节，必须升格为书面契约。

对账的运营形态同样关键：`InsuranceAutoReconJob` 每日 01:30 自动核对前一日，差异数 > 0 自动开运维故障工单（联动第 13 章的 `ops` 工单体系）。对账做成“每天自动跑、有差异有人管”的闭环，才算落地；做成“页面上有个按钮”的对账，三个月后就没人点了。

10.5.3 对账差异的处置

差异单据的处置是运营流程而非纯技术问题，常见类型与处置：

差异形态	常见原因	处置
本地有、通道无结算报文	漏传、留痕缺陷	补传或人工核实医保侧，确认后补留痕
已退费但无冲正报文	冲正失败未回滚（10.3 教训一）	人工发起冲正，追查代码缺陷
通道有、本地无单据	单边账（本地回滚、医保入账）	按医保结算号发起冲正——此时 10.3 教训二的 <code>yb_settle_no</code> 就是救命字段

注意第三类处置对“医保结算号持久化”的依赖——三个知识点在这里闭合成环：结算号回填（10.3）、留痕约定（10.5.2）、差异处置，共同构成资金安全闭环。

10.6 CHS-DRG 入组与支付分析

10.6.1 从按项目付费到按病组付费

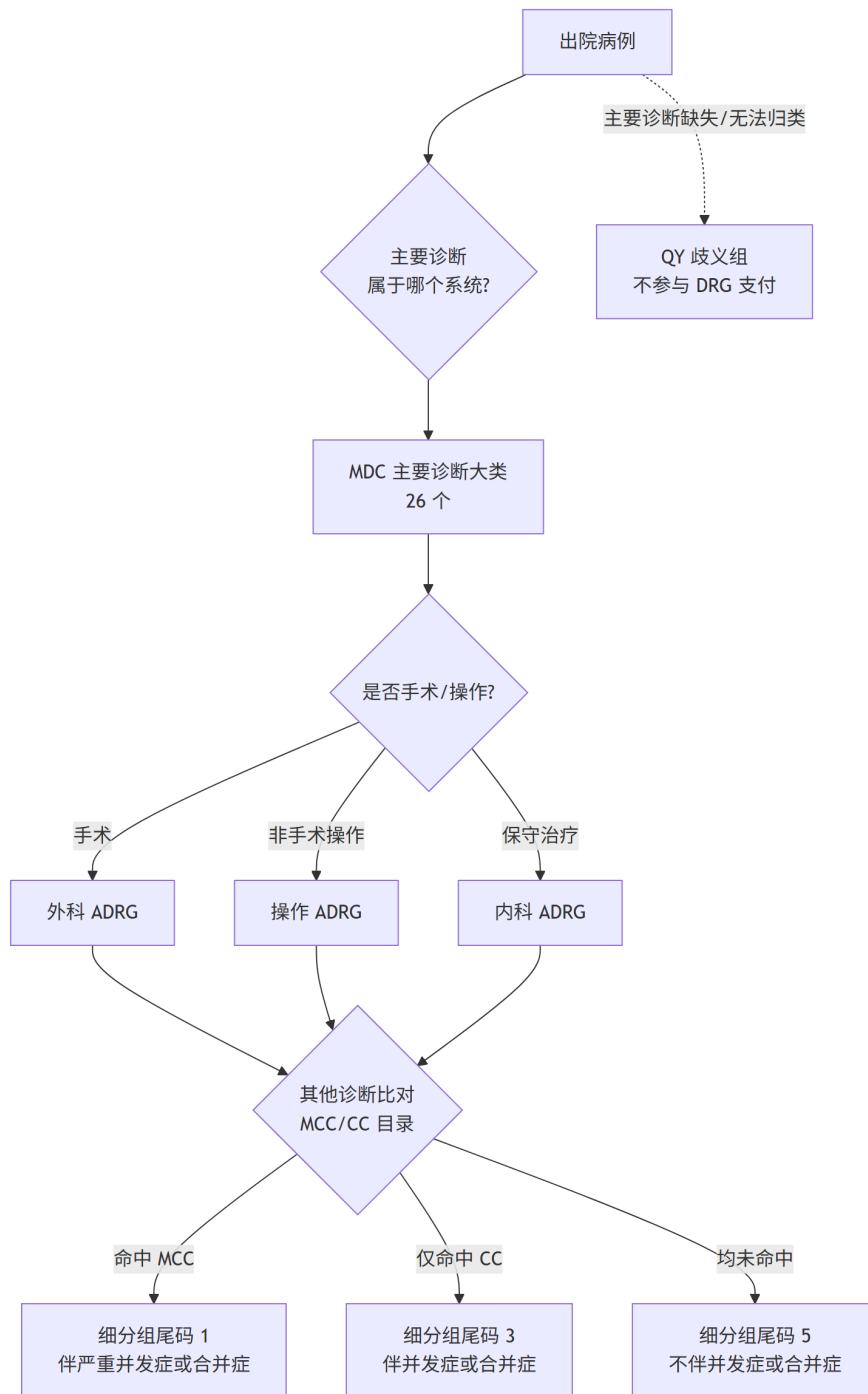
传统医保按项目付费（做多少项目付多少钱）内生激励过度医疗。DRG（Diagnosis Related Groups，疾病诊断相关分组）改变游戏规则：把住院病例按“临床相似、资源消耗相近”分入若干病组，每组一个支付标准——同病组付同价，医院治疗成本低于标准则结余留用，高于标准则自担亏损。激励方向由“多做项目”翻转为“控制成本、提高效率”。

我国的国家版方案为 **CHS-DRG**（国家医保局 2019 年发布分组方案并持续修订），与之并行的还有 **DIP**（按病种分值付费），各统筹区选择其一。本节以 CHS-DRG 为例；DIP 的信息化需求（病案首页质量、费用明细上传）与 DRG 高度同构。

对医院信息系统，DRG 带来三类新需求：入组（每个出院病例分到哪个组）、支付模拟（按病组标准算医保将付多少、与实际成本比结余几何）、运营分析（科室与病组维度的效率指标）。

10.6.2 分组层次：MDC → ADRG → DRG

CHS-DRG 的分组是三层漏斗（图 10-2）：



三层依次是：

1. **MDC**（主要诊断大类，26 个）：按主要诊断的系统分类，如“神经系统疾病及功能障碍”；
2. **ADRG**（核心疾病诊断相关分组，数百个）：在 MDC 内按主要诊断 + 是否手术/操作细分——同样的诊断，手术治疗与保守治疗资源消耗迥异，必须分列（外科组/操作组/内科组）；
3. **DRG** 细分组：在 ADRG 内按病例个体因素（年龄、并发症严重程度）再分。国家规范的尾码约定：**1** = 伴严重并发症或合并症（**MCC**），**3** = 伴并发症或合并症（**CC**），**5** = 不伴，**9** = 未做细分。

其中 MCC/CC 的判定依据是病例的其他诊断（次要诊断）：逐一比对 MCC 目录与 CC 目录，命中 MCC 目录则为“伴 MCC”，仅命中 CC 目录则为“伴 CC”。这就是为什么 DRG 时代病案首页的

其他诊断填写质量直接决定医院收入——漏填一个 MCC 诊断，病例从尾码 1 掉到尾码 5，权重差可达 30% 以上。

每个 DRG 组有一个权重（该组平均资源消耗相对全部病例平均消耗的倍数），统筹区再定一个费率（每权重值多少钱），于是：

病例支付标准 = 该 DRG 组权重 × 统筹区费率

无法入组的病例（主要诊断缺失、编码无法归类）称歧义病例（QY 组），不参与 DRG 支付，通常按项目付费兜底并纳入质控通报。费用极端的病例设高倍率/低倍率豁免通道：实际费用超过支付标准若干倍（如 3 倍）或不足一定比例（如 0.3 倍）的病例，可申请退出 DRG 打包、单议结算——这是给危重疑难病例留的安全阀，防止医院推诿重病人。

案例框 10-6: HIP 的简化分组器

官方分组方案（数百 ADRG、完整 MCC/CC 目录）是付费依据，属外部条件（须从医保局获取）。HIP 在官方目录到位前实现了一个结构完整、目录简化的分组器（`DrgController`，`DrgController.java`），分组逻辑与国家规范同构：

- **ADRG 命中**：`drg_group_def` 表存 ICD 前缀清单 + 是否手术标记 + 基础权重；病例按主要诊断前缀匹配，手术标记精确匹配优先、内科组兜底（有手术的病例优先进外科组，匹配不到时落内科组）；
- **严重程度**：遍历病例其他诊断比对 `drg_cc_list`（前缀 + MCC/CC 级别）， $MCC > CC > \text{无}$ ；
- **细分与权重**：尾码按 1/3/5 拼接，终权重 = 基础权重 × ($MCC \ 1.30 / CC \ 1.15 / \text{无} \ 1.00$)；
- **兜底**：匹配失败入 QY 歧义组，权重 0，单列统计。

简化的是目录内容（示例规则集），不简化的是结构：分组层次、尾码约定、严重程度判定、歧义兜底、高低倍率标记全部按国家规范建模。官方目录到位后，换 `drg_group_def` / `drg_cc_list` 两张表的数据即可，代码不动——这是“外部条件接入点先行”方法（10.7.3）在算法域的应用。另一个值得注意的运营设计：分组器提供 `regroup-all` 全量重算入口，因为规则表更新、病案诊断修正后，历史病例的入组结果必须可重现。

顺带一提入组的数据依赖链：入组读取出院病例的主要诊断（ICD）、其他诊断表、手术记录、住院结算金额、住院天数——分别来自住院模块（第 8 章）与病案管理（第 12 章）。DRG 是典型的“下游消费者”，它的数据质量问题几乎全部要回溯到上游录入环节治理。

10.6.3 运营指标: CMI 与消耗指数

DRG 权重体系派生出一组横向可比的效率指标，是医院运营分析的通用语言：

- **CMI（病例组合指数）** = 总权重 ÷ 总例数。反映收治病例的平均技术难度，CMI 高说明疑难重症占比高。科室间比较绝对费用不公平（ICU 当然比普通内科贵），比较 CMI 校正后的指标才有意义；
- **费用消耗指数** = 某组平均费用 ÷ (组权重 × 全院每权重平均费用)。等于 1 表示与全院平均效率持平，大于 1 表示治同样难度的病花钱更多；
- **时间消耗指数**：同理，把费用换成住院天数。

两个消耗指数把“难度”从“花费”中剥离，回答的是管理者真正关心的问题：这个科室/病组贵，是因为病人难，还是因为效率低？

支付模拟则把入组结果变成财务预演：逐病例算标杆支付（权重 × 费率）与实际费用之差（结余/亏损），按病组、按科室汇总，标记高低倍率病例。在统筹区正式按 DRG 结算前，医院先用自己的数据模拟运行，找出亏损集中的病组提前干预（成本结构、路径管理、编码质量），这正是 DRG 分

析模块的核心价值——HIP 的 `analysis` / `dept-analysis` / `cases` 三个接口分别对应病组、科室、病例三个分析层级，费率走 `sys_config` 的 `drg_rate` 键以便按统筹区配置。

10.6.4 DIP 与医保结算清单

与 CHS-DRG 并行的另一条技术路线是 **DIP**（按病种分值付费）。两者的分组哲学不同：DRG 自上而下——先设计分组框架（MDC/ADRG），再把病例装进去，全国分组框架统一；DIP 自下而上——直接以“主要诊断+手术操作”的真实组合穷举成目录（动辄上万个病种组合），按各组合的历史均费折算分值，年终按区域总额与总分值算出每分值单价再清算。粗略地说，DRG 是“几百个粗组、前瞻定价”，DIP 是“上万个细组、回顾清算”。

对医院信息系统，两者的需求高度同构：入组（DIP 按诊断+操作组合直接查目录，实现比 DRG 分组器更简单）、模拟、病案质量治理。病案首页与费用明细的质量是两条路线共同的生命线——这也是为什么无论统筹区选哪条路线，系统建设的重心是一样的。

数据通道方面，DRG/DIP 付费的官方数据载体是医保结算清单——国家统一格式的出院结算文书，涵盖病案首页核心项+行级费用明细+医保结算信息，经质控校验后上传医保侧作为入组与付费依据。结算清单对系统的要求把本章前后两半连了起来：行级费用（10.2.4 的行级分割）、诊断与手术编码（第 12 章病案管理）、医保结算信息（10.3 的结算号与分割结果），缺一项都生成不出合规清单。实施中常见的返工——“上 DRG 才发现住院费用没有行级归类”——根源都在上游模型，而不在清单模块本身。

10.7 省级 SDK 适配与实施工艺

10.7.1 差异收敛：适配器 SPI

医保接口国家有统一规范，但落到各省是不同的 SDK 包、不同的部署形态（动态库、独立服务、报文网关）、不同的细约定。平台产品不可能为每省维护一个分支——差异必须收敛到一个接口后面。这就是 SPI（Service Provider Interface）模式：平台定义接口与契约，按省做实现包，装配哪个实现由配置决定（第 6 章已介绍该模式，本节看它在医保域的完整应用）。

HIP 的 `InsuranceAdapter` 只有两个方法，这个“小”是刻意的：

```
public interface InsuranceAdapter {
    InsuranceResult uploadSettlement(String chargeNo, BigDecimal amount); // 结算上传
    InsuranceResult uploadRefund(String chargeNo);                       // 退费冲正
    // InsuranceResult(ok, settleNo, message)
}
```

接口面越小，实现方负担越轻、平台被实现细节反向绑架的风险越小。签到、目录下载、批量对账等 SDK 周边能力，约定放在实现包内部自理，不进平台接口——平台只关心结算链路的强一致语义。

10.7.2 契约的书面化：ADR-0002

案例框 10-3 与 10-5 已经展示了两个“实现细节决定平台正确性”的场景。HIP 把这类约定固化为一份架构决策记录（ADR-0002 医保适配器实现约定），要求实现方遵守四条：

1. 结算号回传：`uploadSettlement` 成功必须返回医保结算号，平台回填业务表（冲正与对账依赖）；
2. 留痕约定：每次出站调用须落集成留痕，`channel='YB'`、`ref_no`=本地单号、`payload` 含 `api` 关键字——这是对账的通道账来源，改关键字须同步改对账匹配；
3. 失败语义：不可用/被拒返回 `ok=false` 并带 `message`，不得抛异常吞掉信息；平台据此回滚本地事务。实现内部的重试必须幂等（同单重复上传医保侧不可重复入账）；
4. 同步阻塞边界：调用在收费事务内同步执行，实现须设合理超时；长耗时操作走实现自身的异步通道，不占结算路径。

这份文档的价值在实施阶段兑现：省 SDK 适配开发往往由实施团队甚至第三方完成，没有书面契约，平台团队的隐含假设（“留痕当然要带关键字”）就会在别人的实现里被无声打破，而故障要到对账全红时才暴露。SPI 的接口签名只是契约的一半，行为约定是另一半，且必须落成文字。

10.7.3 假适配器验证法与替换步骤

省 SDK 属外部条件——资质、专线、SDK 包都要等医保局流程。等待期间如何证明“接口一到就能接”？HIP 的做法是假适配器验证：在虚拟医院全量彩排（第 14 章）中，编写一个模拟真实形态的假医保适配器实现类（独立实现包、按 ADR-0002 履行全部契约），完成 Bean 替换装配，并让全部医保 E2E 在该实现下通过。这验证的不是医保业务本身（Mock 早已覆盖），而是替换工艺：装配机制、契约遵守、E2E 的实现无关性。真实 SDK 到位时，工艺已被彩排过一遍，剩下的只是把假逻辑换成真调用。

ADR-0002 同时给出实施期的替换步骤，实为一份可操作的验收清单：

1. 新建实现类（放地区实现包，如 `cn.hip.impl.<region>.insurance`），实现全部契约；
2. Spring 装配替换 Mock（`@Primary` 或条件装配 `hip.insurance.adapter=<region>-sdk`）；
3. 跑医保 E2E 全套（结算/分割/退费/对账断言均不依赖实现细节，实现无关）；
4. 真实环境首日：人工触发一次对账核对，确认无差异后交给每日自动对账。

第 4 步值得体会：再充分的测试也替代不了首日人工核对——它核对的是测试环境无法复现的那部分：真实医保侧的行为。

10.7.4 接入前的调研清单

SDK 适配的返工大户不是编码，而是前期没问清楚。实施进场时应完成一份医保接口现状调研表，至少覆盖：

- 形态：SDK 是动态库、独立服务还是报文网关？部署在院内还是医保前置机？操作系统与位数要求（不少省 SDK 只有 Windows 32 位版本，直接决定前置机选型）；
- 网络：医保专线还是 VPN？前置机 IP 白名单如何申请？超时与带宽实测值；
- 接口清单：结算/冲正之外，签到签退是否必须？参保查询、目录下载、对账文件下载的调用方式与频率限制；
- 测试环境：医保局是否提供联调环境与测试参保人？（没有测试环境的省份，首笔真实结算就是联调，风险策略完全不同）；
- 凭证：机构编码、操作员编码、密钥/证书的申请周期——这些行政流程常常比开发周期更长，必须最早启动。

这份调研表属于实施工艺包（第 14 章）的一部分；医保只是其中一页，LIS 设备、PACS、CA 各有对应页。外部条件的管理原则是把“等待”变成“排队”：技术侧接入点先行完成，行政侧申请最早启动，两条线在日历上可视化对齐。

10.8 延伸：基金监管侧视角

本章始终站在医院端。视角换到医保局端，同样的数据链路服务于完全不同的目标：基金监管。医保局的大数据筛查基于医院上传的结算明细运行监管规则（住院天数异常、费用结构异常、分解住院、挂床住院模式识别），筛出线索后进入飞行检查与案件查办流程——调取病历与费用明细取证、按《医疗保障基金使用监督管理条例》定性、追回资金并作出处理。

医院端系统的建设质量与监管侧直接相关：行级费用明细（10.2.4）、完整留痕（10.3.4）、病案编码质量（10.6）既是医院自我管理的工具，也是接受检查时的原始证据。反过来，理解监管规则有助于医院把审核规则库（10.4）对齐监管口径——监管侧查什么，事前就拦什么。

监管侧业务本身（线索管理、案件流程、法定时限与文书）是另一个独立的产品域，其业务模型、与医院端数据的关系在 HIP 体系中以独立产品（医保案件查办系统）承载，本书不展开，有兴趣的读者可参考其架构决策记录（ADR-0003）。

本章小结

- 医保支付的两个层面：患者结算时的费用分割（甲/乙/丙分类、待遇模型）与机构层面的基金支付（DRG/DIP）。目录对照是一切的前提，对照率是实施上线的硬指标；未对照项目按自费的保守口径处理。
- 分割引擎的三个工程要点：舍入口径全链路一致（新旧算法以配置开关并存兼容）；年度待遇累计的并发控制（惰性建行 + SELECT FOR UPDATE 行锁）；行级分割门诊住院统一（泛化下沉，上游记账模型决定下游能力）。
- 结算与冲正在收费事务内同步执行、失败同回滚，用一致性换延迟。两个真实教训——冲正返回值得被丢弃、医保结算号未持久化——共同指向：Mock 的“永远成功”会掩盖失败路径缺陷，外部调用的返回值必须消费，失败语义必须显式。
- 智能审核走 **WARN** → 调参 → **BLOCK** 的演进路径，阈值参数化，管道自建、规则内容可外采。
- 对账是资金安全的最后防线：业务账 ↔ 通道账逐单核对，每日自动运行，差异自动开工单。对账匹配所依赖的留痕细节必须升格为书面契约。
- CHS-DRG 三层分组（MDC→ADRG→细分组），尾码 1/3/5 由其他诊断比对 MCC/CC 目录判定；CMI 与费用/时间消耗指数把“难度”从“花费”中剥离；支付模拟让医院在正式结算前预演盈亏。简化分组器“结构完整、目录可换”，官方目录到位即插即用。
- DRG 与 DIP 分组哲学相反（前瞻粗组定价 vs 回顾细组清算），但对系统的要求同构：病案首页与行级费用质量是共同生命线，医保结算清单是官方数据载体。
- 省 SDK 差异用 SPI 收敛，接口面刻意做小；行为契约（结算号回传、留痕约定、失败语义、超时边界）书面化为 ADR；用假适配器彩排替换工艺，真实上线首日人工对账。

思考与练习

1. 概念：解释甲类、乙类、丙类费用在分割计算中的不同处理；说明“未对照项目按丙类”为什么是保守口径，保守的代价由谁承担、如何补救。
2. 计算：某居民医保患者住院总费用 8000 元，甲类 5000 元、乙类 2000 元（先行自付 15%）、自费 1000 元；统筹比例 60%，年度起付线 500 元（已用 300 元），封顶线 15 万元（已用 14.85 万元）。计算统筹支付与个人支付，并说明封顶线截断发生在哪一步。
3. 设计：10.2.3 用 SELECT ... FOR UPDATE 解决年度累计的并发竞态。如果改用“乐观锁 + 失败重试”方案，写出累计表需要增加的字段、更新 SQL 与重试逻辑，比较两种方案在窗口收费场景下的优劣。
4. 排障：某医院上线真实医保 SDK 后，每日对账差异数突然从 0 涨到接近全量，但患者结算一切正常。按本章知识列出最可能的原因与排查步骤。（提示：案例框 10-5 的匹配约定。）
5. 推演：病例主要诊断 I21.0（急性心肌梗死），行 PCI 手术，其他诊断含 J96.0（急性呼吸衰竭，设其在 MCC 目录）。设 ADRG 表中“急性心肌梗死伴介入治疗”组基础权重 2.8，费率 9000 元。给出该病例的细分组尾码、终权重、支付标准；若病案漏填 J96.0，支付标准损失多少？
6. 论述：10.3.1 选择医保上传与收费同事务同步执行。论述在日门诊量 8000 人次的三甲医院，这一决策是否仍然成立；若改异步，需要补齐哪些机制（悬挂状态、补偿、对账口径）才能保证资金安全。

7. 实践（配套实训环境）：为 HIP 的审核规则库新增规则 R004“目录外项目连续出现“（同一患者 7 天内自费占比超阈值的结算单达 N 张时提示），阈值走 sys_config，命中落 yb_audit_log。写出表结构改动（如需）、规则 SQL 与测试用例。

第 11 章 临床数据中心与患者 360

本章开启第四篇「数据篇」。前三篇建起来的门诊、住院、医技系统每天生产大量临床数据，但这些数据躺在各自的业务表里，服务的是“把这一单业务办完”；医生调阅患者历史、医院向区域平台上报、科研人员检索病历，需要的是“把一个患者、一类问题的数据聚起来看”。承接这两类诉求的架构部件就是临床数据中心（CDR）。本章从 CDR 与业务库分离的论证讲起，依次展开文档模型、增量同步、患者 360 视图、全文检索、CDA 文档生成与数据订阅推送，贯穿案例为 HIP 的 `datacenter/cdr` 模块——一个刻意做小、但结构完整的 CDR 起步实现。

学习目标

学完本章，你应当能够：

1. 说出 CDR 与业务库（OLTP）分离的两条核心理由，区分 CDR 与 ODR 的职责边界；
2. 解释“患者维度文档快照”模型的设计要点，说明 `docType + refId` 唯一键如何支撑幂等覆盖式同步；
3. 列举数据同步的技术谱系（双写、触发器、日志 CDC、定时增量、事件驱动），按侵入性、时延、运维成本完成一次选型论证；
4. 设计基于水位线的增量同步，并识别水位字段选择、水位推进时机两处正确性陷阱；
5. 描述患者 360 视图的聚合边界如何划定，说明脱敏策略如何与 EMPI 层联动（第 5 章）；
6. 比较病历全文检索的三条实现路径（LIKE / PostgreSQL tsvector / Elasticsearch），说出各自的适用区间；
7. 说出 CDA 文档的头体结构与三级结构化水平，解释“骨架先行”的实施策略；
8. 设计一个面向区域平台的数据订阅推送服务的契约要素（事件类型、留痕、重试与幂等）。

11.1 CDR 的定位：与业务库分离的理由

11.1.1 两类负载的天然冲突

业务库是为联机事务处理（OLTP，Online Transaction Processing）设计的：挂号写一行、收费改一行状态、发药扣一次库存——单笔数据量小、并发高、响应时间以百毫秒计，索引围绕“按单号点查”优化。而临床数据的第二类消费者完全是另一种画像：

- 调阅：医生接诊时要看患者近三年的全部就诊、检验、住院记录——跨表、跨域、按患者聚合；
- 检索：质控与科研要按关键词捞病历——全文扫描，命中范围不可预知；
- 上报与共享：区域平台要文档、医保要清单、卫健要报表——批量导出，高峰在夜间但补数时不挑时间。

这两类负载放在同一个库里，轻则互相抢资源，重则一条科研检索 SQL 的全表扫描把收费窗口拖到超时——查询分析不能拖垮 OLTP，这是数据架构的第一铁律。解法是物理或逻辑上分出一个专供读取分析的数据存储，临床域的这个存储就叫临床数据中心（CDR，Clinical Data Repository）。与之并列的还有运营数据中心（ODR，Operational Data Repository），面向管理运营指标（门诊量、收入、在院数）；两者消费口径不同，但“与业务库分离、单向同步”的架构地位相同。HIP 的分层架构（第 4 章）在数据层明确并列了业务库、CDR、ODR 三个部件，其架构决策表第 3 条写的正是本节标题：“CDR 与业务库分离，通过 CDC/事件同步——查询分析不拖垮 OLTP；对齐互联互通文档注册要求”。

第二条理由“互联互通”值得展开。医院信息互联互通标准化成熟度测评（第 1 章）要求医院建设标准化的电子病历共享文档库，提供文档注册、文档检索、文档调阅等交互服务【成稿核对：互联互通测评方案中文档注册/调阅服务的条目编号与分级要求】。这些服务的语义单位是“文档”（一次门诊的病历文档、一份检验报告文档），而业务库的语义单位是“表行”。从表行到文档的组装如果发生在每次调阅时，既慢又把负载压回业务库；提前组装好存进 CDR，注册、检索、调阅就都是对 CDR 的读操作。CDR 既是性能隔离层，也是“行→文档”的语义转换层——后者常被忽略，却是它区别于简单只读副本（read replica）的本质：只读副本分担的是负载，不改变数据形态；CDR 改变数据形态。

11.1.2 分离的物理形态

“分离”有强弱几档：独立数据库实例（隔离最彻底，运维成本最高）、同实例独立库或 schema、同库独立表。中小医院的现实约束（2-4 台服务器、无专职 DBA，第 4 章）下，HIP 选择最弱的一档：CDR 表与业务表同库共存，靠“单向抽取+独立表+独立索引”实现逻辑分离。这不是妥协掉了架构，而是分阶段兑现——分离的价值大头在数据形态与查询路径的隔离（跨域查询不再 JOIN 七八张业务表），物理隔离留到数据量证明需要时再做，届时 CDR 表整体迁走，读它的患者 360、检索、CDA 接口一行不改。先把边界画对，物理部署可以后补；边界画错，部署再豪华也是一体的泥球。

11.1.3 文档模型：三类起步

CDR 用什么模型存数据？两条路线：明细模型（把业务表按主题域重建为规范化明细，利于统计）与文档模型（把一次临床事件的相关数据组装为一份自包含文档，利于调阅与共享）。互联互通的共享文档库走文档路线；统计分析则偏爱明细。HIP 的 CDR 取文档路线（统计诉求由 ODR 汇总接口与第 12 章的报表引擎承接），全部文档落在一张表里。

案例框 11-1：一张表的 CDR——cdr_document 的设计

HIP 的 CDR 实体（CdrDocument.java，建表见 V11_cdr.sql）只有八列，每列都有明确职责：

```
create table cdr_document (
  id          bigserial primary key,
  patient_id  bigint          not null references empi_patient (id),
  doc_type    varchar(24)     not null,    -- OUTP_ENCOUNTER / LAB_REPORT / INP_SUMMARY
  ref_id      bigint          not null,    -- 源业务主键: 挂号 id / 医嘱 id / 住院 id
  title       varchar(128)    not null,
  doc_time    timestampz      not null,    -- 文档发生时间 (就诊日/报告日/入院日)
  content     varchar(8000)   not null,    -- 结构化内容 JSON
  synced_at   timestampz      not null default now(),
  unique (doc_type, ref_id)
);
create index idx_cdr_patient on cdr_document (patient_id, doc_time desc);
```

三个设计点：其一，`unique (doc_type, ref_id)` 把文档锚定到源业务记录，同一笔业务重复同步永远是覆盖更新而非追加——幂等性建立在数据模型上，而不是同步代码的小心翼翼上（11.2 展开）。其二，`(patient_id, doc_time desc)` 复合索引精确服务患者 360 的唯一查询形态“某患者的文档按时间倒序”，一次索引扫描出结果。其三，`content` 存 JSON 快照而非外键引用——文档是自包含的，调阅不回查业务库，这正是 11.1.1 隔离论证的落点；代价是快照可能过时（同步时滞）与 8000 字符截断（超长病程被截，当前实现直接 `substring`，属于已知妥协）。

三类 `doc_type` 对应三条抽取链：门诊就诊文档（挂号 + 病历 + 诊断 + 医嘱）、检验报告文档（医嘱 + 结果明细）、住院摘要文档（住院 + 病历记录 + 结算）。对照总体规划中“按 WS/T 500 的 53 类 CDA 文档组织”的蓝图【成稿核对：WS/T 500 系列共享文档的类目数】，落地是 3 类——这不是缩水，而是排序：先用最高频的三类文档把“抽取—存储—调阅—输出”的全链路打通，文档类型此后是在稳定管道上做加法。

11.2 数据同步：增量、CDC 与事件驱动的选择

11.2.1 同步技术谱系

把数据从业务库搬进 CDR，业界方案按“变更如何被感知”分五族：

方案	感知机制	时延	对业务侵入	运维成本	典型陷阱
双写	业务代码写两处	无	高（每处写点改代码）	低	两处写的原子性无保障，漏写即不一致
数据库触发器	表上 <code>trigger</code> 捕获 DML	无	中（DDL 侵入）	中	隐藏在库里难审计，拖慢业务事务
日志 CDC	解析 WAL/binlog（如 Debezium、PG 逻辑复制）	秒级	零	高（Kafka/Connect 一套基础设施）	运维复杂度对中小医院常是压垮项
定时增量拉取	按水位字段轮询	分钟级	零（只读查询）	低	水位字段选错则漏数据（11.2.3）
应用事件驱动	业务代码发布事件，监听者入 CDR	近实时	中（发事件点改代码）	低 - 中	事件丢失需发件箱兜底（第 6 章）

CDC（Change Data Capture，变更数据捕获）狭义指日志族——它是理论上最优雅的方案：零侵入、低时延、连删除都能捕获。但 Debezium 一套（Kafka + Connect + 连接器）的运维面，对没有专职中间件工程师的县市级医院是奢侈品；这与第 6 章“不引入消息中间件”的论证是同一笔账。HIP 的现实选择是定时增量拉取为主、全量同步兜底，源码注释写得直白：“MVP 为全量拉取（幂等覆盖）；数据量大后改为按更新时间增量 + CDC”——方案是台阶，不是终点。

11.2.2 HIP 的实现：水位线 + 幂等覆盖

图 11-1 是 HIP 同步链路的全景：

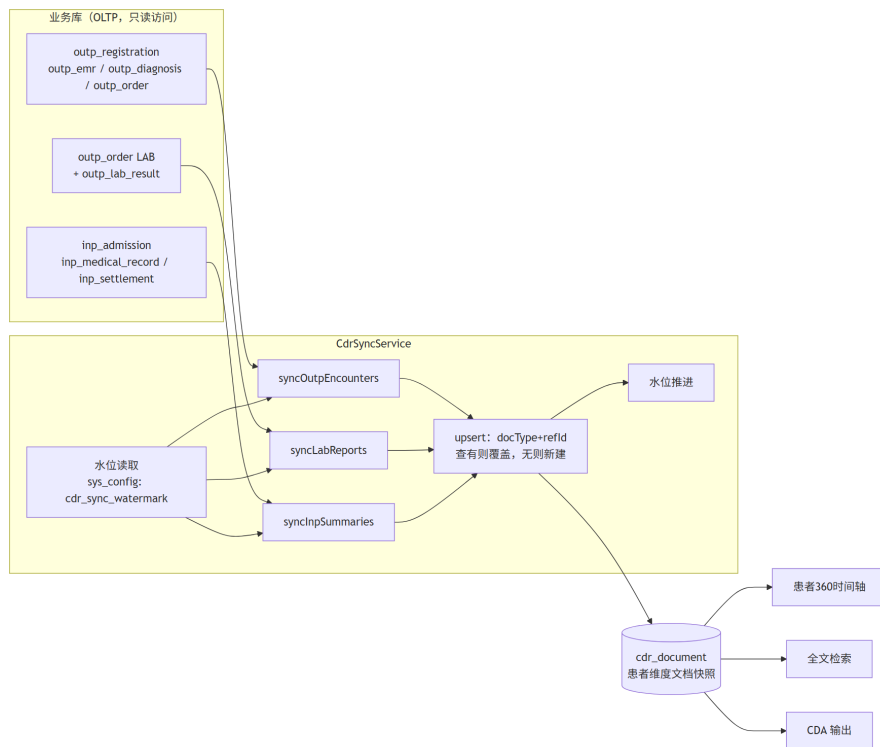


图 11-1 业务库到 CDR 的同步链路

案例框 11-2：水位线存在 sys_config 里

HIP 的同步服务 (CdrSyncService.java) 暴露两个入口：syncAll() 全量（水位取 Instant.EPOCH，即抽全部）与 syncIncremental() 增量。增量的水位线复用了系统参数中心（第 5 章）：

```
@Transactional
public Map<String, Integer> syncIncremental() {
    var rows = jdbc.queryForList(
        "select cfg_value from sys_config where cfg_key = 'cdr_sync_watermark'");
    Instant since = rows.isEmpty() ? Instant.EPOCH
        : Instant.parse((String)
            rows.get(0).get("cfg_value"));
    var result = doSync(since);
    jdbc.update("""
        insert into sys_config(cfg_key, cfg_value, remark)
        values ('cdr_sync_watermark', ?, 'CDR 增量水位')
        on conflict (cfg_key) do update set cfg_value = excluded.cfg_value, updated_at
        = now()
        """, Instant.now().toString());
    return result;
}
```

抽取全部走 JdbcTemplate 原生 SQL 只读业务表（CDR 模块对业务模块零 Java 依赖，边界靠“只 SELECT”守住）；写入统一收口到 upsert()：按 docType + refId 查旧文档，有则改、无则建。于是全量与增量共用同一段写入逻辑，全量重跑永远安全——这就是“幂等覆盖”的含义：任何一次同步失败、水位错乱、数据可疑，管理员在患者 360 页点一次“立即同步”就恢复到正确状态。配置手册将 cdr_sync_watermark 列入“系统内部（勿手改）”段，由同步任务独占维护。

值得指出的诚实口径：当前增量同步是接口触发（`POST /api/cdr/sync-incremental`，ADMIN 权限），尚未任务化为定时作业——对照第 10 章对账“每天自动跑、有差异有人管”的标准，这里还差最后一步调度接入。

11.2.3 水位线的两个正确性陷阱

定时增量看似谱系中最朴素的方案，但它的正确性完全押在水位设计上。HIP 的实现恰好提供了两个真实的反面教材，比任何虚构例子都锋利。

陷阱一：水位字段选错，更新会漏。HIP 业务表（如 `outp_registration`、`inp_admission`）只有 `created_at` / `admit_at`，没有 `updated_at`，增量抽取只能按创建时间过滤。后果是：凡“创建在水位前、变更在水位后”的记录，增量永远抓不到。最典型的是检验链——医嘱开单时 `created_at` 已定，几小时后标本上机、结果发布、状态变为 `EXECUTED`，而增量条件是 `order_type = 'LAB' and status = 'EXECUTED' and created_at > 水位`：报告发布时开单时间早已沉到水位之下，这份新鲜出炉的报告增量同步捞不到。住院同理：入院在水位前、出院在水位后，CDR 里的住院摘要停留在“在院”状态。当前实现靠全量同步兜底修正，而 `CdrController` 上“增量同步（`updated_at` 水位）”的注释与代码实况（`created_at` 过滤）不符——教材如实指出：修这个问题的正道不在同步代码，在上游模型：业务表补 `updated_at` 列（触发器或 ORM 统一维护），水位字段换过去。水位字段的选择，决定了增量的正确性边界。

陷阱二：水位推进过头，窗口期会漏。同步完成后，HIP 把水位推进到 `Instant.now()`——注意不是“本次抽到数据的最大 `created_at`”。在“执行查询”与“记录 now”之间的几秒里新落库的业务数据，`created_at` 小于新水位却未被本次抽走，下次增量也不会再看它：永久漏失。标准解法有二：水位取本批数据的 `max(created_at)`；或保留 now 水位但每次回看一个重叠窗（如水位减 5 分钟）。重叠会重复抽取——而这正是幂等覆盖模型的回报：重复无害，所以水位宁可保守回退，不可激进冒进。增量正确性的三根支柱——可靠的水位字段、保守的水位推进、幂等的写入——HIP 立起了第三根，前两根留作演进（也是本章习题 2、3 的素材）。

还有一处细节：门诊抽取用 `status <> 'CANCELLED'` 排除退号，但已同步后才退号的挂号，其 CDR 文档不会被任何一次同步触碰（源记录被过滤，覆盖不发生），成为残留的“幽灵文档”。删除语义是拉取式同步的天然盲区（日志 CDC 恰恰擅长此处），工程上以软删标记或对账清理弥补——习题 4 请你设计。

11.3 患者 360 视图：跨域聚合与时间轴

11.3.1 聚合边界怎么划

患者 360 指以患者为中心聚合其全部临床数据的调阅视图。“全部”是理想，工程上必须回答：哪些域进视图？判据有三：临床调阅价值（接诊场景最常翻什么）、数据成熟度（源头数据是否完整可信——垃圾聚合进来是负价值）、权限与隐私（精神科、感染科记录的展示需要额外管控）。业界成熟产品的 360 视图通常覆盖就诊、诊断、医嘱用药、检验检查、手术、护理、过敏史等十余域；起步实现则应从“高频 + 数据干净”的域切入。

HIP 的聚合边界即 11.1 的三类文档：门诊就诊、检验报告、住院摘要。边界之外同样值得明说：检查报告（RIS）、手术麻醉、护理记录未进 CDR；检验文档只覆盖门诊医嘱链，住院检验未进标本流转链路（第 9 章已披露该简化）——这些是文档类型清单上的下一批条目，管道不需要为它们改动。

11.3.2 时间轴呈现与脱敏联动

案例框 11-3：患者 360 页面——聚合的消费端

患者 360 页（Patient360View.vue，菜单权限 `cdr:view360`）是典型的“左检索右时间轴”布局：左栏按姓名/患者号/证件/手机检索患者，右栏将 `GET /api/cdr/patients/{id}/documents` 返回的文档渲染为时间轴（Element Plus 的 `el-timeline`），三类文档以颜色区分，单选按钮按 `docType` 过滤（全部/门诊/检验/住院），点“查看”弹出格式化后的 JSON 内容。页头一枚“立即同步”按钮直连全量同步——把 11.2 的兜底能力交到使用者手里。

值得端详的是左栏检索走的是 **EMPI** 的患者接口（PatientController.java），而非 CDR 自建患者查询。这带来两个免费的正确性：一是患者身份口径与全院一致（EMPI 是唯一主索引，第 5 章）；二是脱敏策略自动继承——该接口对非 ADMIN 角色返回的手机号（前 3 后 4）与证件号（前 4 后 3）已打码，360 视图零代码获得等保要求的列表脱敏。复用底座能力而不是各页自造，是脱敏这类横切策略不出豁口的唯一办法。

但豁口仍有一处：`GET /api/cdr/documents/{id}/cda` 输出的 CDA 文档带明文证件号，且该端点未加角色限制——文档内容路径绕开了列表路径的脱敏。这不是疏忽的辩护而是教学的样本：脱敏必须按“数据项”定义策略，而不是按“页面”逐处实现，否则每新增一个数据出口（导出、打印、接口、文档）都是一次重新发明。修复方向见习题 8。

聚合结构全景如图 11-2：

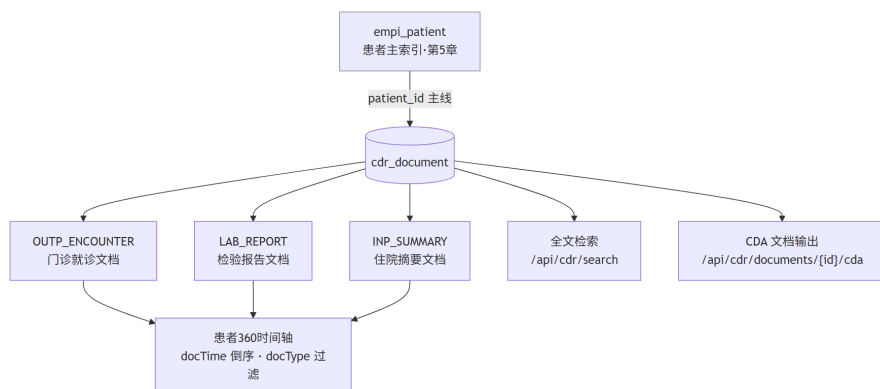


图 11-2 患者 360 的聚合结构：EMPI 定身份，CDR 供文档，三个消费出口共用一份快照

图中的要点是箭头方向：EMPI 与 CDR 之间以 `patient_id` 为唯一主线（总体规划的数据主线原则），三个消费出口（时间轴、检索、CDA）全部只读 `cdr_document`——业务库在这张图上根本不出现，这就是 11.1 隔离论证兑现后的样子。

11.4 全文检索：病历检索的实现路径

11.4.1 三条路径的谱系

“按关键词找病历”的实现路径按数据量与查询复杂度分三档：

1. **LIKE/ILIKE** 子串匹配：零依赖、零同步，任何量级的开发成本都最低；代价是无法走普通索引（前置通配符），每次查询顺序扫描。对中文友好是它被低估的优点——中文字串匹配不需要分词。
2. **PostgreSQL** 全文检索（`tsvector` + GIN 索引）：库内解决，无需新组件；但 PG 内置解析器不支持中文分词，需装 `zhparser/pg_jieba` 之类扩展（信创库上未必可得），且检索语义从“子串”变为“词元”，“心肌梗死”能否命中取决于词典切分。
3. **Elasticsearch/OpenSearch**：专业检索能力（中文分词、相关度排序、高亮、聚合）最完整；代价是引入一整套需要喂内存、管集群、做数据同步的基础设施——CDR 到 ES 的同步管道本身又是一个 11.2 的问题。

选型判据与第 6 章消息中间件之辩同构：用数据量说话。几十万份文档、秒级可接受的场景，LIKE 扫描完全够用；到了千万级文档或需要相关度排序、检索报表时，再上专业引擎。

11.4.2 HIP 的起步版与演进位

案例框 11-4：先把检索入口立住

HIP 的检索实现（CdrDocumentRepository.java）是一条 JPQL：

```
/** 全文检索起步版：ILIKE 匹配标题与内容（中文子串可用；数据量大后迁 OpenSearch） */
@Query("""
    from CdrDocument d where d.title like concat('%', :kw, '%')
    or d.content like concat('%', :kw, '%')
    order by d.docTime desc
    """)
List<CdrDocument> search(@Param("kw") String keyword, Pageable pageable);
```

对外即 GET /api/cdr/search?keyword=，限 50 条。两点剖析：其一，这次全表扫描扫的是 cdr_document 而不是业务库——检索再慢也慢在 CDR 自己身上，收费窗口毫发无损。11.1 的分离论证在此第二次兑现：把数据先聚到一处，低效的查询才有资格存在。其二，检索方案的升级被架构“定了位”：入口（API）、语料（cdr_document 的 title+content）、同步（11.2 管道）都已就位，未来迁 tsvector 是加一列生成列与 GIN 索引，迁 OpenSearch 是给同步管道加一个下游——调用方不感知。注释里“数据量大后迁 OpenSearch”不是一句敷衍，而是有承接点的演进声明；总体规划的技术栈也早为 ES/OpenSearch 留了位（病历检索、EMPI 模糊匹配两个消费方）。

顺带一个诚实订正：注释自称 ILIKE（不区分大小写），JPQL 实际生成的是 LIKE——对中文检索两者无差别，对夹杂的英文药名（“Amoxicillin” vs “amoxicillin”）则有；这类注释与实现的毫米级偏差，正是代码评审该盯住的东西。

11.5 CDA 文档生成：从业务数据到标准骨架

11.5.1 CDA 的层次模型

院内调阅用 JSON 快照足矣，跨机构共享则需要标准格式。CDA（Clinical Document Architecture，临床文档架构）是 HL7 的临床文档标准（第 3 章），一份 CDA 文档分两半：文档头（Header）——文档类型、患者、作者、机构、时间等元数据，高度结构化，支撑文档注册与检索；文档体（Body）——若干章节（section），每节可以是叙述文本，也可以是机器可读的条目（entry）。按文档体的结构化程度，业内分三级：Level 1 整体非结构化（如嵌入 PDF）、Level 2 章节化叙述文本、Level 3 条目级结构化（可直接抽数据）。我国的电子病历共享文档规范（WS/T 500 系列）在 CDA R2 基础上定义了病历概要、门急诊病历、检验报告等数十类文档模板【成稿核对：WS/T 500 系列的准确名称、文档类目数及现行有效版本】，互联互通测评考察的共享文档即以此为准。

理解三级结构的意义在于成本曲线：Header 结构化是文档注册的底线，成本低收益大；Body 到 Level 3 则要求每个观察值映射标准术语（诊断到 ICD、检验到 LOINC——第 12 章术语映射的用武之地），成本陡增。实施策略因此是“骨架先行”：先让每份文档有合规的头和成形的体，条目级结构化按文档类型分批深化。

11.5.2 HIP 的骨架实现

案例框 11-5：一个 text block 生成的 ClinicalDocument

HIP 的 CDA 输出（`CdrSyncService.toCda()`）按文档 id 取出 CDR 快照，回查 EMPI 患者信息，用 Java text block 模板拼出 CDA 样式 XML：

```
return ""
    <?xml version="1.0" encoding="UTF-8"?>
    <ClinicalDocument xmlns="urn:h17-org:v3">
      <typeId root="2.16.840.1.113883.1.3" extension="POCD_MT000040"/>
      <code displayName="%s"/>
      <title>%s</title>
      <effectiveTime value="%s"/>
      <recordTarget><patientRole>
        <id extension="%s"/>
        <patient><name>%s</name><administrativeGenderCode code="%s"/>
          <birthTime value="%s"/></patient>
      </patientRole></recordTarget>
      <component><structuredBody><component><section>
        <text><![CDATA[%s]]></text>
      </section></component></structuredBody></component>
    </ClinicalDocument>
    """.formatted(doc.getDocType(), doc.getTitle(), doc.getDocTime(), ...);
```

按 11.5.1 的坐标系给它定位：Header 有典型骨架（`typeId`、`code`、`effectiveTime`、`recordTarget` 患者信息），Body 是单 section、内容整体装进 CDATA——介于 Level 1 与 2 之间的骨架版。与合规文档的差距同样要列清楚：文档 code 用 `docType` 字符串而非标准文档类型编码及 OID 体系；机构、作者（`author/custodian`）节缺失；无 entry 级条目。接口注释“WS/T 500 结构骨架，字段映射简化版”就是这个口径的自我声明。

还有一个纯工程的教训藏在生成方式里：用字符串拼接生成 XML 没有做转义——患者姓名若含 & 或 <（数据污染、测试数据都可能），输出即非法 XML。玩具阶段无妨，接入区域平台前必须换成 DOM/JAXB 类生成器或至少补转义。“能跑”与“能对外”之间隔着的往往正是这类边角。

从业务数据到标准文档的完整通路如图 11-3——注意 CDA 生成消费的是 CDR 快照，而非现场回查业务库，这使文档输出天然获得同步链路的全部隔离收益：

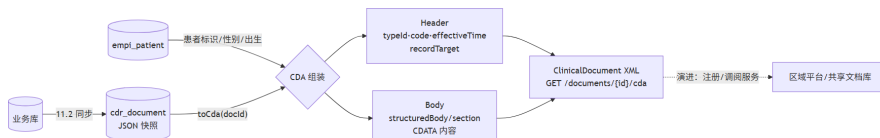


图 11-3 CDA 文档生成流程：快照进、标准骨架出

11.6 数据订阅与推送：向外供数的服务化

11.6.1 拉与推

区域平台、随访系统、商保理赔等第三方要医院的数据，模式无非两种：拉——对方按需调医院的查询/调阅服务（11.5 的文档调阅即属此类）；推——医院在数据事件发生时主动通知订阅方（患者建档、报告发布、出院……）。推模式的标准形态是订阅注册 + Webhook 回调：订阅方登记“我关心什么事件、回调到哪个地址”，医院侧在事件发生时向目标地址 POST 事件负载。工程上要立三根柱子：留痕（每次推送落流水，成功失败可查——与第 10 章“留痕即对账依据”同源）；重试与幂等（对

方接口会抖，重推必然发生，事件负载须带唯一事件 id 供对方去重）；安全（目标地址白名单、签名鉴权、负载按订阅方权限脱敏——向第三方推明文证件号是数据合规事故）。

11.6.2 HIP 的管道先行

案例框 11-6: Mock 推送——又一次“把管道铺通”

HIP 的订阅推送落在数据治理模块（DataStdController.java，建表见 V23__phase26_datastd.sql）：dg_subscription 存订阅（event_type 如 PATIENT_CREATED/LAB_PUBLISHED/DISCHARGE、subscriber、target_url、enabled 启停），dg_push_log 存推送流水（负载、结果 SUCCESS/FAILED/MOCK）。管理接口齐备：登记、列表（带累计推送数）、启停切换、按订阅查流水。而“推”本身是这样的：

```
/** 测试推送 (Mock: 不出网, 落推送流水验证链路) */
@PostMapping("/subscriptions/{id}/test-push")
public R<Map<String, Object>> testPush(@PathVariable Long id) {
    var subs = jdbc.queryForList(
        "select * from dg_subscription where id = ? and enabled", id);
    if (subs.isEmpty()) return R.fail(4633, "订阅不存在或已停用");
    String payload = "{\"event\":\"\" + subs.get(0).get("event_type") + "\",
    \"test\":true}";
    jdbc.update("insert into dg_push_log(subscription_id, payload, result) values
    (?, ?, 'MOCK')",
        id, payload);
    return R.ok(Map.of("pushed", true, "target", subs.get(0).get("target_url")));
}
```

规划与落地的口径要说满：落地的是管道——订阅模型、启停控制、推送留痕、链路验证；未落地的是真实推送——没有业务事件触发（第 6 章的应用事件如检验发布事件是现成的挂接点），没有出网 HTTP 调用，没有重试。这是第 10 章审核雏形“管道自建、内容可换”方法论的重演：区域平台的接入协议、地址、证书是典型外部条件，条件到位前把院内侧的每一块板都钉好，到位后要写的只剩一个真实的推送执行器。习题 7 请你把这块板补上。

顺带指出订阅推送与 CDR 的关系：推送的事件来自业务动作（发布、出院），推送的数据体则宜取 CDR 文档（如 LAB_PUBLISHED 推送该检验的 CDR 快照或 CDA）——事件流与文档库各司其职，这也是把订阅推送放进本章而不是集成章的原因。

本章小结

- CDR 与业务库分离的两条理由：查询分析不拖垮 OLTP（负载画像冲突），以及互联互通要求的“行→文档”语义转换。分离先分逻辑边界（独立表/独立查询路径），物理部署按数据量分期兑现。
- HIP 的文档模型是一张 cdr_document：docType + refId 唯一键把幂等建在数据模型上，(patient_id, doc_time desc) 索引精确服务时间轴；三类文档（门诊/检验/住院）是 53 类蓝图的管道验证版。
- 同步技术谱系五族（双写/触发器/日志 CDC/定时增量/事件驱动），中小医院务实解是定时增量 + 全量幂等兜底。增量正确性三支柱：可靠水位字段（created_at 会漏更新——检验报告发布、住院出院都是“创建在前、变更在后”）、保守水位推进（取 max(created_at) 或回看重叠窗，勿取 now）、幂等写入（重复无害是前两者可以保守的前提）。删除语义是拉取式同步的盲区。

- 患者 360 的聚合边界按“调阅价值 + 数据成熟度 + 权限”划定；患者检索复用 EMPI 接口使脱敏策略自动继承（第 5 章），而 CDA 出口的明文证件号提醒：脱敏要按数据项定义策略，不能按页面逐处实现。
- 全文检索三档路径（LIKE / PG tsvector / ES），按数据量选档。CDR 让低效检索有资格存在（扫描不伤 OLTP），也让检索方案可原位升级（入口、语料、同步管道不变）。
- CDA 分 Header/Body，结构化分三级；实施走“骨架先行”，Header 合规支撑文档注册，条目级结构化按文档类型分批深化。HIP 的 text block 骨架介于 Level 1 - 2，未做 XML 转义是“能跑”到“能对外”之间的典型欠账。
- 订阅推送三根柱子：留痕、重试与幂等、安全（白名单/签名/按订阅方脱敏）。HIP 落地了订阅模型与 Mock 链路，真实推送等外部条件——又一次“管道先行”。

思考与练习

1. 概念：分别用一个具体查询例子说明 CDR 与 ODR 的职责差异；解释“只读副本”为什么不能替代 CDR（提示：负载隔离与语义转换两个维度）。
2. 设计：为 HIP 的 `outp_registration`、`outp_order`、`inp_admission` 补充 `updated_at` 列，给出 DDL 与维护机制（数据库触发器或 ORM 回调，二选一并论证），改写 `syncLabReports` 的增量条件，使“开单在水位前、报告发布在水位后”的检验能被增量捕获。
3. 排障：某医院 CDR 增量同步每 10 分钟一次运行正常，但护士反映“偶尔有患者的门诊记录始终不出现在 360 视图，手工点全量同步就有了”。结合 11.2.3 的水位推进陷阱，给出根因假设、验证方法与两种修复方案的对比。
4. 推演：已同步进 CDR 的挂号随后被退号，产生“幽灵文档”。设计一个清理方案：比较“同步时额外抽取 CANCELLED 记录并打软删标记”与“每日对账扫描 CDR 反查源记录状态”两条路线的成本与时效，说明你的选择。
5. 设计：将 HIP 的检索升级为 PostgreSQL 全文检索：给出 `cdr_document` 增加 `tsvector` 生成列与 GIN 索引的 DDL、查询 SQL，说明中文分词扩展缺失时该方案的退化行为，并给出“何时应当放弃库内方案迁 OpenSearch”的量化判据。
6. 论述：CDA Level 3（条目级结构化）要求诊断、检验结果映射标准术语。结合第 12 章的 `dg_term` 术语映射表，论述把 HIP 的检验报告文档做到 Level 3 需要打通哪些环节（术语覆盖率、缺失映射的兜底、文档模板），成本集中在哪里。
7. 实践（配套实训环境）：为订阅推送补上真实执行器——监听检验发布事件（第 6 章 `LabResultReceivedEvent`），查询 `dg_subscription` 中 `LAB_PUBLISHED` 的启用订阅，向 `target_url` 发起 HTTP POST（负载含唯一事件 id 与 CDR 文档快照），成功/失败落 `dg_push_log`，失败进入至多 3 次的间隔重试。写出事件监听器骨架与幂等约定，并说明推送失败不得影响检验发布主流程的事务边界怎么画。
8. 安全设计：11.3.2 指出 CDA 端点输出明文证件号且未做角色控制。设计一个“按数据项集中定义”的脱敏方案：敏感字段清单与打码规则收口到平台层（参考 `PatientController.mask` 的规则），列表、详情、CDA、订阅推送四类出口统一调用；说明 ADMIN 例外与“对外推送永远脱敏”两条策略如何共存。

第 12 章 数据治理、质控与统计上报

本章属第四篇「数据篇」。如果说第 11 章的 CDR 解决“数据放在哪、如何看”，本章解决的是“数据对不对、如何管、如何报”。数据治理、病案管理、医疗质控、院感监测、统计上报，在组织架构上分属信息科、病案室、质控科、院感科、统计室，在系统里却共享同一批上游数据——它们是同一件事的五个切面：让业务系统沉淀的数据经得起核对、汇总与上报。全章以 HIP 平台的数据中心 11 个功能页为贯穿案例，并深挖一个最有工程张力的设计：自定义报表引擎的 SELECT 白名单。

学习目标

学完本章，你应当能够：

1. 说出数据治理三件套（数据标准、术语映射、质量核查）各自解决的问题，理解卫生信息数据元标准在其中的地位；
2. 解释病案首页在 DRG 支付时代的价值重估，画出“出院—首页汇编—编码—归档”的病案流程，说明编码质量如何逐级传导为医院收入；
3. 设计一组“可归零”的数据质量核查规则，理解反查式 SQL（not exists）在时限质控中的用法；
4. 写出三管千日感染率（VAP/CLABSI/CAUTI）的分子分母定义，说明为什么用“千日率”而不是百分比，并指出数据从哪些护理与医嘱环节来；
5. 画出传染病报卡“报告—审核—上报”的闭环状态机，说出甲乙丙类传染病的报告时限差异；
6. 剖析自助报表引擎的安全模型：说出 SELECT 白名单校验的每一层防线、它挡住了什么、没挡住什么、还能怎样加固；
7. 解释 DDD/AUD 的 WHO 方法学与计算口径，说明日报表与交款核对在财务内控中的作用。

12.1 数据治理框架

12.1.1 从“有数据”到“数据可信”

医院业务系统每天产生海量数据，但“有数据”与“数据可信”之间隔着三个问题：同一个字段各系统叫法、格式、值域不一致（性别是“男/女”还是“1/2”？）；同一个概念各系统编码不一致（本院药品字典的“阿莫西林胶囊”对应哪个标准编码？）；该有的数据没有（出院了没写出院小结、诊断没有编码）。数据治理（Data Governance）就是针对这三个问题的三件套：

治理手段	解决的问题	依据
数据标准（数据元）	字段的名称、格式、值域统一	卫生信息数据元目录（WS 363 系列）与数据元标准化规则（WS/T 303）【成稿核对：标准号与年份】
术语映射	本地编码与标准编码的对照	ICD-10、ICD-9-CM-3、LOINC、ATC 等分类编码体系（第 3 章）
质量核查	完整性、及时性、一致性缺陷的发现与归零	各类上报与评审的数据质量要求

治理不是一次性的“数据清洗项目”，而是嵌在日常运行里的机制：标准在建库时内建（第 3 章“标准内建”策略），映射随字典维护持续更新，核查每天跑、缺陷有人管。

12.1.2 数据元与术语映射

数据元（Data Element）是数据的最小语义单元。国家卫生信息数据元目录给每个数据元一个全局标识（如 DE02.01.039.00 本人姓名）、数据类型（字符/数值/日期/编码）、表示格式（如 A..50 表示最长 50 位字符、YYYYMMDD 表示日期格式）与允许值域。医院把自己的库表字段登记到数据元目录上，互联互通测评（第 1 章）核查的正是这种对齐程度。

术语映射解决“本地叫法 → 标准编码”：诊断映射 ICD-10、检验项目映射 LOINC（Logical Observation Identifiers Names and Codes）、药品映射 ATC（Anatomical Therapeutic Chemical，解剖学治疗学化学分类），映射结果供 CDA 文档生成（第 11 章）、区域平台上传与统计归并使用。

案例框 12-1：HIP 的数据元与术语表

HIP 的数据治理落在 datagov 模块（DataStdController.java），两张核心表（V23__phase26_datastd.sql）：

```
create table dg_data_element (
  code      varchar(32) not null unique, -- 数据元标识 (如 DE02.01.039.00)
  datatype  varchar(16) not null,        -- S 字符/N 数值/D 日期/B 布尔/C 编码
  format    varchar(64),                 -- 表示格式 (如 AN..18 / YYYYMMDD)
  value_set varchar(255),                 -- 允许值/值域说明
  std_ref   varchar(128), ...             -- 引用标准 (WS 363.2、WS/T 303...)
);
create table dg_term (
  category  varchar(32) not null,         -- DIAG/DRUG/LAB/EXAM/PROC
  local_name varchar(128) not null,
  std_code  varchar(64) not null,         -- ICD-10 / LOINC / ATC...
  std_system varchar(32) not null,
  constraint uq_dg_term unique (category, local_name, std_system)
);
```

两个设计点：一是 dg_term 的唯一约束落在（类别 + 本地名称 + 标准体系）——同一个本地术语可以同时映射 ICD-10 和另一套体系，但同一体系内不允许二义映射；二是数据元种子直接引用国标标识（DE02.01.039.00 本人姓名、DE02.01.040.00 性别代码引 GB/T 2261.1 值域），实施时以国标目录为底、按医院实际扩展。注意这只是登记与对照层：真正的价值在第 5 章的字典体系与第 11 章的 CDA 生成消费这份映射时兑现——治理表如果没有消费者，三个月后就是一张死表。

12.1.3 数据质量规则：核查而非报表

数据质量核查与统计报表的区别在于目标值：报表回答“是多少”，核查回答“有多少不该有的”——它的目标永远是零。HIP 内置六条完整性规则（DataGovController.java）：患者缺证件号、患者缺手机号、门诊病历未签名、出院未归档病案、入院诊断未编码、在途未审处方。每条规则是一句 count SQL，返回缺陷数与是否通过（count == 0）。

规则内容朴素，但三条设计原则值得记取：

1. 规则必须“可归零”。“平均住院日 9.2 天”不是质量规则（没有明确的对错），“出院超 3 天未归档 17 份”才是——每一条缺陷都能落到具体单据、具体责任人；
2. 规则挂在治理页，缺陷处理挂在业务页。核查页发现“入院诊断未编码 12 例”，处理动作在住院医生站补录——治理系统是探照灯，不是手术刀；
3. 规则集应随上报要求生长。本章后半的每一类上报（病案、院感、单病种）都会反向催生新的核查规则，12.2 的编码率、12.6 的漏报候选都是这个逻辑。

12.1.4 数据订阅与推送

治理的最后一环是对外供数：区域平台、监管系统、第三方应用订阅医院的数据变更（患者建档、检验发布、出院……）。HIP 用 dg_subscription（事件类型+订阅方+目标 URL+启停）与 dg_push_log（推送流水）承载订阅关系，测试推送在 Mock 阶段不出网、只落流水验证链路——这是第 6 章“外部条件接入点先行”方法的又一次应用。与第 11 章 CDR 的关系可以一句话概括：CDR 管“来取”（查询服务），订阅推送管“送去”（事件驱动），两者共同构成医院数据服务的出口。

12.2 病案管理

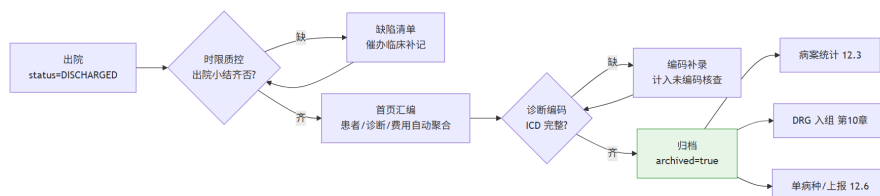
12.2.1 病案首页：被 DRG 重估的一页纸

病案首页是每份住院病案卷首的结构化摘要：患者标识、入出院时间与科室、主要诊断与其他诊断（ICD-10 编码）、手术及操作（ICD-9-CM-3 编码）、费用汇总、离院方式。在按项目付费时代，首页主要服务于病案检索与卫生统计，填错了顶多统计失真；DRG/DIP 时代它变成了结算凭据——第 10 章已经展示了因果链的下游：入组读取首页的主要诊断确定 ADRG，读取其他诊断比对 MCC/CC 目录确定细分组尾码，漏填一个重要合并症，病例从“伴严重并发症”掉到“不伴”，权重损失可达 30%。上游在此、下游在第 10 章：病案室的编码质量管理，从“业务考核指标”升格为“直接决定医院收入的生产环节”。

由此，首页数据的产生方式也在演进：手工誊写 → 系统辅助汇编 → 结构化自动生成。汇编的原则是能从业务数据取的绝不重录——患者信息取自 EMPI、诊断取自医生站、费用取自结算表，病案人员的工作聚焦在核对与编码，而不是抄写。

12.2.2 归档流程

病案从出院到可供利用，要走一条质控链（图 12-1）：



图中三个消费者（统计、DRG、上报）都以“已归档”为口径起点——归档不是把纸放进库房，而是宣告这份病案的数据从此冻结、可被引用。因此归档必须有前置校验（未出院不得归档），归档后修改要走解档流程并留痕（HIP 当前未实现解档，见案例框 12-2 的披露）。

案例框 12-2: HIP 的首页汇编与编码率口径

HIP 的病案功能在 `NursingQualityController.java`: `front-page` 接口把 `inp_admission` (住院)、`empi_patient` (患者)、`inp_settlement` (结算) 三表联查汇编首页数据集, 药费从医嘱表按 `order_type='DRUG' and status='EXECUTED'` 汇总; 归档接口用一条条件更新收口:

```
int n = jdbc.update(
    "update inp_admission set archived = true where id = ? and status = 'DISCHARGED'",
    id);
return n == 0 ? R.fail(9803, "仅出院病历可归档") : R.ok();
```

编码率在病案统计模块 (`MedRecordStatsController.java`) 的 `overview` 中计算: 出院病例中 `admit_diag_icd is not null` 的占比。这里要诚实披露 HIP 的简化: 它以入院诊断的 **ICD** 填充率代表编码率, 且没有独立的出院诊断表与手术操作 (**ICD-9-CM-3**) 编码字段。真实病案室的编码率以出院主要诊断/其他诊断/手术操作的编码完整率为准, 编码工位在归档流程中是独立环节 (专职编码员 + 编码质控抽查)。HIP 的结构留了演进位——第 10 章 **DRG** 入组消费的正是同一诊断字段, 补全出院诊断与手术编码表后, 编码率口径、**DRG** 入组、单病种匹配同步受益, 这再次印证“上游模型决定下游能力”。

与之配套的是 12.1.3 的核查规则“出院未归档病案”与“入院诊断未编码”——病案管理的日常运营就是盯着这两个数字归零。另有一张死亡登记卡 (`mr_death_card`, 直接死因 + 死因链 **B/C/D** 项), 对应死因监测上报的数据集, 属病案室的法定报表之一。

12.2.3 编码率管理的运营含义

编码率管理的抓手不是“月底通报一个百分比”, 而是把未编码病例做成待办清单: 谁的病例、出院几天了、缺什么。指标只帮管理者看趋势, 清单才驱动工作。这也是本章反复出现的模式——12.1.3 的核查规则、12.6 的入组候选、13 章的故障工单, 都是“指标 + 清单”双层结构: 指标示警, 清单派活。

12.3 病案统计

病案统计是医院最古老的统计业务: 以归档病案为数据源, 回答“医院收治了什么病人、做了什么、效率如何”。HIP 的病案统计 (`MedRecordStatsController.java`) 覆盖四张经典报表, 每张背后有一个统计学口径值得点破:

- **疾病谱 TOP20**: 出院主要诊断按 **ICD** 前 3 位归并 (`substring(admit_diag_icd, 1, 3)`)。ICD-10 的前 3 位是“类目”层级 (如 **J06** 急性上呼吸道感染), 归并到类目才能把 **J06.8**、**J06.9** 等亚目合成有管理意义的“病种”; 逐条亚目统计会碎成长尾。
- **ICD 章节构成**: 按首字母归并到 ICD-10 的 22 个章节 (如 **I** 循环系统、**J** 呼吸系统), 用于看学科结构的宏观盘子。
- **科室出院构成**: 出院人次、构成比、平均住院日。平均住院日按 `discharged_at - admit_at` 的日历日均值计算——注意它与第 10 章时间消耗指数的关系: 前者是绝对效率, 后者是难度校正后的相对效率, 管理上应成对看。
- **手术统计**: 按术式汇总例数、近 30 日例数与完成数。

工程上这四张表都是聚合 SQL 直查业务库。对县市级医院的数据量 (年出院数千至数万), 这完全够用; 数据量再大时的演进方向是把聚合切到 12.7 的指标快照或独立统计库——先用最简单的直查, 把复杂度留给被数据证明的瓶颈 (第 4 章的架构态度)。

12.4 医疗质控

12.4.1 时限质控：用反查抓缺陷

病历书写有法定时限：入院记录须在入院后 24 小时内完成，出院小结在出院后规定时限内完成，门诊病历当次完成并签名【成稿核对：《病历书写基本规范》具体时限条款，与第 8 章共用】。时限质控的实现要点是反查——不是查“谁写了”，而是查“到点了还没写的”：

```
select ... from inp_admission a
where a.admit_at < now() - interval '24 hours'
and not exists (select 1 from inp_medical_record r
                where r.admission_id = a.id and r.record_type = 'ADMISSION')
```

HIP 的时限质控接口（NursingQualityController.java 的 `emr-timeliness`）返回三类缺陷：入院超 24h 无入院记录（含超时小时数，便于分级催办）、出院无出院小结、门诊病历未签名，合计为缺陷总数。`not exists` 反查是一切“该做未做”类质控的通用句式，12.6 的单病种漏报候选是同一句式的变体。

12.4.2 抽检评分：把检查变成曲线

病历内涵质控（写得对不对、逻辑通不通）无法用 SQL 判定，靠抽检评分：质控科按标准抽查病历、护理文书、科室管理项目并打分。系统的价值不在打分本身，而在让同一计划下的多次评分连成时间曲线——单次 85 分没有意义，“从 78 爬到 91”或“骤降到 70”才是管理信号。HIP 用 `qc_check_plan`（计划+质控标准+是否临时表单）与 `qc_check_score`（对象+0-100 分+时间）两张表实现（NursingPlusController.java），评分按 `checked_at` 排序即得曲线。这是 PDCA（Plan-Do-Check-Act）循环的最小系统化：计划即 Plan，评分即 Check，曲线拐点触发 Act。

12.4.3 不良事件：匿名上报的制度设计

医疗不良事件（用药错误、跌倒、压疮、非计划再手术……）管理的第一难题不是技术而是报告文化：如果上报等于自认差错，就没人报。国际通行的对策是非惩罚性上报制度，落到系统上是一个具体的设计决策——支持匿名。HIP 的 `qc_adverse_event` 表在匿名上报时把 `reporter_id` 置空（而不是记录后隐藏——数据库里根本没有，才谈得上匿名可信），事件按 1-4 级分级【成稿核对：不良事件分级对应中国医院协会 I-IV 级定义】，按“未处理优先、级别升序”排序呈现，处理走 `NEW → HANDLED` 状态并记录处理意见。分级排序的细节有深意：1 级（最严重）排最前，质控科打开页面看到的永远是最该先管的事。

12.5 医院感染监测

12.5.1 两种时间视角

医院感染（Healthcare-Associated Infection, HAI）监测有两种流行病学视角：发病率监测——连续跟踪一段时间内的新发感染（本节的三管监测）；现患率调查——某个时点横断面清点在院患者中的感染现患（12.5.3）。前者反映过程、可归因到具体环节，后者成本低、适合全院普查与横向比较，两者互补而不可互代。

12.5.2 三管干日率：分子、分母与数据来源

三种侵入性导管相关感染是院感监测的核心专项：VAP（Ventilator-Associated Pneumonia，呼吸机相关肺炎）、CLABSI（Central Line-Associated Bloodstream Infection，中央导管相关血流感染）、CAUTI（Catheter-Associated Urinary Tract Infection，导尿管相关尿路感染）。它们的标准指标都是千导管日感染率：

千日率 = 感染例次数 / 导管留置总日数 × 1000

为什么分母是“导管日”而不是“置管人数”？因为感染风险与暴露时间成正比：留置 30 天与留置 2 天的风险完全不同，按人数算百分比会让长置管科室（ICU）与短置管科室不可比。以导管日为分母是流行病学的人时（**person-time**）发病密度方法——把暴露时间标准化后，不同科室、不同医院的率才可横向比较。分子分母的口径要点：

要素	标准口径	HIP 实现口径
分子	监测期内判定的该类感染例次	<code>inf_catheter</code> 中 <code>status='INFECTED'</code> 的导管数（每根导管至多计 1 例次）
分母	每日清点带管人数累加（导管日）	按置管起止时间折算天数，不足 1 日按 1 日计（ <code>greatest(days, 1)</code> ），符合按日历日登记惯例）
分类	按导管类型分别计算	<code>line_type</code> 三值：VENT/CVC/URINARY，分组统计

案例框 12-3：HIP 的三管监测闭环与数据来源剖析

HIP 的院感专项（`InfectionPlusController.java`）把三管监测做成一个小状态机：置管登记（校验在院、同类型不得重复监测）→ 日评估（每日一次，`on conflict (catheter_id, assess_date) do update` 幂等，评估结论是“是否继续留置”）→ 拔管或判定感染。判定感染是一个事务内的双写：导管置 `INFECTED` 并记感染日期，同时按导管类型自动生成对应部位的院感病例登记（`qc_infection_case`）——三管监测与院感病例库由此保持一致，不需要两头录。

千日率一条 SQL 出结果：

```
select line_type, count(*) filter (where status = 'INFECTED') as infections,
       round(count(*) filter (where status = 'INFECTED') * 1000
             / sum(greatest(extract(epoch from (coalesce(end_at, now()) - start_at))
                             / 86400, 1))::numeric, 2) as per_1000_days
from inf_catheter group by line_type
```

数据从哪里来，是这个专项最值得追问的问题。HIP 当前的置管与日评估由院感专职/护士手工登记——这是可运行的最小闭环，但存在漏登风险（没登记的置管，分母偏小、千日率虚高）。数据其实早已存在于上游：呼吸机使用是医嘱（第 8 章 `inp_order`）、导管护理是护理记录、体温是护理体征（`inp_vital_sign`）。演进方向是从医嘱自动开启监测（呼吸机/置管类医嘱下达即建 `inf_catheter`，停嘱即触发拔管确认），让手工登记退化为例外补录。同一控制器里的术后发热监测已经示范了这种“从既有数据推质控信号”的路子：`inp_surgery` 关联 `inp_vital_sign`，抓术后 72 小时内体温 $\geq 38^{\circ}\text{C}$ 的在院患者，护理体征一次录入、多处消费（体温单在第 8 章，这里是院感线索）。

12.5.3 现患率调查

现患率调查（Point Prevalence Survey）在调查日清点全部在院患者，统计正在发生的院感例数：现患率 = 感染人数 / 在院人数 × 100%。全国医院感染监测网每年组织横断面调查，医院按统一调查日执行【成稿核对：全国现患率调查的组织方式与目标值】。HIP 的 `prevalence` 接口实时生成调查表快照：逐患者列出科室、诊断、是否感染（`exists` 子查询关联院感病例）与感染部位聚合，汇总出在院数、感染数与现患率。注意 HIP 的口径是“任意时刻实时快照”，正式调查要求固定调查日的统一时点口径——使用时应在调查日当天生成并留存。

12.5.4 传染病报卡闭环

法定传染病报告是《中华人民共和国传染病防治法》规定的强制义务：甲类传染病（及按甲类管理的）城镇须在 2 小时内网络直报，乙类 24 小时内【成稿核对：甲/乙/丙类报告时限的现行条款】。院内流程是三步闭环（图 12-2）：临床医生发现即报卡，院感/防保科审核（退回补正或通过），审核通过后上报疾控网络直报系统。



HIP 的实现（NursingPlusController.java，表 `idc_report_card`）用状态机严格约束流转：审核只接受 `REPORTED` 状态，上报只接受 `REVIEWED` 状态（条件更新 + 判定受影响行数的并发定式，与第 7/8/10 章同款），报卡类别用 `CHECK` 约束限定 A/B/C。两处诚实披露：甲类 2 小时时限当前靠前端提示而非系统倒计时预警（习题 5 让你补上）；与国家网络直报系统的接口是外部条件，当前闭环止步于院内 `SUBMITTED` 状态——接入点已留（状态机与时间戳齐备），对接形态届时按第 6 章的适配器方法处理。

12.6 单病种上报与评审指标

单病种质控是国家针对重点病种（急性心肌梗死、脑梗死、心力衰竭、肺炎等）的过程质控制度：每收治一例目标病种，医院须按统一数据集上报诊疗过程关键节点【成稿核对：国家单病种质控平台的病种清单与数据集版本】。上报的第一难题是漏报——临床没意识到某病例属于目标病种。对策与 12.4.1 的反查同源：让系统找人，而不是人找系统。

案例框 12-4：ICD 前缀自动入组

HIP 的单病种模块（`SingleDiseaseController.java`）用一张病种定义表 `sd_disease_def(code, name, icd_prefix)`（种子：AMI→I21、STROKE→I63、CAP→J15、HF→I50），入组候选一条 SQL 扫出来：

```

select ... from inp_admission a
join sd_disease_def d on a.admit_diag_icd like d.icd_prefix || '%'
where not exists (select 1 from sd_case_report c
                  where c.admission_id = a.id and c.disease_code = d.code)
  
```

诊断 ICD 前缀命中病种、且尚未建卡的住院，全部进入候选清单——这是“指标 + 清单”模式的又一实例，也再次踩在病案编码质量的因果链上：诊断没编码，自动入组就是瞎子（12.2 的编码率在此兑现价值）。上报卡走 `DRAFT` → `REPORTED` 状态，上报校验数据集非空；当前 `dataset` 为自由文本，结构化数据集定义与国家平台接口属演进项。

评审指标是另一条上报线：公立医院绩效考核与等级评审要求的运营指标集。HIP 在指标引擎（12.7.2）上内置 9 项：当日门诊人次、药占比、床位使用率、门诊均次费用、在院患者数、平均住院日、抗菌药处方占比、门诊病历签名率、检查报告审核率（`dg_metric_def` 种子 M001 - M009，部分带目标值如药占比 30%、平均住院日 9 天）。指标的技术载体见下节——它们正是指标快照机制的第一批用户。

12.7 自定义报表引擎：SELECT 白名单的安全设计

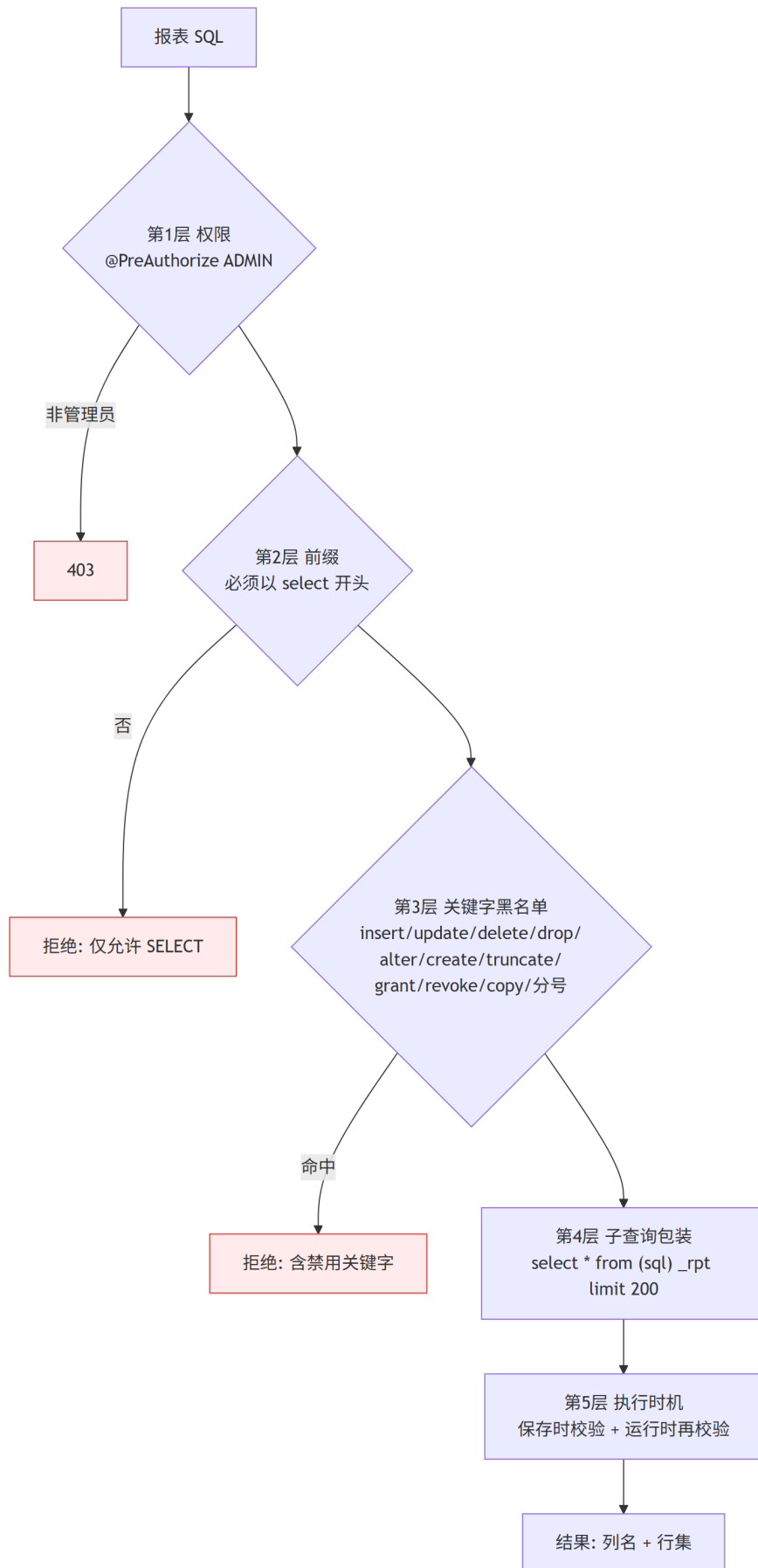
12.7.1 自助查询的需求与风险

统计需求有一条长尾：医务科今天要“按科室的抗菌药占比”，明天要“周四下午的挂号构成”。每个需求都走开发排期，信息科会被报表淹没；于是“自助报表”成为刚需——让管理员直接写 SQL 生成报表。但直接执行用户提交的 SQL，等于把注入攻击的完全体拱手奉上：与第 7 章讲的参数化查询防注入不同（那是防“数据被当成代码”），这里用户提交的本来就是代码。风险具体有三类：写操作破坏数据（update/delete/drop）、越权读取（查 `sys_user` 的口令散列、查患者隐私）、资源耗尽（笛卡尔积联表、pg_sleep 拖死数据库）。

安全模型因此不能靠“过滤坏东西”的心态，而要靠分层收窄：每一层各挡一类风险，任何单层被绕过仍有下一层兜底（defense in depth）。

12.7.2 五层防线的实现与剖析

HIP 报表引擎（DataStdController.java）的校验流程如图 12-3：



案例框 12-5: validateSql 逐层细读——挡住了什么，没挡住什么

```
private static final Set<String> FORBIDDEN = Set.of(
    "insert", "update", "delete", "drop", "alter", "create",
    "truncate", "grant", "revoke", "copy", ";");

private String validateSql(String sql) {
    if (!sql.toLowerCase().trim().startsWith("select")) return "仅允许 SELECT 查询";
    for (String kw : FORBIDDEN)
        if (lower.contains(kw)) return "包含禁用关键字: " + kw;
    return null;
}
// 执行: jdbc.queryForList("select * from (" + sql + ") _rpt limit 200")
```

逐层评估这个设计的攻防账：

- 前缀 **select** + 禁分号合起来封死了“追加第二条语句”的经典注入路径（`select 1; drop table ...`）；禁 **copy** 封死 PostgreSQL 的文件读写通道；
- 子查询包装是精巧的一笔：把用户 SQL 塞进 `from (...)` 里，语法上强制它必须是一个可作为子查询的查询表达式——`select ... into` 建表这类“以 **select** 开头的写操作”会在此处语法报错；同时 `limit 200` 硬性截断结果集；
- 保存时 + 运行时双重校验防的是 TOCTOU（校验与使用分离）：有人绕过接口直改 `dg_report_def` 表里的 SQL，运行时的第二道校验仍会拦截；
- 黑名单用子串匹配，是保守方向的误伤：查询列 `updated_at` 会因含“**update**”被拒。误伤的代价是可用性（换列别名可绕开），换来的是实现只有十行、无解析器依赖——在“仅管理员可用”的场景下，这笔账是划算的；
- 没挡住的：其一，ADMIN 仍可 **SELECT** 读全库，包括 `sys_user` 的口令散列与患者隐私字段——当前模型信任管理员，防的是破坏而非越权读；其二，`limit 200` 只限返回行数不限计算量，恶意或笨拙的笛卡尔积、`pg_sleep()` 仍可拖慢数据库。

由此得出加固路线图（按收益排序）：为报表引擎单独建只读数据库账号（连接层硬隔离，黑名单从“防线”降格为“提示”，这是最该做的一步）；会话级 `statement_timeout` 限死执行时长；表级白名单（只允许查视图层，敏感列在视图中脱敏——与第 13 章按角色脱敏同一思想）。黑名单是启发式，账号权限才是边界：把安全性押在字符串过滤上的系统，迟早某个没想到的关键字上翻车；把过滤当成第一道减速带、把数据库权限当成真正的墙，才是这个设计的正确读法。

12.7.3 指标快照：口径冻结与趋势留存

报表引擎解决“临时问”，指标快照解决“天天看”。HIP 的指标引擎三件套：`dg_metric_def`（指标定义：编码、名称、目标值、`builtin_key`）、代码内的 `BUILTIN` 映射（`builtin_key` → 计算 SQL）、`dg_metric_snapshot`（按日快照，`unique(code, snap_date)`，写入用 `on conflict do update` 幂等覆盖——当日重跑取最新值）。为什么不每次实时算？三个理由：趋势——昨天的药占比实时算不出来，历史值必须当天存下；口径冻结——业务数据会被退费、补录修改，快照留下“当日所见”；性能——运营驾驶舱（ODR 汇总接口把各域今日概况与最新指标快照拼成一页）高频刷新，不应反复全表聚合。计算 SQL 放代码而非配置表，与报表引擎恰成对照：内置指标要口径稳定、随版本受控变更，自助报表要灵活、随需求即时变化——同是“SQL 产出数字”，治理策略因变更频率而分野。

12.8 运营分析

12.8.1 药事分析：DDD 的方法学

比较抗菌药使用强度，不能比金额（价格干扰）也不能比盒数（规格干扰），WHO 的解法是 DDD（Defined Daily Dose，限定日剂量）：ATC 分类下为每个药物设定一个标准日剂量，任何消耗量都可折算为“用了多少个标准日”—— $DDDs = \text{消耗总量} / \text{该药 DDD 值}$ 。在此之上，住院抗菌药使用强度 AUD（Antibacterial Use Density）= 抗菌药 DDDs $\times$ 100 / 同期收治患者床日数，即“每百人天消耗多少个标准日剂量”，专项整治的通行目标值为不超过 40【成稿核对：AUD 目标值出处】。分母用床日数与 12.5 的千日率同理——按暴露时间标准化。

HIP 的药事分析（DrugAnalysisController.java）给出用药 TOP30、中西药构成、月度经费与 AUD 雏形：药品字典 md_drug 带 ddd_per_unit（每最小单位折 DDD 数）， $\text{sum}(\text{qty} \times \text{ddd_per_unit})$ 得 DDDs。代码注释自己披露了口径缺陷：“锥形口径：门诊消耗 + 住院床日”——分子取门诊已收费药量、分母取出院床日，分子分母的人群不一致，算出的数只能看趋势不能对标 40 的目标值。规范口径要求分子分母同取住院：这是“指标先通管道、口径随数据补齐”的过渡形态，教材把它留作习题 3。

12.8.2 日结与交款核对

日结是收费业务的每日闭账：当日全部结算按支付方式、状态汇总，收款员据此与手中现金、支付平台账单核对后交款。HIP 的日结（PrintReportController.java）输出按支付方式 $\times$ 状态的分组汇总与实收/退费/笔数总计，并提供 CSV 导出——导出串首的 `UTF-8 BOM` 是个小而典型的工程细节：没有它，Excel 打开中文 CSV 就是乱码。

交款核对（FinanceController.java）再往前一步，做成按收款员的稽核视图：每人当日收款/退款笔数与金额（来自 `outp_charge.cashier_id`——第 7 章收费落库时记录经手人，此处兑现价值），外加异常明细清单（当日退费单 + 作废支付单 union 呈现）。财务内控的原理是钱与账分离、账与账互核：收款员自报的交款额，要与系统按经手人汇总的净额相符；退费与作废是差错和舞弊的高发区，单列出来供逐笔核对。把这一节与第 10 章的医保对账、第 13 章的审计日志连起来看，医院的资金安全是三层同构的闭环：业务账对通道账（医保/支付对账）、人对账（交款核对）、操作对留痕（审计）。

本章小结

- 数据治理三件套：数据元标准（字段级统一，对齐 WS 363 系列目录）、术语映射（本地编码 $\rightarrow$ ICD/LOINC/ATC，唯一约束防二义）、质量核查（规则必须“可归零”，指标示警、清单派活）；对外供数走订阅推送，与 CDR 构成“来取 + 送去”两个出口。
- 病案首页被 DRG 重估为结算凭据：出院 $\rightarrow$ 时限质控 $\rightarrow$ 首页汇编（能取不录） $\rightarrow$ 编码 $\rightarrow$ 归档，归档即数据冻结；编码质量沿“首页 $\rightarrow$ 入组 $\rightarrow$ 权重 $\rightarrow$ 收入”因果链传导（上游在此、下游在第 10 章）。
- 病案统计的口径要点：疾病谱按 ICD 前 3 位类目归并、章节构成按首字母；平均住院日与消耗指数成对看。
- 医疗质控三式：时限质控用 `not exists` 反查抓“该做未做”；抽检评分连成时间曲线才有管理信号；不良事件匿名上报（`reporter_id` 置空而非隐藏）支撑非惩罚性报告文化。
- 三管千日率 = 感染例次 / 导管日 $\times$ 1000，分母用暴露时间（人时法）才可横向比较；HIP 以“置管登记一日评估—拔管/判感染”状态机承载，判感染同事务生成院感病例；数据来源当前手工登记，演进方向是从医嘱/护理体征自动触发。传染病报卡走“报告—审核—上报”闭环状态机，网络直报为外部条件。
- 单病种上报用 ICD 前缀自动入组解决漏报——系统找人，而非人找系统；其前提又是病案编码质量。

- 报表引擎的安全模型是分层收窄：权限 → select 前缀 → 关键字黑名单 → 子查询包装限行 → 保存/运行双校验；黑名单是启发式，只读数据库账号才是边界。指标快照解决趋势、口径冻结与性能，内置指标与自助报表因变更频率不同而治理策略分野。
- DDD/AUD 把药量折算为标准日剂量、按百人天标准化；日结与交款核对的内控原理是钱账分离、按经手人互核，与医保对账、审计日志三层同构。

思考与练习

1. 概念：说明数据质量核查规则与统计报表的本质区别；用“可归零”标准判断以下哪些适合做核查规则：药占比、出院 3 日未归档病案数、平均住院日、术前讨论缺失数。
2. 计算：某 ICU 本月呼吸机置管 18 例，合计呼吸机日 126 天（其中 1 例仅 10 小时），判定 VAP 2 例次。按 HIP 口径（不足 1 日按 1 日）计算 VAP 千日率；若改按“每日 0 点清点带管人数累加”口径，说明两种分母的差异来源与适用场景。
3. 设计：12.8.1 指出 HIP 的 AUD 锥形口径分子分母人群不一致。基于第 8 章的住院医嘱执行模型（`inp_order` 执行记账），写出规范住院 AUD 的分子 SQL 口径（提示：抗菌药 + 已执行 + 按 `ddd_per_unit` 折算），并说明分母“收治患者床日数”与“出院患者床日数”的取舍。
4. 攻防：针对案例框 12-5 的 `validateSql`，构造两个测试用例：一个会被误伤的合法查询、一个能通过校验但仍造成危害的查询，并分别给出对应的加固措施；论述为什么“只读数据库账号”比扩充黑名单更根本。
5. 设计：为传染病报卡增加甲类 2 小时时限预警：报卡后未在时限内完成上报的卡片须进入督办清单并升级提醒。给出所需的字段/SQL 与调度方式（参考第 13 章定时任务与 12.4.1 的反查句式）。
6. 排障：某院上线 DRG 分析后发现歧义组（QY）病例占比高达 20%。沿本章的因果链（编码率 → 首页 → 入组）列出排查顺序与每步使用的系统功能页。
7. 实践（配套实训环境）：为 HIP 的数据质量核查新增规则“出院超 3 日未归档”（当前规则只查未归档，不查超时），修改 `DataGovController.java` 的 `qualityChecks`，并为该规则补一个单元测试。

第 13 章 安全、合规与运维

本章属第五篇「工程篇」。前面各章讲的是“把功能做出来”，本章讲“让系统配得上上线”——安全合规是医院信息系统的准入资格，运维能力决定它上线后能活多久。本章依次展开等保落地、数据脱敏、备份恢复、运维中心、性能工程与测试体系，贯穿案例是 HIP 平台从演示系统走向试点上线时的一次真实生产化硬化（v15-B 安全基线专项）。

学习目标

学完本章，你应当能够：

1. 说出网络安全等级保护制度的基本框架，理解“从抽象条款到可验证清单”的落地方法，解释密钥外置、默认口令治理、HTTPS 反代为什么是上线前的最小硬化集；
2. 描述数据脱敏的层次模型（传输加密、存储控制、应用层动态脱敏、对外匿名化），说明脱敏为什么必须发生在服务端出口；
3. 定义 RPO 与 RTO，解释“备份不等于可恢复”，设计一套包含自动备份、时效监控、恢复演练留档的备份制度；
4. 说出可观测性三支柱（指标、日志、链路追踪），论证中小医院轻量化观测方案的合理边界，画出故障工单的流转闭环；
5. 掌握以执行计划为依据的索引审计方法，理解“只加不删、注明依据”的索引管理纪律与大表治理策略；
6. 解释测试金字塔在业务信息系统中的变形，论证“Python 脚本打真实 API”的 E2E 策略对中小团队的性价比；
7. 描述 CI 三任务（后端测试、前端构建、端到端回归）的组织方式，理解 CI 全绿、安全基线、备份演练如何共同构成上线门槛。

13.1 等保三级工程落地：从条款到清单

13.1.1 制度框架与医院的定级现实

我国网络安全等级保护制度（下称“等保”）依据《网络安全法》建立，将信息系统按重要程度定为一至五级，级别越高要求越严。现行技术标准体系以《信息安全技术 网络安全等级保护基本要求》（GB/T 22239—2019，业内称“等保 2.0”）为核心，技术要求覆盖安全物理环境、安全通信网络、安全区域边界、安全计算环境、安全管理中心五个层面，另有安全管理制度、机构、人员、建设与运维五类管理要求【成稿核对：等保 2.0 技术/管理要求的分类表述与三级具体控制项条款号】。

医院的核心业务系统（HIS/EMR）承载大量患者诊疗数据且服务不可中断，行业通行做法是按第三级定级备案并定期测评【成稿核对：卫健部门对医院核心系统定级的文件依据】。第 1 章已从政策环境角度介绍过等保与《数据安全法》《个人信息保护法》的关系，本章只回答一个工程问题：拿到一份几百个控制项的等保要求，开发团队从哪里下手？

13.1.2 条款为什么落不了地

等保条款是面向测评的抽象描述，如“应对登录的用户进行身份标识和鉴别，身份标识具有唯一性，身份鉴别信息具有复杂度要求并定期更换”。这类语句无法直接变成任务：复杂度是什么规则？定期是多少天？在哪个系统的哪一层实现？工程团队常见的两种失败方式恰好相反——一种是把等保当成“测评前突击整改”的文档工作，系统本身不动；另一种是照条款逐条堆功能，做出一堆没人用的安全模块。

正确的方法是翻译：把条款翻译成“可执行、可验证”的工程清单，每一项写清实现位置、验证方式、责任环节。判断翻译质量的标准很简单——一个没参加过讨论的实施工程师，拿着清单能否独立完成检查。HIP 的部署手册“安全基线”一节与多医院部署操作指南的上线检查单（- [] HIP_JWT_SECRET 已设置（非开发占位值））就是这种翻译的产物：条款留在测评报告里，清单进入部署流程里。

13.1.3 上线门槛：安全基线的工程化

一个演示环境里安全无虞的系统，拿到真实网络里可能满身是洞——不是代码变了，而是威胁模型变了。演示环境的隐含假设是“访问者都是自己人”；试点上线的那一刻，这个假设失效。因此从演示走向试点，存在一组“不做就不许上线”的最小硬化项，HIP 在 v15 规划中把它命名为安全基线，并明确标注“试点前必做”。这组硬化项的选取标准值得学习：不是等保条款里最难的，而是攻击者最先尝试、修复成本又最低的三件事——密钥、口令、明文传输。

案例框 13-1：v15-B 安全基线三件事（HIP 试点前硬化专项）

HIP 的下一步开发规划 v15-B 将安全基线列为试点准备的第一项，半天级编码，内容三件：

其一，JWT 密钥外置。签发登录令牌的密钥原先是 `application.yml` 里的占位字符串——对演示无害，对生产致命：拿到密钥即可伪造任意用户的令牌，而配置文件随代码仓库流转，等于把万能钥匙印在说明书上。硬化后的配置（`application.yml`）：

```
# 安全基线：生产/试点通过环境变量 HIP_JWT_SECRET 注入（≥32 字符随机串）
jwt-secret: ${HIP_JWT_SECRET:dev-only-secret-change-me-hip-platform-0123456789}
```

注意这个设计的取舍：环境变量未设置时回落到一眼可识别的开发占位值（`dev-only-secret-change-me`），而不是拒绝启动。回落保住了开发与 CI 的零配置便利，风险则转移给部署清单硬卡——检查单里“非开发占位值”一项就是为它设的。这是“密钥不进代码库”原则（十二要素应用方法论中的配置外置）的最小实现；更重的方案（密钥管理服务、定期轮换）留给规模证明其必要时。

其二，默认口令治理。系统初始化自带 `admin/admin123`，演示数据脚本再造 7 个 `Demo1234` 演示账号——默认口令是真实入侵事件中最常见的突破口。治理分两路：生产初始化工具 `init-hospital.py` 把换口令做成断言，配置里不提供新口令直接拒绝执行（`assert new_pwd and new_pwd != 'admin123'`），执行末尾强制更换 `admin` 口令并回验；演示数据脚本 `bootstrap-demo.py` 则被部署手册明令禁止在生产库运行。把“上线要改密码”从叮嘱变成工具里的断言，是清单化思想的又一次体现——人会忘，断言不会。

其三，**HTTPS 反代**。院内网不等于可信网（等保三级对通信保密性有明确要求），登录口令与患者数据不允许明文过线。HIP 的做法是 **nginx 反代终止 TLS**：仓库内的 `nginx.conf` 提供 80 端口的基础配置（静态资源 + `/api` 反代），部署时在其上增加 443 `server` 块与院方证书——证书是逐院资产，不进代码库。

三件事没有一件是新功能，用户完全无感，却构成从“演示”到“生产”的分水岭。它们被排为“试点前必做”的深层理由是：安全欠账的偿还成本随时间递增——上线前改密钥是改一行配置，上线后改密钥是让全院重新登录；上线前禁演示账号是删一行种子，上线后是排查这些账号三个月里干过什么。

13.1.4 认证域的纵深：口令策略、防爆破与审计

密钥与默认口令之外，认证域还需要纵深防御。HIP 在平台底座（第 5 章）中实现了三层，本章从合规视角复核其对应关系：

- 口令复杂度：新建与修改口令要求不少于 8 位且同时含字母和数字，弱口令在服务端被 1102 错误码拒绝【成稿核对：等保三级对口令复杂度与更换周期的具体表述】；
- 登录防爆破：连续 5 次失败锁定账号 15 分钟（错误码 1002），成功登录清零失败计数——这三条行为都有单元测试钉住（`SecurityHardeningTest.java`），安全规则是最值得用测试固化的规则，因为它的破坏往往悄无声息；
- 口令时效：登录响应携带 `passwordAgeDays` 与 `passwordExpireWarning`（超 90 天提示更换），把“定期更换”从制度落到交互。

审计日志（`sys_audit_log`，写操作全量留痕，机制见第 5 章）在合规语境下的身份是证据：等保要求留存网络日志不少于六个月【成稿核对：《网络安全法》第二十一条留存期限表述】，测评会抽查“某敏感操作能否追溯到人”。工程上要注意两点：审计表只增不删（删除审计记录本身就是需要审计的事件），以及审计数据要纳入备份范围——丢了业务数据是事故，丢了审计数据是事故之后还说不清的事故。

13.2 数据脱敏与隐私

13.2.1 法律基线与最小必要原则

《个人信息保护法》将医疗健康信息列为敏感个人信息，处理需具备特定目的和充分必要性；《数据安全法》要求数据分类分级管理（政策脉络见第 1 章）。落到系统设计上，核心是最小必要原则的两个投影：每个角色只看到履职所需的最小数据集（横向），每个场景只输出该场景所需的最小字段集（纵向）。前者靠权限体系（第 5 章 RBAC），后者靠脱敏。

13.2.2 脱敏的层次模型

“脱敏”一词常被笼统使用，工程上应按数据所处环节分层：

层次	手段	防的是谁
传输层	TLS（13.1 反代终止）	网络窃听器
存储层	库表权限、磁盘/备份加密	越权接触数据库的人
应用层（动态脱敏）	按角色/场景在服务端出口遮蔽字段	合法用户的过度浏览
对外发布（静态脱敏/匿名化）	去标识、泛化、限时限量	数据离开系统后的一切人

动态与静态的区别在于数据是否离开系统边界：动态脱敏是“看的时候遮住”，原始数据仍在库中完好；静态脱敏是“给出去之前改掉”，常用于对外分享、测试数据、科研数据集。层次模型的价值

在于逐层追问：讨论“我们做了脱敏”时，先问做在哪一层、防的是谁——只做了传输加密的系统，挡不住一个合法登录的收费员翻遍全院患者手机号。

案例框 13-2：HIP 的三个脱敏场景

场景一：列表按角色动态脱敏。患者检索列表是“过度浏览”风险最高的入口——关键词一搜就是成页的姓名、手机号、证件号。HIP 在服务端出口按角色处理（PatientController.java）：

```
// 等保：列表场景手机号/证件号脱敏，仅 ADMIN 角色见明文
.map(x -> admin ? toDto(x) : maskSensitive(toDto(x)))
// 手机号保留前3后4，证件号保留前4后3
dto.computeIfPresent("phone", (k, v) -> mask(String.valueOf(v), 3, 4));
dto.computeIfPresent("idNo", (k, v) -> mask(String.valueOf(v), 4, 3));
```

保留首尾、遮蔽中段的格式（138****5678）兼顾了核对需求与隐私——窗口人员足以向患者口头确认“尾号 5678 对吗”，却无法抄走完整号码。

场景二：对外分享的匿名化 + 限时。检查报告分享短链（ReportShareController.java）让患者把已审核发布的报告转给外院医生或家属：生成 32 位随机 token 短链，匿名可访问但默认 72 小时过期（4662 失效），姓名静态脱敏为首字 + **，每次访问计数留痕。这里叠加了三种控制——不可枚举的随机标识、限时失效、最小字段集（只出报告所见与结论，不出证件号与费用）——因为数据一旦出了系统边界，就再没有第二次控制机会。

场景三：危急值不外显。患者端（互联网 + 患者服务）能查检验报告，但 HH/LL 危急值结果在患者端被遮蔽为“结果异常，请尽快回院复诊”（PortalController.java，异常标志置为 CRIT）。严格说这不是隐私脱敏，而是告知伦理：危急值意味着病情可能危及生命，应由医生结合病情当面告知并启动处置（危急值院内闭环见第 6 章），而不是让患者深夜独自面对一个吓人的数字。同一份数据、不同受众、不同呈现——这提醒我们：脱敏规则的主语不是数据，而是“数据 × 场景”。

13.2.3 工程讨论：位置、粒度与张力

三个场景共同的实现位置是服务端出口。前端脱敏（拿到明文再遮蔽显示）在浏览器开发者工具面前形同虚设，任何在响应报文里出现过的明文都视同已经泄露——这是脱敏实现的第一铁律。

粒度上，HIP 当前用“是否 ADMIN”一刀切，是保守起步的策略：宁可让少数合法需求多走一步（找管理员查明文），不可默认放开。真实演进方向是把“查看明文”独立成权限点授给确需角色（如收费窗口核对证件），并对明文查看行为落审计——授权与留痕必须同时到位，只授权不留痕，等于给过度浏览发了通行证。脱敏与业务便利永远存在张力，处理张力的原则是：放开一个口子时，问清楚谁需要、看什么、留什么痕。

13.3 备份与恢复

13.3.1 RPO 与 RTO：先给灾难定价

备份方案的两个基本指标：**RPO**（Recovery Point Objective，恢复点目标）——灾难发生时最多允许丢失多长时间的数据；**RTO**（Recovery Time Objective，恢复时间目标）——从灾难到恢复服务最多允许多久。两者都是业务决策而非技术决策：医院丢一天的收费与病历记录意味着什么？门诊停摆四小时意味着什么？答完这两个问题，技术方案只是推论——每日全量备份给出“最坏丢一天”的 RPO；要把 RPO 压到分钟级，就要在全量之上加 WAL 归档（PostgreSQL 的预写日志连续归档，可恢复到任意时间点）；RTO 则取决于恢复流程的熟练度，而熟练度只能靠演练获得。

经典的 **3-2-1** 原则（至少 3 份副本、2 种介质、1 份异地）是防“备份与生产一起毁”的底线。HIP 试点形态的备份落在应用服务器本地目录，满足的是日常误操作与单库故障场景；机房级灾难的防护（异地副本）属于按院实施时的基础设施议题，方案上应明确交代而不是含糊带过——说清楚现在防到哪一级，比声称什么都防更专业。

13.3.2 备份不等于可恢复

备份领域流传一句反复被事故验证的话：没有经过恢复演练的备份，等于没有备份。三种经典的死法：备份任务某天起静默失败，无人知晓，直到需要时才发现最后一份可用备份是三个月前的；备份文件按时生成但损坏或不完整（磁盘满、导出中断），文件列表看起来一切正常；备份完好，但会做恢复操作的人已离职，文档过期，在最需要冷静的时刻现场摸索。三种死法对应三种制度设计：时效监控盯“是否还在成功备份”，恢复演练盯“备份是否真能恢复”，演练留档盯“恢复的手艺是否还在组织里”。

案例框 13-3：HIP 的备份-恢复演练闭环

HIP 的备份制度由三个脚本加一张台账构成，全部在仓库内可查：

- `schedule-backup.ps1`：向 Windows 任务计划注册 HIP-DailyBackup（每日 02:30），把备份从“有人记得跑”变成“系统按时跑”——v15 规划将其列为“备份制度落地”，v17 又要求挂载实操留档，任务化本身也要留下证据；
- `db-backup.ps1`：`pg_dump` 归档格式导出到 `deploy/backups/`，随后调用平台接口 `POST /api/ops/backups` 把文件名、大小、状态写入 `ops_backup_log` 台账——备份行为在业务系统里留痕，而不是只散落在文件系统里；
- `db-restore.ps1`：恢复演练脚本。三个细节体现了制度思考：默认恢复到独立校验库 `hip_restore_check`；目标库参数若指向生产库 `hip` 直接拒绝执行（防呆——演练工具在设计上就不允许碰生产）；恢复后自动校验关键表行数（`sys_user > 0`）给出通过判定。演练通过后，对应备份记录状态可标为 `VERIFIED`，台账即演练档案。

闭环的最后一块是时效监控：运维中心健康告警（13.4）中有一条——

```
select count(*) from ops_backup_log
where status in ('SUCCESS','VERIFIED') and created_at > now() - interval '24 hours'
-- 计数为 0 则告警："24 小时内无成功备份"
```

注意告警的方向：不是“备份失败时告警”——失败的任务可能连告警都发不出（脚本没跑、机器关机、任务被禁用），而是“24 小时内没有成功记录就告警”。监控存在性而非监控失败事件，是所有周期性任务监控的正确姿势：它把“没消息”从默认的好消息变成坏消息。

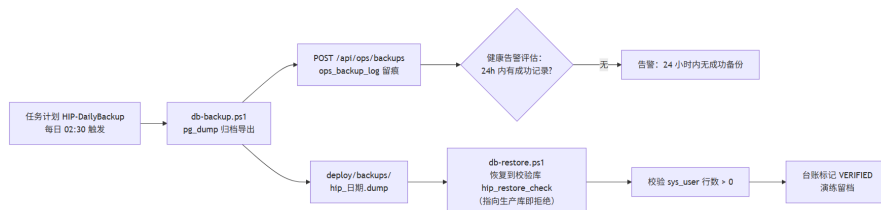


图 13-1 备份-恢复演练闭环：自动备份、台账留痕、时效告警、演练验证四环相扣

把图 13-1 与部署手册的回滚策略连起来看：应用回滚靠保留上一版 jar（Flyway 迁移向前兼容、不自动降级，所以发版前必先备份库——第 14 章的升级演练正是拿备份库做 N-1→N 前滚），数据回

滚靠每日全量 + WAL 归档。备份在这里显出它的第二身份：不仅是灾难保险，也是每一次发版的后悔药。

13.4 运维中心

13.4.1 可观测性三支柱与轻量化取舍

可观测性(Observability)领域的通用框架是三支柱：指标(Metrics, 聚合数值的时序)、日志(Logs, 离散事件明细)、链路追踪(Traces, 一次请求跨组件的路径)。互联网行业的标准配置——Prometheus + Grafana 采指标、ELK 聚日志、OpenTelemetry 做追踪——对县市级医院的单体部署形态(第 4 章)往往是负资产：三套系统本身就是三个要运维的对象，而院内信息科的运维人手常常只有一两人。

HIP 的取舍是把观测数据落进业务数据库、用业务页面呈现：慢接口写 `ops_slow_api` 表，健康状态一个接口聚合，日志走本地滚动文件(`pilot profile` 配置 $50\text{MB} \times 30$ 个，见 `application-pilot.yml`)，不引入任何观测中间件。这不是“简陋”，而是与部署形态匹配的架构一致性：模块化单体的观测就该是单体的。边界同样要说清——多实例部署、需要跨服务追踪、或指标量大到拖累业务库时，就是升级到专业观测栈的时机：届时 `ops_slow_api` 的数据口径可以平移成 Prometheus 指标，页面平移成 Grafana 面板，先轻后重的路径是通的。

13.4.2 健康告警与慢接口观测

运维中心(`OpsController.java`, V19 迁移建表)的健康概览接口一次聚合四类基础信号：数据库连通性、运行时长与堆内存、磁盘剩余、告警清单。告警清单按规则现算：数据库不可用、存在未处理的 HIGH 级故障工单、24 小时内无成功备份(13.3)、今日慢接口超过 50 次。这是拉式评估——管理员打开页面即得实时结论，实现零依赖；它的局限也直白：没人打开页面就没有告警。演进方向是把同一套评估逻辑挂到定时任务并对接外发通道(短信/企业微信)，评估规则不变，只是多了一个触发时机——这正是“规则与通道分离”带来的演进便利。

慢接口观测的实现是一个 Servlet 过滤器(`SlowApiFilter.java`)：每个 `/api/` 请求计时，超过阈值(`hip.ops.slow-ms`，开发默认 500ms，试点 `profile` 收紧到 300ms)落一行 `ops_slow_api`。两个细节是观测代码的通用纪律：落库失败静默吞掉(`catch (Exception ignored)`)——观测永远不许伤害业务，宁可丢观测数据；阈值走配置——不同环境的“慢”标准不同，试点收紧阈值是主动放大观测灵敏度，让问题在数据量小时就露头。运维中心页面按 7 天窗口聚合(次数、最大/平均耗时)，排序即优化工单的候选清单——13.5 的索引审计正是消费这份清单的下游。

13.4.3 故障工单与巡检

故障管理的重量级方案是 ITIL 流程与专业 ITSM 工具，轻量级的底线则是一张有状态的台账：何时报的、什么级别、谁在管、怎么了结。HIP 的 `ops_fault_ticket` 就是这条底线：标题、级别(LOW/MEDIUM/HIGH)、状态(OPEN/RESOLVED)、上报人、处理说明、时间戳。工单的价值不在表结构，在入口的汇聚——HIP 的工单有三个来源(图 13-2)：管理员人工上报；系统自动开单(医保每日对账差异 > 0 时，`InsuranceAutoReconJob.java` 以 `system` 名义插入 MEDIUM 工单，第 10 章已从对账视角讲过这条联动)；试点运行期的用户反馈(v15-C 规划明确“问题一律进 `ops_fault_ticket` 台账 → 分级 → 按需开小版本”，工单台账成为试点反馈闭环的载体，见第 14 章)。

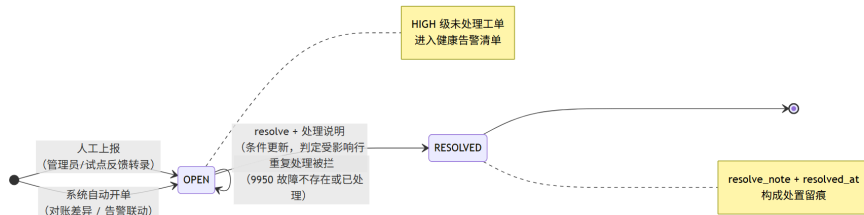


图 13-2 故障工单流转：多来源汇聚一张台账，闭环状态机 + 处置留痕

结单接口值得看一眼实现：`update ... set status='RESOLVED' where id=? and status='OPEN'`，受影响行数为 0 则返回“9950”故障不存在或已处理。这是全书反复出现的“条件更新 + 判定受影响行数”并发定式（第 7、8、10 章的占号、占床、结算是同一模式）在运维域的应用——两个管理员同时结同一单，只有一人成功，台账不会出现覆盖。

巡检台账 `ops_inspection`（项目、PASS/FAIL、说明、巡检人）看似只是个记录本，制度含义却与恢复演练留档同构：周期性人工检查必须留下可核查的痕迹，等保测评对运维管理的检查（巡检记录、故障处置记录）落到系统里就是这两张表。运维体系的全貌至此闭合：观测发现问题（健康告警、慢接口）→ 工单承接问题 → 巡检与演练证明制度在运转。

13.5 性能工程

13.5.1 测量先于优化

性能工程的第一原则是测量先于优化：没有测量数据支撑的优化，多半优化错了地方。医院系统的性能需求也不均匀——窗口业务（挂号、收费、发药）是排队场景，P95 响应时间直接决定队伍长度，敏感度最高；报表统计类允许慢一些，但不允许慢到拖垮别人（一条全表扫描的报表 SQL 可以让收费窗口一起卡住，这是共享数据库的连带代价）。因此测量体系要能回答两个问题：哪些接口在变慢（13.4 的慢接口台账），以及慢在哪里（数据库执行计划）。

13.5.2 索引审计：以执行计划为依据

索引缺失是业务系统性能问题的第一大户，但“多建索引”不是答案——每个索引都有写入代价与维护成本，凭感觉建的索引常常既没命中查询又拖慢插入。正确的方法是审计：拿真实查询清单逐条看执行计划（PostgreSQL 的 `EXPLAIN ANALYZE`），确认走了全表扫描且数据量将增长的，才补索引。

案例框 13-4：V39 索引审计（HIP 技术债清偿专项）

HIP 在 v14 技术债专项中对 38 个迁移版本积累的大表查询做了一次系统审计，产出迁移 `V39_indexes.sql`。这份 27 行的 SQL 是索引管理纪律的样板，值得整体读：

```

-- 技术债二：索引审计补缺（只加不删）
-- 依据：EXPLAIN ANALYZE 实测 E2E 常用查询，补高频点查/时序查询缺失索引。
-- 医保对账 msgExists 逐单点查 (channel+status+ref_no)
create index if not exists idx_int_log_ref_no on int_message_log (ref_no);
-- 体温单/生命体征趋势按入院时序拉取
create index if not exists idx_inp_vital_adm_time on inp_vital_sign (admission_id,
measured_at);
  
```

```
-- 审计按用户查最近操作 (order by id desc limit 200)
create index if not exists idx_audit_user_id on sys_audit_log (username, id desc);
```

三条纪律：其一，只加不删——删索引的风险评估远重于加，单独立项处理；其二，每条索引注明它服务的查询——索引与查询的对应关系写进注释，后人才能判断“这个索引还有没有存在的理由”，没有出处的索引会永远无人敢动；其三，审计的依据是实测——用 E2E 套件（13.6）跑出的真实查询路径做 EXPLAIN ANALYZE，而不是对着表结构想象。

这次审计还留下一个诚实的不作为记录：outp_charge 日结查询按 created_at::date 过滤，timestampz 转 date 不是 immutable 函数，无法直接建函数索引；正规解法是把查询改写为范围条件（>= 当日零点 and < 次日零点），但改写涉及多处 SQL，而日结是小结果集日报，试点量级实测可接受——于是记入技术债交接单待议区，数据量触发才动。性能工程的成熟不在于消灭所有慢查询，而在于知道哪些慢是当下值得付钱修的。

13.5.3 大表治理

医院系统的表按增长速度分两类：主档表（患者、字典、账号）增长缓慢，流水表（订单行、留痕、审计、观测）随业务线性增长。HIP 里增长最快的几张——outp_order（每诊次数行）、int_message_log（每次集成调用一行、含完整报文）、sys_audit_log（每次写操作一行）、ops_slow_api、inp_vital_sign——一年数据量在百万到千万行级。治理手段按代价升序：

1. 索引 + 时间窗口：一切流水查询强制带时间范围（慢接口页面查 7 天、工单查最近 200 条），配合时序索引，查询代价与表总量解耦——这是 HIP 当前的主力手段；
2. 归档：超过在线保留期的数据移入归档表/归档库。注意与合规的交叉：审计日志、医保留痕有法定/协议留存期（13.1），归档可以，删除不行——第 10 章“留痕只增不减”的原则在容量维度的对偶命题是“移走但不销毁”；
3. 分区：PostgreSQL 原生分区按月切流水表，归档变成挂摘分区的秒级操作——量级到达前不必预支这份复杂度。

13.5.4 压测的双重目的

压力测试的常规目的是验证吞吐与响应时间。HIP 的压测报告（bench-registration.py，10 并发 × 100 请求打并发挂号，号源容量 50）给出：成功 50、满号拦截 50、错误 0，吞吐 128 req/s，挂号 P95 84ms。这组数字里最有信息量的不是吞吐，而是 50/50/0——号源容量 50 的排班在 100 个并发请求下恰好成功 50 笔、拦截 50 笔、无一超卖：压测在这里同时是并发正确性测试，验证第 7 章原子占号的防超挂语义在真实并发下成立。对含额度类资源（号源、床位、库存、医保额度）的系统，压测脚本都应该带这类守恒断言——吞吐达标但多卖了一个号的系统，性能再好也不能上线。

13.6 测试体系

13.6.1 测试金字塔与业务系统的现实

经典测试金字塔主张：底层大量单元测试（快、稳、定位准），中层适量集成测试，顶层少量端到端测试（慢、脆、维护贵）。这个模型诞生于算法与组件逻辑密集的软件；医院信息系统的价值分布却不同——大量业务规则长在跨模块流程与 SQL 里（挂号→开单→收费→发药是四个模块的接力），纯内存可测的算法单元占比不高。对这样的系统，机械追求单测覆盖率会产出大量“mock 了所有依赖、只测了转发”的低价值测试。

HIP 的测试形态是对现实的适配：55 个单元测试集中在算法与规则密度最高的域——医保分割与待遇模型、CDSS 与合理用药规则、密码策略与防爆破（13.1）、HL7 报文解析、模块开关——这些

域输入输出清晰、错一处就错一片，单测性价比最高；19 套 E2E 脚本（`tools/e2e-*.py`）承担业务流程的正确性，每套脚本走通一条真实业务链。两层之间没有传统的“集成测试层”，因为 E2E 直接打真实 API、连真实数据库，已经覆盖了装配与 SQL 的集成风险。

13.6.2 E2E：用 Python 脚本打真实 API

HIP 的 E2E 不用浏览器自动化（Selenium/Playwright），而是 Python 脚本直接调用后端 API。这个选择对中小团队的性价比值得展开：

- 稳定：UI 测试脆在选择器——改个按钮布局就红一片；API 契约的变更频率远低于界面，测试跟着契约走；
- 快且零依赖：只用标准库 `urllib`，无浏览器、无驱动、无 `npm` 依赖树，CI 里裸 Python 就能跑，本地复现一条命令；
- 失败信息直白：断言失败直接打印接口响应（错误码 + 消息），而不是一张“元素未找到”的截图；
- 脚本即文档：一套 E2E 就是一条业务流程的可执行描述——`e2e-outpatient.py` 从挂号写到发药，新人读它比读操作手册更接近系统真相。

代价也要诚实交代：API 级 E2E 测不到前端本身。HIP 用 CI 的前端构建任务（`vue-tsc` 类型检查 + `vite` 构建）兜住前端的静态正确性，交互层面依赖人工与试点反馈——对表单增删改查为主的业务前端，这是经过计算的风险接受，而非疏忽。

案例框 13-5：e2elib 公共库与 E2E 的数据纪律

19 套脚本曾各自复制登录、调用、断言代码，v14 技术债专项把公共逻辑抽成 `e2elib.py`——不到 70 行，却浓缩了 E2E 工程的几条经验：

```
def ok(r, step):
    assert r['code'] == 0, f'{step} 失败: {r}'    # 失败即打印完整响应

def new_patient(token, name, sex='F', **extra):
    """建 E2E 专用患者（同患者同排班重复挂号会被 3002 拦截，各期造数据须新建专用患者）"""

def find_free_bed(token, ward_type='NURSING'):
    """遍历护理单元找空床；找不到说明有 E2E 在院患者未收尾出院"""

def discharge_cleanup(token, admission_id, pay_method=None):
    """收尾出院释放床位，防止 E2E 在院患者耗尽病区空床"""
```

注释里全是踩坑记录：E2E 打的是真实业务规则，业务防重复挂号，测试就必须每次新建专用患者；测试占了床位不出院，跑几轮后全院无空床，后续套件全部误报——所以“收尾出院”成为公共库函数。E2E 的测试数据必须自管理（自己造、自己清、幂等可重跑），这是它与单元测试（事务回滚即净）最大的工程差异。

这套脚本还有一重身份：实施验收工具。第 14 章的虚拟医院全量彩排要求“19 套 E2E 在新装实例复跑”，医保适配器替换验收要求“医保 E2E 全套在新实现下通过”（第 10 章），部署手册的上线验证一步就是改 BASE 地址跑 `e2e-outpatient.py`。同一套断言，在开发期是回归，在实施期是验收，在生产演练期是冒烟——测试资产被复用了三次，这是 API 级 E2E 额外的红利。

13.6.3 CI 三任务

HIP 的持续集成（`ci.yml`）在每次 `push` 与 `PR` 上并行跑三个任务（图 13-3）：

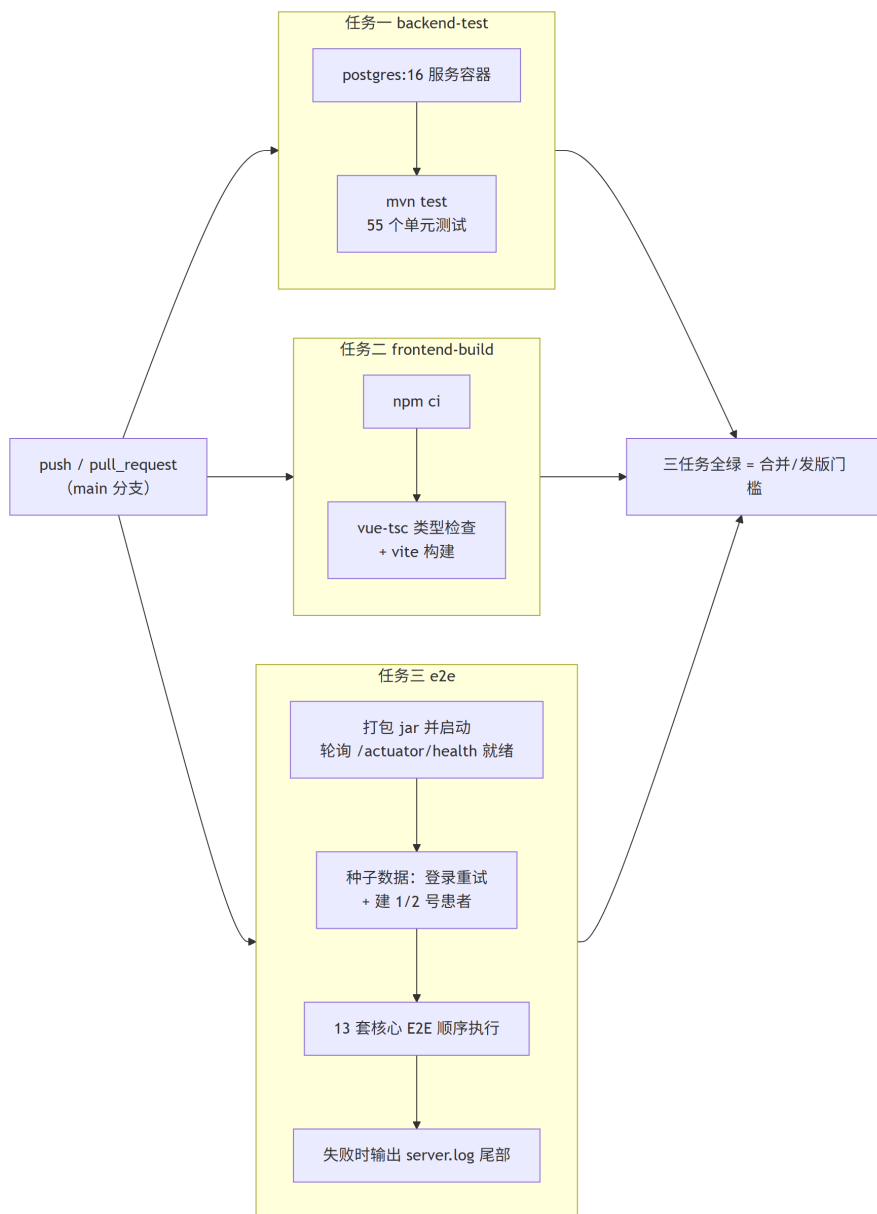


图 13-3 CI 三任务流水线：单测、前端构建、完整后端端到端回归并行把关

e2e 任务最接近真实部署：起 postgres 容器、打包 jar、像生产一样启动、轮询健康端点就绪后，先做种子数据（全新库需要 1、2 号患者），再顺序跑 13 套核心 E2E 套件，失败时自动输出服务端日志尾部辅助定位。两个工程细节有普遍参考价值：登录带重试——服务刚就绪时连接池预热等瞬时因素曾造成误报，重试 5 次消除了这类“狼来了”（CI 的假失败比真失败更危险，它教会团队忽略红灯）；这条容器化任务本身就是部署形态的持续验证——部署手册明确写道，CI 的 postgres 容器 workflow 即容器路径的日常演练，每次提交都在证明 compose 部署形态仍然成立。

13.6.4 质量门槛的合成

把本章的线索收拢：一次真正的“可以上线”，是多个门槛的合取——CI 三任务全绿（功能没退化）、安全基线清单逐项勾掉（13.1，威胁模型切换完成）、备份任务挂载且恢复演练留档（13.3，后悔药已备）、健康告警阈值按试点值核对（13.4，观测已就位）。HIP 的 v15 规划把这四件事排进“试点上线准备”同一个方向，不是巧合：安全、备份、观测、测试在日常里是四个话题，在上线决策这一刻是同一张检查单。而这张检查单本身如何演化成可复制的实施工艺，是第 14 章的主题。

本章小结

- 等保落地的方法是翻译：把抽象条款译成写明实现位置与验证方式的工程清单，清单进入部署流程。从演示到试点的最小硬化集（安全基线）抓的是攻击者最先尝试、修复成本最低的三件事：密钥外置（环境变量注入 + 部署清单硬卡占位值）、默认口令治理（初始化工具断言强制改密、演示数据禁入生产）、HTTPS 反代（nginx 终止 TLS）；安全欠账的偿还成本随时间递增，故为“试点前必做”。
- 认证纵深：口令复杂度服务端拒绝、5 次失败锁定 15 分钟、90 天更换提醒，全部有单元测试钉住；审计日志在合规语境下是证据，只增不删且纳入备份。
- 脱敏按层次讨论：传输、存储、应用层动态脱敏、对外匿名化，防的对象各不相同。脱敏必须发生在服务端出口；HIP 三场景——列表按角色遮蔽（保留首尾）、分享短链（随机 token + 限时 + 最小字段）、危急值不外显（告知伦理）——说明脱敏规则的主语是“数据 × 场景”。
- 备份先定 RPO/RTO 再选方案；备份不等于可恢复，制度设计三件套：时效监控（监控“无成功记录”而非“失败事件”）、恢复演练（默认打校验库、拒绝生产库的防呆脚本）、演练留档（台账 VERIFIED 状态）。备份同时是发版回滚的前提。
- 运维观测的轻量化路径：观测数据落业务库、页面呈现、不引入观测中间件，与单体部署形态保持架构一致；慢接口过滤器“观测不伤业务、阈值走配置”；故障工单一张台账汇聚人工、系统自动（对账差异）、试点反馈三类来源，条件更新防并发覆盖。
- 性能工程测量先于优化：慢接口台账指出“哪里慢”，EXPLAIN ANALYZE 回答“慢在哪”；索引审计三纪律——只加不删、注明服务的查询、依据实测；大表治理按代价升序（时间窗口 + 索引 → 归档不删除 → 分区）；压测对额度类资源要带守恒断言，它同时是并发正确性测试。
- 测试形态适配业务系统现实：单测集中在算法与安全规则域，业务流程交给 Python 脚本打真实 API 的 E2E——稳定、零依赖、失败信息直白、脚本即文档，且同一套资产复用为回归、实施验收与生产冒烟；E2E 测试数据必须自管理。CI 三任务（单测/前端构建/完整后端 E2E）全绿与安全基线、备份演练、告警核对合成上线门槛。

思考与练习

1. 概念：任选等保三级的三条抽象要求（如“身份鉴别信息具有复杂度要求并定期更换”），按 13.1.2 的方法各写出一条“可执行、可验证”的清单项（实现位置 + 验证方式），并说明什么样的清单项算不合格。
2. 设计：HIP 的 JWT 密钥在环境变量未设置时回落开发占位值（案例框 13-1）。请设计另一种策略——pilot profile 下未设置 HIP_JWT_SECRET 即拒绝启动：写出实现思路（提示：@Value 校验或 ApplicationRunner 断言 + profile 条件），并比较两种策略在开发便利、CI、误配置风险三个维度的优劣。
3. 排障：某日健康页出现“24 小时内无成功备份”告警，但 deploy/backups/ 目录里明明有当天凌晨的备份文件。按图 13-1 的闭环列出至少三种可能原因与对应排查步骤。（提示：留痕接口调用条件、台账 status 字段取值、服务器时钟。）
4. 设计：收费窗口角色确有核对证件号明文的业务需求。为其设计一个“查看明文”方案：权限点如何定义、接口如何改造、审计如何配套；并从最小必要原则论证为什么“直接把收费角色加入 ADMIN 明文白名单”是错误做法。
5. 计算与论述：HIP 每日 02:30 全量备份。给出最坏情形下的 RPO（灾难发生在什么时刻最坏）；若叠加 WAL 归档（归档延迟按分钟级计），RPO 变为多少；RTO 由哪些环节构成，说明为什么 RTO 只能靠恢复演练测得而无法纸面推算。

6. 排障：慢接口台账显示某报表接口 7 天内出现 40 次、平均耗时 2100ms。写出完整排查步骤：如何取到它执行的 SQL、如何用 `EXPLAIN ANALYZE` 判断是否缺索引、什么情况下应选择“改写查询”或“接受现状记入待议区”而不是加索引（参考案例框 13-4 的 `created_at::date` 案例）。
7. 实践（配套实训环境）：用 `e2elib` 编写一套 `e2e-ops.py`，覆盖故障工单闭环：创建 HIGH 工单 → 健康概览出现对应告警 → resolve 结单 → 再次 resolve 断言返回 9950 → 健康概览告警消失。要求测试数据自管理、脚本幂等可重跑。
8. 论述：13.6.1 称机械追求单测覆盖率对业务信息系统性价比不高，13.6.2 又承认 API 级 E2E 测不到前端交互。结合测试金字塔，论述 HIP 当前“55 单测 + 19 套 E2E + 前端只做构建检查”的形态中，最值得增加投入的下一层测试是什么（候选：Service 层集成测试、前端组件测试、契约测试），并给出判断依据。

第 14 章 产品化与实施方法论

本章属第五篇「工程篇」，也是全书的收束。前三十三章讲的是“把一套医院信息系统做出来”，本章讲“把它变成可以装进第二家、第三家医院的标准产品”——这是两件难度不同的事。全章依次展开产品化总原则、四层标准化模型、部署形态论证、种子数据与初始化工程、模块开关、实施定制层规约、发布工艺、实施全流程、偏离管理、试点反馈闭环，最后以“虚拟医院全量彩排”作为验收方法论收尾。本章的案例素材大多来自 HIP 从 v14 到 v17 四轮规划的真实决策记录——包括“什么时候停止开发功能”这类教科书很少讨论的判断。

学习目标

学完本章，你应当能够：

1. 解释单院系统与标准产品的本质差异，说出“配置优先、扩展次之、禁止分叉”的含义与理由；
2. 用四层标准化模型（标准产品层/配置层/SPI 扩展层/实施定制层）为一个具体的个性化需求判层，并说明每层的变更主体与升级影响；
3. 论证县市级医院场景下单院独立部署与多租户 SaaS 的取舍，理解“标准化收益来自同一产物而非共库”这一判断；
4. 设计种子数据的三类分层（平台必需/行业基线/演示数据）与新医院初始化路径，说明为什么历史迁移不能改；
5. 说出实施定制层的隔离要素（代码包名、Flyway 版本段、表前缀、路由前缀、错误码段）及升级兼容底线；
6. 描述语义化版本约定与“停服—备份—换产物—前滚—回归—回滚”的升级工艺，解释 $N-1 \rightarrow N$ 升级演练验证的是什么；
7. 理解需求矩阵逐条应答的工程化方法（自动评审、证据包、偏离表冻结），以及“功能开发边际判断”的依据；
8. 说明虚拟医院全量彩排作为产品化验收手段的设计逻辑：它证明的是路径，而不是功能。

14.1 从单院系统到标准产品

14.1.1 定制化泥潭：行业的老病

医院信息行业有一个流传很广的说法：HIS 厂商“卖一家医院，养一个分支”。每进一家医院，实施团队在标准版上改代码满足院方个性化需求，改完的代码留在该院的“专属版本”里。三五年后，厂商手里有几十个互不兼容的代码分支：新功能要在每个分支重复开发，缺陷要在每个分支重复修复，

升级成本高到大多数医院常年停留在多年前的老版本上。这就是定制化泥潭——每一次“为客户改代码”在当时都是合理的商业决策，累积起来却摧毁了产品。

软件工程对这个问题的一般解法早有共识：把变化点从代码里外置出去。变化点可以外置为配置（参数、开关）、数据（字典、规则库）、接口实现（插件/适配器），实在外置不了的，隔离到边界清晰的扩展区。核心代码对所有客户保持同一份，这是“产品”与“项目集合”的分水岭。

14.1.2 总原则：配置优先，扩展次之，禁止分叉

把上述共识落成可执行的纪律，是一条个性化需求的处理优先级：

改配置 > 换数据 > 实现 SPI > 实施层扩展模块；任何情况下不 fork 平台核心代码。

这条阶梯的排序依据是变更成本与升级摩擦：改配置当场生效、升级零影响；换数据（字典、规则）同样不碰代码；SPI 实现（第 6 章的适配器模式）按省/按供应商做实现包，可跨院复用；实施层扩展要写代码，但被规约限制在隔离区内（14.7）。每往下走一级，交付慢一分、升级摩擦大一分，所以必须自上而下逐级排除——只有上面三级都装不下的“真定制”，才允许落到第四级。

案例框 14-1：HIP 的产品化立项决策

HIP 最初是“面向峨眉山市人民医院的单院系统”。在功能开发接近尾声（v14 - v15，见 14.11）后，产品化方案（v16 立项）把目标改写为：升级为同级别同类医院（县市级综合医院）可复用的标准产品，并给出一个等式作为总纲：

每家医院 = 标准产品（同一构建产物）+ 该院的配置与数据 + 少量实施层扩展。

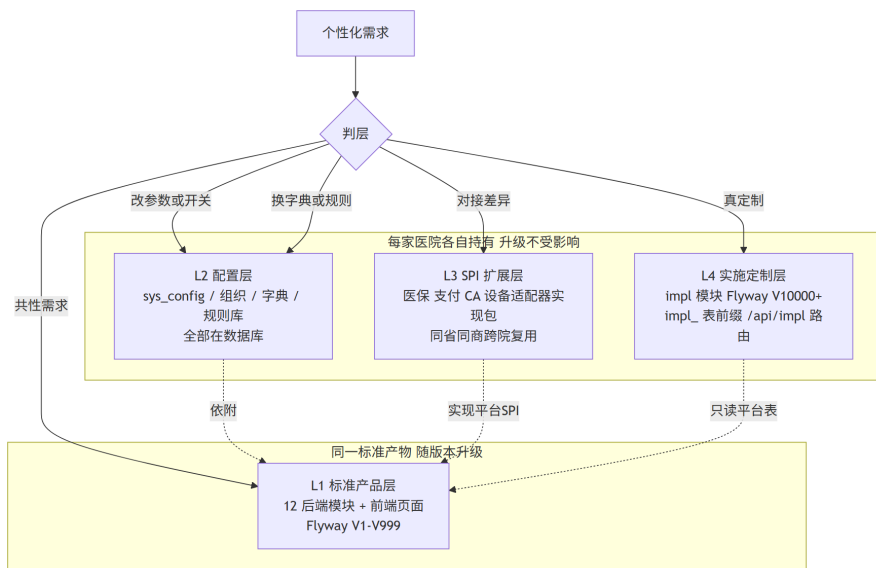
注意措辞的两个细节：一是“同一构建产物”而不是“同一套源码”——所有医院运行的是同一个 jar 与前端 dist，差异不允许在编译期出现；二是“少量实施层扩展”——扩展被预期为少量，多了就说明前三层的机制有缺口，应该反哺产品（把共性需求做进标准层），而不是在扩展层膨胀。已有的第一家医院被重新定位为“产品的第一个实施案例”：它的配置与定制沉淀反过来校准了分层清单。这个“从第 N 个客户反推产品边界”的动作，正是单院系统产品化的标准起手式。

14.2 四层标准化模型

14.2.1 按变更主体分层

产品化的核心工程问题是：系统里的每一样东西，归谁改、随什么节奏改？答案不同的东西必须物理分层，否则变更会互相纠缠。HIP 的四层模型如下（图 14-1）：

层	内容	变更主体	变更节奏
L1 标准产品层	12 后端模块 + 前端 65+ 页 + Flyway V1 - V999 + 必需种子	产品团队	随版本发布
L2 配置层	sys_config（院名/比例/开关）、组织结构、角色菜单、字典、规则库、报表定义	实施工程师	纯数据，随时改
L3 SPI 扩展层	InsuranceAdapter / PayAdapter / SignatureAdapter / int_adapter 路由 / ai-service	地区/供应商实现包	按对接项目，跨院复用
L4 实施定制层	该院独有的页面/接口/报表，独立 Maven 模块	实施项目组	随该院需求



这个模型不是 HIP 独创——它与 ERP 行业的“标准包 + 客户化配置 + 用户出口 (User Exit)”、SaaS 行业的“core + config + extension”是同一思想的不同表述。真正有产品性质差别的是配套纪律：**L2/L3/L4** 全部活在数据库和独立模块里，标准产物的替换动不到它们——这是升级敢做、敢频繁做的结构性前提（14.8）。

14.2.2 运行期的判层铁律

分层模型不只约束开发，更约束运行期的问题处置。多医院部署操作指南把它写成一条“每个反馈先判层再动手”的铁律：

改配置（当场生效）→ 换数据/规则（当场生效）→ 适配器缺陷（改实现包，同省同修）→ 平台缺陷（提产品需求进版本，所有医院受益）→ 真定制（进该院 **impl** 模块）。禁止为单院改平台核心。

这条链的价值在最后一句。实施现场最常见的诱惑是“就改两行代码，比走流程快”——而每一次屈服都是向定制化泥潭迈一步。判层铁律把“快”从选项里拿掉，逼所有人回答同一个问题：这个诉求本质上属于哪一层？

14.3 单院独立部署 vs 多租户 SaaS

标准产品要服务多家医院，部署形态有两条路线。多租户 **SaaS**：所有医院共用一套运行实例与数据库（按租户隔离数据），厂商集中运维，升级一次全体生效。单院独立部署：每家医院一个实例一个库，产物相同、实例独立。互联网行业的直觉偏向前者——运维集中、成本摊薄、版本永远最新。但把县市级公立医院的现实约束逐条摆出来，结论会反转：

1. 数据边界是合规问题而非技术问题。医院是独立法人，患者诊疗数据的等保定级、审计追责都以院为边界（第 13 章）；共库多租户下“逻辑隔离”在等保测评与院方安全评估面前说服力有限；
2. 部署位置：医院核心业务系统普遍要求部署在院内网，与医保专线、检验设备、PACS 同网段——“云端集中部署”在很多地市根本不具备网络条件；
3. 故障半径：HIS 停机是全院性事故，多租户把 N 家医院绑上同一个故障半径；
4. 定制隔离：L4 定制按院加载，单实例天然隔离；多租户下需要复杂得多的租户级插件机制。

于是 HIP 的形态铁律是：一院一实例一库，全部医院使用同一个标准发布产物。关键在认清一点——标准化的收益来自“同一产物 + 分层配置”，不来自共库。多院并行管理因此变成一种“车队管理”：实施资料库维护装机登记表（院名 | 版本 | 配置 JSON | 适配器实现及版本 | **impl** 模块 | 联系人），新版本发布后逐院滚动升级，先升试点院观察再推其余。

独立部署的真正代价不是运维工作量，而是版本漂移：各院升级窗口不同步，车队里会同时存在多个版本。管理手段有三：装机登记表让“谁在哪个版本”随时可查；语义化版本（14.8）让“落后多少”可度量；滚动升级策略给漂移设上限——原则上不允许任何医院落后超过一个次版本，否则跨版本升级的迁移前滚与 `impl` 兼容验证的风险都会叠加。这套做法放弃了 SaaS 的集中运维红利，换来的是每家医院合规自治、故障独立、升级窗口可各自协商——对县市级公立医院市场，这笔账是划算的。【成稿核对：不同地区对公立医院核心系统上云的政策口径存在差异，成稿时按最新监管文件校订本节表述】

14.4 种子数据分层

14.4.1 问题：迁移脚本里的“演示患者”

数据库迁移（第 5 章的 Flyway 纪律）管结构，但系统可运行还依赖种子数据：菜单、角色、默认参数、基础字典。单院开发期常见的做法是把种子连同演示数据一股脑写进迁移——反正只装这一家。产品化时这就成了雷：新医院装机，库里冒出“张三”测试患者和演示账号；更隐蔽的是行业字典（ICD 简表、示例药品）与平台骨架（菜单树）混在一起，实施工程师分不清哪些能换、哪些不能动。

通用解法是按数据的归属与生命周期分类。HIP 的种子数据分层清单分三类：

- **A 平台必需**：菜单树与角色绑定、默认参数键、数据元/术语框架——平台骨架，随产品版本演进，留在 Flyway；
- **B 行业基线**：药品/收费项目/ICD 简表、医保对照种子、DRG 简化目录、CDSS 示例规则——让系统“默认可用”，实施时可被院方目录整表替换；
- **C 演示数据**：演示患者、演示账号、体检套餐——只进演示环境脚本（`bootstrap-demo.py`），生产库禁止出现。

14.4.2 历史迁移不改：用路径实现分层

分层清单确定后，一个诱人的做法是回头把 43 个历史迁移里的 B/C 类数据清理出去。HIP 明确不这样做——历史迁移不改是 Flyway 纪律的底线（改了校验和，已装机的库会前滚失败）。分层通过新库初始化路径实现：

Flyway 自动跑（A + B 默认）→ `init-hospital.py`（院方数据覆盖 B + 组织/账号/开关）→ 不跑 `bootstrap-demo`（C）

案例框 14-2：一次种子数据审计

产品化一期对全部迁移做了逐个审计，结论写进分层清单：**43 个迁移中不存在 C 类数据**——演示数据从第一天起就走脚本不走迁移，这次审计是确认而非清理；A/B 混排存在但无害，因为 B 类每张表都有明确的覆盖途径（字典走 `init` 工具 CSV `upsert`、医保对照走导入接口、DRG 目录官方数据整表替换后 `regroup-all` 重算）。审计还标注了一个边界案例：迁移里种的“内科门诊+内科病区 6 床”属于“最小可运行样例”，实施按院方结构补建后，样例保留无害。这个案例框的方法论价值大于结论本身：产品化改造前先审计存量，把“要改的”与“只需确认的”分开——很多预想中的重构，审计后发现只需要一条路径约定。审计的另一个产出是把「生产库未运行过 `bootstrap-demo`（无演示账号）」写进了上线前检查清单（14.9），让纪律变成可勾选的检查项。

14.5 医院初始化工程

14.5.1 从“点页面”到“跑工具”

新医院开荒的传统方式是实施工程师照着手册在管理页面逐项点击：建科室、建账号、改参数、导字典——一两天的机械劳动，且无法复核“到底配了什么”。工程化的目标是把开荒变成一份配置文件 + 一次工具执行：配置文件即实施档案，可评审、可版本管理、可重放。

HIP 的 `init-hospital.py` 是这个思路的落地：读取一份院配置 JSON（模板 `hospital.example.json`），经平台开放的实施 API 依次完成七步——登录 → 院名与品牌（`hospital_name` 及 `extraConfig` 任意配置键）→ 模块开关 → 科室与病区床位（`type: NURSING` 带 `beds` 数自动建床）→ 业务账号与角色 → 字典 CSV 导入（药品/收费项目/医保对照三个模板）→ 强制更换 `admin` 口令并用新口令回验。

三个设计点值得展开：

1. 幂等：科室/账号按名跳过、字典按编码 `upsert`、配置覆盖——工具可重复执行，中途失败重跑即可，不需要“断点续传”逻辑。幂等是一切实施工具的第一美德，因为现场执行环境永远比实验室脏；
2. 安全基线内置：脚本开头即断言 `adminNewPassword` 必填且不等于默认口令——“生产初始化必须更换默认口令”不靠手册提醒，靠工具拒绝执行（第 13 章的口令治理在实施工艺中的落点）；
3. 配置即档案：执行完的院配置 JSON 存入实施资料库，每院一份——它同时是重装依据、审计依据和 14.3 装机登记表的内容来源。

字典导入接口的容错与回执是另一个容易被低估的细节。院方提供的目录 CSV 几乎必然有脏行（缺列、格式错、编码重复），导入接口的正确行为是逐行校验、坏行拦截、好行入库，并返回结构化回执（入库数、错误数、错误示例）——`init` 工具把回执直接打印在执行日志里。产品化 E2E 甚至为此专门造了一条坏行断言“1 入库 1 拦截”：如果一条坏行让整批导入失败，实施工程师将陷入“改一行、重传全量、再撞下一行”的循环；如果坏行被静默吞掉，账实不符要到对照率核查时才暴露。批量工具的验收标准不是“数据全对时能跑”，而是“数据半对时行为可预期”。

14.5.2 配置手册：让每个键都有出处

工具解决“怎么配”，还需要一册文档回答“有哪些可配、各是什么含义”。HIP 的配置手册把配置分成两层：部署配置（`application.yml` / 环境变量，起服务前定，如 `HIP_JWT_SECRET`、MLLP 开关、慢接口阈值）与运行配置（`sys_config` 表，管理页随时改、即时生效，24 键分机构品牌/模块开关/医保待遇/业务参数四组）。手册末尾一行纪律比正文更重要：新增配置键必须随迁移插入默认值、`remark` 必填、手册同步补一行——配置项失控的系统里，没人敢改任何参数。

14.6 模块开关与品牌配置化

14.6.1 功能开关的一般分类

功能开关（Feature Toggle）在业界按用途分为发布开关（灰度新功能）、实验开关（A/B 测试）、运维开关（降级保护）与授权开关（按客户购买范围启停功能）。产品化场景的模块开关属最后一类，它有两个特殊要求：粒度是“业务域”而非单个功能点；关闭必须彻底——医院没采购 DRG 模块，不能只是菜单上看不见，接口还留在那里可被直接调用（授权边界、等保测评都不允许）。

案例框 14-3: ModuleGate 的双路关闭

HIP 的模块开关以一张集中注册表实现（`ModuleGate.java`）：六个可开关域（`drg/cdss/insurance/blood/hr/surgery`），每个域登记菜单路径与 API 前缀：

```
public static final Map<String, ModuleDef> MODULES = Map.of(
    "drg",          new ModuleDef("/drg",          List.of("/api/drg")),
    "insurance",    new ModuleDef("/insurance",    List.of("/api/insurance")),
```

```

        "blood",      new ModuleDef("/inpatient/blood", List.of("/api/inpatient/
blood")),
        "surgery",    new ModuleDef("/surgery",          List.of("/api/inpatient/
surgeries", "/api/anes")), ...);

```

开关值存 `sys_config ( module.<key>.enabled`，缺省视为启用)，双路生效：登录接口组装菜单树时过滤被停用域的菜单路径（入口消失）；`ModuleGateFilter` 拦截命中前缀的 API 请求直接返回 404——过滤器源码注释里的一句话就是本节论点：「只藏菜单不拦接口是伪关闭」。注意 `surgery` 域登记了两个 API 前缀（手术 + 麻醉）——注册表模式的好处正在于此：域与路由的对应关系集中在一处声明，而不是散在各控制器的条件注解里。配套的 E2E（`e2e-product1.py`）断言了完整闭环：停用后菜单消失且 API 实测 404，恢复后立即回归——“关闭要彻底，恢复要即时”，两头都要验。

14.6.2 品牌配置化

比模块开关朴素但同样必要的是品牌参数化：院名、简称、票据抬头、咨询电话全部走 `sys_config`，登录页、单据打印、患者端联动取值。检验是否配置彻底的方法很简单：初始化工具改一次院名，全端走查一遍看有没有地方还写着老名字——HIP 的彩排（14.12）正是这么验的。

14.7 实施定制层规约

14.7.1 扩展机制的设计要素

允许实施团队写代码，又要保证升级不被拖住，扩展机制必须在每个可能冲突的维度上划出命名空间。通用清单是：代码位置、数据库变更、数据表、对外路由、错误码——每个维度都要回答“扩展用哪段、平台用哪段、越界怎么发现”。

HIP 的 ADR-0003 实施定制层规约（ADR 方法见第 4 章）给出全套答案：

维度	平台段	实施段	越界防线
代码	平台模块	<code>impl/<hospital></code> Maven 模块，包名 <code>cn.hip.impl.<hospital>.*</code>	启 动 器 <code>scanBasePackages=cn.hip</code> 自动装配，无需改平台代码
Flyway	V1 - V999	V10000+	同库合并执行，版本段互不冲突
表	平台表	只建 <code>impl_</code> 前缀自有表	禁止修改平台表结构（升级兼容底线）；要平台数据就读平台表、写自己的表
路由	<code>/api/**</code>	<code>/api/impl/<hospital>/**</code> ；前端 <code>/impl/*</code> 懒加载	不改平台页面源码
错误码	平台段	20000+	避开平台码段

两条配套纪律使规约闭环：标准仓库的根 `pom` 永远不含任何 `impl` 模块——启用 `impl` 是实施仓库的改动，标准构建产物里没有一行定制代码；平台升级后先对新版重编译 `impl` 模块，编译失败即兼容性破裂信号，先修 `impl` 再上线——把“升级会不会弄坏定制”从上线后的惊吓变成上线前的编译检查。

14.7.2 模板先行

规约要配模板才可执行。`impl/template-hospital` 是一个可编译的最小样例：示例控制器（`/api/impl/template/ping` 与 `notes` 读写，带权限注解与 20000 段错误码）、V10000 示例迁移（建 `impl_template_note` 表）。实施新院的定制从复制模板开始，改 `artifactId` 与包名即可——模板是规约的可执行版本，没有模板的规约会被每个项目组各自诠释一遍。

14.8 发布工艺

14.8.1 语义化版本与发布物

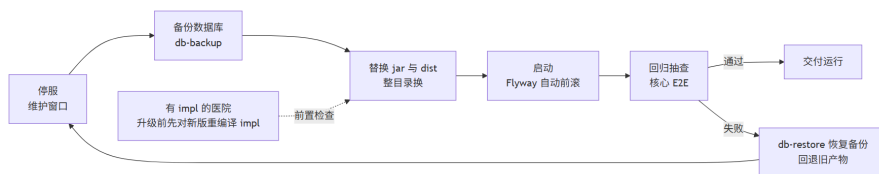
多院并行后，“给医院装的是哪个版本”必须有严格答案。业界通行语义化版本（Semantic Versioning）约定：主版本.次版本.修订号，主版本表破坏性变更、次版本表向后兼容的功能增加、修订号表缺陷修复。HIP 在产品化二期定下 1.0.0 起点，此后试点反馈的小版本走修订号（如 1.0.1，见 14.10）。

发布物的完整性同样是工艺问题。release.py 打包的发布目录不止 jar: server jar + 前端 dist + deploy 编排 + CHANGELOG + 八份实施文档（功能清单/部署指南/配置手册/操作手册/培训脚本/两份 ADR 等）+ 六件实施工具（init-hospital/import-patients/备份恢复脚本等）+ CSV 模板目录 + 自动生成的升级说明。逻辑很直接：实施工程师到现场只应该带一个 zip——文档与工具跟产物走同一个版本，才不会出现“拿着 1.0 的手册装 1.1 的包”。

发布物里两份小文档值得单独说。CHANGELOG 不是流水账：HIP 的 1.0.0 条目除功能范围外，还写明版本纪律（迁移段划分、升级流程指引）与已知边界——外部条件项接入点就绪待资质、配套产品项见偏离表。把“这个版本不含什么”写进变更日志，验收与商务沟通就有了统一口径。升级说明由打包脚本按版本自动生成，内容是六步操作（停服→备份→换产物→启动前滚→回归抽查→失败恢复回滚）外加一行针对性提醒（有 impl 定制的医院先重编译）——它是升级流水线（14.8.2）的“随箱说明书”，保证任何一个没参与过开发的工程师拿到包就能执行。

14.8.2 升级：15 分钟的流水线

四层模型在升级时兑现红利：配置、字典、规则库、impl 段迁移全在数据库里，产物替换动不到它们。单院升级因此收敛为一条约 15 分钟的流水线（图 14-2）：



14.8.3 升级演练：把“应该没问题”变成“演过一遍”

升级流程写在纸上不算数。HIP 在定版前做了一次脚本化的 N-1→N 升级演练：建一个演练库跑上一版（V42）全量迁移并造入业务数据 → 备份 → 换新版产物启动 → Flyway 自动前滚 V43 → 验证旧数据完好、新配置就位、接口正常。演练验证的不是新功能（E2E 早已覆盖），而是升级工艺本身：前滚会不会撞上存量数据、备份恢复链路是否真的可用。这与第 10 章的“假适配器验证法”是同一方法论：对“路径”的验证必须真的走一遍路径，不能用对“点”的验证代替。

14.9 实施全流程

把前述工件串起来，就是单院实施的完整流程。多医院部署操作指南把它定为七步，每家医院重复执行：

1. 装机：设置 HIP_JWT_SECRET 环境变量（必做）→ 建空库启动，Flyway 自动建全部结构与 A/B 种子 → nginx 挂 dist、反代、TLS；
2. 初始化：填写院配置 JSON → 跑 init-hospital → 配置 JSON 归档；
3. 字典与数据迁移：药品/收费项目/医保对照 CSV 导入（对照率达标是上线门槛，第 10 章）、存量患者导入（同证件号幂等）、DRG 官方目录整表替换；
4. 接口适配：先填接口现状调研表（医保/LIS 设备/PACS/CA/支付商与联系人），再按 SPI 逐项对接；未到位的适配器保持 Mock——业务闭环完整可用，条件到位无缝切换（第 6 章的外部条件工程化）；
5. 真定制（如有）：按 ADR-0003 复制模板开工；
6. 制度化运行：备份任务计划注册 + 一次恢复演练留档、健康告警阈值核对、医保每日自动对账确认开启（第 13 章）；

7. 培训与试运行：8 角色真人培训，试运行问题一律进故障工单台账。

流程之外有两个工程细节。其一是环境分离：试点/生产用独立的 `pilot profile`（连接串走环境变量、慢接口阈值从 500ms 收紧到 300ms、MLLP 按院启停、日志滚动归档）——开发环境的宽松默认值不允许带进生产。其二是上线前检查清单，逐项打勾且多数条目是“实测”而非“确认”：`admin` 旧口令登录须返回错误码、未采购模块直连 API 实测 404、生产库无演示账号、备份完成过一次恢复演练。检查清单的每一项都应当是一次可执行的验证动作，而不是一句“已完成”的自我声明。

实施文档的收尾是一张常见问题速查表，它的组织方式本身就体现分层思想——每个现象的处置都指回对应的层与规约：

现象	处置指向	所属层/规约
启动即失败	查 Flyway 日志，确认库空或版本连续；实施段仅允许 V10000+	L1 迁移纪律 / ADR-0003
某模块整页消失	检查 <code>module.*.enabled</code> ——可能实施时按未采购关闭	L2 配置
医保对账出现差异	核对适配器留痕是否符合 ADR-0002 关键字约定	L3 适配器契约
升级后 <code>impl</code> 接口报错	实施仓库对新版重编译 <code>impl</code> 模块，修兼容后再上线	L4 定制规约

支持工程师照表处置的同时，也在被表训练成按层思考——好的运维文档是组织结构的教材。

14.10 需求响应与偏离管理

14.10.1 矩阵：4062 行的应答义务

政府采购的医院信息化项目，技术需求书动辄数千条参数，投标与验收都要求逐条应答：满足/部分满足/不满足（偏离）。这份“参数矩阵”贯穿项目全生命周期——投标时是响应承诺，验收时是核对底稿，扯皮时是仲裁依据。HIP 面对的是 107 个模块、4062 条参数的矩阵，其收官过程是一套可复用的方法：

1. 机器初评：`matrix-autoreview.py` 用规则库对全量条目自动归类初评，人工只抽审存疑主题——4062 条逐条纯人工评审在成本上不成立；
2. 多轮滚动重评：每完成一批开发就把新能力喂给评审器重跑（初评符合率 74%→78%→80%→81%→83%），矩阵同时反向充当开发排期的输入——待评条目的主题聚类直接生成了后续各期的功能清单；
3. 证据包收口：终审剩 241 条待人工签认，逐条生成“证据包”再交人工；
4. 偏离表定稿：`export-deviation.py` 导出四栏响应底稿，冻结为技术偏离表定稿——冻结意味着此后的响应口径不再随开发漂移，验收谈判有了稳定文本。

最终口径：模块口径 107 项中软件可推进部分全覆盖（√≈99、外部条件≈6、服务项 2）；参数口径正偏离/无偏离 3388（83%），其余明确归入部分响应 152、配套产品 210、外部条件 28、非参数 43。注意这个结构：“不满足”不是一个筐——属配套产品的（CA、影像后处理、商用知识库）写明“平台接入点已留，参数由所购产品响应”；属外部条件的写明接入点与触发条件。把偏离分类到“谁、在什么条件下补齐”，偏离表才从扣分项清单变成实施路线图。

案例框 14-4：证据包——AI 辅助签认的形态

241 条待签认逐条人工从零判断仍然太贵。HIP 的做法是先机器生成证据包（`matrix-evidence.py`）：一组带优先级的规则，每条规则是三元组（正则模式，建议结论，证据/理由），例如：

```
(r'折旧', '功能等效', '资产直线折旧/净值/价值调整/折旧明细 (/hrp/assets,
as_value_adjust) '),
(r'耐药|ESBL|MDR|...', '部分响应', '院感专项已备: 三管千日率/现患率一键调查/报卡闭环;
耐药谱与手卫生依从率需检验微生物数据源, 属专业院感监测软件深化'),
```

输出是每条参数的「建议结论 + 指向具体功能页/表/错误码的证据指认」，规则命中不了的保持“真待议”交人工裁决。这个设计的要点有三：结论必须带证据（指认到 /hrp/assets 页和 as_value_adjust 表，签认者可当场核验，而不是信一句“已支持”）；机器只建议、人只签认——签字责任不可自动化，自动化的是把 241 条的判断成本压成几十条真待议 + 两百条核验；规则库即口径档案——为什么某类参数归配套产品，理由就写在规则里，半年后复盘有据可查。

与之配套的是 152 条部分响应的成本筛选（部分响应筛选）：逐条评估补全成本，挑出 8 条“一句话缺口”（如排班加上/下周快捷切换、审方队列上限键）排入定版后的第一个小版本 1.0.1，其余 144 条按成本因由归类维持部分响应——不硬刷条目数。且这 8 条快赢明确不进 1.0 冻结窗，保证彩排与打 tag 时产物稳定：响应率的每一个百分点都要用交付质量去换，换不起就诚实写偏离。

14.11 试点运行与反馈闭环

14.11.1 功能开发的边际判断

产品开发最难的决定之一是停下来。需求矩阵永远填不满，判断依据不应是“还有多少条没做”，而是新增开发对真实覆盖的边际贡献。HIP 在 v14 的决策记录是一个可引用的范本：当时剩余 532 条待评参数，团队没有直接排期，而是先做了主题聚类与抽样审读，发现其中 515 条是“既有模块的一句话变体”（如「输入床号换床」≈已有的转科换床）、平台型产品参数或外部能力，遂写下结论：“继续新增功能已无法有效提升真实覆盖，刷条目数只会稀释质量。”此后 v15 - v17 三轮规划的主要产出不再是功能，而是“签认结论、可上线的部署、运行中的反馈闭环”。

边际判断的信号可以一般化：新需求大多是既有能力的措辞变体；剩余条目集中在配套产品与外部条件；继续开发挤占的恰恰是硬化（安全基线、备份、演练）与工艺（初始化、发布）的工时——而后者才是上线真正缺的东西。

14.11.2 小版本模式与外部条件排队

试点期的开发形态随之切换为小版本模式：问题一律进运维故障工单台账（第 13 章）→ 分级 → 按需开小版本修复——不预设功能期，让真实运行反馈驱动排期，而不是让排期去猜。同时维护一张外部条件排队表：与采购方逐项确认省医保 SDK、CHS-DRG 官方目录、商用知识库、CA、微信资质等的可获得性，技术侧接入点全部先行就绪（第 6、10 章），行政侧申请最早启动——“到位即接”。定版后的开发被明确收敛为两种形态：版本迭代（进标准产品，所有医院受益）与 impl 定制（进单院隔离区），除此之外没有第三种写代码的理由——这正是 14.2 判层铁律在时间维度上的延伸。

14.12 验收方法论：虚拟医院全量彩排

14.12.1 为什么需要一次“假实施”

产品化的各项机制——种子分层、初始化工具、模块开关、impl 规约、适配器替换——每项都有各自的测试。但“能装进第二家医院”是这些机制组合作用的结论，组合的缝隙恰恰是单项测试照不到的地方：init 工具在 impl 迁移段存在时还幂等吗？关掉的模块会不会被某个 E2E 隐式依赖？替换了医保适配器，对账还认得留痕吗？回答组合问题的通用方法只有一个：端到端彩排——完整执行一次真实流程的全部步骤，只是对象是假的。飞行员用模拟机、灾备用演练，产品化用虚拟医院。

案例框 14-5: HIP 1.0.0 定版前的全量彩排

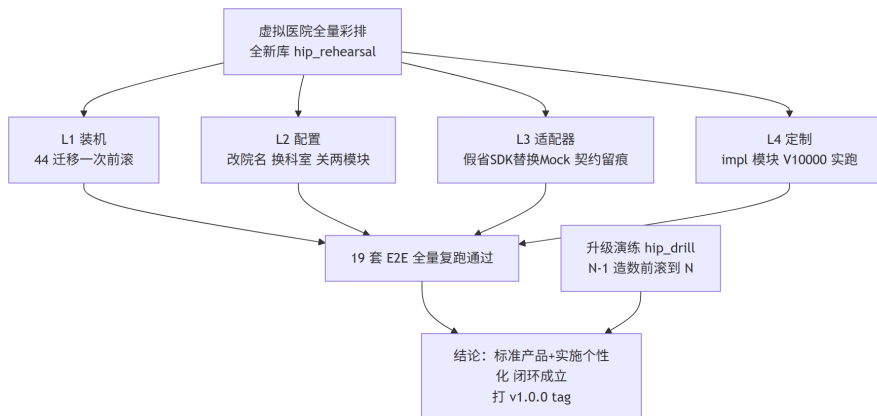
彩排在一个全新库（hip_rehearsal）上执行完整实施流程，逐项对应产品化机制（提交 5f51a80，2026-08-07）：

- 装机：44 个迁移（V1 - V43 平台段 + V10000 实施段）一次前滚成功——验证平台段与实施段在同库合并执行互不冲突；
- 初始化：init-hospital 改院名为「彩排第二医院」、建不同的科室结构——验证品牌与组织的全联动；
- 模块开关：关闭 blood/surgery 两域，逐一实测 API 404 后恢复——验证授权边界真实生效；
- 实施定制：impl 模板模块挂载实跑（ping 接口 + notes 表读写）——验证 V10000 段迁移与 /api/impl/* 路由的完整链路；
- 适配器替换：假省 SDK 医保适配器（FakeSdkInsuranceAdapter.java）以 @Primary 替换 Mock，严格履行 ADR-0002 契约——彩排产生 20 条 fake-sdk 留痕、YBSDK 前缀结算号正确回填业务表，医保业务与对账对实现替换零感知（第 10 章假适配器验证法的实施侧闭环）；
- 全量回归：19 套 E2E 在彩排实例上全部通过——这是彩排的判定条件：标准测试资产不加修改地在“另一家医院”上通过。

彩排通过后打 v1.0.0 tag。两个收尾细节体现纪律：彩排为挂载 impl 临时改过的根 pom 已还原——标准仓库保持不含任何 impl 模块（ADR-0003 底线）；彩排中发现两个 E2E 脚本隐式依赖默认院名/取列表首行，当场改为参数化语义与唯一名回查——彩排修的往往不是产品，而是测试资产里潜伏的“单院假设”，这类假设不彩排永远发现不了。

14.12.2 彩排证明了什么

图 14-3 概括彩排的验证矩阵——它的对象是四层模型的每一层机制及其组合：



要点在于分清彩排证明与不证明的东西：它不证明功能正确——那是 55 个单元测试与 19 套 E2E 平时的职责；它证明的是路径——“标准产物 + 分层配置 + 隔离定制”这条实施路径端到端走得通，且走完之后标准测试资产原样可用。对产品化而言这恰是最该验收的命题：功能是资产，可复制的实施路径才是产品。定版由此获得含义：1.0.0 不是“功能做完了”的里程碑，而是“复制路径被证明了”的里程碑。

本章小结

- 单院系统产品化的总原则是配置优先、扩展次之、禁止分叉：个性化需求按“改配置 > 换数据 > 实现 SPI > 实施层扩展”逐级排除，任何情况下不 fork 核心；每家医院 = 同一构建产物 + 该院配置数据 + 少量隔离定制。

- 四层标准化模型按变更主体分层：标准产品层随版本发布，配置层/SPI 层/定制层全部活在数据库与独立模块里——产物替换动不到它们，是升级敢做的结构性前提。运行期配套“先判层再动手”的处置铁律。
- 县市级医院场景选择单院独立部署而非多租户 SaaS：法人数据边界、院内网形态、故障半径都指向单实例；标准化收益来自同一产物，不来自共库。多院并行是“车队管理”：装机登记表 + 逐院滚动升级。
- 种子数据分平台必需/行业基线/演示数据三类；历史迁移不改，分层靠“新库初始化路径”实现：Flyway 照跑、init 工具覆盖行业基线、演示脚本生产禁跑。
- 初始化工程的三美德：幂等（现场可重跑）、安全基线内置（工具拒绝默认口令）、配置即档案（院配置 JSON 归档）。模块开关必须双路关闭——只藏菜单不拦接口是伪关闭。
- 实施定制层靠全维度命名空间隔离（impl 模块/V10000+ 段/impl_ 表前缀//api/impl 路由/20000+ 错误码），底线是禁改平台表结构；“升级后 impl 编译失败即兼容性破裂信号”把兼容检查前移到编译期。
- 发布工艺 = 语义化版本 + 完整发布物（文档工具随产物同版）+ 15 分钟升级流水线（停服→备份→换产物→前滚→回归→失败回滚）；升级工艺本身要用 N-1→N 演练验证。
- 需求矩阵的收官方法：机器初评滚动重评→证据包辅助签认（结论必须带证据，机器建议、人签认）→偏离分类到“谁在什么条件下补齐”→定稿冻结。部分响应按补全成本筛选，快赢进小版本，不硬刷响应率。
- 功能开发要做边际判断：当新需求多为既有能力的措辞变体时果断收束，转入“小版本 + 外部条件排队”的试点节奏；定版后只剩版本迭代与 impl 定制两种开发形态。
- 虚拟医院全量彩排是产品化的验收方法：在全新实例上完整执行实施路径（装机/初始化/关模块/换字典/impl 实践/适配器替换），以“全量 E2E 原样通过”为判定条件——它验收的是可复制的路径，而非功能本身。

思考与练习

1. 概念：解释“配置优先、扩展次之、禁止分叉”中每一级的含义，并说明为什么排序依据是升级摩擦而不是开发工作量。举一个看似该改代码、实际应落在配置层的医院个性化需求。
2. 判层：某医院提出四个需求：① 门诊统筹比例按当地政策调整；② 检验结果对接本院已有的 X 品牌生化仪；③ 增加一张该院独有的“院领导每日晨会报表”；④ 住院押金不足时希望系统允许“欠费医嘱”继续执行（平台当前拦截）。用四层模型逐个判层，说明理由；其中哪个应当反哺标准产品层、走什么流程？
3. 设计：为一个规划中的区域检验集中化系统论证部署形态：多家乡镇卫生院 + 一家县医院共用检验中心。单实例与多租户的论证条件与本章 14.3 有何不同？给出你的结论与关键假设。
4. 排障：某医院升级到新版本后，实施工程师报告“定制的报表接口全部 500”。按 ADR-0003 的规约与升级流程，列出排查顺序，并指出最可能被跳过的那个升级前置步骤。
5. 设计：参照 14.5，为“病区搬迁”（某病区整体迁往新楼，科室代码不变、床位重新编号）设计一个幂等的实施工具：写出输入配置结构、执行步骤与幂等策略，并说明哪些动作必须拒绝自动化、留给人工确认。
6. 论述：14.10 的证据包方法把机器放在“建议”位、人放在“签认”位。论述在验收场景下这样分工的必要性；如果直接让模型输出最终结论并自动填表，会在工程与责任两个层面引入什么风险？
7. 论述：14.11 引用的边际判断依据是“新增开发无法有效提升真实覆盖”。有观点认为投标场景下响应率每高一个百分点都有商务价值，应当继续刷条目。结合偏离表的分类口径（部分响应/配套产品/外部条件）与 1.0.1 快赢包的取舍，评析这两种立场，给出你的裁决标准。

8. 实践（配套实训环境）：完成一次微型彩排：建全新库，用 `init-hospital` 初始化一家“实验医院”（不同院名、关闭 `hr` 模块），复制 `impl` 模板为 `impl/lab-hospital`（改包名与路由前缀），执行 `V10000` 段迁移并实测定制接口；随后跑 `e2e-product1` 与 `e2e-outpatient` 两套回归。记录全过程发现的“单院假设”（如有），并按 14.12 的口径写一段彩排结论。

附录 A AI 在医院信息化中的应用

本书正文多处提到“AI 属外部条件”（第 6、9、14 章）。本附录把这句话展开：以 HIP 平台两个真实存在的“智能”功能为标本，先看清它们的实现形态与边界设计，再推及影像 AI、CDSS 知识库与大模型应用的接入谱系。附录的立场与全书一致——如实写：HIP 在本书成稿时没有接入任何真实模型，它交付的是接入的工程结构，这恰恰是最值得讲的部分。

A.1 两个“智能”的真实形态：读码确认

智能导诊是患者端(/portal)的功能：输入症状，系统推荐就诊科室。读其实现(PortalController.java 的 /api/portal/guide)会发现它既不是模型也不是外部服务，而是一条 SQL——按症状关键词 LIKE 匹配 triage_guide 规则表（V27 迁移内置 12 条种子，如“胸痛→急诊科，勿自驾车”“发热→发热门诊，请佩戴口罩”），返回科室与就医建议。它属于种子数据分层中的 C 类演示数据：业务页可维护、按院增删。一张几十条的规则表撑起的“智能”，恰是行业里大量“智能导诊”产品的真实底色——关键词规则起步，知识图谱与对话模型是后话。

导诊的患者端集成方式也值得一看：入口在患者端 H5 首页，与预约挂号、报告查询并列；接口挂在 /api/portal 下，走独立的 PORTAL 令牌与 ROLE_PORTAL 角色（与院内接口完全隔离，第 5 章）。推荐结果直接衔接“预约挂号”动作——导诊的产品价值不在推荐算法多聪明，而在把“不知道挂哪科”的患者引导进正确的号源，因此它天然属于患者端而非医生端。

ai-service 则是平台外的独立 HTTP 服务：一个 FastAPI 骨架（main.py，全部实现 60 行 Python），定义了两个接口——/analyze/lab 检验结果智能解读、/analyze/image 影像风险评分。当前实现是规则版 mock：检验解读按异常标志（HH/H/L/LL）查一张四项指标的规则字典给出建议文本，影像评分返回固定值；两个接口的响应都带 model: rule-mock-v0 字段，/health 自报 mode: rule-mock——服务在协议层就诚实声明自己是假的（第 6 章“诚实的 Mock”范式）。其文件头注释写明了设计意图：真实模型到位后替换实现即可，主平台通过 hip.integration.ai-url 指向本服务，接口契约不变。

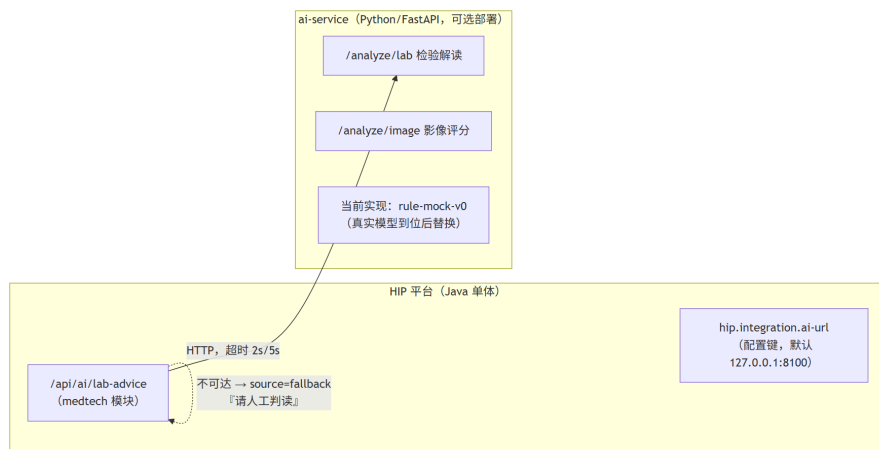
A.2 边界设计：为什么 AI 能力放平台外

ai-service 是全平台唯一的 Python 组件（Java 主体之外），且是部署表中唯一标注“可选（缺席自动降级）”的服务（多医院部署操作指南）。这两个“唯一”背后是四条理由：

1. 技术栈异质。AI 生态（PyTorch、模型运行时、GPU 驱动）整个生长在 Python 侧，塞进 Java 单体只会两头别扭；
2. 生命周期异步。模型按周迭代、平台按版本发布，独立进程各自升级互不牵连；
3. 资源形态不同。推理需要 GPU 的那天，独立服务可以单独换机器，平台不动；

4. 供应商现实。医院的 AI 能力多为外采成品（影像 AI 一体机、商用 CDSS），本来就是外部系统——平台内建 AI 反而是少数派。

整体结构如图 A-1：



平台侧的调用代码（MedTechController.java 的 `/api/ai/lab-advice`）体现了对“可缺席”的完整处理：连接超时 2 秒、请求超时 5 秒，任何异常不报错，而是返回 `source: fallback` 与一条建议——“AI 子服务不可用，请人工判读”。优雅降级的要点在语义：AI 建议是增强项，缺席时业务照常，医生看到的是“此路暂不可用”而非系统故障；返回体里的 `source` 字段让前端能标注建议来源，也让排障者一眼分清“AI 说的”与“兜底说的”。

A.3 医院 AI 应用的接入形态谱系

影像 AI。商用形态多为厂商一体机/云服务：从 PACS 侧订阅 DICOM 影像流，推理后把结构化结果（病灶标注、风险分级）回写 PACS 或以报告形式推给 RIS。对 HIS 平台，接入点通常只有两个：报告界面的调阅链接、结果的结构化回传。HIP 的形态（第 9 章 9.4）与此对齐：平台与 PACS 只以一个 URL 松耦合，`/analyze/image` 接口预先定义了“送描述、回评分”的契约——真实影像 AI 到位时，替换的是 ai-service 的实现，平台调用代码不动。

CDSS 知识库。第 7 章的合理用药五重拦截走自建规则表（`cdss_rule`，FORBID/CAUTION 分级），第 10 章的医保审核同构。自建规则库的天花板是内容规模：数万条药品相互作用、适应症限定规则依赖专业知识库团队持续维护，这是商用知识库（外部条件）的价值。工程答案与第 10.4 节一致：管道自建、内容外采——规则表结构、触点嵌入、分级干预、命中留痕自己建好，商用知识库接入时灌内容即可。

大模型（LLM）应用。病历文书辅助生成、报告解读、导诊问答是当前热点，但合规边界必须先于功能画清：《互联网诊疗监管细则（试行）》明确处方应由接诊医师本人开具，严禁使用人工智能等自动生成处方【成稿核对：文号与条款】；生成式 AI 服务另受《生成式人工智能服务管理暂行办法》约束【成稿核对：适用口径】；若产品宣称辅助诊断功能，还可能落入医疗器械软件注册管理范围【成稿核对：SaMD 分类界定】。工程上的保守姿态是：LLM 输出定位为供医师采信的草稿与提示，最终签名责任始终在人；面向患者的输出尤须克制——HIP 患者端连危急值都遮蔽为“请尽快回院复诊”（避免患者误读延误，见第 7 章），让模型直接向患者解释病情，审慎程度只应更高。

A.4 「AI 属外部条件」的工程化

把 A.1 - A.3 收拢，会发现 AI 接入完全落在第 6 章“外部条件工程化”的框架里：接入点先行（HTTP 契约与配置键先定义）、规则版兜底（rule-mock 让页面、调用链、E2E 在无模型环境下成立）、条件到位即接（换实现不换契约）。ai-service 的 HTTP 契约，本质上是医保 `InsuranceAdapter` SPI（第 10 章）的跨语言等价物——SPI 用 Java 接口收敛差异，AI 用 HTTP 接口收敛差异，思想同源。

两者有一处关键差异值得作为本附录的结束语：医保适配器失败必须同事务回滚（记账类依赖，宁停勿错），AI 服务失败只需降级提示（增强类依赖，宁缺勿扰）。为外部依赖选择正确的失败语义，比接入本身更能体现架构判断——这也是“AI 属外部条件”这句朴素结论背后，真正的工程内容。

附录 B 实训环境搭建指南

本附录面向使用本书开课的院校与培训机构：在一台普通机器上搭起完整的 HIP 平台，初始化一所虚拟医院，跑通一条门诊全流程，并把各章的实践题落到实处。装机细节以 部署手册 与 多医院部署操作指南 为准，本附录只写实训场景的关键节点与常见坑。

B.1 环境要求

单机即可完成全书实验（后端 + 数据库 + 前端同机），建议内存 8GB 起。两种组织方式：教师机集中部署、学员浏览器访问（省事，适合业务章节）；或每组一套独立实例（必需，适合第 4、5、14 章要动源码与数据库的实验——一院一实例一库本就是产品的部署形态）。

软件	版本	说明
操作系统	Windows 10/11 或 Linux	生产形态为 Linux/信创麒麟，实训 Windows 即可
JDK	21（Temurin）	平台为 Java 21 + Spring Boot 3.5.5（勿用 17，编译不过）
Maven	3.9+	构建后端多模块工程
PostgreSQL	16	Docker、WSL 或系统安装三选一
Node.js	LTS 版本	仅前端开发/构建期需要（Vue 3 + Vite）
Python	3.10+	初始化工具与 E2E 回归脚本（仅标准库，无需 pip 依赖）
ai-service	可选	FastAPI (pip install fastapi uvicorn)，缺席自动降级，不影响任何实验

B.2 装机步骤与常见坑

按 README「本地启动」三步走：起 PostgreSQL → mvn -pl server -am spring-boot:run → cd frontend/shell && npm install && npm run dev，浏览器打开 http://localhost:5173，admin/admin123 登录。关键节点与坑：

- 数据库先行且库要空。建库 hip、用户 hip；首次启动 Flyway 自动执行全部迁移（V1 - V43，含平台必需与行业基线种子）并创建 admin/admin123。启动即失败十有八九是库不空或迁移版本不连续——实训中“推倒重来”的正确姿势是删库重建，让 Flyway 重跑，而不是手工补表；

2. **Windows 下 Docker Desktop** 不稳时用 **WSL PostgreSQL**(仓库开发即此形态, `tools\dev-start.ps1` 一键起); Linux 直接 `cd deploy && docker compose up -d`;
3. 端口清单: 后端 8080、前端开发 5173、MLLP 设备模拟 2575、ai-service 8100、Orthanc/PACS 8042。教学机房如有端口管控需提前报备;
4. 登录锁定: 密码连错 5 次锁 15 分钟——演示“防爆破”很直观, 但学员集体输错时请等待或换账号, 平台无解锁后门;
5. **JWT** 密钥: 未设 `HIP_JWT_SECRET` 时回落开发占位值, 实训可接受; 借实训环境向学员演示第 13 章安全基线时, 应现场设置一次并说明“生产严禁占位值”。

B.3 初始化一所虚拟医院

实训环境走两条路线, 对应两个教学目的:

快路线(业务实验用): 后端启动后执行 `python tools/bootstrap-demo.py` ——幂等注入演示患者、8 角色演示账号(统一密码 Demo1234)、体检套餐等引导数据, 配合 培训脚本 即可分角色开练。注意此脚本仅限实训/演示库, 生产库严禁运行(上线检查清单专门查它)。

工艺路线(第 14 章实验用): 用生产同款初始化工具 `init-hospital.py` 走一遍真实施: 复制 `tools/init-templates/hospital.example.json`, 填写院名、模块开关(六域: `drg/cdss/insurance/blood/hr/surgery`)、科室与病区床位(NURSING 类型自动建床)、业务账号、药品/收费项目/医保对照三张字典 CSV, 然后 `python tools/init-hospital.py 配置.json`。工具依次完成登录→机构配置→模块开关→科室床位→账号→字典导入→强制更换 **admin** 口令七步, 幂等可重复。给虚拟医院起个像样的名字——院名会联动登录页、票据抬头与患者端, 学员能直观看到“配置优先”的含义。

B.4 验证清单: 跑通一条门诊全流程

装机与初始化完成后, 手工走通下表(对应第 2 章业务链与第 7 章各节), 全绿即环境可用:

#	步骤	入口	预期
1	患者建档	患者管理	身份证号自动推算性别生日；同证件号重复建档幂等返回原档
2	排班与挂号	门诊挂号	选号源挂号成功；余号减一，满号拦截
3	收挂号费	门诊收费	挂号费自动并入待收（REG 订单行）
4	接诊与病历	门诊医生站	候诊列表见患者；病历保存后出现 CDSS 建议条
5	开单	医生站处方/检验	开一条药品 + 一条血常规；试开“头孢+藿香正气”体验配伍拦截（4015）
6	收费	门诊收费	订单转已收费；选医保支付可观察费用分割明细
7	检验执行	LIS 工作台	采样打码→核收→录入结果发布；录 HH 值触发危急值弹窗，医生站须确认
8	发药	药房发药	发药成功、库存核减；重复发药被拦
9	患者端	/portal	患者号+手机号登录，可见挂号、报告（危急值遮蔽为回院提示）、智能导诊

自动化对照：`python tools/e2e-outpatient.py` 覆盖同一闭环外加退费/退号拦截等分支，30 秒内应全部通过。E2E 脚本本身就是教材——每个断言都是一条业务规则（第 13 章 13.6）。

B.5 各章实践题与实训环境对应表

各章“思考与练习”的实践题所需的模块与工具如下（模块开关默认全开，若做过关闭实验记得恢复）：

章	实践内容	依赖模块/组件	相关工具
2	8 角色全流程走查	全模块默认配置	bootstrap-demo、培训脚本
4	写一篇 ADR（叫号队列选型）	无（读源码）	docs/adr/ 现有 ADR 为范本
5	新增 sys_config 配置键全流程	源码构建环境	Flyway 迁移、配置手册
6	内容路由正则匹配改造	集成平台	e2e-integration.py、e2e-mlp.py
7	急诊占床并发防重	门急诊	两终端并发复现
8	移动工作台“待执行医嘱”页签	住院线	/m 移动端
9	CSSD 器械包状态机设计	medtech（参照 LIS 建模）	—
10	医保审核规则 R004	module.insurance.enabled=1	e2e-insurance.py
11	CDR 水位与脱敏修补	datacenter	e2e-phase1821.py
12	数据质量核查新规则 + 单测	datagov	mvn test
13	编写 e2e-ops.py 工单闭环	运维中心	tools/e2elib
14	微型彩排：新库+init+impl 定制	独立实例与空库	init-hospital.py、e2e-product1.py

运行期遇到问题，先查 多医院部署操作指南 第五节“常见问题”，再按“问题分层归位”判层——这个排障习惯本身就是第 14 章要教的内容。

附录 C 术语表

收录全书高频核心术语，按主题分组；每条给出一句话定义与首次系统讲解的章节。

C.1 业务与临床

术语	定义	详见
HIS (Hospital Information System)	医院信息系统, 覆盖挂号收费、医嘱、药事等核心业务流程的系统总称	第 4 章
电子病历 (EMR)	以电子方式记录、存储与调阅的诊疗文书体系, 含门诊病历与住院三段文书	第 8 章
EMPI (Enterprise Master Patient Index)	患者主索引, 用唯一患者标识归并跨系统就诊记录, 建档以证件号幂等	第 5 章
CDSS (Clinical Decision Support System)	临床决策支持系统, 在开单等触点给出拦截或建议, 分级干预 (FORBID/CAUTION)	第 7 章
LIS (Laboratory Information System)	检验信息系统, 管理标本采集、上机、结果审核发布的全流程	第 9 章
RIS (Radiology Information System)	检查(放射)信息系统, 管理检查登记、报告书写与审核发布	第 9 章
PACS (Picture Archiving and Communication System)	影像归档与通信系统, 负责医学影像的存储与调阅	第 9 章
CDR (Clinical Data Repository)	临床数据中心, 与业务库分离的诊疗文档汇聚库, 支撑患者 360 与检索	第 11 章
HRP (Hospital Resource Planning)	医院资源计划, 资产、物资、人事等运营后勤管理系统	第 2 章
统一订单行	处方、检查、检验、治疗共用一张订单行模型, 一体化开单与计费	第 7 章
记账式医嘱	医嘱执行即产生费用记账的住院费用模型, 行级费用的来源	第 8 章
危急值 (Critical Value)	危及生命须立即处置的检验结果 (HH/LL), 触发通知一确认一留痕闭环	第 6 章
TAT (Turnaround Time)	检验周转时间, 从采样(或核收)到发布的时长考核口径	第 9 章
Braden/Morse 量表	压疮/跌倒护理风险评估量表, 评分自动分级并进预警清单	第 8 章
临床路径 (Clinical Pathway)	按病种预定义的标准诊疗流程模板, 入径批量开嘱、变异留痕	第 8 章
预检分诊	急诊按病情紧急程度分级 (I - IV 级) 排队的制度	第 7 章
病案首页	住院病案的结构化摘要文书, DRG 入组与统计上报的数据源	第 12 章
HAI (Healthcare-Associated Infection)	医院感染, 住院期间获得的感染, 院感监测的对象	第 12 章
三管千日率 (VAP/CLABSI/CAUTI)	呼吸机/中心静脉导管/导尿管相关感染率, 按千置管日计算	第 12 章
现患率调查 (PPS)	某时点在院患者感染情况的横断面调查	第 12 章
DDD/AUD	限定日剂量及由其折算的抗菌药物使用强度 (国考 ≤ 40 参考线)	第 12 章

C.2 架构与工程

术语	定义	详见
模块化单体 (Modular Monolith)	单进程部署、内部按模块严格分界的架构形态, HIP 的总体选型	第 4 章
微服务 (Microservices)	按业务能力拆分为独立部署服务的架构风格, 运维成本随之上升	第 4 章
SOA/ESB	面向服务架构及其企业服务总线中枢, 医院集成平台的上一代主流形态	第 4 章
Monorepo	前后端与工具共用一个代码仓库的工程布局	第 4 章
ADR (Architecture Decision Record)	架构决策记录, 以背景—决策—后果结构留痕重大技术选择	第 4 章
SPI (Service Provider Interface)	平台定义接口、按需替换实现的扩展模式, 收敛医保/支付/签名等外部差异	第 6 章
端口与适配器 (Ports & Adapters)	业务核心只依赖抽象端口、外设经适配器接入的架构模式, SPI 的理论原型	第 6 章
Mock 实现	模拟外部系统行为的替身实现, 须履行留痕契约并写明能力边界	第 6 章
事务发件箱 (Transactional Outbox)	业务与待发消息同事务落库、异步投递的最终一致模式	第 6 章
JWT (JSON Web Token)	自包含签名令牌, 无状态会话认证的载体	第 5 章
RBAC (Role-Based Access Control)	基于角色的访问控制, 用户—角色—权限 (菜单) 三级授权模型	第 5 章
Flyway	数据库版本化迁移工具, 历史迁移不改是纪律红线	第 5 章
ModuleGate	HIP 的模块开关机制: 未启用模块菜单过滤 + API 404 双路关闭	第 14 章
四层标准化模型	标准产品/配置/SPI 扩展/实施定制四层, 医院差异的分层归位框架	第 14 章
种子数据分层	初始数据按平台必需/行业基线/演示三类管理, 决定初始化路径	第 14 章
语义化版本 (Semantic Versioning)	主.次.修订号的版本约定, 破坏性变更升主版本	第 14 章
外部条件	依赖院方资质、采购或第三方到位才能落地的对接项	第 6 章
接入点先行	外部条件未到位时先完成接口契约、配置键与 Mock 的方法	第 6 章
虚拟医院彩排	用一次完整的假实施 (新库初始化到 E2E 回归) 验证产品化闭环	第 14 章
幂等	同一操作重复执行结果不变的性质, 初始化与导入工具的基本要求	第 5 章
行级锁 (SELECT FOR UPDATE)	事务内锁定单行防“读—改—写”竞态, 号源/床位/年度额度的通用解法	第 10 章
E2E 测试	经真实 HTTP 接口串起业务闭环的端到端回归, 断言即业务规则	第 13 章

C.3 标准与数据

术语	定义	详见
HL7 V2	医疗消息交换标准，以竖线分隔的段结构报文，设备对接主流	第 6 章
MLLP (Minimal Lower Layer Protocol)	HL7 V2 的 TCP 传输封装协议	第 6 章
DICOM	医学影像的存储与通信标准，文件自描述、三层模型 (Study/Series/Instance)	第 9 章
CDA (Clinical Document Architecture)	HL7 临床文档架构，头+体结构的 XML 病历文档标准	第 11 章
ICD-10 / ICD-9-CM-3	国际疾病分类(诊断)与手术操作分类编码	第 12 章
LOINC	检验观测指标的国际标识编码体系	第 12 章
ATC (Anatomical Therapeutic Chemical)	药品解剖学-治疗学-化学分类编码，DDD 的挂靠体系	第 12 章
数据元 (WS 363)	卫生信息数据元标准，字段定义与值域代码的国家规范	第 12 章
OLTP	联机事务处理，业务库的负载形态，与分析负载分离是 CDR 的立论	第 11 章
CDC (Change Data Capture)	从数据库日志捕获变更的增量同步技术	第 11 章
医保编码贯标	国家医保局统一 15 项医保编码并要求医院落地对照的工作	第 10 章
FHIR (Fast Healthcare Interoperability Resources)	HL7 新一代互操作标准，以 RESTful 资源模型交换医疗数据	第 3 章
WS/T 500 系列	国家电子病历共享文档规范，基于 CDA 的文档模板家族	第 3 章

C.4 医保与 DRG

术语	定义	详见
三大目录	医保药品、诊疗项目、医疗服务设施三个可报销目录	第 10 章
目录对照	院内编码与医保编码建立映射的实施工作，对照率是上线门槛	第 10 章
甲/乙/丙类	全额纳入报销/先行自付后纳入/目录外自费的费用分类口径	第 10 章
先行自付	乙类项目个人先按比例自付、余额再进报销范围的规则	第 10 章
起付线/封顶线	进入统筹前的年度自付门槛与统筹基金年度支付上限	第 10 章
费用分割	把结算金额按医保规则拆为统筹支付与个人支付等部分的计算	第 10 章
冲正	退费时撤销医保侧入账的逆向操作，与本地退费同事务	第 10 章
医保结算号	医保侧返回的结算流水号，冲正与对账的必备关联键，必须持久化	第 10 章
对账（业务账/通道账）	本地结算单与集成留痕报文逐单核对，资金安全的最后防线	第 10 章
智能审核（WARN/BLOCK）	事前费用合规审查，从提醒留痕起步、精度达标后升级拦截	第 10 章
DRG（Diagnosis Related Groups）	疾病诊断相关分组，按病组打包付费的医保支付方式	第 10 章
CHS-DRG / DIP	国家版 DRG 方案与按病种分值付费两条并行技术路线	第 10 章
MDC / ADRG	主要诊断大类（26 个）与其下按诊断+手术细分的核心分组层	第 10 章
MCC/CC	严重并发症或合并症/并发症或合并症，决定细分组尾码（1/3/5）	第 10 章
CMI（病例组合指数）	总权重÷总例数，反映收治病例平均技术难度	第 10 章
费用/时间消耗指数	难度校正后的费用与住院日效率指标，1 为全院平均水平	第 10 章
高倍率/低倍率	实际费用相对支付标准的极端病例豁免通道，退出打包单议	第 10 章
QY 歧义组	无法入组病例的兜底组，不参与 DRG 支付并纳入质控	第 10 章
医保结算清单	DRG/DIP 付费的国家统一出院结算文书，要求行级费用明细	第 10 章

C.5 安全与运维

术语	定义	详见
等保三级	网络安全等级保护第三级，医院核心系统的合规基线	第 13 章
数据脱敏	按角色遮蔽证件号、电话等敏感字段的展示层防护	第 13 章
审计日志	写操作全量留痕（操作人/接口/内容），可追溯与举证依据	第 5 章
RPO/RTO	恢复点目标（可容忍丢多少数据）与恢复时间目标（可容忍停多久）	第 13 章
可观测性（Observability）	以指标、日志、链路三类遥测数据理解系统运行状态的能力	第 13 章
故障工单	运行期问题的登记—处理—结单台账，对账差异自动开单入此	第 13 章
问题分层归位	反馈先判层（配置/数据/适配器/平台/定制）再动手的运行铁律	第 14 章

C.6 政策与评价体系

术语	定义	详见
电子病历系统应用水平分级评价	卫健委主导的电子病历能力 0-8 级评价，公立医院普遍以 4 级为达标线	第 1 章
互联互通标准化成熟度测评	以标准化数据与文档共享为核心的医院信息互联互通分级测评	第 1 章
智慧服务/智慧管理分级	面向患者服务与医院管理两条线的智慧医院评估体系	第 1 章
三医联动	医疗、医保、医药改革协同推进的政策框架，医保支付改革的语境	第 1 章
信创	信息技术应用创新，国产化软硬件替代趋势，选型的备选约束	第 4 章

后记

这本书不是先有提纲再找例子写成的，而是从一个真实系统里“长”出来的。

它的源头是一个具体得不能再具体的工程任务：为县市级综合医院做一套买得起、跑得动、接得上医保的一体化信息平台。从第一张患者建档表到医保费用分割引擎，从深夜排查一笔对不平的结算差异，到用一所“虚拟医院”从零装机彩排整个实施流程——平台迭代到 v1.0.0 定版时回头看，那些反复出现的问题与反复奏效的解法，已经自己排成了一本书的目录。我要做的只是把它们如实写下来。

“如实”二字，是写作全程唯一的纪律，也是最难的部分。写书的人天然想把系统写得比实际更完整、把决策写得比当时更英明。本书选择了另一条路：以代码为最终事实。宣传口径与代码不符的，按代码写；文档声称有而代码没有的，写“规划与落地的差距”；当时判断失误造成返工的，把返工过程原样呈现。书中那些“教训案例”——被丢弃的冲正返回值、没地方存的医保结算号、抓不全增量的水位线——都曾是真实的深夜与冷汗。把它们印在书里，比印十条最佳实践更有价值，因为读者迟早会在自己的系统里遇见它们。

写作过程本身也是一次对平台的全面审计。为了确保每个案例可对照源码，全书的类名、表名、字段名都经过逐一核对；这个过程反过来揪出了平台里十几处文档与代码不符、乃至若干处真实的并发与安全缺陷——它们已进入修复流程。如果说这本书对 HIP 平台有什么回馈，这份“以写促审”的问题清单算是其中之一。

本书的写作与排版依托了一条完全开源可复现的工具链：Markdown 书稿、Mermaid 制图、Pandoc 转换、Typst 排版；平台本身则站在 Java、Spring Boot、Vue、PostgreSQL 及整个开源生态的肩膀上。向这些工具背后的社区致意。

书中仍有约六十处标注【成稿核对】的政策文号与标准条目待正式出版前逐条核定，也一定还有作者未能自察的错漏。医院信息化是一个政策、技术与临床三线并进的领域，书出版之日就是内容开始过时之时——唯愿书中那些不随版本更迭的东西：对业务的敬畏、对一致性的执着、对“如实”的坚持，能比具体的技术细节活得更久。

欢迎批评指正。

徐剑

2026 年 8 月